

Экзаменационный билет 2

1. Классификация операционных систем. Особенности ОС каждого типа. Декомпозиция системы на уровне иерархии: иерархическая машина (Медника, Донована), иерархическая структура Unix BSD, архитектуры ядер ОС - определение, отличительные особенности, ОС с монолитным ядром - система прерываний. Аппаратные прерывания: диаграмма "Запрос ввода/вывода в приложении". ОС с микроядром - концепция, ОС Mach и ОС QNX.

Ответ:

Операционная система - комплекс программ, которые совместно управляют ресурсами вычислительной системы и процессами, которые используют эти ресурсы

Иерархическая машина Медника-Донована

- появилась в конце 60-70 годов, но актуально и сейчас
- именно на его основе выделены функции, которые могут быть выполнены в режиме пользователя и те, которые необходимо выполнить в режиме ядра. -Компьютер - программно управляемое устройство. Железо и софт неотъемлемые части => уровни иерархии выстроены относительно аппаратной составляющей, которую когда-то кто-то из классиков назвал "голая машина".

2 билет

2.1 Классификация операционных систем. Особенности ОС определенных типов. Виртуальная машина и иерархическая машина – декомпозиция системы на уровни иерархии, иерархическая структура Unix **BSD**, архитектуры ядер ОС – определение, примеры.

Классификация операционных систем.

1. Однопрограммная пакетной обработки
В оперативной памяти может быть только 1 прикладная программа.
2. Мультипрограммной пакетной обработки
В оперативной памяти одновременно много программ. Загрузка – перфокартами. Программа располагается в памяти целиком; процессорное время выделяется по принципу приоритетов.
3. Системы разделения времени
В оперативной памяти одновременно находится большое число программ, процессорное время квантуется, чтобы обеспечить гарантированное время ответа системы. Время ответа системы ≤ 3 секунд.
4. Системы реального времени
Реальное время в операционных системах – это способность операционной системы обеспечить требуемый уровень сервиса в определенный промежуток времени, величина которого определяется особенностями работы внешнего по отношению к вычислительной системе устройства или процесса.
В отличие от систем общего назначения ОСРВ обеспечивает ответ системы (или сервис системы) за определенный промежуток времени, то есть обслуживание запроса. Это всегда запрос или внешний по отношению к системе или внешнего процесса.
2 процесса реального времени в наших компьютерах – видео и аудио.
Жесткое реальное время – это когда интервал установлен жестко и не может быть превышен.
Мягкое реальное время – возможность небольших отклонений от величины интервала.
5. Персональные операционные системы

Виртуальная машина – кажущаяся, возможная.

Виртуальная машина – набор команд и функций, необходимых пользователю для получения сервиса операционной системы

Иерархическая машина (МЕДНИКА-ДОНОВАНА) – ОС разбивается на функции и определяется место этих функций по удаленности от аппаратной части.



Управление процессами (нижний уровень) – выделение кванта времени.

Управление процессами (верхний уровень) – создание, уничтожение, взаимодействие при помощи сообщений.

Управление информацией – хранение, уничтожение файлов, файловая система.

Интерфейс – функции, которые нижние уровни предоставляют верхним уровням.

- прозрачные - позволяет обращаться через уровни
- полупрозрачные - какие-то обращения через уровни возможны (какие-то только к низлежащим уровням)
- непрозрачные - обращения возможны только к низлежащим уровням

Таблица 1: Структура ядра ОС UNIX 4.4 BSD

Системные вызовы				Сис. выз.		
Управление терминалом	1	2	3	4	Сокеты	Обработка сигналов
Необработанный телетайп	Обработанный телетайп	Файловая система	Виртуальная память	Сетевые протоколы	Создание и завершение процесса	
	Дисциплины линий связи	Буферный КЭШ	Страничный КЭШ	Маршрутизация		
Драйверы символьных устройств		Драйверы блочных устройств		Драйверы сетевых устройств	Планирование процессов	
					Диспетчеризация процессов	

- 1- Символьный уровень
- 2- Именованье файлов
- 3- Отображение адреса
- 4- Страничные прерывания

///- Аппаратные и эмулируемые прерывания

Архитектура ядер операционных систем

Существует два типа ядер:

1. Монолитные ядра
2. Микроядра

Монолитное ядро

Это программа имеющая модульную структуру, то есть состоящая из подпрограмм. Системы Windows, Unix, Linux имеют монолитные ядра. Unix имеет минимизированное ядро (вынесен графический интерфейс).

Микроядро

В микроядерной архитектуре все компоненты являются самостоятельными программами, которые выполняются в собственных адресных пространствах. В силу этого взаимодействие между компонентами ОС выполняется с помощью посылки и приема сообщений, причём механизм посылки и приема сообщений обеспечиваются специальным модулем ядра, который называется микроядром. Mach – первая ОС с микроядром. Классическим примером микроядерной ОС является Symbian OS.

Unix bsd (Berkeley System Distribution).

ОС с монолитным ядром и с микроядром

Структура ядра ОС

- Две базовые архитектуры ядер:
 - **монолитное** ядро
 - программа, имеющая модульную структуру, то есть состоящая из подпрограмм
 - каждое изменение ядра требует перекомпиляции
 - микроядерная архитектура
 - реализовано микроядро, в которое включены только самые низкоуровневые действия, выполняемые ядром
 - остальные компоненты реализованы в виде отдельных модулей, которые могут выполняться в режиме пользователя
- UNIX Linux — системы с **монолитным** ядром

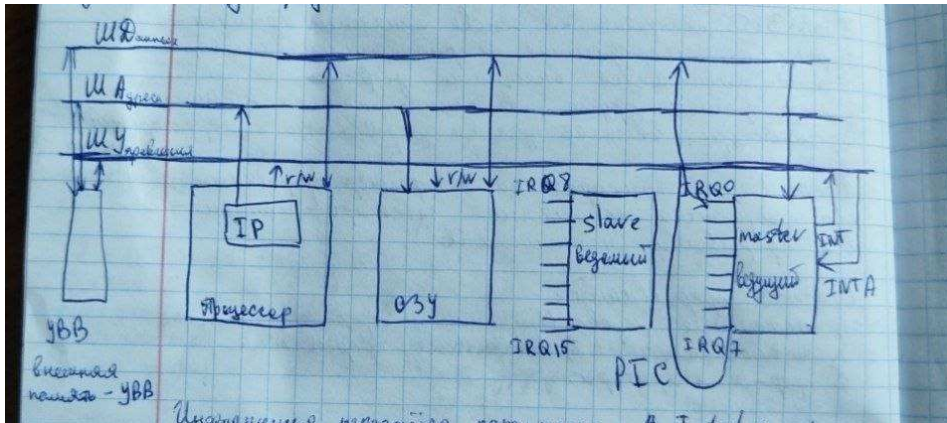
Unix/Linux имеют минимизированное ядро. То есть какие-то действия вынесены из ядра.

В Windows же, например, GUI входит в ядро, а в UNIX/LINUX из ядра вынесено.
- Изначально все операционные системы строились как операционные системы с **монолитными** ядром.
- Соответственно, в какой-то момент потребовалось определить места каждых важнейших в системе действий по отношению друг к другу

Аппаратные прерывания

Типы аппаратных прерываний:

- Асинхронные события в системе. Также имеют различный характер в системе. Самое массовое – прерывания от внешних устройств.
- Второй тип – прерывание от системного таймера, которое выполняется по тикку (выделенный импульс) 18,3 раз в секунду.
- Третий тип – прерывание от действий оператора (ctrl+alt+del, ctrl+C = завершение процесса в unix)



IRQ1 – сигнал от клавиатуры.

Когда завершается операция ввода/вывода, в буфер клавиатуры записывается код, контроллер отправляет его на ножку IRQ1. Формируется запрос INT и по шине управления поступает в процессор.

В конце цикла выполнения каждой команды процессор проверяет наличие сигнала прерывания и переходит в обработчик, прежде посылая через INT A сигнал.

Получив INT A контроллер формирует вектор прерывания, который по шине данных поступает в процессор. Вектор используется для адресации обработчика.

У контроллера прерываний есть порт, значит он адресуется.

3.1.2 Ввод-вывод

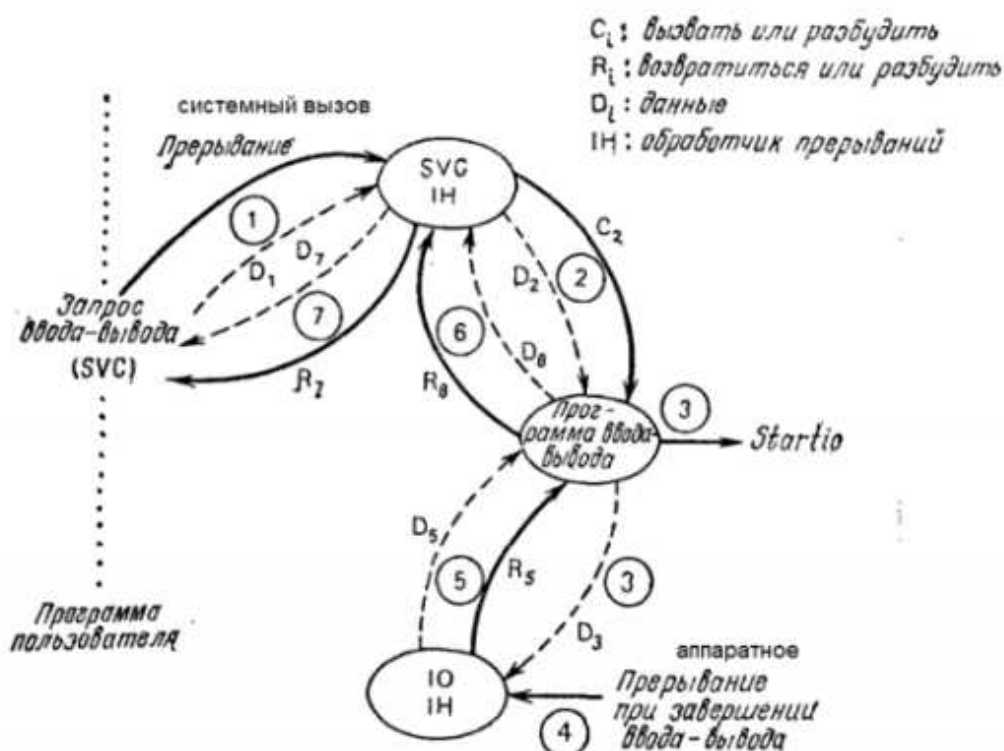


Рис. 1: Управление прерываниями в последовательности ввода-вывода

1 – системный вызов
 D1 – передача данных

D5 – данные сохраняются в буфере ядра

Ю ИН – выполняется на высоком уровне привилегий (должно быстро завершиться)

Программа ввода/вывода – драйвер

ОС в стадии выполнения – супервизор.

Обработчик системного вызова вызывает драйвер – код, который имеет много точек входа.

Драйвер предназначен для управления внешним устройством, его инициализация.

По шине данных посылается информация контроллеру.

Любой обработчик аппаратного прерывания входит в драйвер и является его точкой входа.

Mach - первая ОС с микроядром. Классическим примером микроядерной ОС является Symbian OS.

Ядро **Mach** управляет пятью основными абстракциями:

1) Процессы

Является основной абстракцией. Именно процесс владеет ресурсами. Он **не владеет приоритетом**, но имеет его, ему он назначается. Для пользовательских процессов значение приоритета изменяется.

2) Потоки

127

В Unix потоки имеют те же приоритеты, что и процессы. Если дополнительных процессов не создаётся, `main` рассматривается как один поток. Если создаются, то новые потоки рассматриваются как вспомогательные, но для системы это всё потоки. Приоритеты потоков меняются в зависимости от того, на чём поток был блокирован, сколько времени ожидал в очереди готовых потоков, но поскольку базовым понятием является процесс, никто не переписывает всё на потоки.

Потоки — часть кода программы, которая может выполняться параллельно с другими частями программы. Они не имеют собственного адресного пространства и выполняются в адресном пространстве процесса.

При этом поток владеет аппаратным контекстом (счетчиком команд). При выполнении они запрашивают ресурсы, но владельцем ресурсов будет процесс.

С памятью возникают проблемы, связанные с потоками.

Дополнение ядра `mach`: поддерживаются задачи и нити (`threads`). Под задачей понимает процесс, фактически статический объект, занимаемый адресное пространство и набор системных ресурсов.

3) Объекты памяти

Вводится понятие объекта памяти. Память страничная, но объекты памяти могут занимать одну или несколько страниц, то есть страницы в одной последовательности адресов в виртуальном адресном пространстве. Память страничная, есть пул страниц в виртуальном адресном пространстве.

При этом также реализованы страничные прерывания. Если при обращении к объекту оказалось, что он не находится в физической памяти, возникнет страничное прерывание.

При этом в отличие от монолитных систем, микроядро для загрузки в физическую память нужной страницы сформирует запрос. И этот запрос поступит на сервер режима пользователя, на сервер памяти. Соответственно, сервер памяти выполнит соответствующий запрос, может вызвать ещё какой-то код ядра. В итоге нужная страница будет загружена в физическую память. Процесс, обратившийся к объекту памяти, продолжит своё выполнение, поскольку это исправимое исключение.

При этом в современных Linux, они загружают сразу несколько страниц (пул) с тем расчётом, что дальше будет обращение к следующим страницам. Это не всегда происходит так. Тогда происходит проигрыш, поскольку в память загружены неиспользуемые страницы.

4) Порты

Это средство обмена сообщениями, то есть порт поддерживает упорядоченный список сообщений. Процесс владеет каким-то количеством портов для обмена

сообщениями с серверами или с другими процессами, поскольку параллельные процессы взаимодействуют.

Соответственно, для взаимодействия реализована возможность того, что процесс может предоставить другому процессу возможность посылать или получать сообщения из одного из своих портов

5) Сообщения (тут что-то ничего не сказала)

- **QNX** Neutrino RIOS v.6.2
 - Можно пользоваться бесплатно, устанавливается в качестве учебной. Ограничение: нельзя продавать изделия, в которой установлена эта система, зарубеж.
 - Вся система построена на микроядре:
минимизированное микроядро, из которого вынесены в отдельные процессы файловая система, доступ в сеть, ...
 - Основа любой микроядерной архитектуры — модель клиент-сервер, основана на передаче сообщений
 - Ядро обеспечивает передачу сообщений в любой системе
 - Предоставленные алгоритмы планирования:
 - FIFO
 - RR
 - sporadic scheduling (случайное, но на самом деле возникающее особенно)
(может быть соб. в ОС, кот. не имеет никакого периода)

2. Взаимодействие параллельных процессов: передача сообщений, диаграмма - три состояния блокировки процесса при передаче сообщений, очереди сообщений System V: правила получения сообщения из очереди, системные вызовы, пример кода.

Ответ:

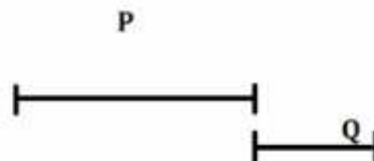
8 Взаимодействие параллельных процессов

8.1 Уровни наблюдения

Выделяют три варианта

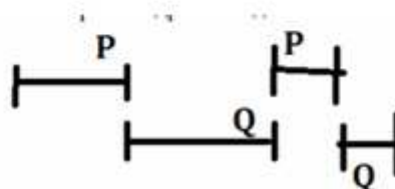
1) Последовательное выполнение

- Сначала выполняется первая программа от начала до конца, после этого начинает вторая от начала до конца и так далее.



2) Квазипараллельное выполнение

- Связан с квантованием процессорного времени
- Для первой программы выделен квант, он истёк. Квант выделяется для второй программы и так далее.
- На уровне команд это последовательное выполнение, но пользователю кажется, что оно параллельное, отсюда и название.
- Это характерно для однопроцессорных систем (но в какие-то моменты может быть и в многоядерных системах).



3) Реальная параллельность

63

- Выполнение кодов совпадает по времени.
-

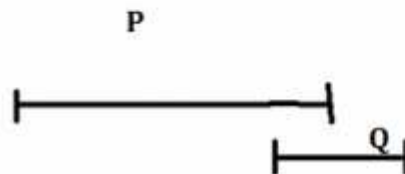
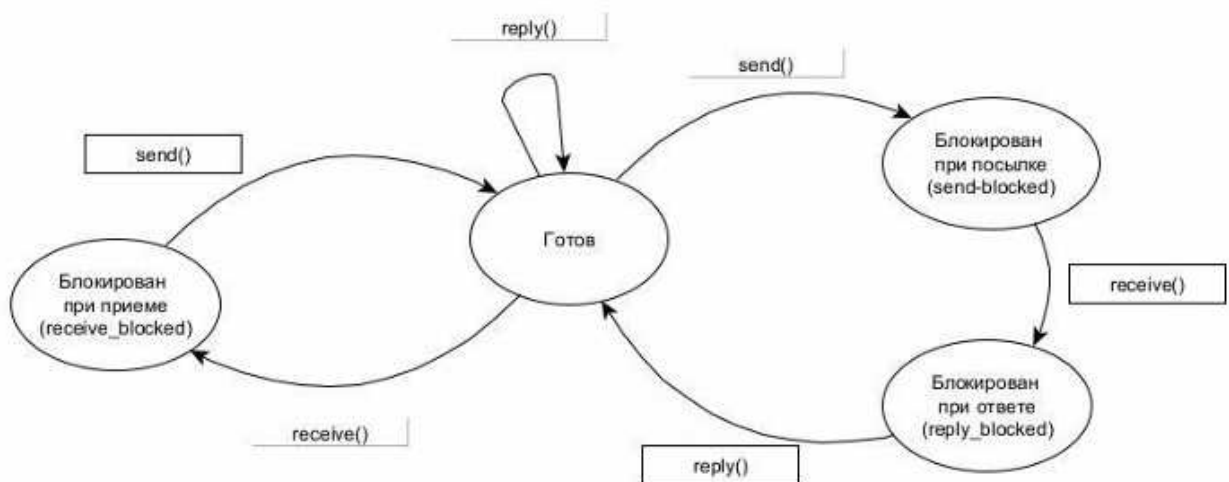


Диаграмма 3 состояния блокирования процесса при передаче сообщений

- **3 состояния блокировки процесса при передаче сообщения**



- Существуют разные средства, исключающие часть этих блокировок
- Диаграмма для самого строгого протокола взаимодействия
- Блокировки неизбежны, когда необходимо строгое взаимодействие. Первый процесс запросил данные, и он заинтересован в их получении, иначе ему нечего делать.

Очереди сообщений System V

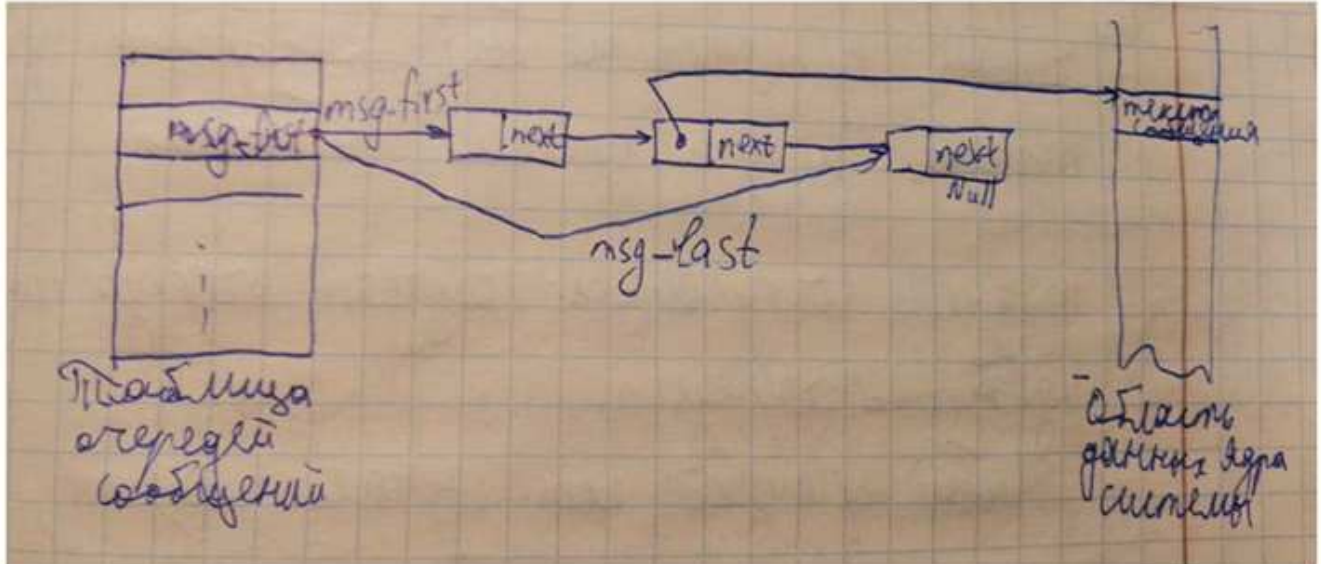
IPC (inter process communication)

К средствам межканального взаимодействия в System V относятся:

- сигналы
- семафоры
- именованные и неименованные программные каналы
- сегменты разделяемой памяти
- очереди сообщений
- файлы, отображающиеся в память (принято относить к более поздним средствам взаимодействия)

Очереди сообщений

- Сообщение — наиболее общее средство передачи информации
- Очереди в IPC System V реализованы на отдельно стоящих машинах.
- В системе имеется таблица очередей сообщений. Каждый элемент этой таблицы описывает одну очередь.



```
struct msgid_ds
msgget()
msgctl()
msgsnd()
msgrcv()
```

```
struct msgbuf
{
    long mtype; // тип сообщения
    char mtext[MSGMAX];
}
```

- Данные могут быть любыми, размер ограничен максимальным размером сообщения в системе
- Очереди сообщений построены таким образом, чтобы процессы могли не блокироваться при отправке или приеме сообщений
- Системный вызов `msgget` создает очередь сообщений, идентифицирует эту очередь.
После этого любой процесс, который знает идентификатор очереди, может отправить или взять сообщение

- Когда процесс передает сообщение в очередь(указав ее id), ядро создает для него новую запись и перемещает в конец списка записей указанной очереди
- В каждой такой записи указывается
 - тип сообщения
 - число байт данных
 - указатель на фактическую область данных ядра, где будет находиться сообщение
- Ядро копирует сообщение из адресного пространства отправителя в пространство ядра, после чего отправитель может завершиться, а сообщение остается доступным для чтения другим процессом
- Когда процесс выбирает сообщение из очереди, ядро копирует сообщение в адресное пространство процесса, после чего запись удаляется
- Итого: **очереди сообщений** предполагают, что процессы могут не блокироваться при отправке и при получении сообщения. Отправил и забыл.
- Сравним **очереди сообщений** и сегменты разделяемой памяти
 - очереди:
 - 2 копирования: при отправке (из пространства пользователя в пространство ядра) и при получении (из пространства ядра в пространство пользователя)
 - При использовании очередей сообщений может быть реализована динамическая связь.
- В System V сообщения можно извлекать не только по принципу FIFO, но и в некотором порядке, который может определяется приоритетами, но это потребует дополнительных настроек и действий.
- В IPC System V имеются совершенные средства взаимодействия процессов.
- UNIX BSD предоставляет одно универсальное средство взаимодействия — сокеты

Экзаменационный билет 3

1. Прерывания: система прерываний, особенности каждого типа прерываний. Аппаратные прерывания: виды и особенности каждого вида. Диаграмма состояний процесса. Последовательность действий при выполнении запроса приложения ввода-вывода с комментариями. Программируемый контроллер прерываний (PIC), маскируемые и немаскируемые прерывания. Адресация аппаратных прерываний - простейшая схема (16-разрядный режим).

Ответ:

Прерывания

6.2 События, переводящие систему в режим ядра

Фактически эта информация полностью коррелирует с системой прерываний. Три типа события, которые переводят систему из режима пользователя в режим ядра:

- 1) Системные вызовы (программные **прерывания**, Software Interrupts, Windows — traps, ловушки).
Это набор функций, которые предоставляются пользователю операционной системой, это не произвольные функции (говорили в POSIX).
- 2) Исключения (исключительные ситуации).
Исключительные ситуации в современных системах бывают двух типов:
 - 2.1) Исправимые — страничные **прерывания**. После того, как оно обработано системой, программа продолжает выполняться.
 - 2.2) Неисправимые — ошибки кода или системы (сбои). При них программа завершается, как мы говорим — аварийно.
- 3) Аппаратные **прерывания**
Принято различать три типа:
 - 3.1) **прерывания** системного таймера (единственные периодическое прерывание системы)

Диаграмма состояний процесса

6.3 Диаграмма состояний процессов Unix.

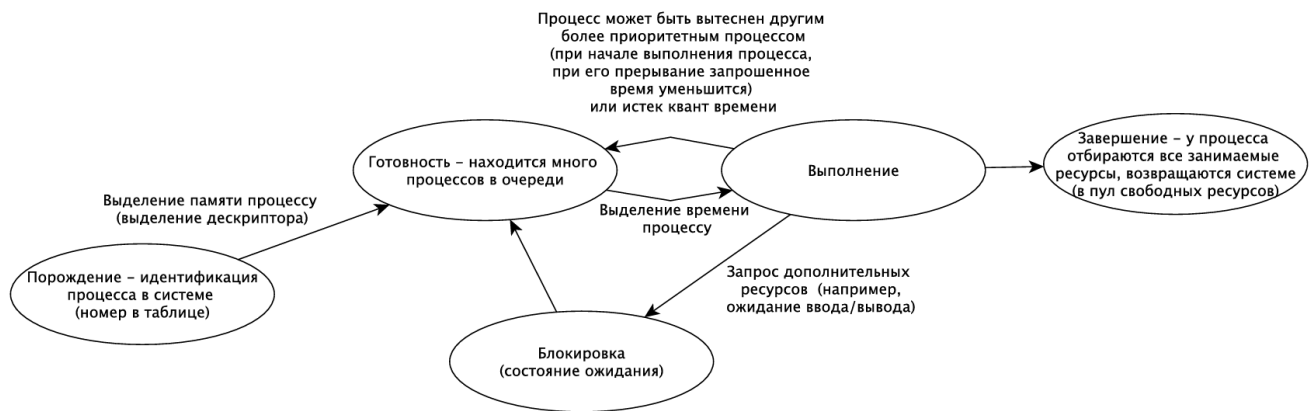
Постоянно происходит переключение в режим ядра и обратно. Это связано с переключением аппаратного контекста (регистров процессора). Переключение с помощью pusha и popa поддерживается аппаратно, потому что они выполняются очень часто.

На диаграмме есть очень много ситуаций, когда выполняется переход в режим ядра. Любой системный вызов приводит к переключению процесса в режим ядра. Есть два базовых системных вызова: read, write. Ни одна система не позволяет приложениям напрямую обращаться к устрой-

46

ствам ввода/вывода. Это базовый подход, обеспечивающий безопасность ОС.

1.1.3 Диаграмма состояний процесса



Аппаратные прерывания

- 3) **Аппаратные прерывания**. Их принято делить на
- 3.1) Прерывания системного таймера.
 - 3.2) Прерывания от устройств ввода-вывода.
 - 3.3) Прерывания действия оператора.

Про **аппаратные прерывания**[new]. Каналы – программируемые, задача - управлять работой внешних устройств, освободить процессор от непроизводительного управления внешними устройствами (внешние - механические, значит медленные). Что значит процессор управляет? Значит, что процессор, например, ждет от внешнего устройства данные (запросил их) и он будет опрашивать готовность

¹Под interrupt понимается именно аппаратное прерывание.

устройства, например, клавиатуры, когда же запрошенные данные окажутся в буфере данной клавиатуры. Т.е. будет тратить кванты процессорного времени на опрос внешнего устройства и делать другую работу не может. Выполняется непроизводительная работа. Поэтому реализовано распараллеливание функций и управление устройствами передано каналам (канальная архитектура). В результате появилась группа аппаратных прерываний от устройств ввода вывода. При этом прерывание от устройств ввода вывода информирует процессор о **завершении** операции ввода вывода.

Также включены прерывания действий оператора.

Ввод-вывод

3.1.2 Ввод-вывод

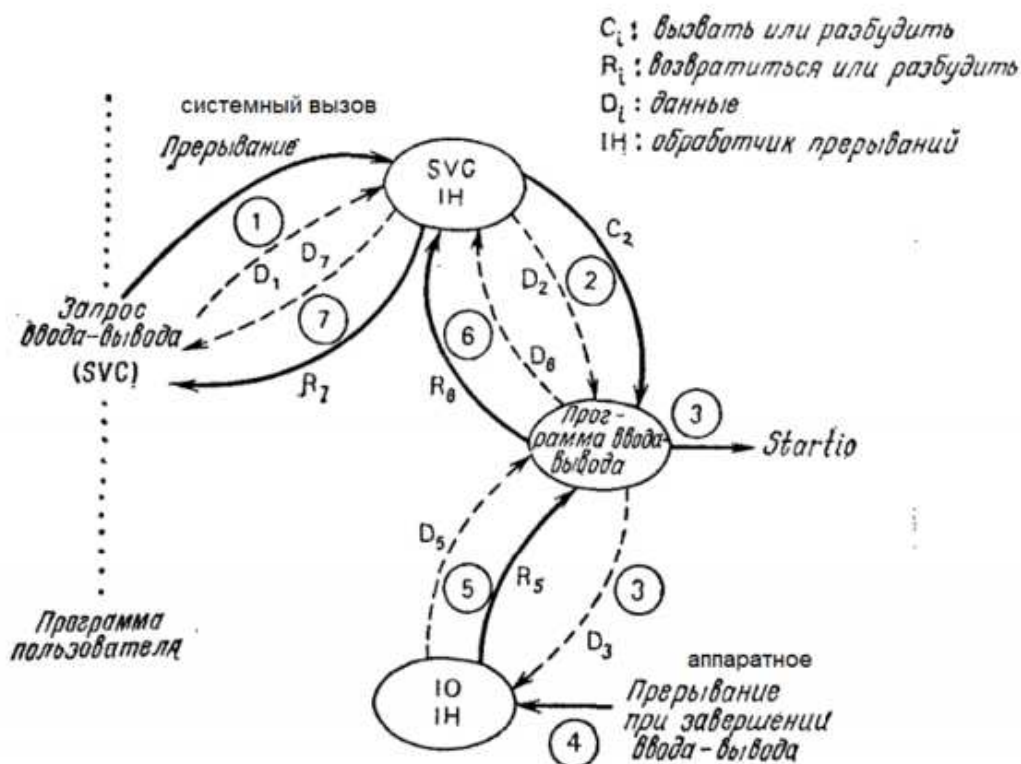


Рис. 1: Управление прерываниями в последовательности ввода-вывода

1 – системный вызов
D1 – передача данных

D5 – данные сохраняются в буфере ядра

Ю III – выполняется на высоком уровне привилегий (должно быстро завершиться)

Программа ввода/вывода – драйвер

ОС в стадии выполнения – супервизор.

Обработчик системного вызова вызывает драйвер – код, который имеет много точек входа.

Драйвер предназначен для управления внешним устройством, его инициализация.

По шине данных посылается информация контроллеру.

Любой обработчик аппаратного прерывания входит в драйвер и является его точкой входа.

10.23 Последовательность операций при запросе приложением ввода вывода Системный вызов является одним из типов прерываний в системе. Пунктиром в данной диаграмме обозначается передача данных. Любой системный вызов представляет из себя вызов функции. И у этой функции есть формальные параметры. Соответственно, при вызове эти параметры получают фактические значения. То есть осуществляется передача данных.	Поскольку приложение запросило ввод - вывод, ну, возмём простейший понятный вариант - чтение или запись из файла. Файл находится на внешнем устройстве, на внешнем запоминающем устройстве, device. Соответственно, в результате уже выполнения кода ядра будет вызван драйвер. Драйвера входят в код ядра, являются частью кода ядра (говорим о низкоуровневом драйвере, который
187	188
В какой-то момент времени приложение выполняет запрос (read-write - апи, системный вызов, переводит систему в режим ядра). Система переходит в режим ядра. (Эта схема из шоу, поэтому тут svc, вызов супервизора). То есть в результате вызова апи read-write система переходит в режим ядра, и это и есть вызов супервизора (Superviser, супернадсмотрщик). ИH - interrupt handler, системный вызов. У ядра есть такой слой функций, который вызывается сразу непосредственно при переходе в режим ядра, например, switch. Поскольку приложение запросило ввод - вывод, ну, возмём простейший понятный вариант - чтение или запись из файла. Файл находится на внешнем устройстве, на внеш-	не непосредственно управляет работой внешнего устройства). В результате драйвер сформирует из переданных параметров код, который по шине данных будет передан на контроллер, скажем, диска и в результате это будет инициализация вода вывода (на диаграмме startio). в соответствии с принципом распараллеливания функции (передача управления внешними устройствами специальным процессором для того, чтобы освободить по тем временам центральный процессор от задачи управления внешними устройствами, медленными внешними устройствами). В результате передачи по шине данных команд контроллеру внешнего устройства, контроллер внешнего устройства, поскольку у нас с вами не канальная архитектура, а шинная, начинает управлять работой внешнего устройства. Процессор отключается от управления внешним устройством. По завершение операции ввода-вывода (в схеме написано IO IN, IO interrupt handler). Прерывания от внешних устройств формируются контроллерами прерываний по завершении операции ввода-вывода. D1, D2, D3 - данные, то есть таким образом как бы показано, что запрос ввода-вывода, который привёл к вызову драйвера, драйвер переведёт данные запроса в формат (по-детски) понятному устройству (хз что именно тут она пыталась донести). Это будет передано по шине данных контроллеру. Вот этот разрыв (не помню, про что именно она говорила, наверное, про 3) означает, что процессор отключается и не
	189

Программируемый контроллер прерываний

управляет внешним устройством, внешним устройством управляет (в шинных архитектурах) контроллер. И по завершении ввода-вывода контроллер сформирует сигнал прерывания. Этот сигнал придёт на выделенную ножку контроллера прерываний. В результате контроллер прерываний сформирует сигнал int (сокращение от interrupt), будет по шине управление - это сигнал передан на выделенную ножку процессора. В конце цикла выполнения каждой команды процессор проверяет наличие сигнала на выделенной ножке. Если сигнал пришёл, то процессор отправляет сигнал int. Получив этот сигнал, контроллер формирует вектор прерывания. Процессор этот вектор прерывания в простейшей схеме шестнадцатизрядной использует как смещение в таблице векторов прерываний. Получив адрес обработчика прерывания процессор переходит на выполнение кода обработчика прерывания. При этом, поскольку обработчики прерывания выполняются на самом высоком уровне приоритета, вся выполняемая работа будет прервана, процессор перейдёт на выполнение обработчика. И, соответственно, устройство выполнит запрос. Например, это команда read, тогда какие-то данные должны будут быть записаны в буфер ядра. После этого опять же драйвер вызовет callback(?) функцию (точки прерывания устройств является одной из точек входа драйверов устройств). Драйвер устройства имеет один обработчик прерывания на функцию (есть многофункциональные внешние устройства, как бы набор из трёх типов внешних	управляет внешним устройством, внешним устройством управляет (в шинных архитектурах) контроллер. И по завершении ввода-вывода контроллер сформирует сигнал прерывания. Этот сигнал придёт на выделенную ножку контроллера прерываний. В результате контроллер прерываний сформирует сигнал int (сокращение от interrupt), будет по шине управление - это сигнал передан на выделенную ножку процессора. В конце цикла выполнения каждой команды процессор проверяет наличие сигнала на выделенной ножке. Если сигнал пришёл, то процессор отправляет сигнал int. Получив этот сигнал, контроллер формирует вектор прерывания. Процессор этот вектор прерывания в простейшей схеме шестнадцатизрядной использует как смещение в таблице векторов прерываний. Получив адрес обработчика прерывания процессор переходит на выполнение кода обработчика прерывания. При этом, поскольку обработчики прерывания выполняются на самом высоком уровне приоритета, вся выполняемая работа будет прервана, процессор перейдёт на выполнение обработчика. И, соответственно, устройство выполнит запрос. Например, это команда read, тогда какие-то данные должны будут быть записаны в буфер ядра. После этого опять же драйвер вызовет callback(?) функцию (точки прерывания устройств является одной из точек входа драйверов устройств). Драйвер устройства имеет один обработчик прерывания на функцию (есть многофункциональные внешние устройства, как бы набор из трёх типов внешних
	190
устройства; соответственно, там каждый тип будет иметь свой обработчик прерывания). Драйвер переведёт полученные данные в соответствующий формат и как бы в обратном порядке. В итоге данные поступят приложению (перед этим приложение должно быть разблокировано). Все это время, начиная с перехода в режим ядра, приложение заблокировано. Оно разблокируется системой, когда система передаст ему данные из ядра в буфер пользователя. И это называется в общей классификации ввода-вывода	

Прерывания

- Система прерываний:
 - системные вызовы (программные прерывания)
 - исключения

130

- аппаратные прерывания
 - от системного таймера (единственное периодическое)
 - от внешних устройств
 - от действий оператора (Ctrl+C)
- Система прерываний организуется через контроллер прерываний.
- Контроллер прерываний **PIC**(program interrupt controller) состоит из 2 микросхем, каждая по 8 входов, каскадное соединение. Входы контроллера – линии IRQ(interrupt request) (линии прерываний). Имеют номера. IRQ2 используется для каскадного соединения.
 - Линии IRQ связаны с внешними устройствами
 - IRQ – interrupt request
 - IRQ0 – тики
 - IRQ1 – от клавиатуры PC/2
 - IRQ2 – от мыши
- 1 контроллер – ведущий, master
- 2 контроллер – ведомый, slave

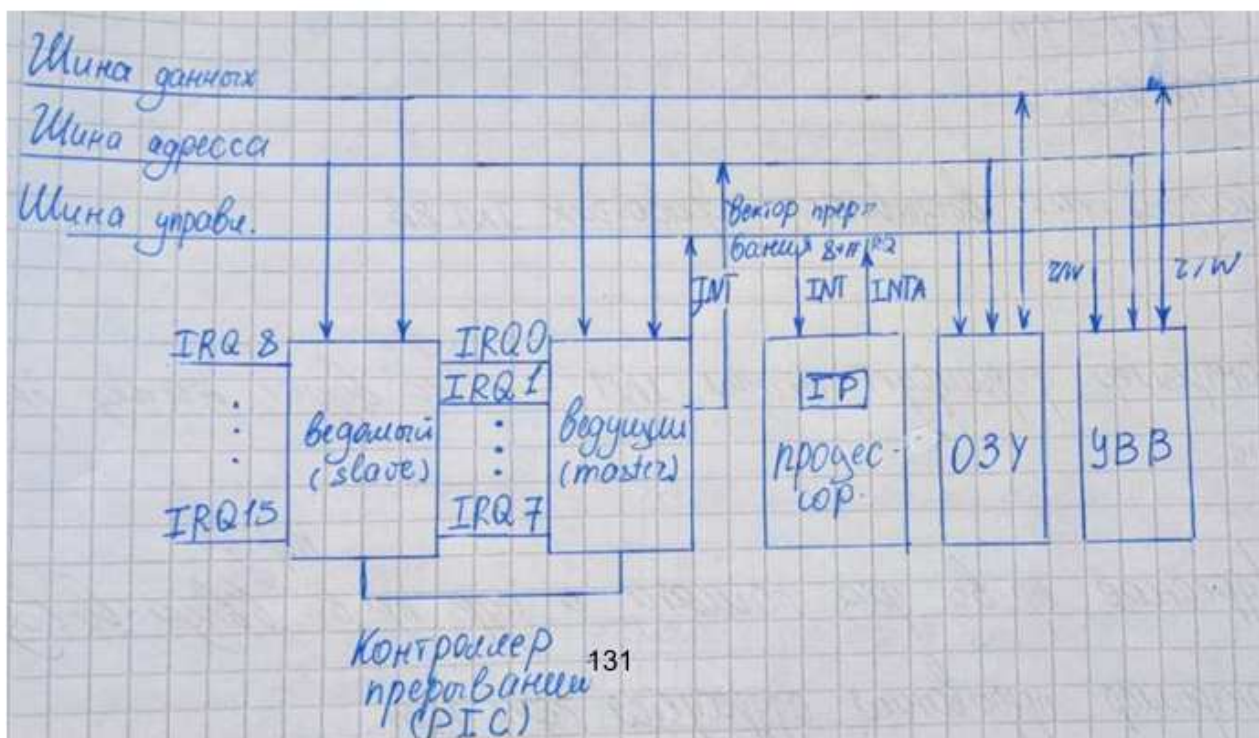
Прерывания

- Система прерываний:
 - системные вызовы (программные прерывания)
 - исключения

130

- аппаратные прерывания
 - от системного таймера (единственное периодическое)
 - от внешних устройств
 - от действий оператора (Ctrl+C)
- Система прерываний организуется через контроллер прерываний.
- Контроллер прерываний **PIC**(program interrupt controller) состоит из 2 микросхем, каждая по 8 входов, каскадное соединение. Входы контроллера – линии IRQ(interrupt request) (линии прерываний). Имеют номера. IRQ2 используется для каскадного соединения.
 - Линии IRQ связаны с внешними устройствами
 - IRQ – interrupt request

- IRQ – interrupt request
- IRQ0 – тики
- IRQ1 – от клавиатуры PC/2
- IRQ2 – от мыши
- 1 контроллер – ведущий, master
- 2 контроллер – ведомый, slave
- Когда на контроллер прерываний приходит прерывание от внешнего устройства, контроллер формирует INT, которые по шине управления поступает на выделенную ножку процессора.
- Цикл выполнения команды
 - → Выборка → Дешифрация → Выполнение
- В конце цикла выполнения команды ЦП проверяет наличие сигнала INT на выделенной ножке. Если он присутствует, то процессор по шине управления посылается INT A(answer)
- Получив этот сигнал от процессора(информирует о готовности обл. с к. д.) контроллер по шине данных процессору посылает вектор, который формируется как сумма базового вектора + номер линии IRQ.
- вектор = базовый вектор + №IRQ
- В реальном режиме базовый вектор ведущего контроллера равен 8, отсюда
INT8h — 8+0=8;
INT9h — 8+1=9; клавиатура
- В реальном режиме базовый вектор ведущего контроллера равен 8, отсюда
INT8h — 8+0=8;
INT9h — 8+1=9; клавиатура



- Аналогично маскированию прерываний в контроллере прерываний, маскированные прерывания, которые пришли из PIC, в MSI могут быть замаскированы.

Все это указано в драйвере устройства.

2. Задача "Обедающие философы" - модели действий философов и множественные семафоры, псевдокод одной из моделей. Множественные семафоры UNIX System V: поддержка в системе, системные вызовы, пример использования из лабораторной работы "читатели-писатели"

Ответ:

9.4.2 Задача **обедающих** философов

В современных операционных системах реализованы наборы считающих семафоров. Работу наборов демонстрирует задача **обедающих** философов, которую также сформулировал Дейкстра

На круглом столе стоят 5 тарелок, между которыми лежит по одному прибору. Философ может поесть только если ему удалось захватить сразу два прибора: левый и правый. Иначе он будет «голодать». Он думает некоторое время, потом пытается поесть. То есть пытается захватить приборы, захватив, ест. После этого освобождает приборы, опять какое-то время думает.

Существуют 3 варианта действия философов:

- 1) Каждый философ пытается захватить сразу два прибора: левый и правый. Если ему удаётся, он может начать есть. Поев, он освобождает сразу оба прибора.
- 2) Каждый философ сначала захватывает сначала правый прибор. Если ему это удалось, захватывает левый.
- 3) Каждый философ пытается захватить левый прибор. Если удалось, захватить, начинает есть. Если не удалось — освобождает левый.

Первый вариант может привести к «голодной смерти». Умереть может только 1 философ.

Скажем, сначала начальный момент времени второму и пятому удалось захватить оба прибора. Соответственно, первый, третий и четвёртый захватить оба прибора уже не могут. Поев, скажем, второй, освобождает оба прибора. Значит, третий может захватить оба прибора. Второй, первый, четвёртый захватить не мог. Если третий освободит, то, скажем, второй опять может захватить. Четвёртый не может, потому что пятый удерживает. Понятно? Если пятая освобождает, то четвёртый может захватить. Поев, освободит пятый может захватить. Значит, в итоге вот он первый, он никак не может захватить. То второй удерживает, то пятый.

Второй вариант. Каждому удаётся взять правые приборы, левые взять не могут. Всё расхватили. Тупик, блокируют друг друга.

И третий вариант. Взяли правый прибор, не могут взять левый, кладут правый. Опять взяли правый, не могут взять левый, кладут. Захваты, освобождения одних и тех же ресурсов.

И третий вариант. Взяли правый прибор, не могут взять левый, кладут правый. Опять взяли правый, не могут взять левый, кладут. Захваты, освобождения одних и тех же ресурсов.

Первый вариант - это бесконечное откладывание. Процесс не может захватить необходимые ему ресурсы. Второй вариант - это тупик. И третий вариант - захват и освобождения одних и тех же ресурсов. Работа выполняется системой, но полезные действия при этом отсутствуют. Ничего полезного не делается, всё тратится на захват и освобождение одних и тех же ресурсов.

9.4.3 Решение с помощью наборов семафоров

Все философы действуют, как вы видели, единообразно. Одинаковые действия выполняют. Соответственно, подробно мы запишем действие пятого. Ну, можно по-простому написать.

```
1 chops: array[1..5] of semaphore;  
2 i: int  
3 begin  
4   i := 5  
5   repeat  
6     chops[i] := 1  
7     i := i - 1
```

```

8  until i = 0;
9  cobegin
10     1: var left, right:
11         begin
12             left := 1;
13             right := 5;
14             ...
15         end;
16     ...
17  5:
18     begin
19     var
20         left, right: 1..5
21     begin
22         left := 5;
23         right := 1;
24     repeat
25         // Размышляет
26         P(chops[left], chops[right]);
27         // Ест
28         V(chops[left], chops[right]);
29     forever
30     end; // 5
31 coend;
32 end;

```

Листинг 10. Какой-то алгоритм?

Мы видим, что захват и освобождение выполняется сразу на несколькими семафорами набора. Ну, для пятого философа, значит, для пятого и первого приборов сразу, захват левого и правого прибора.

Самое важное свойство семафоров: одной неделимой командой можно изменить все или часть семафоров набора. Если в программе используется большое число семафоров, которые освобождаются и объявляются в разных функциях, очень сложно отладить программу и найти когда она входит в тупик.

Win, Unix, современный Linux поддерживают наборы считающих семафоров, они обладают важнейшим свойством: одной неделимой операцией можно изменить все или часть семафоров набора.

Если какая-либо операция не может быть выполнена над одним семафором, то неудачной считается операция над всеми семафорами.

Семафоры поддерживаются в ядре таблицей семафоров. Каждая строка описывает 1 набор из всех созданных в системе наборов семафоров.

1. Имя – целое число, присваивается процессам создавшим набор семафоров. Другие процессы могут по имени открыть.
2. UID – идентификатор создателя набора семафора и его группы. Если UID процесса совпадает с UID создателя, то может удалять набор и изменять его управляющие параметры.
3. Права доступа
4. Количество семафоров в наборе
5. Время изменения одного или нескольких значений семафора последним процессом
6. Время изменения управляющих параметров
7. Указатель на массив семафоров

О каждом семафоре известно:

1. Значение семафора
2. ID процесса, который оперировал с семафором в последний раз
3. Число процессов заблокированных на семафоре

В отличие от семафора Дейкстры, на семафоре в Unix определено 3 операции:

- Захват, декремент
- Освобождение, инкремент
- Проверка семафора на 0. Процесс выполняющий такую проверку переходит в состояние ожидания освобождения ресурса.

В системе определены структуры для работы с семафорами: `<sys/sem.h>`

```
struct sembuf
{
    n_short sem_num; // индекс
    short sem_op; // операция
    short sem_flg; // флаги
}
```

3 операции:

- `sem_op < 0`. Захват, декремент. Если не может то блокируется в операции.

- `sem_op > 0`. Освобождение, инкремент
- `sem_op = 0`. Проверка семафора на 0.

На семафорах определены флаги:

`IPC_NOWAIT` – информирует ядро о нежелании процесса переходить в состояние ожидания.

Это позволяет избежать очереди к семафору, в случае аварийного завершения или `kill`.

В силу того, что `kill` невозможно прекратить, убиваемый процесс не может осуществить освобождение семафора.

`SEM_UNDO` – указывает ядру, что необходимо отслеживать значение семафора, в результате завершения процесса, вызвавшего `sem_op`, ядро отменяет все изменения, чтобы процессы не были заблокированы навсегда.

1. Взаимоисключения, организация монопольного доступа процесса к разделяемой переменной.

Чистое взаимодействие реализуется в задаче читателя-писателя осуществляется монопольный доступ писателя.

2. Синхронизация, когда процесс заинтересован в действиях другого процесса.

Задача производства потребления, если потребитель работает быстрее производителя возникнет ситуация когда буфер пуст, потребитель будет ожидать когда производитель положит что-нибудь в буфер, это видно при передаче сообщений, сообщения несут информацию которая интересует процесс, для того чтобы продолжить свое выполнение.

Читатели-писатели

```
#define WAIT_WRITERS 0
#define WAIT_READERS 1
#define ACTIVE_WRITER 2
#define ACTIVE_READERS 3

struct sembuf start_read[] = {
    {WAIT_READERS, 1, 0},
    {ACTIVE_WRITER, 0, 0},
    {WAIT_WRITERS, 0, 0},
    {WAIT_READERS, -1, 0},
    {ACTIVE_READERS, 1, 0},
};

struct sembuf stop_read[] = {
    {ACTIVE_READERS, -1, 0}
};

struct sembuf start_write[] = {
    {WAIT_WRITERS, 1, 0},
    {ACTIVE_READERS, 0, 0},
    {ACTIVE_WRITER, 0, 0},
    {WAIT_WRITERS, -1, 0},
    {ACTIVE_WRITER, 1, 0}
};

struct sembuf stop_write[] = {
    {ACTIVE_WRITER, -1, 0}
};

int flag = 1;

void sig_handler(int signal) {
    flag = 0;
}
```

```

    printf("catch sig: %d -- pid: %d\n", signal, getpid());
}

void reader(int semfd, char *val);
void writer(int semfd, char *var);

int main(void) {
    int readers = 5;
    int writers = 3;
    int status;
    pid_t wchild;
    pid_t wchilds[readers + writers];
    int perms = S_IRWXU | S_IRWXG | S_IRWXO;

    if (signal(SIGALRM, sig_handler) == (void (*)(int)) -1) {
        perror("signal");
        exit(1);
    }

    key_t key = ftok("./key.txt", 1);
    if (key == -1) {
        perror("ftok");
        exit(1);
    }

    int semfd = semget(IPC_PRIVATE, 4, IPC_CREAT | perms);
    if (semfd == -1) {
        perror("semget");
        exit(1);
    }

    int shmfd = shmget(IPC_PRIVATE, 256, IPC_CREAT | perms);
    if (shmfd == -1) {
        perror("shmget");
        return 1;
    }

    char *buf = shmat(shmfd, NULL, 0);
    if (buf == (void *)-1) {
        perror("shmat");
        exit(1);
    }
    *buf = 'a';
    if (semctl(semfd, 3, SETVAL, 0) == -1) {
        perror("semctl");
        exit(1);
    }
    if (semctl(semfd, 2, SETVAL, 0) == -1) {
        perror("semctl");
        exit(1);
    }
    if (semctl(semfd, 0, SETVAL, 0) == -1) {
        perror("semctl");
        exit(1);
    }
}

```

```

}
if (semctl(semfd, 1, SETVAL, 0) == -1) {
    perror("semctl");
    exit(1);
}

for (int i = 0; i < readers + writers; ++i) {
    if ((wchlds[i] = fork()) == -1) {
        perror("fork");
        exit(1);
    }
    if (wchlds[i] == 0) {
        alarm(5);
        if (i % 2 == 0)
            reader(semfd, buf);
        else
            writer(semfd, buf);
        exit(0);
    }
}

for (int i = 0; i < readers + writers; ++i) {
    errno = 0;
    wchild = wait(&status);
    if (wchild == -1) {
        perror("Can't wait");
        printf("Errno: %d\n", errno);
    } else {
        if (WIFEXITED(status))
            printf("wchild: PID = %d exited with status = %d\n", wchild, WEXITSTATUS(status));
        else if (WIFSIGNALED(status))
            printf("wchild: PID = %d exited with status = %d\n", wchild, WTERMSIG(status));
        else if (WIFSTOPPED(status))
            printf("wchild: PID = %d exited with status = %d\n", wchild, WSTOPSIG(status));
        else
            printf("wchild: PID = %d exited with unknown status\n", wchild);
    }
}

if (shmdt(buf) == -1) {
    perror("shmdt");
    exit(1);
}

if (shmctl(shmfd, IPC_RMID, 0) == -1) {
    perror("shmctl");
    exit(1);
}

if (semctl(semfd, IPC_RMID, 0) == -1) {
    perror("semctl");
    exit(1);
}

```

```

    exit(0);
}

void reader(key_t semfd, char *var) {
    srand(getpid());
    while (flag) {
        sleep(rand() % 2 + 1);
        if (semop(semfd, start_read, 5) == -1) {
            if (errno == EINTR) {
                printf("PID %d, Errno \n", getpid());
                exit(1);
            }
            perror("semop");
            exit(1);
        }

        printf("reader: %d read %c\n", getpid(), *var);

        if (semop(semfd, stop_read, 1) == -1) {
            perror("semop");
            exit(1);
        }
    }
}

void writer(key_t semfd, char *var) {
    srand(getpid());
    while (flag) {
        sleep(rand() % 2 + 1);
        if (semop(semfd, start_write, 5) == -1) {
            if (errno == EINTR) {
                printf("PID %d, Errno \n", getpid());
                exit(1);
            }
            perror("semop");
            exit(1);
        }

        (*var)++;
        if (*var == 'z' + 1)
            *var = 'a';

        printf("writer: %d wrote %c\n", getpid(), *var);

        if (semop(semfd, stop_write, 1) == -1) {
            perror("semop");
            exit(1);
        }
    }
}

```

1. Тупики: типы ресурсов в теории тупиков и их особенности. Обнаружение тупиков для повторно используемых ресурсов методом редукции графа с демонстрацией действий на графе. Способы представления графа, алгоритмы обнаружения тупиков. Пример анализа состояния системы методом редукции графа (псевдокод). Методы восстановления работоспособности системы.

Тупики

Мы рассматривали с вами примеры, когда процессы захватывают семафоры в обратном порядке. При этом каждый процесс ждёт освобождения нужного ему семафора, удерживая уже захваченный. Ни один из процессов не может продолжить своё выполнения. При этом в системе они существуют, занимают выделенные им ресурсы, но при этом никакой работы не выполняют. Этот пример можно перефразировать следующим образом. То есть процессы запрашивают, запрашивают ресурсы, полученный ресурс удерживают и запрашивают следующий ресурс, который, как показано в данном примере, занят другим процессом.

1	P1:
2	...
3	Запросил R1
4	Получил R1
5	...
6	Запросил R2

1	P2:
2	...
3	Запросил R2
4	Получил R2
5	...
6	Запросил R1

- 1) P1 запрашивает R1 и получает
- 2) P2 запрашивает R2 и получает
- 3) P1 запрашивает R2 и не может получить
- 4) P2 запрашивает R1 и не может получить

Процессы запрашивают ресурс, задерживают его и запрашивают следующий, который в данном примере занят другим процессом.

Наиболее наглядно тупики демонстрируются с помощью графов. Как правило, процесс обозначаются кружочками, а ресурсы — прямоугольниками.

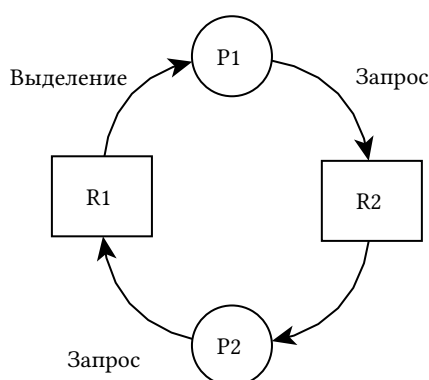


Рис. 19.

Стрелка из вершины процесса — запрос ресурса, а стрелка из вершины ресурса к процессу — выделение. Перед нами двудольный (бихроматический) направленный граф — граф Хольта.

Ресурсы 1 и 2 — единичные. Возможна ситуация, когда ресурсы содержат несколько единиц. Тогда этот же граф представляется следующим образом:

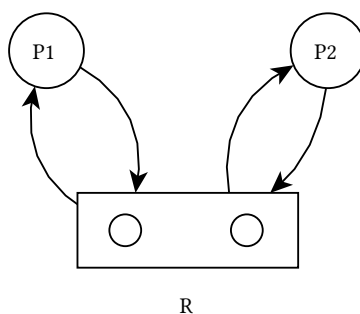


Рис. 20.

Процесс P1 запросил и получил единицу ресурса R. Процесс P2 запросил и получил также единицу ресурса R. Больше нет свободных единиц ресурса R. Процесс P1 для того, чтобы продолжить своё выполнение, запрашивают ещё одну единицу. В результате они не могут освободить занимаемые им единицы ресурса R и блокируют друг друга, они удерживают единицы ресурса R.

Явно видна **замкнутая цепь запросов**.

рисунок верхний

Определение 10.5 (Тупик / тупиковая ситуация)

Тупик или тупиковая ситуация — ситуация, которая возникает в результате монопольного использования разделяемых ресурсов, когда процесс, владея ресурсом, запрашивает другой ресурс, занятый непосредственно или через цепочку запросов другим процессом, ожидающим освобождение ресурса, занятого первым процессом.

Наиболее большую опасность тупики представляют для систем реального времени, для систем, которые предназначены для управления внешними системами, или устройствами, или процессами (внешними по отношению к самой системе). Как правило, это системы, которые управляют какими-то важными объектами, например, атомным реактором.

С точки зрения тупиков ресурсы делятся на два типа:

1) Повторно используемые ресурсы

- 1.1) Аппаратная составляющая компьютера (процессоры, физическая память, устройства ввода/вывода)
- 1.2) Реентерабельные коды ОС
- 1.3) Объекты ядра (семафоры, программные каналы, разделяемая память, очереди сообщений)

Получение процессом такого ресурса никак не влияет на этот ресурс. В результате использования его характеристики не меняются, в систему он возвращается в том же виде, в котором был получен.

Таких ресурсов в системе очень много, причём они разнообразны.

Количество ресурсов ограничено, их количество постоянно для некоторого периода взаимодействия процессов (например, для семафоров).

2) Потребляемые

- 2.1) Сообщения

В процессе получения эти ресурсы перестают существовать.

Наибольшую проблему представляет в распределённых системах, когда может существовать большое количество процессов, создающих сообщения и принимающих их.

Любой процесс может создать и потребить любое количество сообщений

Существуют системы, которые попадать в тупик не могут {не в плане что никогда не попадут, а в плане что для корректной работы не должны попадать, как я поняла}.

{Есть системы, в которых тупик может возникнуть в комбинации повторно используемых ресурсов и потребляемых, но это отдельные случаи, и, как правило, в таких ситуациях не бывает неограниченного числа сообщений.}

Методы борьбы с тупиками

1) Предотвращение тупиков (исключение тупиков)

Необходимо создать в системе такие условия, при которых возникновение тупиков невозможно.

2) Обход тупиков (недопущение тупиков)

Тупики теоретически возможны (для этого есть условия), но система выполняет такие действия, чтобы тупик не возник. Выполняемые действия позволяют обойти тупиковую ситуацию.

У этого метода есть классический способ обхода, который был предложен Дейкстрой — алгоритм банкира (раздел 10.13.4).

3) Обнаружение тупиков и восстановление работоспособности системы

Обнаружение всегда требует восстановления работоспособности системы попавшей в тупиковую ситуацию.

Обнаружение тупиков

Для обнаружения тупиков используется графовая модель Холта (когда когда начали рассматривать эту тему). Это бихроматический или двудольный направленный граф, в котором имеется два непересекающихся множества вершин. Множество вершин делится на два непересекающихся подмножества: множество процессов и множество ресурсов. Граф направленный, поскольку имеет значение направления дуги. Если дуга выходит из вершины-процесс в вершину-ресурс, это запрос. Если дуга выходит из вершины-ресурс в вершину-процесс - это выделение. Ни одна дуга не соединяет вершины одного и того же подмножества.

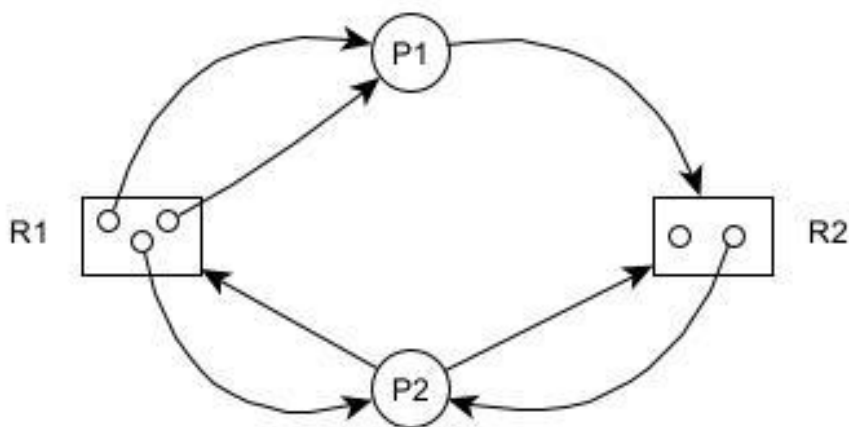
Рассмотрим классический пример такого двудольного графа.

Пусть у нас есть ресурс R и у этого ресурса три единицы. И скажем, два процесса P2, P1. При этом процесс P1 запросил и получил одну единицу ресурса R1 и запрашивает ещё одну единицу. Процесс P2 запросил и получил одну единицу ресурса R. Значит, вот это приобретение или получение - это запрос. При этом мы видим на этом простейшем графе, что запрос процесса P1 может быть удовлетворён, потому что у системы есть свободная единица ресурса R. То есть нет никакой проблемы. Но так бывает не всегда. Количество ресурсов всегда ограничено. Оно меньше числа процессов, процессы конкурируют в системе за ресурсы и вот могут попадать в такие ситуации, когда ни один из взаимодействующих процессов не может продолжить своё выполнение. Для обнаружения тупиковой ситуации используется метод редукции графа.

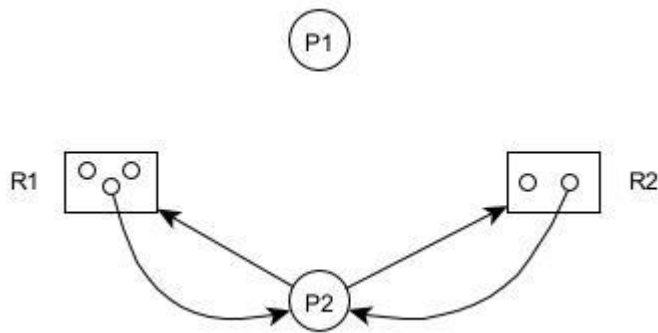
Метод редукции (сокращения) графа. Этот метод позволяет обнаружить тупиковую ситуацию.

Метод редукции графа

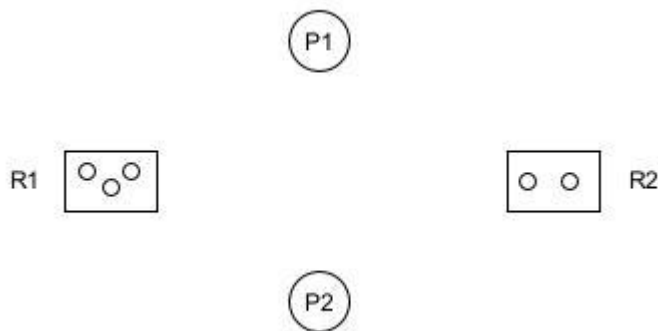
- Реализация на основе двудольного графа
- Есть два множества вершин:
 - множество вершин процессов
 - множество вершин ресурсов
- Граф является направленным
- С помощью дуг соединены вершины различных подмножеств
 - “Выделение” — дуга, выходящая из вершины подмножества ресурсов и входящая в вершину подмножества процессов
 - “Запрос” — дуга, выходящая из вершины подмножества процессов и входящая в вершину подмножества ресурсов
- Из графа удаляются все ребра, которые отражают, что ресурс может быть выделен процессу
- В результате после удаления ребер, если остаются только изолированные вершины — система не в тупике
- Пример:



- У системы есть еще свободная единиц ресурса R2 и если она будет выделена P1, он сможет завершиться

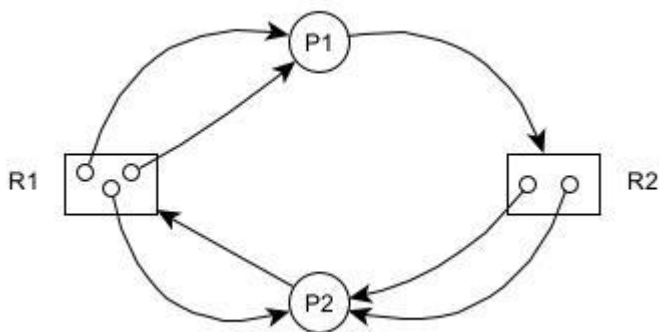


- Следующий шаг:



- Система не в тупике

Пример 2 (замкнутая цепь запросов)



Данный граф нельзя сократить ни по P1, ни по P2, так как у системы нет свободных ресурсов, требуемых P1 и P2

- Теорема 1: Граф является полностью сокращаемым, если есть последовательность сокращений, которая сокращает все дуги
- Теорема 2: Цикл в графе повторно используемых ресурсов является необходимым условием тупика
- Теорема 3: Если состояние S не является состоянием тупика и выполняется переход из состояния S1 в состояние S2 в результате

действия процесса P_i , причем $S2$ является тупиковой, то операция, выполняемая процессом P_i , является запросом

- Тупик возникает только в результате запроса

- Для представления двудольного графа минимум необходимы две матрицы и два связанных списка

- матрица распределения

p/r	1	2	3
1	0	1	1
2	1	3	0
3	1	5	0
	2	9	1

- матрица запроса

p/r	1	2	3
1	1	1	0
2	0	0	1
3	1	1	2

- Вектор свободных единиц ресурсов

$$F = \begin{array}{|c|c|c|c|} \hline r & 1 & 2 & 3 \\ \hline & 4 & 0 & 2 \\ \hline \end{array}$$

- $$R_{ij} = \sum_{i=1}^n b_{ij} + F_j$$

- $R = (6, 9, 3)$

- $P2$ имеет строку меньше $< F \Rightarrow$ он может завершиться

- Матрица распределения

p/r	1	2	3
1	0	1	1
2	0	0	0
3	1	5	0

- $F = (5, 3, 2)$

- $P1$ завершился

p/r	1	2	3
1	0	0	0
2	0	0	0
3	1	5	0

-
- $F=(5,4,3)$
- Система не находится в тупике $\Rightarrow$ завершился P3

p/r	1	2	3
1	0	0	0
2	0	0	0
3	0	0	0

-
- $F=(6, 9, 3)$
- Системы:
 - известны процессы, формирующие сообщения и процессы, потребляющие сообщения
 - известны производители, неизвестны потребители
 - оба неизвестны

Теорема 10.6 (Теорема 1, полностью сокращаемый граф)

Граф является полностью сокращаемым, если существует такая последовательность сокращений, которая устраняет все дуги. Если граф нельзя полностью сократить, то анализируемое состояние является тупиковым. (Состояние графа S является состоянием тупика тогда и только тогда, когда граф повторно используемых ресурсов в состоянии S не является полностью сокращаем.)

Теорема 10.7 (Теорема 2, о состоянии графа)

Цикл в графе повторно используемых ресурсов является необходимым условием тупика.

Тупик возникает в результате запроса процесса на ресурс, который не может быть немедленно удовлетворён, потому что у системы нет необходимого количества свободных единиц запрашиваемого ресурса.

Теорема 10.8

Если состояние системы S не является состоянием тупика, а состояние T , в которое перешла система из состояния S , является тупиковым состоянием, то операция, которая перевела систему из состояния S в состояние T является запросом.

Другими словами, тупик может возникнуть только в результате запроса процесса на ресурс. Если такой запрос удовлетворить нельзя, то, значит, нельзя сократить граф, то есть нельзя эту дугу удалить. Но в результате вот таких запросов всегда возникает замкнутая цепь запроса.

Чтобы программно исследовать состояние системы необходимо представлять граф. И наиболее очевидной формой представления графа является матричная форма. При этом необходимо иметь обязательно две матрицы: матрицу текущего распределения и матрицу запросов.

Матрица распределения. Каждый элемент матрицы определяет, сколько единиц ресурса выделено процессу P .

Определение 10.11 (Различия предотвращения и обнаружения тупиков)

А обнаружение тупиков фактически означает, что анализируется текущее состояние системы, а предотвращение это предварительный анализ состояния системы. То есть, процесс делает запрос на ресурс и прежде, чем этот ресурс процессу будет выделен, анализируется состояние системы. Если в результате запроса будет определено, что данный запрос приведёт к тупиковой ситуации, этот запрос не будет удовлетворён. А обнаружение связано с констатацией факта, что система находится в тупике (или не находится или находится). Вот в этом разница.



Способы предотвращения тупиков (или восстановления системы?)

Существует 3 основных подхода для предотвращения тупиков:

1) Перезапустить систему

Плох тем, что будет потеряна вся работа всей совокупности процессов.

2) Последовательно удалять/завершать/отбирать ресурсы у процессов, которые попали в тупик (замкнутая цепь запросов). Можно последовательно завершать процессы попавшие в тупик, значит, отбирать у них ресурсы, возвращать их системе. И очень возможно, что эти ресурсы позволят другим процессам, попавшим в тупик, продолжить своё выполнение.

Проблема может быть решена откатом. Процесс не завершается, а переводится в то состояния, в котором он был до запроса.

3) Отбирать ресурсы у процессов, которые не попали в тупиковую ситуацию. При этом в том предположении, что отобранные ресурсы позволят удовлетворить запросы соответствующих ресурсов для процессов, которые попали в тупик.

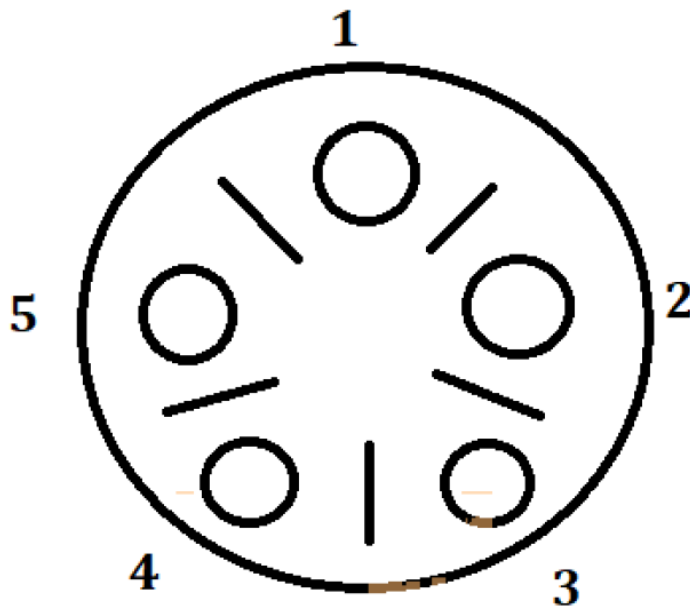
Какие процессы выбирать для завершения? Например, можно завершать самые низкоприоритетные процессы (например, вычислительные).

Все эти решения требуют скрупулёзного анализа программистом. Не существует систем, которые не могли бы попасть в тупик.

2. Задача «Обедающие философы» - модели запросов ресурсов философами и распределения ресурсов вычислительной системы, псевдокод одной из моделей. Множественные семафоры UNIX System V: поддержка в системе, системные вызовы, пример использования из лабораторной работы «производство-потребление».

Обедающие философы

За круглым столом сидят 5 философов. Перед каждым философом стоит тарелка. Между тарелками лежит по 1 палочке (для еды нужно 2).



Существует три способа взаимодействия философов:

- 1) Каждый философ пытаются взять сразу оба прибора (левый и правый) и начать есть. После, возвращает оба прибора
- 2) Каждый из философов берет правый прибор и пытается взять левый, удерживая правый
- 3) Если не может взять левый, возвращает правый

Проблемы:

- 1) Приводит к бесконечному откладыванию
 - a) 1 философ умрет голодной смертью
 - b) 2 и 5 берут оба прибора (1, 3, 4 ждут)
 - c) 5 положил, 4 может взять (1, 3, 5 ждут)
 - d) 2 положил, 3 может взять (1, 2, 4 ждут)
 - e) 1 постоянно ждет
- 2) Все взяли правый прибор, пытаются взять левый — классический пример блокировки deadlock
- 3) Все взяли правый, не могут взять левый => возвращают правый и повторяют эти действия.
 - a) В системе процессы захватывают и освобождают одни и те же ресурсы => полезная работа не выполняется

```

var
  chops: array[1..5] of semaphore;
  i: integer;
begin
  repeat
    chops[i] := 1;
    i := i - 1
  until i = 0;
parbegin
  1: begin
    left := 1; right := 2;
    ...
  end;
  5: begin
    left := 5; right := 1;
    repeat
      //размышляет
      P(chops[left], chops[right]); // захватывает
      // ест
      V(chops[left], chops[right]); // освобождение
    forever
  end; // 5
parend
end.

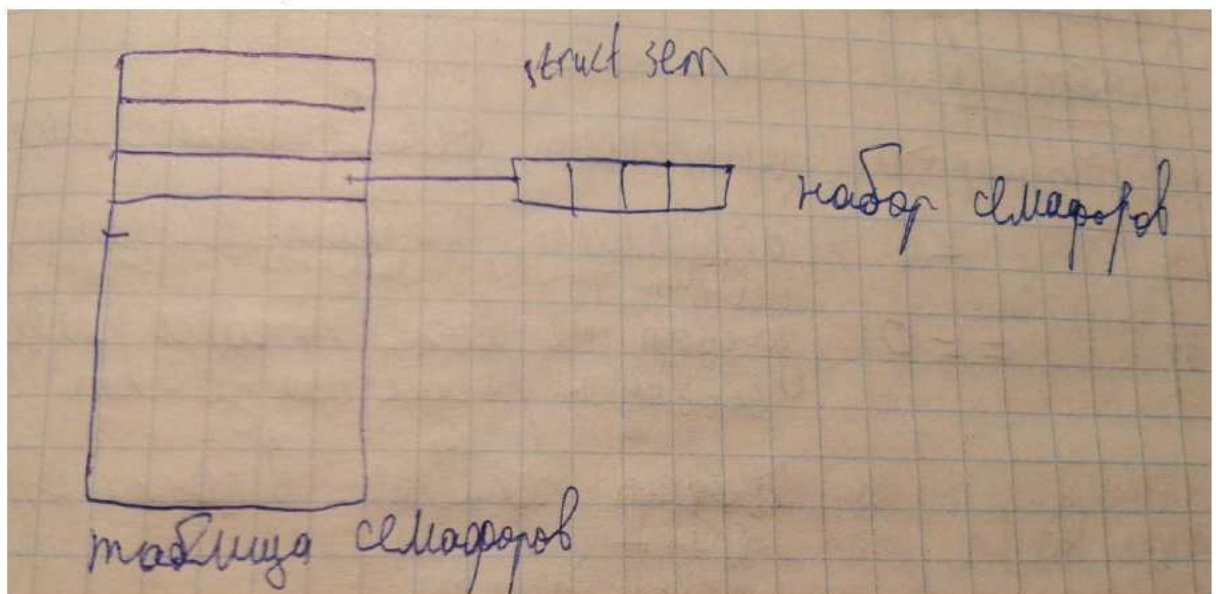
```

Семафоры UNIX

- Семафоры UNIX вносят дополнения в семафоры Дейкстры.
- В современных системах поддерживаются наборы семафоров
- Наборы семафоров представлены массивами семафоров
- В UNIX LINUX семафоры поддерживаются системной таблицей(находится в адресном пространстве системы) семафоров. Каждый элемент таблицы описывает один набор семафоров (то есть это дескриптор)
- Дескриптор набора семафоров хранит:
 - имя (целое число) – присваивается процессом, который создал набор семафоров. Другие процессы по этому имени могут получить доступ к нему.
 - uid
 - идентификатор создателя наборов семафоров и идентификатор его группы.

Процесс, `uid` которого совпадает с `uid` созд. может удалить набор и изменить его существующие права(?)

- права доступа
- количество семафоров в наборе
- время изменения 1 или нескольких семафоров последним процессом
- время последнего изменения управляющих параметров набора каким-либо процессом
- указатель на массив семафоров
=> в наборе семафоров семафоры обозначаются индексами начиная с нулевого



- На семафорах определены системные вызовы:
 - `semget()` — создание набора семафоров
 - `semctl()` — изменение управляющих параметров набора семафоров
 - `semop()` — операции на наборе семафоров или на семафоре
- Для определения операций на семафорах определена структура

```
struct sembuf
{
    ushort sem_num; //index
    short sem_op; // операция
    short sem_fl; // флаги
}
```

- В отличие от семафоров Дейкстры, на семафорах UNIX определено 3 операции
- sem_op:
 - > 0 — инкремент, освобождение семафора
 - < 0 — декремент, захват семафора
 - блокирование процесса, если декремент невозможен
 - $== 0$ — захват не выполняется, процесс проверяет, свободен ресурс или занят
 - будет переведен в состояние ожидания освобождения ресурса. После его освобождения декремент не выполн.
- Флаги sem_fl:
 - IPC_NOWAIT — информирует систему о нежелании процесса переходить в состояние ожидания (освобождения семафора)
 - Наличие данного флага объясняется желанием избежать блокирования всех процессов, находящихся в очереди процессов, в случае, если захвативший ресурс процесс завершится аварийно или получит сигнал kill
 - Так как kill невозможно перехватить, процесс не может освободить семафор
 - Блокированный процесс не получает процессорного времени
 - SEM_UNDO — данный флаг указывает ядру, что необходимо отслеживать изменение значения семафора в результате вызова semop.
 - Указывает ядру, что при завершении процесса, вызвавшего semop, необходимо ликвидировать сделанные изменения, чтобы другие процессы не были заблокированы на семафорах навечно
- semget возвращает файловый дескриптор созданного набора семафоров
- Семафоры могут служить как источник информации — процесс освобождает семафор как событие. Но это одно значение — свободен или занят. Но есть и другие средства передачи данных.

Производство-потребление

C

```
#include <stdlib.h>
#include <string.h>
#include <sys/errno.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/time.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#define SB 0
#define SE 1
#define SF 2
#define P -1
#define V 1

struct sembuf start_produce[2] = {{SE, P, 0}, {SB, P, 0}};
struct sembuf stop_produce[2] = {{SB, V, 0}, {SF, V, 0}};
struct sembuf start_consume[2] = {{SF, P, 0}, {SB, P, 0}};
struct sembuf stop_consume[2] = {{SB, V, 0}, {SE, V, 0}};
int count = 0;

int flag = 1;
void sig_handler(int sig_num) {
    flag = 0;
}

void producer(int semfd, char **pr_ptr, char *letter) {
    while (flag) {
        if (semop(semfd, start_produce, 2) == -1) {
            printf("Producer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }

        if (*letter == 'z') {
            *letter = 'a';
            count++;
        } else {
            (*letter)++;
        }
        (*pr_ptr)++;
        if (count == 2) {
            if (kill(0, SIGALRM) == -1) {
                perror("kill");
                exit(1);
            }
            exit(0);
        }

        **pr_ptr = *letter;
    }
}
```

```

        printf("Producer %d - %c\n", getpid(), **pr_ptr);

        if (semop(semfd, stop_produce, 2) == -1) {
            printf("Producer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }
    }

    exit(0);
}

void consumer(int semfd, char **cn_ptr) {
    while (flag) {
        if (semop(semfd, start_consume, 2) == -1) {
            printf("Consumer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }

        (*cn_ptr)++;
        printf("Consumer %d - %c\n", getpid(), **cn_ptr);

        if (semop(semfd, stop_consume, 2) == -1) {
            printf("Consumer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }
    }

    exit(0);
}

int main() {
    int shmfd, semfd;
    int perms = S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH;
    char *addr;
    char **pr_ptr;
    char **cn_ptr;
    char *letter;
    pid_t chpid[7];

    semfd = semget(100, 3, IPC_CREAT | perms);
    if (semfd == -1) {
        perror("semget");
        exit(1);
    }

    if (semctl(semfd, SB, SETVAL, 1) == -1) {
        perror("semctl");
        exit(1);
    }

    if (semctl(semfd, SE, SETVAL, 256) == -1) {
        perror("semctl");
        exit(1);
    }

    if (semctl(semfd, SF, SETVAL, 0) == -1) {
        perror("semctl");
        exit(1);
    }

    shmfd = shmget(101, 256, IPC_CREAT | perms);
    if (shmfd == -1) {
        perror("shmget");
        exit(1);
    }

    addr = shmat(shmfd, NULL, 0);
    if (addr == (char *)-1) {

```

```

        perror("shmat");
        exit(1);
    }

    pr_ptr = (char **)addr;
    cn_ptr = pr_ptr + 1;
    letter = (char *) (cn_ptr + 1);

    *cn_ptr = letter + 1;
    *pr_ptr = *cn_ptr;
    *letter = 'a' - 1;

    if (signal(SIGALRM, sig_handler) == SIG_ERR) {
        perror("signal");
        exit(1);
    }

    for (int i = 0; i < 3; i++) {
        chpid[i] = fork();
        if (chpid[i] == -1) {
            perror("fork");
            exit(1);
        }

        if (chpid[i] == 0) {
            producer(semfd, pr_ptr, letter);
        }
    }

    for (int i = 3; i < 7; i++) {
        chpid[i] = fork();
        if (chpid[i] == -1) {
            perror("fork consumer");
            exit(1);
        }

        if (chpid[i] == 0) {
            consumer(semfd, cn_ptr);
        }
    }

    for (int i = 0; i < 7; i++) {
        int status;
        pid_t child = wait(&status);

        if (WIFEXITED(status))
            printf("Child: PID = %d exited with status = %d\n", child, WEXITSTATUS(status));
        else if (WIFSIGNALED(status))
            printf("Child: PID = %d killed by signal = %d\n", child, WTERMSIG(status));
        else if (WIFSTOPPED(status))
            printf("Child: PID = %d stopped by signal = %d\n", child, WSTOPSIG(status));
        else
            printf("Child has finished: PID = %d\n", child);
    }

    if (shmdt(addr) == -1) {
        perror("shmdt");
        exit(1);
    }

    if (semctl(semfd, 0, IPC_RMID, NULL) == -1) {
        perror("semctl");
        exit(1);
    }

    if (shmctl(shmfd, IPC_RMID, NULL) == -1) {
        perror("shmctl");
        exit(1);
    }

```

```
}

return 0;

}
```

Теория

Формулировка: Есть два типа процессов: первый тип — процессы-производители, которые только производят единицу данных и помещают в ячейки буфера, второй тип — процессы-потребители, которые могут только выбирать данные из буфера.

Производители-потребители — асинхронные процессы, каждый выполняется с собственной скоростью. Если буфер (хранилище) заполнено, то производитель будет заблокирован. Если производитель запаздывает, то потребитель ждет производителя. Если ячейка освобождена, то второй раз взять значение из этой же ячейки невозможно.

Семафор — защищенная неотрицательная переменная, на которой определены две операции: P — декремент, захват семафора, V — инкремент, освобождение семафора.

Виды:

1. Бинарный семафор (0/1).
2. Считающий > 0 , int.

Алгоритм Деикстры решения задачи производства-потребления:

Se, Sf — считающие семафоры, Sb — бинарный.

C ▾

Producer:

```
P(Se)
P(Sb)
N = N + 1;
V(Sb)
V(Sf)
```

Consumer:

```
P(Sf)
P(Sb)
N = N - 1;
V(Sb)
V(Se)
```

Sb обеспечивает монопольный доступ к буферу.

Основная особенность семафоров UNIX в том, что одной неделимой операцией можно изменить все или часть семафоров набора.

Система должна предоставить соответствующие системные вызовы, система переходит в режим ядра, что связано с переключением аппаратного контекста.

Структуры для работы с семафорами — структуры ядра. В ядре поддержка с помощью таблицы семафоров, содержащей дескриптор набора (semid_ds)

1. Виртуальная память: определение, 3 схемы преобразования адресов в страницах. Управление памятью страницами по запросам в архитектурах x86-64 (расширенное преобразование (PAE)) - схема преобразования, обоснование использования данной схемы - достоинства и недостатки. Анализ страничного поведения процессов: свойство локальности, рабочее множество, графики, показывающие рабочие множества (или подтверждающие теорию рабочих множеств)

10.15.2 Виртуальная память

Программа не должна целиком находиться в физической памяти. В физ.памяти должна находиться та часть программы, с которой производится работа в данный момент времени. То есть в памяти должны находиться те команды программы, к которым в текущий момент времени обращается процессор и данные, соответственно.

Это отправная точка идеи виртуальной памяти. Виртуальный — кажущийся, возможный.

Существует 3 схемы управления виртуальной памятью:

- 1) Страницами по запросу
- 2) Сегментами по запросу
- 3) Сегментами, поделёнными на страницы по запросу

В стремлении избавиться от фрагментации, от необходимости выделения непрерывных адресных пространств в физической памяти, пришли к идеи виртуальной памяти.

Но это очень затратная схема, очень много дополнительных действий необходимо выполнять для адресации команды данных. Какой от этого выигрыш, если это так затратно? Увеличение мультизадачности (мультипрограммности (multitasking)), это значит, что больше одной программы может одновременно находиться в физической памяти. Они могут выполняться, могут ждать своей очереди, но это и есть мультизадачность.

Это огромные затраты, но и колоссальный выигрыш. Но от идеи использования виртуальной памяти никто не отказался.

10.16 Управление памятью страницами по запросу

Во всех трёх частях содержится общая часть: «по запросу», то есть тогда, когда процессор в них возникает необходимость, когда процессор обращается к данным, отсутствующим в физической памяти. Реализуется это с помощью прерываний, в

частности, если мы говорим об управлении памятью страницами по запросу, то это будет страничное прерывание. Причём страничное прерывание в системах отнесено к исправимым исключениям.

В начальный момент времени, когда процесс порождён, то есть он получил идентификатор, часть его полей дескриптора проинициализирована системой, процессу выделяется физическая память. В современных системах это какое-то минимально необходимое для начала выполнения количество страниц физической памяти. Условно говоря, 3 страницы: **сегмента кода, сегмента данных и сегмента стека.**

После этого по диаграмме состояний, процесс переходит в очередь готовых процессов для получения процессорного времени. Получив процессорное время, процесс начинается выполняться команда за командой. В какой-то момент времени процессор обращается к данным, отсутствующим в физической памяти (под данными в общем случае понимаются и команды и собственно данные). Возникает страничное прерывание, в результате которого нужная страница будет загружена в физическую память, после чего процесс начнёт своё выполнение. То есть к затратам ещё добавляется обслуживание страничных прерываний, затраты ещё больше.

10.17 Схемы преобразования адресов в страничном распределении

10.17.1 Прямое отображение

Процессы имеют виртуальные адресные пространства. Виртуальный адрес делится на две части: **номер страницы и смещение.** В составе регистра процесса обязательно должен быть регистр базового адреса таблицы страниц.

Каждый процесс имеет собственную таблицу страниц, в которой находятся дескрипторы страниц.

Фактически номер страницы виртуального адреса служит смещением к дескриптору в таблице страниц конкретного процесса (?). Если страница загружена в физическую память, то из этой таблицы можно получить её адрес. Смещение одно и то же. Причём смещение берётся из программы, выполняется преобразование и получается физический адрес.

При этом такая таблица страниц находится в физической памяти. **То есть для того, чтобы выполнить преобразование вот в соответствии с этой схемой, сначала надо обратиться к таблице страниц, получить физический адрес страницы и затем выполнить преобразования для получения физического адреса данных.** Такое преобразование будет выполняться на каждой команде, а то и несколько раз — крайне затратные действия.

Обращение к физической памяти требует большого количества тактовых импульсов, так называемых циклов обращения к памяти. Это затратное действие. Стремясь

уменьшить эти накладные расходы, разработчики систем, основываясь на своих технических знаниях, предложили использовать ассоциативную память.

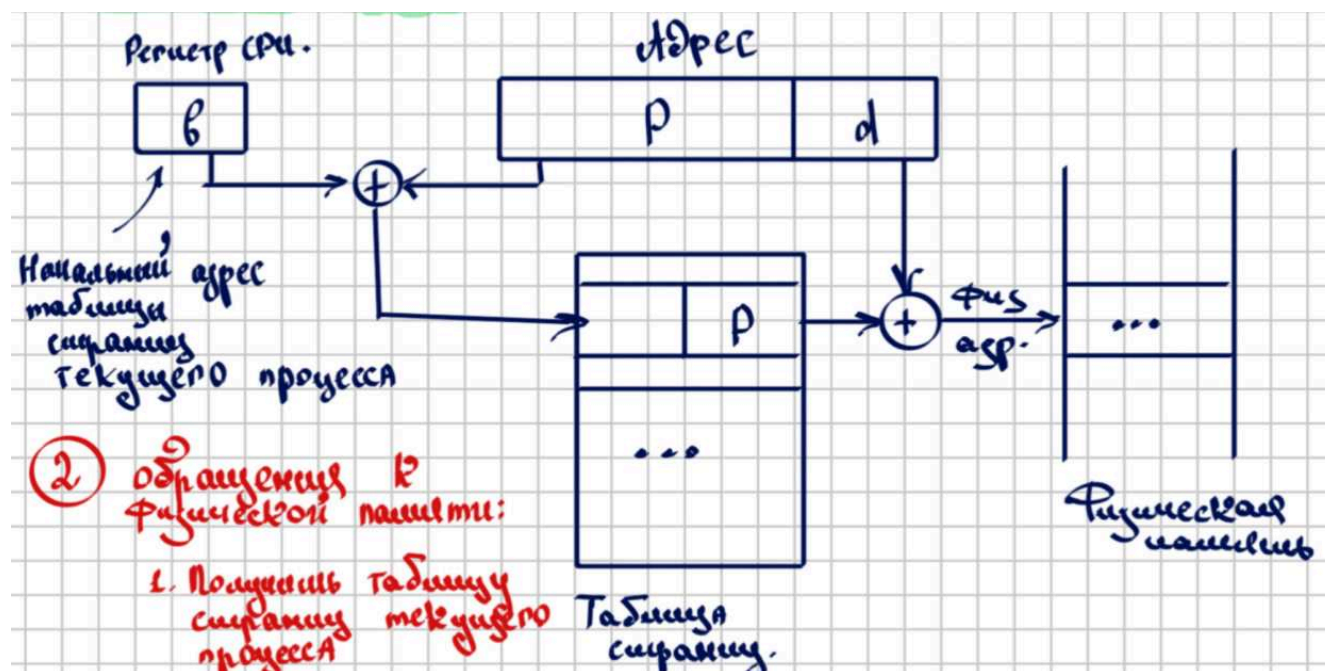


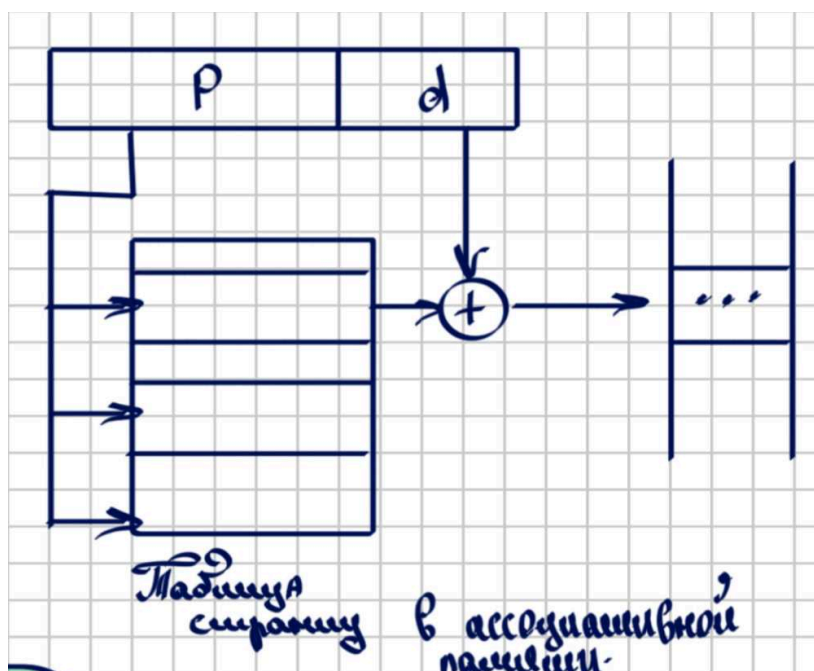
Рис. 36.

10.17.2 Ассоциативное отображение

Таблица страниц загружается в ассоциативную память.

Особенностью ассоциативной памяти является возможность получения данных за 1 такт за счёт схемы реализации ассоциативной памяти.

Важно понимать, почему эта схема не нашла своё физическое воплощение, а лишь частичное. Ассоциативная память очень дорогая, это всегда регистровая память.



Номер виртуальной страницы является ключом, ключ — 4 разряда. При совпадении ключа формируется импульс, который позволяет считать информацию. Каждый разряд каждого регистра дополнительно имеет схему совпадения (тут схема архитектурская).

Мало того, что это регистровая память (количество схем увеличивается в 2 раза), также увеличивается количество соединений. Это говорит о дороговизне ассоциативной памяти.

А с ростом размеров программ таблицы страниц увеличиваются, они становятся очень большими. Значит, необходимо иметь ассоциативную память очень большого размера, специальную ассоциативную память. То есть теоретически это возможно, но практически это сталкивается с огромными затратами, и с огромной стоимостью такой памяти.

10.17.3 Ассоциативно-прямое отображение

⚠ ВНИМАНИЕ

На картинке таблицы ассоциативной памяти написано, а она говорила просто ассоциативная память

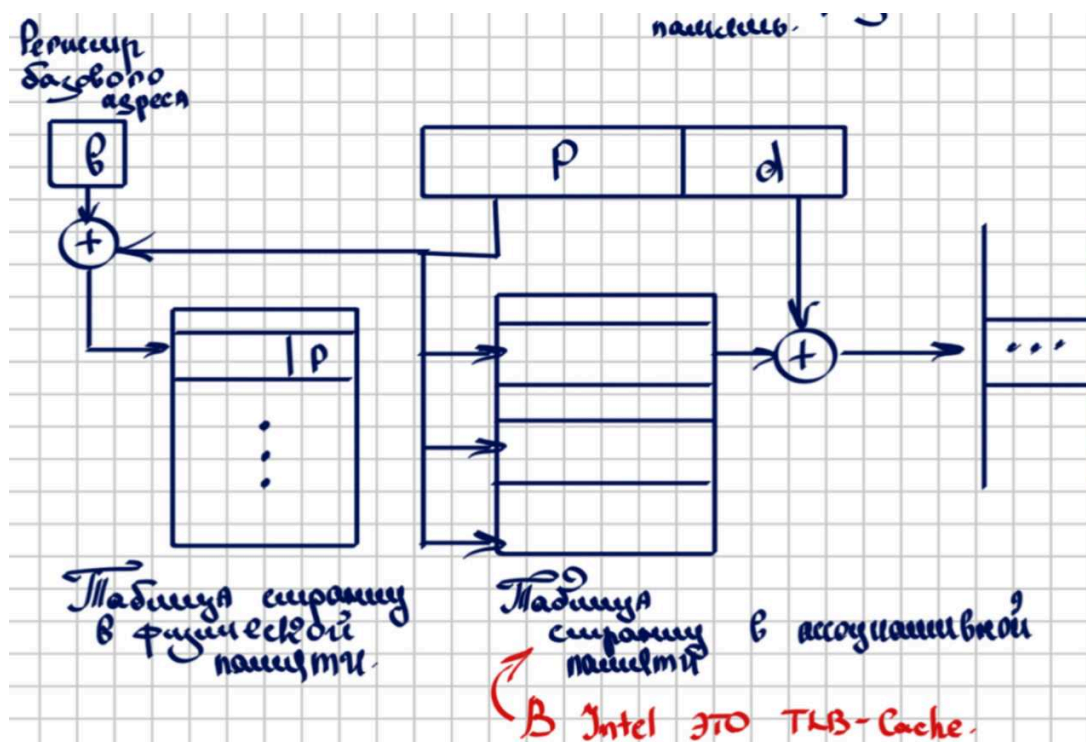


Рис. 38.

В физической памяти располагается таблица страниц. В процессоре есть регистр базового адреса таблицы страниц и ассоциативная память небольшого размера, в которое может находиться только небольшое количество дескрипторов страниц.

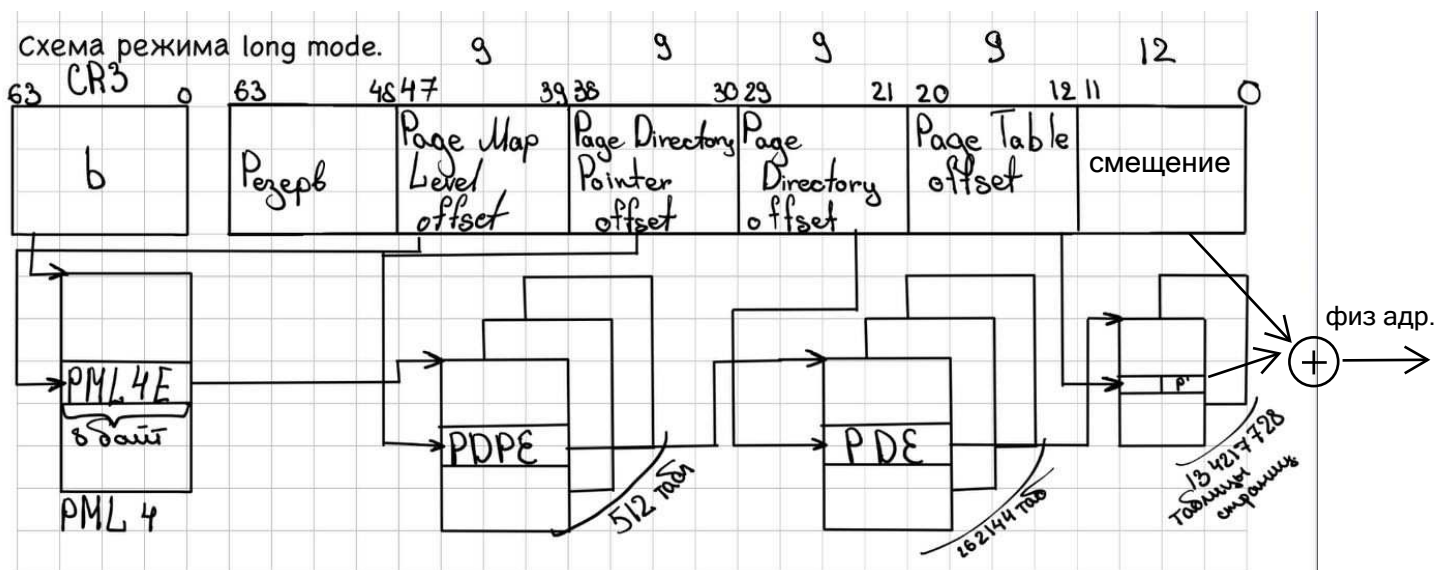
При этом в такой ассоциативной памяти хранятся только дескрипторы страниц, к которым были последние обращения. Связано это со свойством локальности наших программ: если обращение было к странице, то наиболее вероятно, что следующие обращения будут к этой же странице. Это следует из того, что команды, данные хранятся в памяти в последовательных адресах, при этом наиболее вероятной структурой является следование (выполнение программы именно команды за командой). Ветвление выполняется в программах реже, сюда же относим и ветвление в цикле. Кроме того, данные также хранятся в последовательных адресах. Но очевидно, что на странице никогда не выполняются все команды, как правило это от $\frac{1}{3}$ до $\frac{2}{3}$ команд. Даже кеш небольшого размера (от 16 строк) обеспечит скоростные показатели, составляющие 90% скоростных показателей при реализации полностью ассоциативной памяти.

КАК ПРОИСХОДИТ РАБОТА: Сначала страница ищется в ассоциативной памяти. Если её там нет, то обращение происходит к таблице страниц физической памяти. При этом будет выполнено замещение дескриптора (если дескриптора не оказалось в ассоциативной памяти в результате обращения, то будет выполнено замещение). В современных системах — Кеш TLB (Translation Lookaside Buffer), фирмы IBM, ассоциативный по множеству четырёхнаправленный кеш.

Размеры программ растут, размеры таблиц страниц увеличиваются. Решением задачи увеличения размеров стала схема с гиперссылками.

- В режим long реализовано только управление памятью страницами по запросу.
- В режиме long — схема PAE (physical address extension)

PAE



- PTE — таблицы страниц
- PDE — каталоги таблиц страниц —
- PDPE — указатель на каталоги таблиц страниц — максимум 512 табл.
- PML4E — указатель на указатель — такая таблица в системе 1
- Дескрипторы инициализируются по мере надобности
- Размер дескриптора — 8 байт
- Таблицы по 2^9 строк $\Rightarrow$ каждая таблица в памяти занимает одну страницу
- Когда процесс создается — ему выделяется минимально необходимое число страниц.
- То есть выполняется отложено.

Например, при создании процесса создается 3 страницы: точка входа, страница сегмента данных, страница сегмента стека. Все остальные страницы создаются по мере надобности при возникновении системных прерываний.

10.19 Теория рабочего множества

В 1968 году Деннинг предложил в качестве локальной меры производительности взять число страниц, к которым обращается процесс за интервал времени Δt . Рабочее множество (working set) — функция $W(t, \Delta t)$. Величина показывает количество страниц, к которым обращается процесс. При этом выяснилось, что для эффективного выполнения процесса (процесс эффективно выполняется, когда нет страничных прерываний, то есть нет блокировок в ожидании загрузки нужной страницы), если в памяти находятся все страницы, к которым процесс обращается за некоторый интервал Δt , то есть всё рабочее множество.

Исследование IBM было проведено, потому что несмотря на затраты, никто не отказался от реализации идеи виртуальной памяти, потому что это существенно повышает уровень мультизадачности. При этом возникает вопрос: сколько физической памяти потенциально нужно предоставлять каждому процессу, то есть сколько физических страниц понадобится процессу для его эффективного выполнения, чтобы он смог загрузить в память своё рабочее множество. Эта теория отвечает на этот вопрос.

В результате исследований было подтверждено наличие рабочего множества. При этом в начальный момент времени, когда процесс порождён,

Процессу, чтобы он мог начать выполняться, выделяется минимальное необходимое количество страниц. Скажем, ему выделяется по одной странице кода, стека и данных. После этого процесс переходит в состояние готовности. Получив процессорное время, программа начинает выполняться, и процесс начинает интенсивно подгружать нужные ему страницы, то есть будут возникать страничные прерывания, система будет их обрабатывать и загружать нужные страницы в память.

В результате в какой-то момент времени процесс загрузит в память своё первое рабочее множество. Этот интервал Δt , в течение которого процесс будет обращаться к одним и тем же страницам, ни предсказать, ни измерить невозможно, { процессы выполняются по разным траекториям, зависят от динамических параметров.} Длительность тоже можно определить только в процессе выполнения. Затем процесс перейдёт к своему второму множеству. И так далее.

Существуют такие интервалы во времени жизни процесса, когда процесс выполняется без страничных прерываний. Для этого процесс должен иметь возможность загрузить в память всё своё рабочее множество, то есть количество физических страниц которые выделены процессу, например, как квота, должно позволять загружать ему рабочее множество. Если процесс не может загрузить в память своё рабочее множество, возникнет интенсивная подкачка одних и тех же страниц в результате страничных прерываний. Это получило название **trashing**.

В Windows рабочее множество — это квота. Каждому процессу изначально устанавливается квота — 10 физических страниц (то есть ему выделено 10 кадров, он может

загрузить 10 страниц.). Любая система контролирует число страничных прерываний. Если число страничных прерываний у процесса резко возрастает, это значит, что процесс не может загрузить в память всё своё рабочее множество. В этом случае квота может быть увеличена. Для этого нужно иметь возможность, например, отложить процесс на какое-то время, пока система не сможет добавить к его квоте ещё некоторое количество страниц, чтобы процесс смог загрузить в память своё рабочее множество.

10.19.1 hit rate

??

TLB Hit Rate?

hit rate paging — доля обращений, не потребовавших смена страницы.

Первый график отражает влияние изменения размера страниц при сохранении объёма оперативной памяти на число прерываний.

график, похожий на «купол» нормального распределения

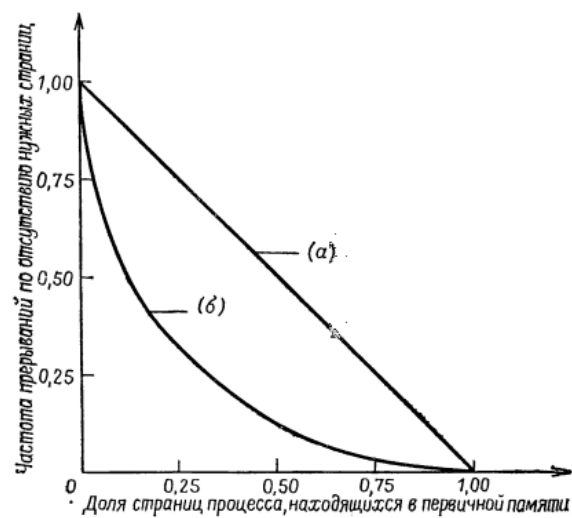
Рис. 43. первый график

Мы видим, что размер страницы увеличивается (в итоге может стать размером с память). Также видим, что число страничных прерываний растёт, сначала не очень интенсивно, затем резко, а потом падает. Рост числа страничных прерываний связан с тем, что чем больше страница, тем меньшее количество команд будет выполнено на этой странице. Когда страница достигнет размера, сопоставимого с размером команды, число страничных прерываний сократится.

Второй график — кривая времени жизни, график зависимости длительности периода между прерываниями. Размер программы — сегмент кода, сегмент данных, сегмент стека.

Теория рабочего множества

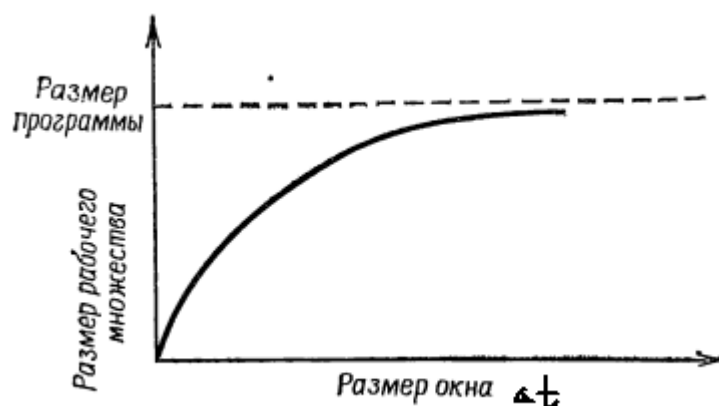
- Рассмотрим зависимость числа страничных прерываний от объема физической памяти



- Является мерой обобщенного поведения программ
- Неформально: рабочее множество – подмножество страниц, к которым процесс активно обращается. Для эффективного выполнения программы необходимо, чтобы ее рабочее множество находилось в первичной памяти
- В 1968 году Деннинг предложил в качестве локальной меры производительности взять число страниц, к которым обращается процесс за время Δt
- То есть, рабочее множество страниц процесса $W(t, \Delta t)$ в момент времени $t + \Delta t$ — набор страниц, к которым этот процесс обращается в течение интервала времени процесса Δt .
Время процесса – время, в течение которого процесс имеет в своем распоряжении центральный процессор.



- Чем больше Δt , тем к большему числу страниц обращается процесс
- При увеличении Δt размер рабочего множества стремится к пределу



- Программа должна иметь возможность загрузить в память рабочее множество страниц
 - То множество страниц, к которому в текущий интервал времени обращается процесс
- Если программа не имеет такой возможности, то пробуксовка (trashing)
 - Пробуксовка — загрузка и выгрузка одних и тех же страниц
- (Совершенная ОС стремится использовать все процессорное время на себя саму, а на приложения времени не остается – нужен компромисс)
- Page — виртуальная страница
- Frame — физическая страница

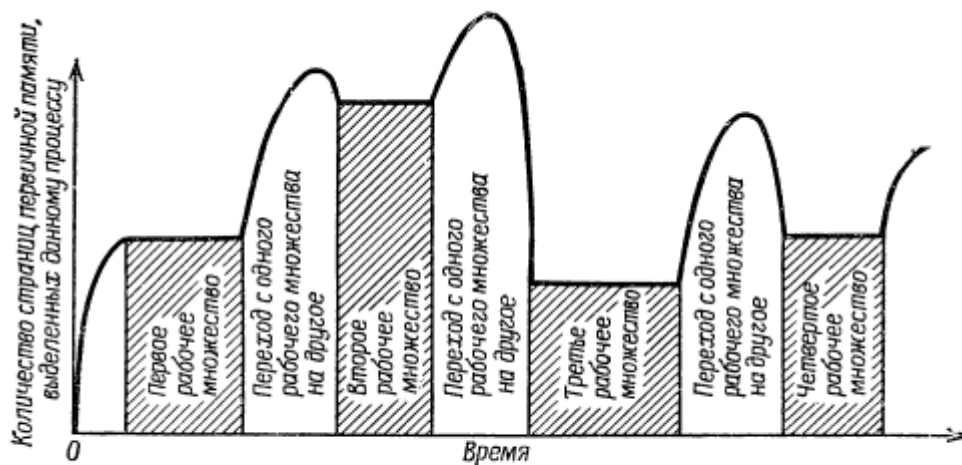


Рис. 9.6 Выделение основной памяти при управлении памятью на основе рабочих множеств.

1. Вначале, поскольку процесс запрашивает страницы своего рабочего множества не все сразу, а по одной, он постепенно получается достаточный объем памяти для размещения текущего рабочего множества.
2. Затем на какой-то период времени этот объем стабилизируется, поскольку процесс интенсивно обращается к страницам своего первого рабочего множества.
3. Со временем процесс перейдет на следующее рабочее множество, что показывает кривая линия от 1 ко 2 раб. множеству. Вначале эта кривая поднимается выше уровня первого рабочего набора, поскольку для процесса по его запросам быстро производится подкачка страниц нового рабочего множества.

При этом система не может сразу определить, то ли процесс просто расширяет свое рабочее множество, то ли процесс переходит на новое.

4. После того, как процесс начнет стабильно обращаться к страницам своего следующего рабочего множества, система видит, что в течение выбранного окна происходит обращение к меньшему количеству страниц, чем ранее, и сокращает объем памяти, выделяемой процессу, до числа страниц этого второго рабочего множества.
5. Каждый раз, когда осуществляется переход с одного рабочего множества на другое, кривая линия поднимается, а затем спадает, показывая, каким образом система адаптируется к подобному переходу.

Проблема: рабочие множества меняются во времени, причем следующее рабочее множество процесса может существенно отличаться от предыдущего. Стратегия управления памятью должна обязательно учитывать этот факт, чтобы избежать перегрузок первичной памяти и возникающего вследствие этого трешинга.

График зависимости длительности периода между прерываниями (кривая линии жизни)

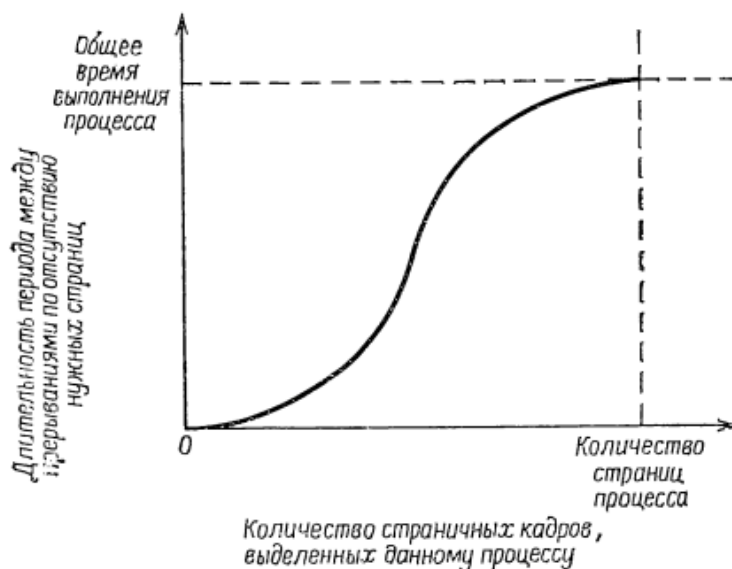
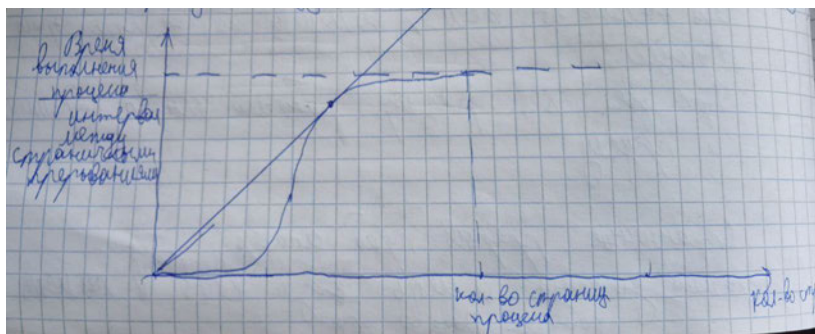


Рис. 9.12

- На графике показано, каким образом меняется среднее время между прерываниями по отсутствию нужной страницы в памяти при увеличении числа фреймов, выделяемых процессу.
- Кривая монотонно растет – чем больше фреймов имеет процесс, тем больше интервал между прерываниями по отсутствию нужных страниц.
- Перегиб происходит, так как в памяти оказывается все рабочее множество процесса
- Поначалу величина интервала между прерываниями растет очень быстро, поскольку растет часть рабочего множества, которая оказывается в первичной памяти.
- После того, как выделенный объем первичной памяти оказывается достаточным для размещения всего рабочего множества, кривая делает перегиб, указывая на то, что выделение дополнительных

фреймов не приведет к существенному увеличению интервала между прерываниями.

Влияние размера страниц при одном и том же размере физической памяти

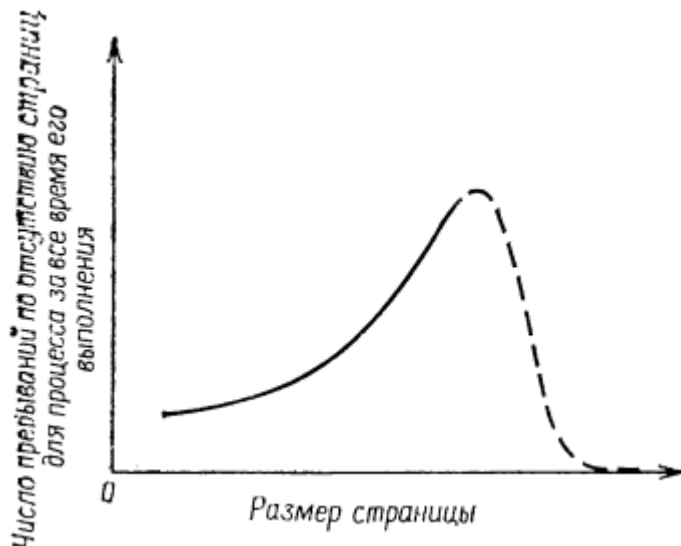
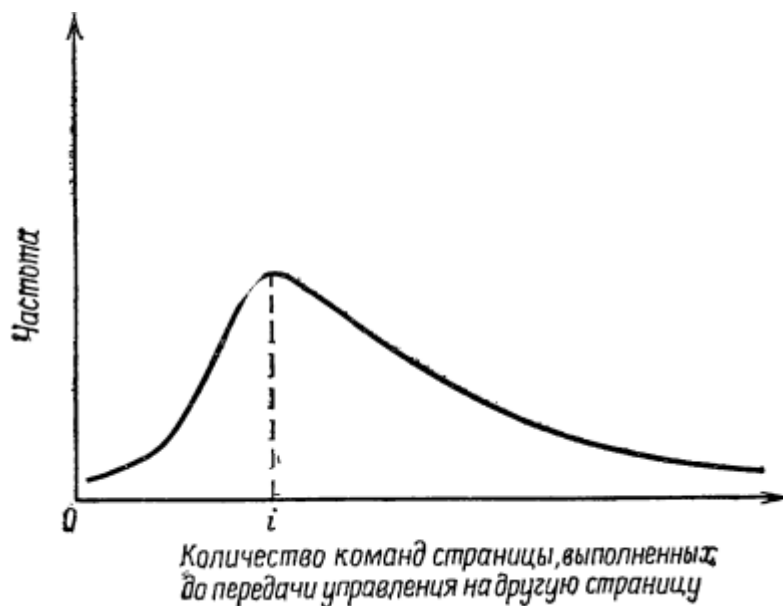


График показывает влияние размера страниц на число страничных прерываний

- С ростом размера страниц сначала число прерываний растет, так как в память попадет большее число процедур и данных, к которым не будет обращений. Так как никогда не выполняются все команды на странице, всегда выполняется только какой-то процент.
- Чем больше страница, тем меньше будет выполнено команд
- Падает, так как размер страницы стремится к размеру программы

Зависимость частоты обращения к странице от количества команд на странице, выполненных до передачи управления на другую страницу



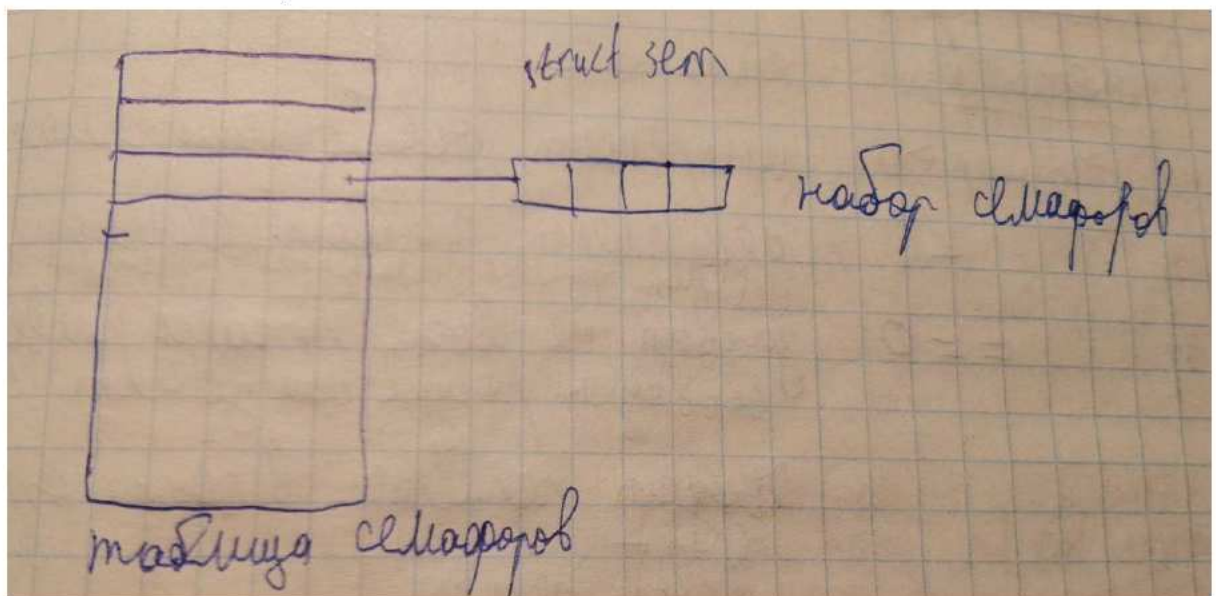
- На графике показано, какой процент команд данной страницы выполняется до того, как управление будет передано другой странице.
- Максимум указанной кривой пришелся на точку, соответствующую 200 командам страницы размеров 4096Б.
- По-видимому нужно уменьшать размер страниц – Дейтелл
- Рабочее множество от Windows называется квота
- Так как предсказать размер рабочего множества невозможно, то каждому процессу перед началом выполнения выдается квота — число страниц, которые процесс может запросить в ОЗУ.
 - Вопрос: сколько страниц выдавать?
- Производится контроль за числом страничных прерываний
 - Если идет рост страничных прерываний, то система работает неэффективно, так как страничные прерывания — затратные действия => квота для данного процесса мала
 - При малейшей возможности квота увеличивается, чтобы процесс мог их загрузить в память, то есть загрузить все рабочее множество.

Семафоры UNIX

- Семафоры UNIX вносят дополнения в семафоры Дейкстры.
- В современных системах поддерживаются наборы семафоров
- Наборы семафоров представлены массивами семафоров
- В UNIX LINUX семафоры поддерживаются системной таблицей(находится в адресном пространстве системы) семафоров. Каждый элемент таблицы описывает один набор семафоров (то есть это дескриптор)
- Дескриптор набора семафоров хранит:
 - имя (целое число) – присваивается процессом, который создал набор семафоров. Другие процессы по этому имени могут получить доступ к нему.
 - uid
 - идентификатор создателя наборов семафоров и идентификатор его группы.

Процесс, `uid` которого совпадает с `uid` созд. может удалить набор и изменить его существующие права(?)

- права доступа
- количество семафоров в наборе
- время изменения 1 или нескольких семафоров последним процессом
- время последнего изменения управляющих параметров набора каким-либо процессом
- указатель на массив семафоров
=> в наборе семафоров семафоры обозначаются индексами начиная с нулевого



- На семафорах определены системные вызовы:
 - `semget()` — создание набора семафоров
 - `semctl()` — изменение управляющих параметров набора семафоров
 - `semop()` — операции на наборе семафоров или на семафоре
- Для определения операций на семафорах определена структура

```
struct sembuf
{
    ushort sem_num; //index
    short sem_op; // операция
    short sem_fl; // флаги
}
```

- В отличие от семафоров Дейкстры, на семафорах UNIX определено 3 операции
- sem_op:
 - > 0 — инкремент, освобождение семафора
 - < 0 — декремент, захват семафора
 - блокирование процесса, если декремент невозможен
 - $== 0$ — захват не выполняется, процесс проверяет, свободен ресурс или занят
 - будет переведен в состояние ожидания освобождения ресурса. После его освобождения декремент не выполн.
- Флаги sem_flg:
 - IPC_NOWAIT — информирует систему о нежелании процесса переходить в состояние ожидания (освобождения семафора)
 - Наличие данного флага объясняется желанием избежать блокирования всех процессов, находящихся в очереди процессов, в случае, если захвативший ресурс процесс завершится аварийно или получит сигнал kill
 - Так как kill невозможно перехватить, процесс не может освободить семафор
 - Блокированный процесс не получает процессорного времени
 - SEM_UNDO — данный флаг указывает ядру, что необходимо отслеживать изменение значения семафора в результате вызова semop.
 - Указывает ядру, что при завершении процесса, вызвавшего semop, необходимо ликвидировать сделанные изменения, чтобы другие процессы не были заблокированы на семафорах навечно
- semget возвращает файловый дескриптор созданного набора семафоров
- Семафоры могут служить как источник информации — процесс освобождает семафор как событие. Но это одно значение — свободен или занят. Но есть и другие средства передачи данных.

Производство-потребление

C

```
#include <stdlib.h>
#include <string.h>
#include <sys/errno.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/time.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#define SB 0
#define SE 1
#define SF 2
#define P -1
#define V 1

struct sembuf start_produce[2] = {{SE, P, 0}, {SB, P, 0}};
struct sembuf stop_produce[2] = {{SB, V, 0}, {SF, V, 0}};
struct sembuf start_consume[2] = {{SF, P, 0}, {SB, P, 0}};
struct sembuf stop_consume[2] = {{SB, V, 0}, {SE, V, 0}};
int count = 0;

int flag = 1;
void sig_handler(int sig_num) {
    flag = 0;
}

void producer(int semfd, char **pr_ptr, char *letter) {
    while (flag) {
        if (semop(semfd, start_produce, 2) == -1) {
            printf("Producer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }

        if (*letter == 'z') {
            *letter = 'a';
            count++;
        } else {
            (*letter)++;
        }
        (*pr_ptr)++;
        if (count == 2) {
            if (kill(0, SIGALRM) == -1) {
                perror("kill");
                exit(1);
            }
            exit(0);
        }

        **pr_ptr = *letter;
    }
}
```

```

        printf("Producer %d - %c\n", getpid(), **pr_ptr);

        if (semop(semfd, stop_produce, 2) == -1) {
            printf("Producer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }
    }

    exit(0);
}

void consumer(int semfd, char **cn_ptr) {
    while (flag) {
        if (semop(semfd, start_consume, 2) == -1) {
            printf("Consumer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }

        (*cn_ptr)++;
        printf("Consumer %d - %c\n", getpid(), **cn_ptr);

        if (semop(semfd, stop_consume, 2) == -1) {
            printf("Consumer %d, errno - %d, error - %s\n", getpid(), errno, strerror(errno));
            exit(1);
        }
    }

    exit(0);
}

int main() {
    int shmfd, semfd;
    int perms = S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH;
    char *addr;
    char **pr_ptr;
    char **cn_ptr;
    char *letter;
    pid_t chpid[7];

    semfd = semget(100, 3, IPC_CREAT | perms);
    if (semfd == -1) {
        perror("semget");
        exit(1);
    }

    if (semctl(semfd, SB, SETVAL, 1) == -1) {
        perror("semctl");
        exit(1);
    }

    if (semctl(semfd, SE, SETVAL, 256) == -1) {
        perror("semctl");
        exit(1);
    }

    if (semctl(semfd, SF, SETVAL, 0) == -1) {
        perror("semctl");
        exit(1);
    }

    shmfd = shmget(101, 256, IPC_CREAT | perms);
    if (shmfd == -1) {
        perror("shmget");
        exit(1);
    }

    addr = shmat(shmfd, NULL, 0);
    if (addr == (char *)-1) {

```

```

        perror("shmat");
        exit(1);
    }

    pr_ptr = (char **)addr;
    cn_ptr = pr_ptr + 1;
    letter = (char *) (cn_ptr + 1);

    *cn_ptr = letter + 1;
    *pr_ptr = *cn_ptr;
    *letter = 'a' - 1;

    if (signal(SIGALRM, sig_handler) == SIG_ERR) {
        perror("signal");
        exit(1);
    }

    for (int i = 0; i < 3; i++) {
        chpid[i] = fork();
        if (chpid[i] == -1) {
            perror("fork");
            exit(1);
        }

        if (chpid[i] == 0) {
            producer(semfd, pr_ptr, letter);
        }
    }

    for (int i = 3; i < 7; i++) {
        chpid[i] = fork();
        if (chpid[i] == -1) {
            perror("fork consumer");
            exit(1);
        }

        if (chpid[i] == 0) {
            consumer(semfd, cn_ptr);
        }
    }

    for (int i = 0; i < 7; i++) {
        int status;
        pid_t child = wait(&status);

        if (WIFEXITED(status))
            printf("Child: PID = %d exited with status = %d\n", child, WEXITSTATUS(status));
        else if (WIFSIGNALED(status))
            printf("Child: PID = %d killed by signal = %d\n", child, WTERMSIG(status));
        else if (WIFSTOPPED(status))
            printf("Child: PID = %d stopped by signal = %d\n", child, WSTOPSIG(status));
        else
            printf("Child has finished: PID = %d\n", child);
    }

    if (shmdt(addr) == -1) {
        perror("shmdt");
        exit(1);
    }

    if (semctl(semfd, 0, IPC_RMID, NULL) == -1) {
        perror("semctl");
        exit(1);
    }

    if (shmctl(shmfd, IPC_RMID, NULL) == -1) {
        perror("shmctl");
        exit(1);
    }

```

```
}

return 0;

}
```

Теория

Формулировка: Есть два типа процессов: первый тип — процессы-производители, которые только производят единицу данных и помещают в ячейки буфера, второй тип — процессы-потребители, которые могут только выбирать данные из буфера.

Производители-потребители — асинхронные процессы, каждый выполняется с собственной скоростью. Если буфер (хранилище) заполнено, то производитель будет блокирован. Если производитель запаздывает, то потребитель ждет производителя. Если ячейка освобождена, то второй раз взять значение из этой же ячейки невозможно.

Семафор — защищенная неотрицательная переменная, на которой определены две операции: P — декремент, захват семафора, V — инкремент, освобождение семафора.

Виды:

1. Бинарный семафор (0/1).
2. Считающий > 0 , int.

Алгоритм Деикстры решения задачи производства-потребления:

Se, Sf — считающие семафоры, Sb — бинарный.

C ▾

Producer:

```
P(Se)
P(Sb)
N = N + 1;
V(Sb)
V(Sf)
```

Consumer:

```
P(Sf)
P(Sb)
N = N - 1;
V(Sb)
V(Se)
```

Sb обеспечивает монопольный доступ к буферу.

Основная особенность семафоров UNIX в том, что одной неделимой операцией можно изменить все или часть семафоров набора.

Система должна предоставить соответствующие системные вызовы, система переходит в режим ядра, что связано с переключением аппаратного контекста.

Структуры для работы с семафорами — структуры ядра. В ядре поддержка с помощью таблицы семафоров, содержащей дескриптор набора (semid_ds)

1. Управление виртуальной памятью: три способа управления виртуальной памятью, общие принципы и сравнение способов управления виртуальной памятью. Распределение памяти сегментами, поделенными на страницы: отложенные действия по обработке страничного прерывания в такой системе - последовательность возникающих прерываний. Сопоставление со страничным преобразованием в архитектуре x86-64.

10.15.2 Виртуальная память

Программа не должна целиком находиться в физической памяти. В физ.памяти должна находиться та часть программы, с которой производится работа в данный момент времени. То есть в памяти должны находиться те команды программы, к которым в текущий момент времени обращается процессор и данные, соответственно.

Это отправная точка идеи виртуальной памяти. Виртуальный — кажущийся, возможный.

Существует 3 схемы управления виртуальной памятью:

- 1) Страницами по запросу
- 2) Сегментами по запросу
- 3) Сегментами, поделёнными на страницы по запросу

В стремлении избавиться от фрагментации, от необходимости выделения непрерывных адресных пространств в физической памяти, пришли к идеи виртуальной памяти.

Но это очень затратная схема, очень много дополнительных действий необходимо выполнять для адресации команды данных. Какой от этого выигрыш, если это так затратно? Увеличение мультизадачности (мультипрограммности (multitasking)), это значит, что больше одной программы может одновременно находиться в физической памяти. Они могут выполняться, могут ждать своей очереди, но это и есть мультизадачность.

Это огромные затраты, но и колоссальный выигрыш. Но от идеи использования виртуальной памяти никто не отказался.

10.16 Управление памятью страницами по запросу

Во всех трёх частях содержится общая часть: «по запросу», то есть тогда, когда процессор в них возникает необходимость, когда процессор обращается к данным, отсутствующим в физической памяти. Реализуется это с помощью прерываний, в

частности, если мы говорим об управлении памятью страницами по запросу, то это будет страничное прерывание. Причём страничное прерывание в системах отнесено к исправимым исключениям.

В начальный момент времени, когда процесс порождён, то есть он получил идентификатор, часть его полей дескриптора проинициализирована системой, процессу выделяется физическая память. В современных системах это какое-то минимально необходимое для начала выполнения количество страниц физической памяти. Условно говоря, 3 страницы: **сегмента кода, сегмента данных и сегмента стека.**

После этого по диаграмме состояний, процесс переходит в очередь готовых процессов для получения процессорного времени. Получив процессорное время, процесс начинается выполняться команда за командой. В какой-то момент времени процессор обращается к данным, отсутствующим в физической памяти (под данными в общем случае понимаются и команды и собственно данные). Возникает страничное прерывание, в результате которого нужная страница будет загружена в физическую память, после чего процесс начнёт своё выполнение. То есть к затратам ещё добавляется обслуживание страничных прерываний, затраты ещё больше.

Управление сегментами по запросу

- Было реализована в первых UNIX
- Связано было с тем, что на уровне страниц теряется принадлежность страниц к процессу и возникает проблема.
- Проблема коллективного использования страниц — несколько процессов заинтересованы в одних и тех же страницах
 - Одни в исходном содержимом
 - Другие в редактировании
- =>дополнительные расходы: приходилось указывать, что страница разделяемая и выдавать права доступа процессам. Нужно тщательно выполнять контроль.

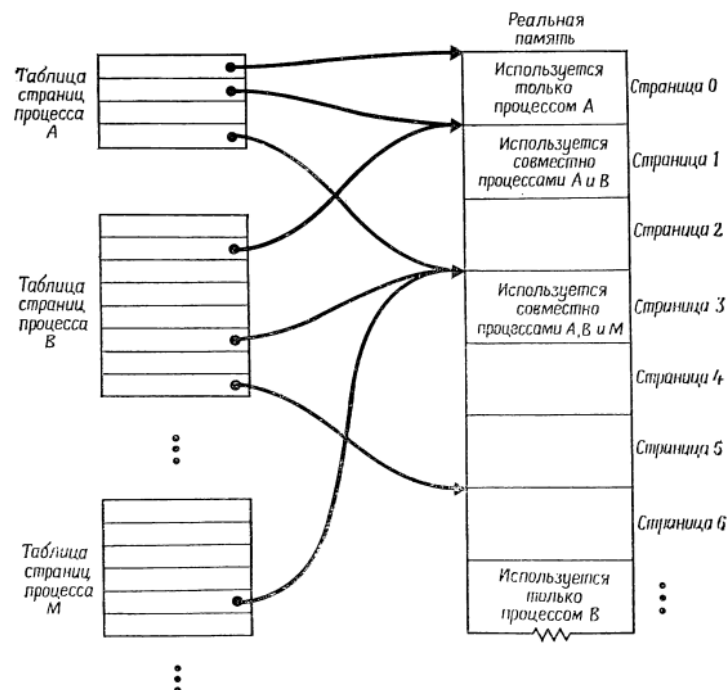
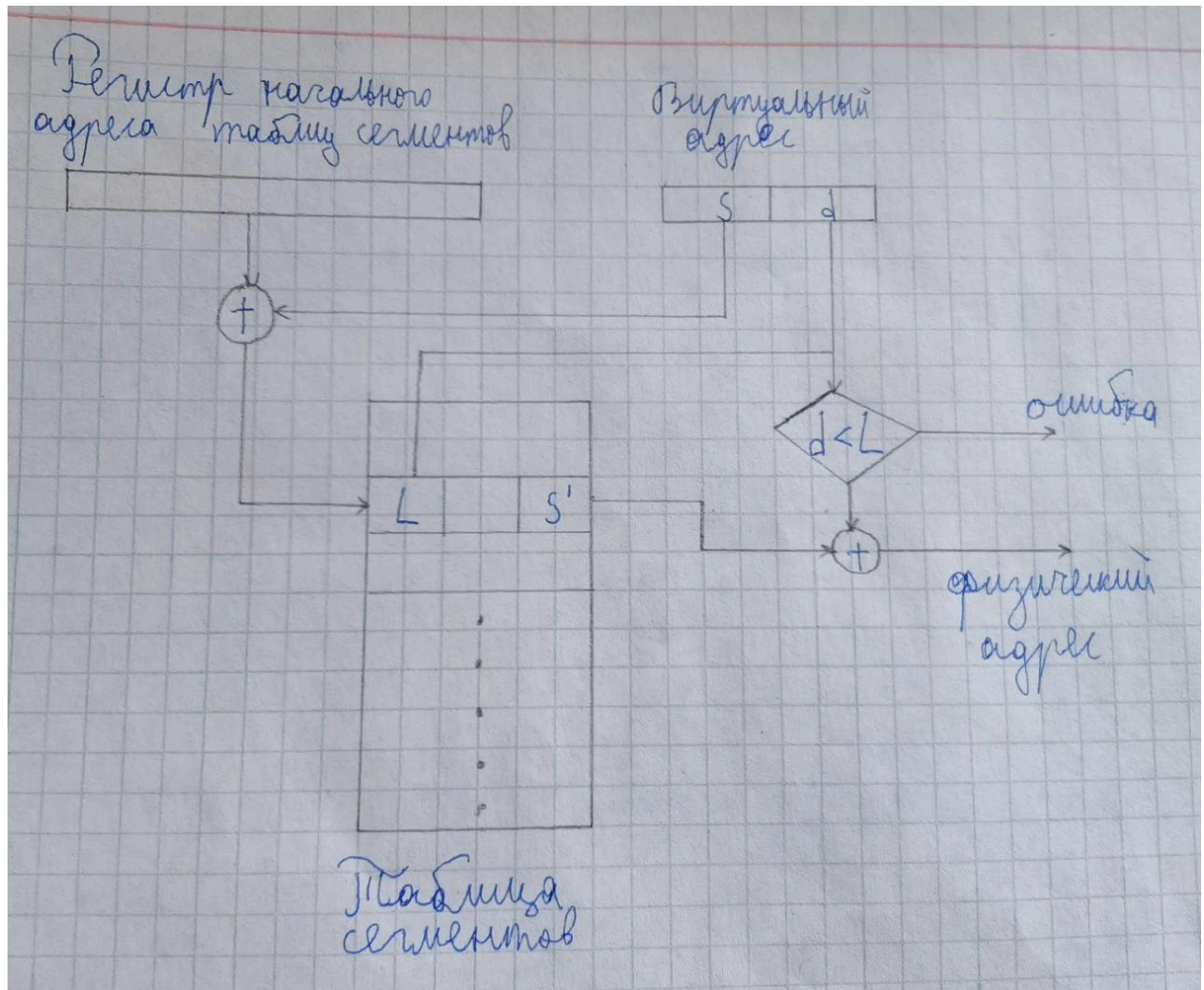


Рис. 8.15 Коллективное использование программ и данных в чисто страничной системе.

- Несмотря на то, что при управлении памятью страницами по запросу, страница меняется на страницу, так как страница является физической единицей, то есть при замене страницы проблем не возникает, но проблемы с коллективным использованием делают более привлекательной схему управления сегментами по запросу.

- Сегмент является единицей логического деления памяти, размер определяется размером программного кода
- Коллективное использование сегментов логично.
Данные сегмента разделяются несколькими процессами, так как в этом коде заинтересованы разные процессы.(?)



- ОС должна иметь информацию о размере сегмента, чтобы контролировать его выход за собственное адресное пространство.
- В таблице сегментов - дескриптор сегментов.
Дескриптор содержит размер, флаги, адрес сегмента
- Размер сегмента в любой системе ограничен
- При управлении сегментами по запросу преобразования выполнялись по дескрипторам
 - S — номер сегмента
 - S' — начальный адрес сегмента
 - L — размер сегмента

- Смещение 20 разрядов
- Если $D \geq L$ – то прерывание по выходу за пределы сегмента, иначе получаем физический адрес
- В режиме long сегментация не используется вообще, но глобальная таблица осталась
- (В сегментные 16-разрядные регистры в защищенном режиме записываются смещения к дескриптору)

Проблемы с управлением сегментами по запросу

- 1) Сегменты – единицы логического деления, имеют разные размеры. При замещении сегмента ОС должна выбрать сегмент для замещения. Используются те же алгоритмы замещения, как и для страниц – LRU — вытесняются те, к которым давно не было обращений, по свойству локальности.
- 2) Если есть свободный сегмент — нужно выбрать, в какой свободный сегмент загрузить сегмент. Стратегии выбора раздела те же: первый подходящий, самый тесный, самый широкий.

3) При swapping

Нашли давно не используемый сегмент. Может оказаться, что он недостаточного размера для загрузки нашего сегмента => придется выгрузить еще один сегмент — в этом огромная проблема.

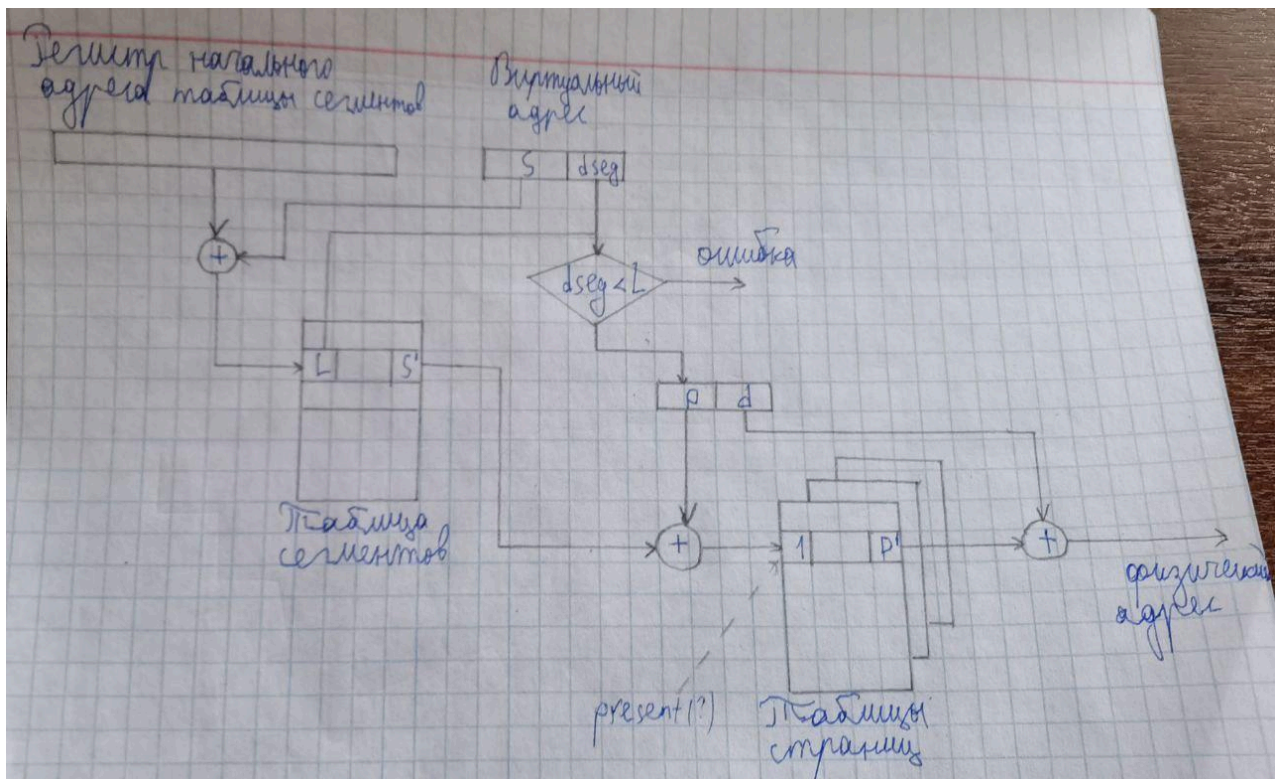
Также возникает фрагментация которая требует передвижений сегментов — дополнительные расходы.

(В страницах существует скрытая фрагментация — последняя страница заполнена не полностью — статистически на $\frac{1}{3}$. Но при использовании(?) страниц на это не обращают внимания)

Таким образом, swapping сегментов затруднен из-за разных размеров.

Управление сегментами, разделенными на страницы, по запросу

- Попытка объединения преимуществ двух подходов(страницами по запросу и сегментами по запросу)
- Каждый процесс имеет одну таблицу сегментов и много таблиц страниц
- При этом размер сегмента должен быть кратен размеру страницы
- Сегментно-страничная организация была реализована в системе Multics



- Обозначения:
 - s – номер сегмента
 - p – номер страницы
 - d – смещение в рамках страницы, по которому находится нужный элемент
- Логика получения адреса:
 - $b+s$ – получаем адрес строки для сегмента s в таблице сегментов первичной памяти. В ней указан адрес базовый адрес s' таблицы страниц для сегмента s
 - $p+s'$ – адрес строки в таблице страниц для страницы p сегмента s . Строка содержит номер фрейма p' .
 - $p'+d$ – реальный адрес
- Просмотр таблицы сегментов может показать, что сегмента s в первичной памяти нет — прерывание по отсутствию сегмента. ОС найдет сегмент во внешней памяти, сформирует для него таблицу страниц и загрузит соответствующую страницу в первичную память. (Возможно вместо какой-то существующей)
- Когда сегмент находится в первичной памяти, обращение к таблице страниц может показать, что нужной страницы в первичной памяти нет — прерывание по отсутствию страницы. ОС найдет страницу во внешней памяти и загрузит ее в первичную память.
- Адрес виртуальной памяти может выйти за рамки сегмента и произойдет прерываний по выходу за пределы сегмента.
- Системные таблицы

Размеры приложений достаточно большие

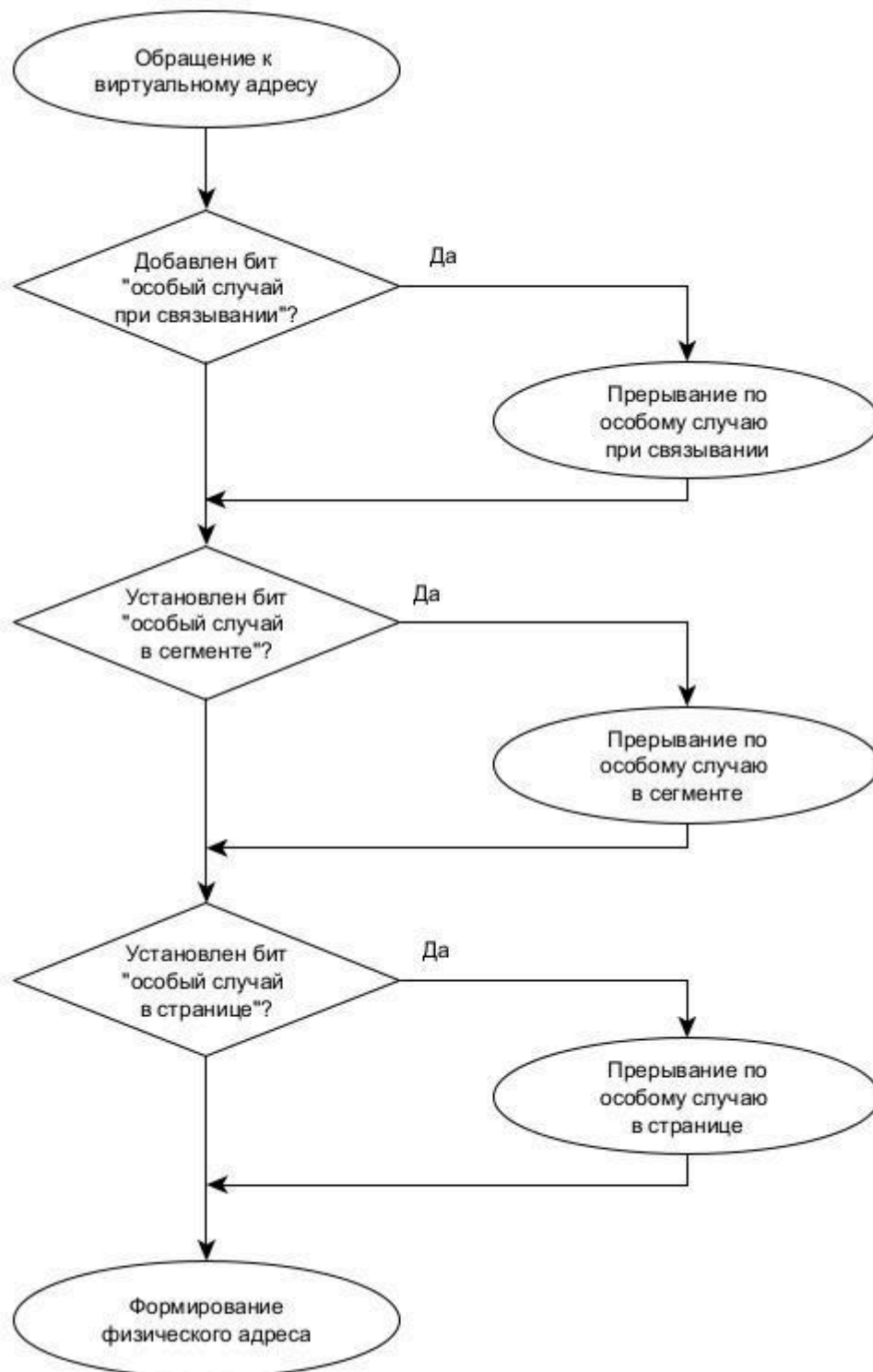
В результате таблица сегментов могла быть большой и сама занимать несколько страниц физической памяти.

При этом таблица сегментов процесса должна находиться в физической памяти.

Если сегмент большого размера, то он мог описываться несколькими таблицами страниц.

И если к сегменту осуществлялось обращение, то страницы тоже должны были находиться в физической памяти.

Вопрос динамического связывания и вопрос отложенного связывания



- Система делает только необходимое в текущий момент действие. Если к сегменту не было обращений, то ему не создается дескриптор

— отложенное действие. Заранее ОС ничего не делает, ведь может оказаться, что обращений к данному сегменту(странице) вообще не будет.

- 1) Особый случай при связывании: когда к сегменту выполняется обращение и оказывается, что нет дескриптора данного сегмента, создается дескриптор
Только при первом обращении!
- 2) Особый случай в сегменте:
сегмент имеет дескриптор, но для данного дескриптора не созданы таблицы страниц
 - а) Создается таблица страниц
 - б) Адрес созданной таблицы страниц будет записан в дескриптор сегмента
(на схеме в дескрипторе сегмента находится начальный адрес таблицы страниц)
- 3) Особый случай в странице:
страница не находится в физической памяти
 - а) должны быть выполнены все необходимые действия по загрузке страницы в физическую память — надо найти свободный фрейм, если нет — вытеснять

Возникают:

- 1) Стр.
- 2) Сегм.
- 3) Деск.

Обрабатываются в обратном порядке.

Огромные затраты!

Поздно выгружать страницы из памяти, когда нужно загружать новые. Это делается заранее. Демон страниц — вызывается по таймеру и анализирует таблицы страниц, чтобы найти те, к которым не было обращений.

Это делается, чтобы сократить время обработки страничного прерывания.

- В современных системах страничное управление, т.к. решили вопрос коллективного использования страницы с помощью флага `copy-on-write`

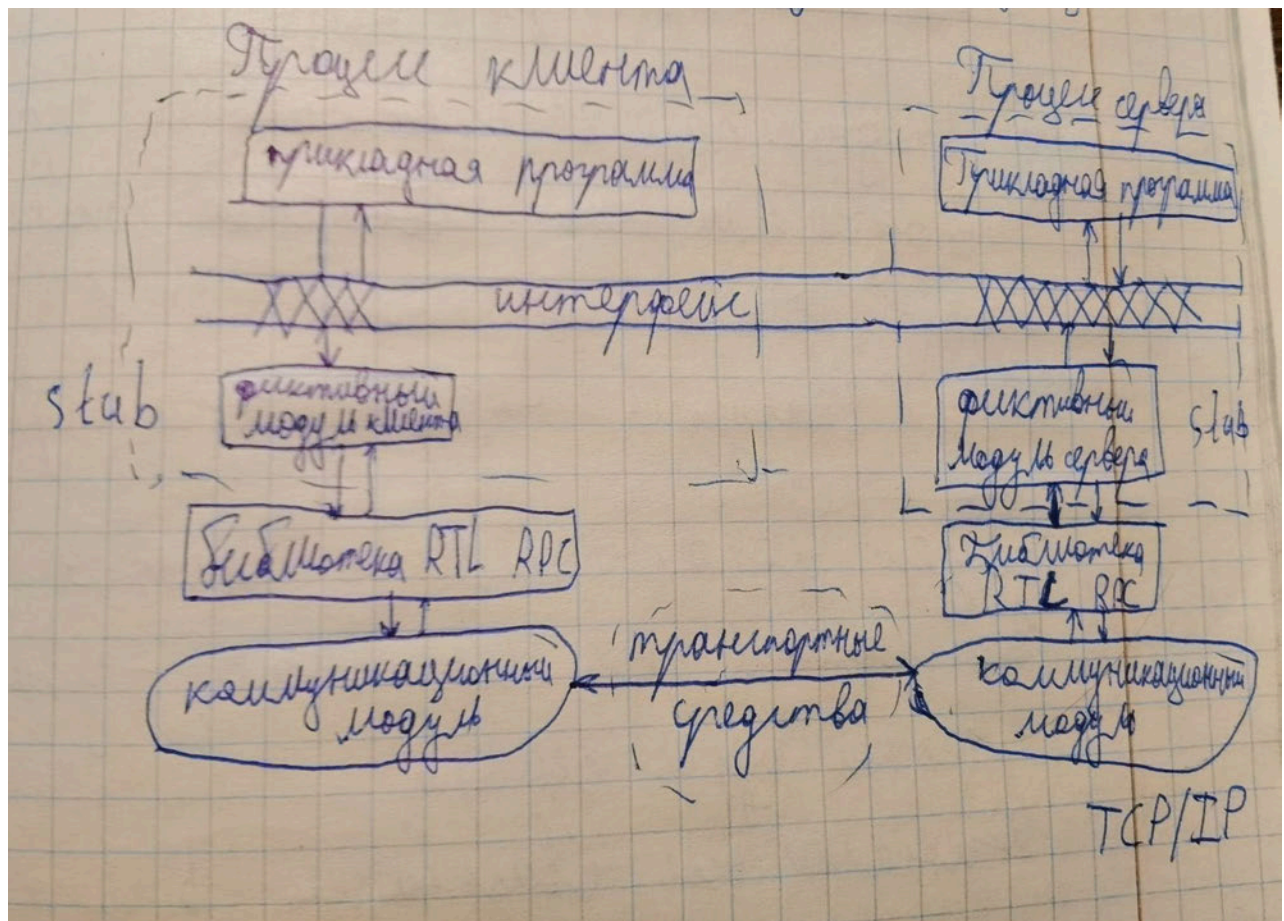
Совместно использование страницы — изменяются права на ONLY READ и устанавливается флаг COPY ON WRITE.

- В режим long реализовано только управление памятью страницами по запросу.
- В режиме long — схема PAE (physical address extension)

2. RPC: концепция и реализация в Linux - rpcgen, файл <name>.x, генерируемые файлы. Пример лабораторной работы по RPC - алгоритм «Булочная».

RPC

- Remote Procedure Calls
- Механизм, с помощью которого процесс инициализирует другой процесс на этой же или на другой машине от своего имени
- Напоминает вызов локальной функции, а именно процесс вызывает функцию и передает ей параметры, а затем ожидает, когда она вернет результат
- Специфика: вызываемую функцию выполняет другой процесс
- Такое взаимодействие обязательно выполняется по модели клиент-сервер
 - процесс, инициализирующий RPC — клиент
 - процесс, выполняющий вызов RPC-функции — сервер
- Смысл RPC — удаленный вызов выполняется как будто локальный
 - чтобы скрыть от пользователя все механизмы удаленного доступа



- Обращение из приложения через интерфейс транслируется в обращение к клиентскому стабу (заглушке)
- Выполняется через сокеты, но так как все механизмы от пользователя закрываются, сокеты не упоминаются
- Закрытие от пользователя всех особенностей взаимодействия дает возможность обращаться как к обычной программе
- Фиктивный модуль клиента:
 - входные параметры от клиента преобразуются в одно или несколько сообщений (пакет)
- Фиктивный модуль сервера
 - то же самое наоборот


```
//bakery.x
```

```
program BAKERY_PROG
{
    version BAKERY_VER
    {
        int GET_NUMBER() = 1;
        long BAKERY_PROC(int) = 2;
    } = 1;
} = 0x20000001;
```

```
//bakery.h
```

```
#ifndef _BAKERY_H_RPCGEN
#define _BAKERY_H_RPCGEN

#include <rpc/rpc.h>
#include <pthread.h>

#ifdef __cplusplus
extern "C" {
#endif

#define CLIENT_COUNT 6

struct BAKERY_RESULT {
    long time;
};
typedef struct BAKERY_RESULT BAKERY_RESULT;

#define BAKERY_PROG 0x20000001
#define BAKERY_VER 1

#define GET_NUMBER 1
#define BAKERY_PROC 2

extern enum clnt_stat get_number_1(void *argp, int *clnt_res, CLIENT *clnt);
extern enum clnt_stat bakery_proc_1(int *argp, struct BAKERY_RESULT *clnt_res, CLIENT *clnt);

extern bool_t get_number_1_svc(void *argp, int *result, struct svc_req *rqstp);
extern bool_t bakery_proc_1_svc(int *argp, struct BAKERY_RESULT *result, struct svc_req *rqstp);

extern bool_t xdr_BAKERY_RESULT(XDR *xdrs, BAKERY_RESULT *objp);

extern int bakery_prog_1_freeresult(SVCXPRT *transp, xdrproc_t xdr_result, caddr_t result);

#ifdef __cplusplus
}
#endif

#endif
```

```
//bakery_xdr.c
```

```
#include "bakery.h"
```

```
bool_t
xdr_BAKERY_RESULT(XDR *xdrs, BAKERY_RESULT *objp)
{
    if (!xdr_long(xdrs, &objp->time))
        return FALSE;
    return TRUE;
}
```

```
//bakery_svc.c
```

```
#include "bakery.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <rpc/pmap_clnt.h>
#include <sys/socket.h>
#include <netinet/in.h>

typedef struct {
    struct svc_req rqstp_copy;
    SVCXPRT *transp;
} thread_data_t;

static void *get_number_worker(void *ptr)
{
    thread_data_t *td = (thread_data_t *)ptr;

    int result = 0;
    bool_t ok = get_number_1_svc(NULL, &result, &td->rqstp_copy);

    if (ok && !svc_sendreply(td->transp, (xdrproc_t)xdr_int, (char *)&result))
        svcerr_systemerr(td->transp);

    (void)bakery_prog_1_freeresult(td->transp, (xdrproc_t)xdr_int, (caddr_t)&result);

    free(td);
    return NULL;
}

static void
bakery_prog_1(struct svc_req *rqstp, register SVCXPRT *transp)
{
    union {
        int bakery_proc_1_arg;
    } argument;

    union {
        int get_number_1_res;
        struct BAKERY_RESULT bakery_proc_1_res;
    } result;

    xdrproc_t _xdr_argument = NULL;
    xdrproc_t _xdr_result = NULL;

    switch (rqstp->rq_proc) {
    case NULLPROC:
        (void)svc_sendreply(transp, (xdrproc_t)xdr_void, (char *)NULL);
        return;

    case GET_NUMBER:
        _xdr_argument = (xdrproc_t)xdr_void;
        _xdr_result = (xdrproc_t)xdr_int;

#ifdef ASYNC_GET_NUMBER && (ASYNC_GET_NUMBER == 1)
        {
            thread_data_t *td = (thread_data_t *)malloc(sizeof(thread_data_t));
            if (!td) {
                svcerr_systemerr(transp);
                return;
            }
            td->rqstp_copy = *rqstp;
            td->transp = transp;

```

```

        pthread_t th;
        pthread_attr_t attr;
        pthread_attr_init(&attr);
        pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);

        if (pthread_create(&th, &attr, get_number_worker, td) != 0) {
            pthread_attr_destroy(&attr);
            free(td);
            svcerr_systemerr(transp);
            return;
        }
        pthread_attr_destroy(&attr);
        return;
    }
#else
    {
        bool_t ok = get_number_1_svc(NULL, &result.get_number_1_res, rqstp);

        if (ok && !svc_sendreply(transp, _xdr_result, (char *)&result))
            svcerr_systemerr(transp);

        (void)bakery_prog_1_freeresult(transp, _xdr_result, (caddr_t)&result);
        return;
    }
#endif

    case BAKERY_PROC:
        _xdr_argument = (xdrproc_t)xdr_int;
        _xdr_result = (xdrproc_t)xdr_BAKERY_RESULT;
        break;

    default:
        svcerr_noproc(transp);
        return;
}

memset((char *)&argument, 0, sizeof(argument));
if (!svc_getargs(transp, _xdr_argument, (caddr_t)&argument)) {
    svcerr_decode(transp);
    return;
}

bool_t ok = bakery_proc_1_svc(&argument.bakery_proc_1_arg, &result.bakery_proc_1_res,
rqstp);

if (ok && !svc_sendreply(transp, _xdr_result, (char *)&result))
    svcerr_systemerr(transp);

if (!svc_freeargs(transp, _xdr_argument, (caddr_t)&argument)) {
    fprintf(stderr, "errorfreeargs\n");
    exit(1);
}

(void)bakery_prog_1_freeresult(transp, _xdr_result, (caddr_t)&result);
}

int
main(int argc, char **argv)
{
    (void)argc; (void)argv;

    SVCXPRT *transp;

    pmap_unset(BAKERY_PROG, BAKERY_VER);

    transp = svctcp_create(RPC_ANYSOCK, 0, 0);
    if (!transp) {

```

```

        fprintf(stderr, "errorrtcp\n");
        exit(1);
    }
    if (!svc_register(transp, BAKERY_PROG, BAKERY_VER, bakery_prog_1, IPPROTO_TCP)) {
        fprintf(stderr, "errorregtcp\n");
        exit(1);
    }

    svc_run();
    fprintf(stderr, "svcrun\n");
    exit(1);
}

```

//bakery_server_join.c

```

#include "bakery.h"
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <rpc/rpc.h>
#include <stdint.h>
#include <stdlib.h>

#ifndef GET_WORK_USEC
#define GET_WORK_USEC 50000
#endif

#ifndef SERVICE_USEC
#define SERVICE_USEC 100000
#endif

static int client_num = 0;

static int current_ticket = 0;

typedef struct {
    int ticket;
} join_worker_ret_t;

static void *join_worker(void *arg)
{
    (void)arg;

    usleep(GET_WORK_USEC);

    join_worker_ret_t *ret = (join_worker_ret_t *)malloc(sizeof(join_worker_ret_t));
    if (!ret) return NULL;

    ret->ticket = client_num++;
    return ret;
}

bool_t
get_number_1_svc(void *argp, int *result, struct svc_req *rqstp)
{
    (void)argp;
    (void)rqstp;

    pthread_t th;
    if (pthread_create(&th, NULL, join_worker, NULL) != 0)
        return FALSE;

    void *pret = NULL;
    if (pthread_join(th, &pret) != 0 || pret == NULL)
        return FALSE;

    join_worker_ret_t *jr = (join_worker_ret_t *)pret;
    *result = jr->ticket;
}

```

```

    free(jr);
    return TRUE;
}

bool_t
bakery_proc_1_svc(int *argp, struct BAKERY_RESULT *result, struct svc_req *rqstp)
{
    (void)rqstp;

    const int ticket = *argp;
    result->time = 0;

    if (ticket != current_ticket)
        return TRUE;

    struct timespec ts;
    if (clock_gettime(CLOCK_REALTIME, &ts) == 0)
        result->time = (long)ts.tv_nsec;

    usleep(SERVICE_USEC);
    current_ticket++;
    return TRUE;
}

int
bakery_prog_1_freeresult(SVCXPRT *transp, xdrproc_t xdr_result, caddr_t res)
{
    (void)transp;
    xdr_free(xdr_result, res);
    return 1;
}

```

```

//bakery_server_mutex_async.c

#include "bakery.h"
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <rpc/rpc.h>
#include <stdio.h>

#ifdef GET_WORK_USEC
#define GET_WORK_USEC 50000
#endif

#ifdef SERVICE_USEC
#define SERVICE_USEC 100000
#endif

static pthread_mutex_t num_mutex = PTHREAD_MUTEX_INITIALIZER;
static int client_num = 0;
static int current_ticket = 0;

bool_t
get_number_1_svc(void *argp, int *result, struct svc_req *rqstp)
{
    (void)argp; (void)rqstp;

    usleep(GET_WORK_USEC);

    pthread_mutex_lock(&num_mutex);
    *result = client_num++;
    pthread_mutex_unlock(&num_mutex);

    return TRUE;
}

```

```

bool_t
bakery_proc_1_svc(int *argp, long *result, struct svc_req *rqstp)
{
    (void)rqstp;
    const int ticket = *argp;
    *result = 0;

    if (ticket != current_ticket)
        return TRUE;

    struct timespec ts;
    if (clock_gettime(CLOCK_REALTIME, &ts) == 0)
        *result = (long)ts.tv_nsec;

    usleep(SERVICE_USEC);
    current_ticket++;

    return TRUE;
}

int
bakery_prog_1_freeresult(SVCXPRT *transp, xdrproc_t xdr_result, caddr_t res)
{
    (void)transp;
    xdr_free(xdr_result, res);
    return 1;
}

```

```

//bakery_clnt.c

#include "bakery.h"
#include <string.h>

static struct timeval TIMEOUT = { 25, 0 };

enum clnt_stat
get_number_1(void *argp, int *clnt_res, CLIENT *clnt)
{
    return clnt_call(clnt, GET_NUMBER,
                     (xdrproc_t)xdr_void, (caddr_t)argp,
                     (xdrproc_t)xdr_int, (caddr_t)clnt_res,
                     TIMEOUT);
}

enum clnt_stat
bakery_proc_1(int *argp, struct BAKERY_RESULT *clnt_res, CLIENT *clnt)
{
    return clnt_call(clnt, BAKERY_PROC,
                     (xdrproc_t)xdr_int, (caddr_t)argp,
                     (xdrproc_t)xdr_BAKERY_RESULT, (caddr_t)clnt_res,
                     TIMEOUT);
}

```

```

//bakery_client.c
#include "bakery.h"
#include <stdlib.h>
#include <stdio.h>
#include <signal.h>
#include <errno.h>
#include <time.h>
#include <sys/types.h>
#include <unistd.h>

int flag = 1;
void sig_handler(int sig_num)
{

```

```

    flag = 0;
    printf("PID: %d\n", getpid());
}

void
bakery_prog_1(char *host)
{
    CLIENT *clnt;
    enum clnt_stat retval_1;
    int result_1;
    char *get_number_1_arg;
    enum clnt_stat retval_2;
    struct BAKERY_RESULT result_2;
    int bakery_proc_1_arg;

    char buff[100];

#ifdef DEBUG
    clnt = clnt_create (host, BAKERY_PROG, BAKERY_VER, "tcp");
    if (clnt == NULL) {
        clnt_pcreateerror (host);
        exit (1);
    }
#endif /* DEBUG */

    retval_1 = get_number_1((void*)&get_number_1_arg, &result_1, clnt);
    if (retval_1 != RPC_SUCCESS) {
        clnt_perror (clnt, "call failed");
    }
    bakery_proc_1_arg = result_1;
    result_2.is_recieved_time = 0;
    while (result_2.is_recieved_time == 0)
    {
        retval_2 = bakery_proc_1(&bakery_proc_1_arg, &result_2, clnt);
        if (retval_2 != RPC_SUCCESS) {
            clnt_perror (clnt, "call failed");
            exit(1);
        }
        if (result_2.is_recieved_time)
        {
            struct timespec ts;
            clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
            time_t time = ts.tv_sec * 1000000000 + ts.tv_nsec;

            printf("Client #%d got result: %.ld\n", bakery_proc_1_arg, time);
        } else {
            printf("Client #%d is waiting\n", bakery_proc_1_arg);
        }
    }
#ifdef DEBUG
    clnt_destroy (clnt);
#endif /* DEBUG */
}

int
main (int argc, char *argv[])
{
    char *host;
    if (argc < 2) {
        printf ("usage: %s server_host\n", argv[0]);
        exit (1);
    }
    host = argv[1];
    if (signal(SIGINT, sig_handler) == SIG_ERR)
    {
        perror("signal");
        printf("errno: %d\n", errno);
        exit(1);
    }

```

```

    }
    srand(getpid());
    while(flag)
    {
        bakery_prog_1 (host);
    }
    exit (0);
}

```

```

//bakery_client_bench.c
#include "bakery.h"
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

static long long mono_ns(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
    return (long long)ts.tv_sec * 1000000000LL + (long long)ts.tv_nsec;
}

int main(int argc, char *argv[])
{
    if (argc < 2) {
        printf("Usage: %s host [id]\n", argv[0]);
        return 1;
    }
    char *host = argv[1];
    int client_id = (argc >= 3) ? atoi(argv[2]) : (int)getpid();
    (void)client_id;

    CLIENT *clnt = clnt_create(host, BAKERY_PROG, BAKERY_VER, "tcp");
    if (!clnt) {
        clnt_pcreateerror(host);
        return 1;
    }
    int ticket = -1;

    long long t0 = mono_ns();
    enum clnt_stat st = get_number_1(NULL, &ticket, clnt);
    long long t1 = mono_ns();

    if (st != RPC_SUCCESS) {
        clnt_perror(clnt, "GET_NUMBER failed");
        clnt_destroy(clnt);
        return 1;
    }

    printf("ticket=%d latency=%.3f ms\n", ticket, (double)(t1 - t0)/1e6);
    clnt_destroy(clnt);
    return 0;
}

```

Экзаменационный билет 8

1. Взаимоисключение и синхронизация процессов и потоков. Семафоры Дейкстры: определение, виды. Семафор, как средство взаимного исключения и синхронизации. Семафоры UNIX/LINUX system V: поддержка семафоров в системе - дескриптор набора семафоров, системные вызовы, определенные на семафорах, примеры реализации решения задач с помощью семафоров: «Производство-потребление» и «Читатели-писатели»

Ответ:

9.1 Определение семафора

Dijkstra EW.

Работа, связанная с семафорами и взаимным исключением была опубликована в 1965 году. Уже в 1965 году решение проблем взаимодействия параллельных процессов назрело, стало очень важным.

Определение 9.1 (Семафор)

Семафор — это неотрицательная защищённая переменная, на которой определены 2 операции:

- 1) P — Первая буква нидерландского слова «Proberen» (пытаться) — декремент, захват семафора. Можно выполнить только если $S > 0$. Иначе процесс блокируется в ожидании освобождения семафора.

$$S = S - 1 \quad S > 0$$

- 2) V — Первая буква нидерландского слова «Verhogen» (поднимать выше) — инкремент. Разблокирует процесс, ожидающий на семафоре.

$$S = S + 1$$

S — Semaphore.

Семафоры бывают:

- 1) Бинарные (0 и 1), определены только две неделимые операции (P и V)
- 2) Считающие (целые положительные значения и 0)

9.3 Взаимоисключение с помощью семафоров

1	S: Semaphore
2	P1:
3	P(S)
4	CR1
5	V(S)
6	PR1
7	P2:
8	P(S)
9	CR2
10	V(S)
11	PR2

Листинг 7. Взаимоисключение с помощью семафоров

P2 получил инициативу, выполнил декремент, P1 получив инициативу, будет блокирован, пока P2 не инкриментирует S. Это взаимоисключение с использованием бинарных семафоров.

1	S1, S2: Semaphore
2	P1:
3	P(S1)
4	P(S2)
5	CR1
6	P(S2) // как будто должно быть V(S2)
7	P(S1) // как будто должно быть V(S1)
8	PR1
9	P2:
10	P(S2)
11	P(S1)
12	CR2
13	P(S1) // как будто должно быть V(S1)
14	P(S2) // как будто должно быть V(S2)
15	PR2

Листинг 8. Пример взаимной блокировки процессов (deadlock)

Процесс изменяет несколько семафоров. 2 семафора. В обоих семафорах нуждаются оба процесса. Каждый из процессов захватил семафор, но для продолжения работы нужно захватить ещё один семафор, который занят другим процессом. Никто свой семафор освободить не может, это тупик.

В современных системах определены **наборы** считающих семафоров. Семантически наборы определены в ОС как массивы.

Задачи синхронизации:

1. Задача производства --- потребления

9.4.1 Задача производства — потребления (Producer — Consumer)

Есть 2 типа процессов:

- 1) Процессы-производители, которые производят единицу каких-то данных и помещают их в ячейки буфера / хранилища, т.е. производители могут только производить и помещать произведенные данные в буфер.
- 2) Процессы-потребители, которые могут только выбирать данные из буфера. В результате ячейка освобождается, и второй раз из этой же ячейки ничего взять невозможно.

Решение задачи предложил Дейкстра. Его решение основывается на 3 семафорах: двух считающих и одном бинарном.

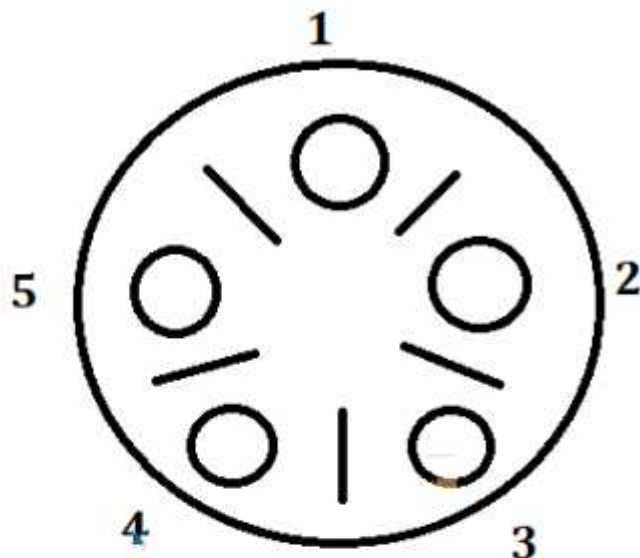
- Считающие семафоры: `Empty`, `Sfull`
- Бинарные семафор: `Sbinary`

```
1 // Начальные значения:
2 //   Empty  = N
3 //   Sfull   = 0
4 //   Sbinary = 1
5 producer:
6   P(Empty)
7   P(Sbinary)
8   N = N + 1
9   V(Sbinary)
10  V(Sfull)
11
12 consumer:
13  P(Sfull)
14  P(Sbinary)
15  N = N - 1
16  V(Sbinary)
17  V(Empty)
```

2. Обедаящие философы

Обедающие философы

За круглым столом сидят 5 философов. Перед каждым философом стоит тарелка. Между тарелками лежит по 1 палочке (для еды нужно 2).



Существует три способа взаимодействия философов:

- 1) Каждый философ пытаются взять сразу оба прибора (левый и правый) и начать есть. После, возвращает оба прибора
- 2) Каждый из философов берет правый прибор и пытается взять левый, удерживая правый
- 3) Если не может взять левый, возвращает правый

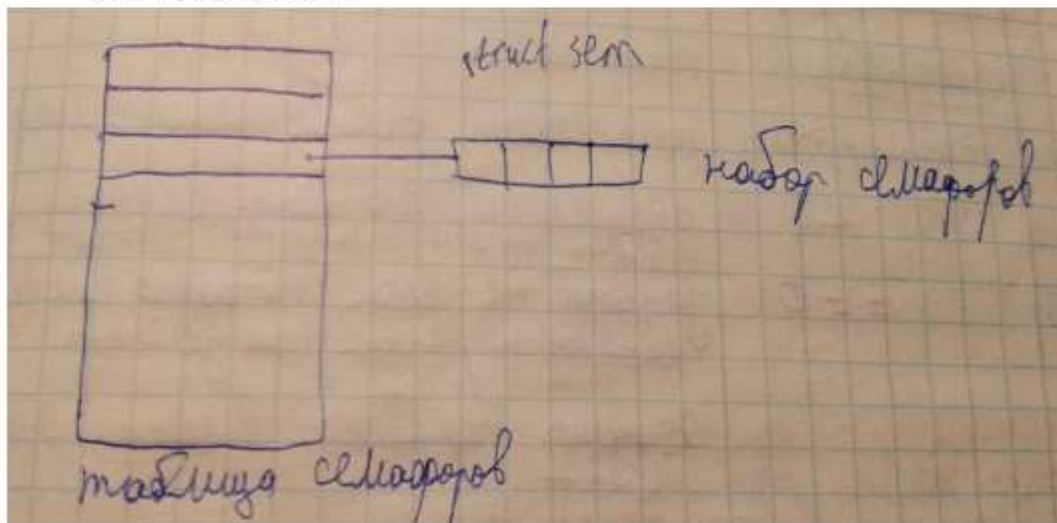
Проблемы:

- 1) Приводит к бесконечному откладыванию
 - a) 1 философ умрет голодной смертью
 - b) 2 и 5 берут оба прибора (1, 3, 4 ждут)
 - c) 5 положил, 4 может взять (1, 3, 5 ждут)
 - d) 2 положил, 3 может взять (1, 2, 4 ждут)
 - e) 1 постоянно ждет
- 2) Все взяли правый прибор, пытаются взять левый — классический пример блокировки deadlock
- 3) Все взяли правый, не могут взять левый => возвращают правый и повторяют эти действия.
 - a) В системе процессы захватывают и освобождают одни и те же ресурсы => полезная работа не выполняется

Семафоры UNIX

Процесс, uid которого совпадает с uid созд. может удалить набор и изменить его существующие права(?)

- права доступа
 - количество семафоров в наборе
 - время изменения 1 или нескольких семафоров последним процессом
 - время последнего изменения управляющих параметров набора каким-либо процессом
 - указатель на массив семафоров
- => в наборе семафоров семафоры обозначаются индексами начиная с нулевого



Семафоры UNIX

- Семафоры UNIX вносят дополнения в семафоры Дейкстры.
- В современных системах поддерживаются наборы семафоров
- Наборы семафоров представлены массивами семафоров
- В UNIX LINUX семафоры поддерживаются системной таблицей(находится в адресном пространстве системы) семафоров. Каждый элемент таблицы описывает один набор семафоров (то есть это дескриптор)
- Дескриптор набора семафоров хранит:
 - имя (целое число) – присваивается процессом, который создал набор семафоров. Другие процессы по этому имени могут получить доступ к нему.
 - uid
 - идентификатор создателя наборов семафоров и идентификатор его группы.

- На семафорах определены системные вызовы:
 - `semget()` — создание набора семафоров
 - `semctl()` — изменение управляющих параметров набора семафоров
 - `semop()` — операции на наборе семафоров или на семафоре
- Для определения операций на семафорах определена структура

```
struct sembuf
{
    ushort sem_num; //index
    short sem_op; // операция
    short sem_fl; // флаги
}
```

- В отличие от семафоров Дейкстры, на семафорах UNIX определено 3 операции
- sem_op:
 - > 0 — инкремент, освобождение семафора
 - < 0 — декремент, захват семафора
 - блокирование процесса, если декремент невозможен
 - $= 0$ — захват не выполняется, процесс проверяет, свободен ресурс или занят
 - будет переведен в состояние ожидания освобождения ресурса. После его освобождения декремент не выполн.
- Флаги sem_flg:
 - IPC_NOWAIT — информирует систему о нежелании процесса переходить в состояние ожидания (освобождения семафора)
 - Наличие данного флага объясняется желанием избежать блокирования всех процессов, находящихся в очереди процессов, в случае, если захвативший ресурс процесс завершится аварийно или получит сигнал kill
 - Так как kill невозможно перехватить, процесс не может освободить семафор
 - Блокированный процесс не получает процессорного времени
 - SEM_UNDO — данный флаг указывает ядру, что необходимо отслеживать изменение значения семафора в результате вызова semop.
 - Указывает ядру, что при завершении процесса, вызвавшего semop, необходимо ликвидировать сделанные изменения, чтобы другие процессы не были заблокированы на семафорах навечно
- semget возвращает файловый дескриптор созданного набора семафоров
- Семафоры могут служить как источник информации — процесс освобождает семафор как событие. Но это одно значение — свободен или занят. Но есть и другие средства передачи данных.

«производство-потребление»

```
int flag = 1;
void sig_handler(int signal)
{
    flag = 0;
```

```
    printf("catch sig: %d, PID: %d\n", signal, getpid());
}

// Структуры для операций с семафорами:
// sem[0] - свободные места (n)
// sem[1] - занятые места (0)
// sem[2] - мьютекс (1)

// Захват свободного места и мьютекса
struct sembuf start_prod[2] = { { 0, -1, 0 }, { 2, -1, 0 } };
// Освобождение мьютекса и увеличение занятых
struct sembuf end_prod[2] = { { 2, 1, 0 }, { 1, 1, 0 } };
// Захват занятого места и мьютекса
struct sembuf start_cons[2] = { { 1, -1, 0 }, { 2, -1, 0 } };
// Освобождение мьютекса и увеличение свободных
struct sembuf end_cons[2] = { { 2, 1, 0 }, { 0, 1, 0 } };

void producer(int semid, char **prod_ptr, char *symb)
{
    while (flag)
    {
        // Захват семафоров для производителя
        if (semop(semid, start_prod, 2) == -1)
        {
            printf("semop, %d, errno: %d\n", getpid(), errno);
            exit(1);
        }

        // Критическая секция - производство
        **prod_ptr = *symb;
        printf("producer: %d, %c\n", getpid(), **prod_ptr);
        // ...

        // Освобождение семафоров
        if (semop(semid, end_prod, 2) == -1)
        {
            printf("semop, %d, errno: %d\n", getpid(), errno);
            exit(1);
        }
    }
}

void consumer(int semid, char **cons_ptr)
{
    while (flag)
    {
        // Захват семафоров для потребителя
        if (semop(semid, start_cons, 2) == -1)
        {
            printf("semop, %d, errno: %d\n", getpid(), errno);
            exit(1);
        }

        // Критическая секция - потребление
```

```
    printf("consumer: %d, %c\n", getpid(), **cons_ptr);
    // ...

    // Освобождение семафоров
    if (semop(semid, end_cons, 2) == -1)
    {
        printf("semop, %d, errno: %d\n", getpid(), errno);
        exit(1);
    }
}

int main()
{
    ...

    // Установка обработчика сигнала для корректного завершения
    if (signal(SIGALRM, sig_handler) == (void (*)(int)) -1)
    {
        printf("signal, errno: %d\n", errno);
        exit(1);
    }

    // Создание массива семафоров (3 семафора)
    if ((semid = semget(key, 3, IPC_CREAT | perms)) == -1)
    {
        printf("semget, errno: %d\n", errno);
        exit(1);
    }

    // Инициализация семафоров:
    if (semctl(semid, 0, SETVAL, n) == -1)    // Свободные места = n
    if (semctl(semid, 1, SETVAL, 0) == -1)    // Занятые места = 0
    if (semctl(semid, 2, SETVAL, 1) == -1)    // Мьютекс = 1 (свободен)

    // Создание дочерних процессов
    for (int i = 0; i < 6; ++i)
    {
        if ((childs[i] = fork()) == -1) { ... }
        else if (childs[i] == 0)
        {
            if (i % 2 == 0)
                producer(semid, prod_ptr, symb);
            else
                consumer(semid, cons_ptr);
            exit(0);
        }
    }

    // Ожидание завершения всех процессов
    ...

    // Удаление семафоров
    if (semctl(semid, 0, IPC_RMID, NULL) == -1)
```

```
{  
    printf("semctl, errno: %d\n", errno);  
    exit(1);  
}  
  
return 0;  
}
```

«читатели-писатели»

```
#include <sys/sem.h>  
#include <sys/shm.h>  
#include <stdio.h>  
#include <unistd.h>  
#include <stdlib.h>  
#include <signal.h>  
  
// Определение семафоров:  
#define WAIT_WRITERS 0 // Ожидающие писатели  
#define WAIT_READERS 1 // Ожидающие читатели  
#define ACTIVE_WRITER 2 // Активный писатель (0 или 1)  
#define ACTIVE_READERS 3 // Количество активных читателей  
  
// Операции для читателя:  
struct sembuf start_read[] = {  
    // Увеличить счетчик ожидающих читателей  
    {WAIT_READERS, 1, 0},  
    // Проверить, что нет активного писателя (ожидание = 0)  
    {ACTIVE_WRITER, 0, 0},  
    // Проверить, что нет ожидающих писателей (ожидание = 0)  
    {WAIT_WRITERS, 0, 0},  
    // Уменьшить счетчик ожидающих читателей  
    {WAIT_READERS, -1, 0},  
    // Увеличить счетчик активных читателей  
    {ACTIVE_READERS, 1, 0},  
};  
  
struct sembuf stop_read[] = {  
    // Уменьшить счетчик активных читателей  
    {ACTIVE_READERS, -1, 0}  
};  
  
// Операции для писателя:  
struct sembuf start_write[] = {  
    // Увеличить счетчик ожидающих писателей  
    {WAIT_WRITERS, 1, 0},  
    // Проверить, что нет активных читателей (ожидание = 0)  
    {ACTIVE_READERS, 0, 0},  
    // Проверить, что нет активного писателя (ожидание = 0)  
    {ACTIVE_WRITER, 0, 0},  
    // Уменьшить счетчик ожидающих писателей  
    {WAIT_WRITERS, -1, 0},  
};
```

```
// Установить активного писателя (увеличить с 0 до 1)
{ACTIVE_WRITER, 1, 0}
};

struct sembuf stop_write[] = {
    // Сбросить активного писателя (уменьшить с 1 до 0)
    {ACTIVE_WRITER, -1, 0}
};

int flag = 1;

void sig_handler(int signal) {
    flag = 0;
    printf("catch sig: %d\n", signal);
}

void reader(key_t semfd, char *var) {
    while (flag) {
        // Захват ресурса для чтения
        // (могут читать несколько читателей одновременно)
        if (semop(semfd, start_read, 5) == -1) {
            if (errno == EINTR) {
                printf("errno EINTR: PID %d\n", getpid());
                exit(1);
            }
            printf("semop: %s\n", strerror(errno));
            exit(1);
        }

        // Критическая секция чтения
        printf("reader: %d read %c\n", getpid(), *var);

        // Освобождение ресурса
        if (semop(semfd, stop_read, 1) == -1) {
            printf("semop: %s\n", strerror(errno));
            exit(1);
        }
    }
}

void writer(key_t semfd, char *var) {
    while (flag) {
        // Захват ресурса для записи (только один писатель)
        if (semop(semfd, start_write, 5) == -1) {
            if (errno == EINTR) {
                printf("errno EINTR: PID %d\n", getpid());
                exit(1);
            }
            printf("semop: %s\n", strerror(errno));
            exit(1);
        }

        // Критическая секция записи (эксклюзивный доступ)
        (*var)++;
    }
}
```

```
    if (*var == 'z' + 1)
        *var = 'a';
    printf("writer: %d wrote %c\n", getpid(), *var);

    // Освобождение ресурса
    if (semop(semfd, stop_write, 1) == -1) {
        printf("semop: %s\n", strerror(errno));
        exit(1);
    }
}

}

int main(void) {
    ...

    // Установка обработчика сигнала
    if (signal(SIGALRM, sig_handler) == (void (*)(int)) -1) {
        printf("signal: %s\n", strerror(errno));
        exit(1);
    }

    // Создание массива из 4 семафоров
    int semfd = semget(IPC_PRIVATE, 4, IPC_CREAT | perms);
    if (semfd == -1) {
        printf("semget: %s\n", strerror(errno));
        exit(1);
    }

    // Инициализация семафоров:
    if (semctl(semfd, 3, SETVAL, 0) == -1) // ACTIVE_READERS = 0
    if (semctl(semfd, 2, SETVAL, 0) == -1) // ACTIVE_WRITER = 0
    if (semctl(semfd, 0, SETVAL, 0) == -1) // WAIT_WRITERS = 0
    if (semctl(semfd, 1, SETVAL, 0) == -1) // WAIT_READERS = 0

    // Создание процессов читателей и писателей
    for (int i = 0; i < readers + writers; ++i) {
        if ((wchlds[i] = fork()) == -1) {
            printf("fork: %s\n", strerror(errno));
            exit(1);
        }
        if (wchlds[i] == 0) {
            if (i % 2 == 0)
                reader(semfd, buf);
            else
                writer(semfd, buf);
            exit(0);
        }
    }

    // Ожидание завершения процессов
    ...

    // Удаление семафоров
    if (semctl(semfd, IPC_RMID, 0) == -1) {
```

```
        printf("semctl: %s\n", strerror(errno));
        exit(1);
    }

    exit(0);
}
```

2. Аппаратные прерывания, задачи аппаратных прерываний. Прерывание от системного таймера: основные функции обработчика прерывания от системного таймера в системах разделения времени.

Ответ:

1.5.2 Система прерываний

Изменение ПО было кардинальным – появилось полноценная система прерываний, которая соответствует современной системе прерываний. Это значит, что если операция ввода-вывода направляет? управляет? канал, то необходимо сообщить процессору, что операция ввода-вывода завершена. Для этого были включены {аппаратные прерывания от внешних устройств прерывания от внешних устройств, которые относятся к аппаратным прерываниям}. Т.е. по завершении операции ввода-вывода, канал информирует процессор о завершении ввода-вывода с помощью аппаратного прерывания, прерывания от устройства ввода-вывода.

Два типа каналов:

- мультиплексорный, позволяет обслуживать несколько медленных устройств, переключаясь между ними;
- селекторный, позволяет в каждый момент времени обслуживать одно быстрое внешнее устройство.

1.5.3 Про аппаратные прерывания

Каналы – программируемые, задача – управлять работой внешних устройств, освободить процессор от непроизводительного управления внешними устройствами (внешние – механические, значит медленные).

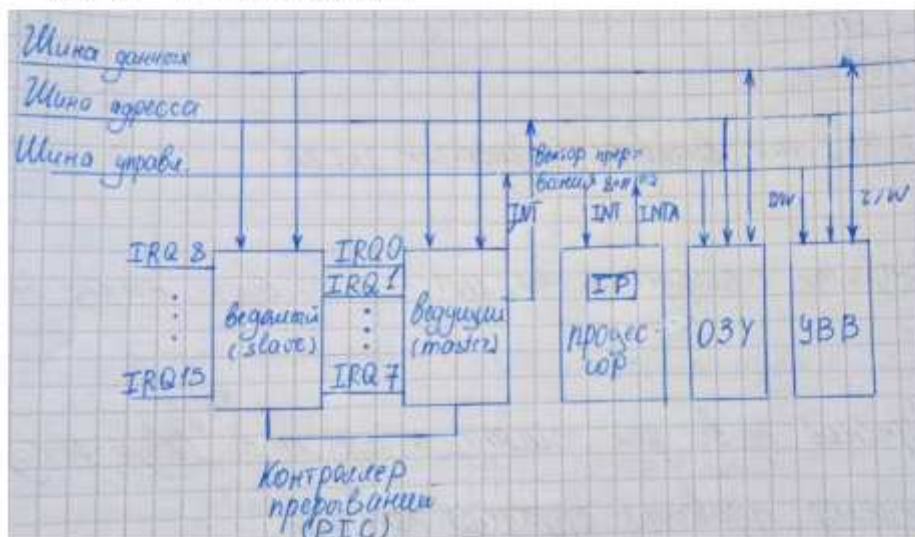
Что значит процессор управляет? Значит, что процессор, например, ждет от внешнего устройства данные (запросил их) и он будет опрашивать готовность устройства, например, клавиатуры, когда же запрошенные данные окажутся в буфере данной клавиатуры. Т.е. будет тратить кванты процессорного времени на опрос внешнего устройства и делать другую работу не может. Выполняется непроизводительная работа. Поэтому реализовано распараллеливание функций и управление устройствами передано каналам (канальная архитектура). В результате появилась группа аппаратных прерываний от устройств ввода вывода. При этом прерывание от устройств ввода вывода информирует процессор о **завершении** операции ввода вывода.

Также включены прерывания действий оператора.

Прерывания

- Система прерываний:
 - системные вызовы (программные прерывания)
 - исключения

- аппаратные прерывания
 - от системного таймера (единственное периодическое)
 - от внешних устройств
 - от действий оператора (Ctrl+C)
- Система прерываний организуется через контроллер прерываний.
- Контроллер прерываний PIC(program interrupt controller) состоит из 2 микросхем, каждая по 8 входов, каскадное соединение.
Входы контроллера – линии IRQ(interrupt request) (линии прерываний). Имеют номера. IRQ2 используется для каскадного соединения.
 - Линии IRQ связаны с внешними устройствами
 - IRQ – interrupt request
 - IRQ0 – тики
 - IRQ1 – от клавиатуры PC/2
 - IRQ2 – от мыши
- 1 контроллер – ведущий, master
- 2 контроллер – ведомый, slave
- Когда на контроллер прерываний приходит прерывание от внешнего устройства, контроллер формирует INT, которые по шине управления поступает на выделенную ножку процессора.
- Цикл выполнения команды
 - → Выборка → Дешифрация → Выполнение
- В конце цикла выполнения команды ЦП проверяет наличие сигнала INT на выделенной ножке. Если он присутствует, то процессор по шине управления посылается INT A(answer)
- Получив этот сигнал от процессора(информирует о готовности обл. с к. д.) контроллер по шине данных процессору посылает вектор, который формируется как сумма базового вектора + номер линии IRQ.
- вектор = базовый вектор + №IRQ
- В реальном режиме базовый вектор ведущего контроллера равен 8, отсюда
INT8h — $8+0=8$;
INT9h — $8+1=9$; клавиатура



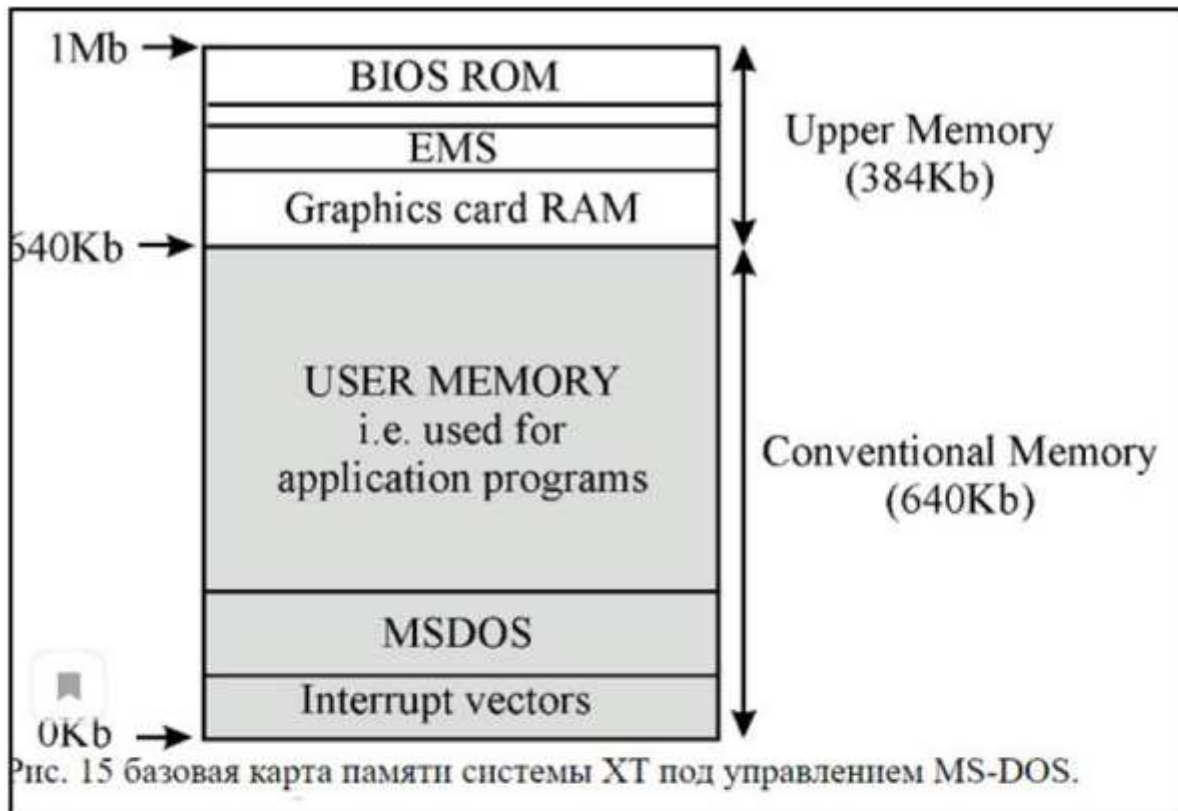


Рис. 15 базовая карта памяти системы XT под управлением MS-DOS.

- На контроллер прерываний посылаются маскируемые прерывания
 - процессор отправляет маску на контроллер, чтобы запретить прерывание
 - Маска посылается по шине данных на контроллер => нужно указать адрес =>
- Контроллер имеет порт (то есть адресуемое устройство)
- В наших компьютерах смешанное адресное пространство
- Системная шина — локальная шина памяти. ЦП и ОС соединены отдельной быстродействующей шиной. Память более чем на порядок медленнее, чем процессор.
- => В процессоре 4 кэша (L1-L3, TLB)
 - L1 и L2 — у каждого ядра. L3 — общий.
- В кэше — команды и данные, к которым были последние обращения. ЦП обращается к ним, и если там нет, лезет в ОЗУ.
- ЦМОС-микросхема
 - энергонезависимая
 - питается от аккумуляторной батарейки
 - благодаря этой микросхеме верное время после выключения компьютера

- Стартуют компьютеры в реальном режиме, чтобы сократить объем стартового кода. Потом система уже загружается в ОЗУ.
- ОС – программа, лежащая на диске => ОС должна быть загружена в ОЗУ.
 - никогда не называется файлом
 - на диске во вторичной памяти

Функции INT8h

- 1) инкремент счетчика реального времени (
 - под реальным временем подразумевается физическое суточное поясное время (время, в котором мы живем).
 - Счетчик находится в области данных BIOS по известному адресу. Сегментный адрес 40h. Это оперативная память (энергозависимые). Когда компьютер выключается, память теряется.
 - 2) Декремент счетчика реального времени до отключения моторчика дисководов
 - Первые компьютеры на INtel 80 не содержали винчестеры, поэтому всю информацию писали на гибкие диски, которые нужно раскрутить до определенной скорости. Это выполнял моторчик дисководов.
 - На то, чтобы раскрутить нужно время. Остановился, нужно записать? Снова раскручиваем. Поэтому контроль моторчика был возложен на счетчик. В него писали 2 секунды, после чего посылался сигнал отключения в порт дисководов.
 - Порты используются для адресации внешних устройств.
 - 3) Вызов пользовательского прерывания 1CH
 - В чистом виде заглушка, чтобы разработчики вешались на него, а не на 8-го прерывания.
 - Если повесят слишком много на 8-е прерывание: это прерывание самое приоритетное, его никто не может прервать. В результате страдает так называемая **отзывчивость системы**.
- Когда вызывается обработчик 8 прерывания? По тикеру

- Тактовый генератор. У компьютера указана тактовая частота. Он генерирует сигнал, в котором длительность низкого напряжения равна длительности высокого напряжения.
 - Из него с помощью делителей и линий задержки выделяются импульсы с определенным периодом.
 - Тик в ДОСе сформирован так, что он повторяется 18.2 раза в секунду. По этому тикку и происходит вызов обработчика 8 прерывания.
 - По каждому тикку вызывается обработчик, который в первую очередь инкрементирует счетчик реального времени
-
- Прерывание от таймера - аппаратное прерывание, единственное периодическое прерывание в системе.
 - Вывод: в обработчиках прерываний циклов быть не может!!! Их нужно завершать как можно быстрее, иначе будет страдать отзывчивость системы. Это общее правило для всех обработчиков аппаратных прерываний.

Экзаменационный билет 9

1. Виртуальная память: управление памятью страницами по запросу - три схемы преобразования виртуального адреса к физическому. Реализация страничного преобразования в компьютерах на базе процессоров Intel (x86_64) - PAE: размеры таблиц и их количество на каждом этапе преобразования, кэш TLB. Обоснование применения данной схемы преобразования. Алгоритмы вытеснения страниц. Дескриптор страницы: основные флаги для работы со страницами

Ответ:

Виртуальная память

- **Виртуальная память** — память, объем которой превышает объем физического адресного пространства
- Устранилось требование загружать в память все адресное пространство задания: в памяти должна находиться та часть адресного пространства задания, к которой обращается процессор
- Управление виртуальной памятью:
 - страницами по запросу
 - сегментами по запросу
 - сегментами, разделенными на страницы, по запросу
- **Виртуальный адрес**:
 - номер страницы
 - смещение

3 способа преобразования:

Прямое преобразование

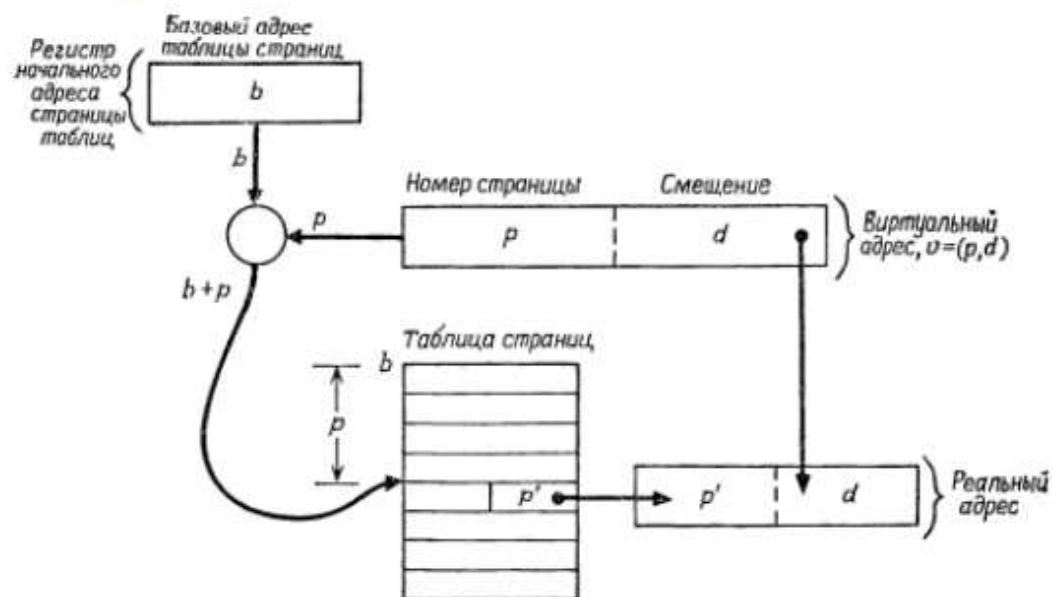


Рис. 8.12 Преобразование адреса страницы путем прямого отображения.

- Виртуальный адрес делится на 2 части: номер страницы и смещение
 - Регистр базового адреса таблицы страниц = регистр начального адреса таблицы страниц
 - Номер страницы – смещение к начальному адресу таблицы страниц
-

- Таблицы имеют один и тот же размер => добавляем смещение и формируем физический адрес
- Таблица страниц в оперативной памяти (в ней дескриптор страницы) Системная таблица => находится в адресном пространстве ядра системы
- Если процесс имеет в виртуальной памяти n страниц, то таблица при прямом отображении для этого процесса содержит последовательные строки для страницы 0, страницы 1, страницы 2, ..., страницы $n-1$.
- Такое преобразование выполняется на каждой команде обращения к оперативной памяти, а то и несколько раз.
 - Считать команду
 - Операнд – получить адрес операнда
 - Косвенный адрес – преобразовать адрес операнда
- Обращение к памяти — затратное действие
- Адрес команды, адрес операнда, м.б. косвенный адрес

Ассоциативное отображение

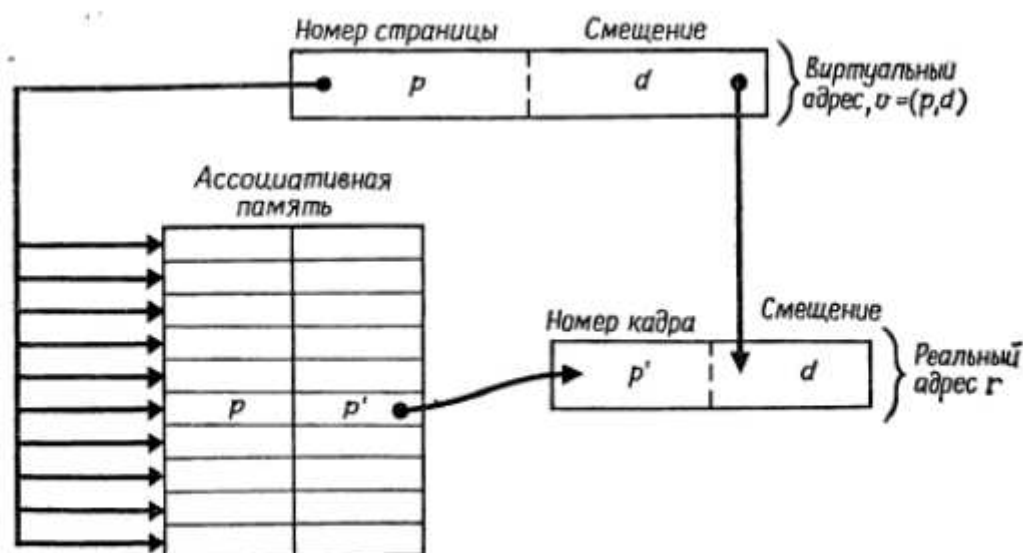


Рис. 8.13 Преобразование адреса страницы при чисто ассоциативном отображении.

- Вся таблица преобразования страниц размещается в ассоциативной памяти (которая работает на порядок быстрее первичной)
- Ключом является номер страницы:
все строки таблицы, хранящейся в ассоциативной памяти, одновременно сравниваются с адресом страницы p .
- Ассоциативная память выдает значение p' как адрес страничного кадра, соответствующего странице p
- Ассоциативная память позволяет сравнивать ключ со своим номером в таблице
- Минусы: стоимость ассоциативной памяти высока
- Что из себя представляет ассоциативная память:

Ассоциативное прямое отображение

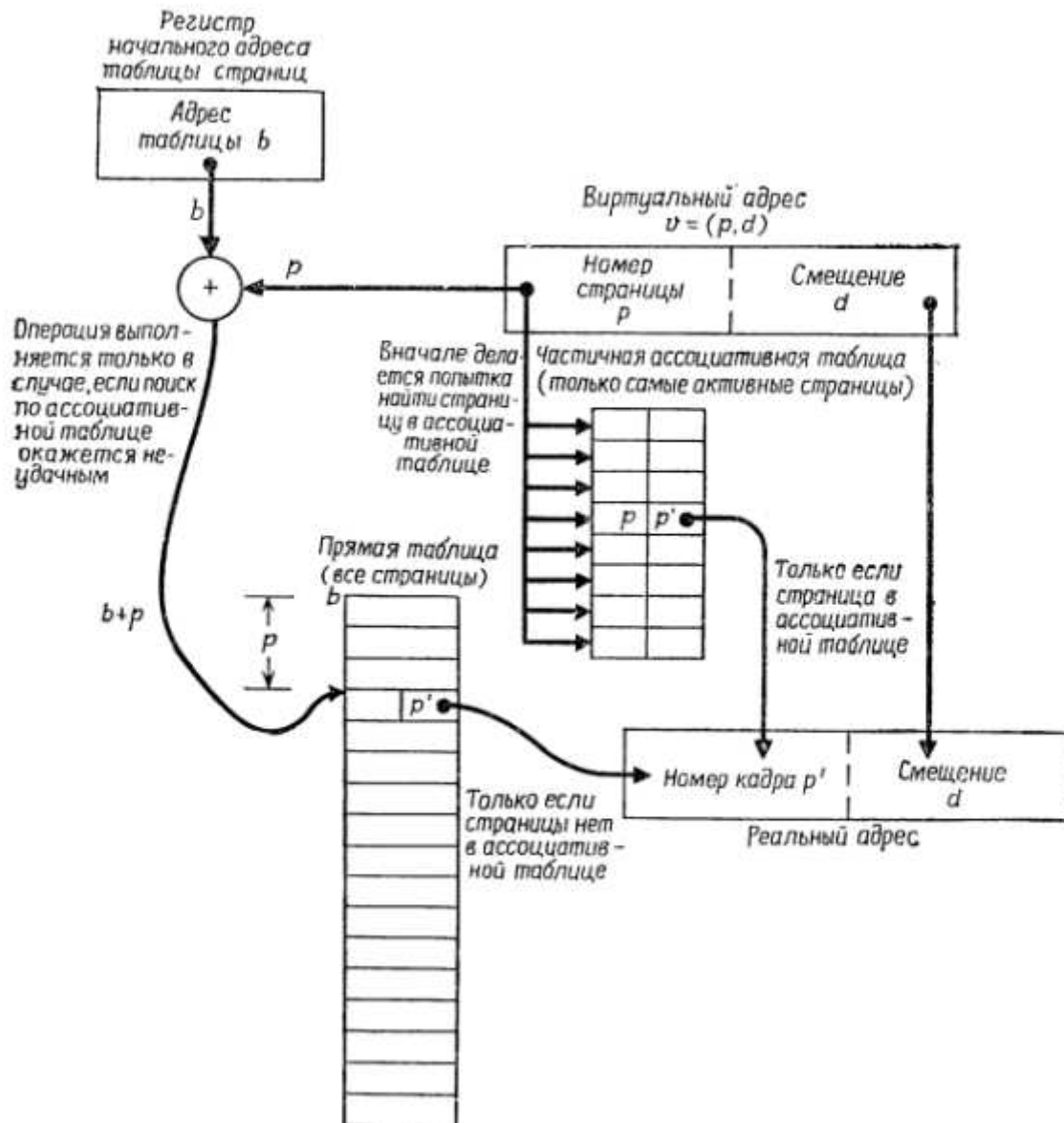
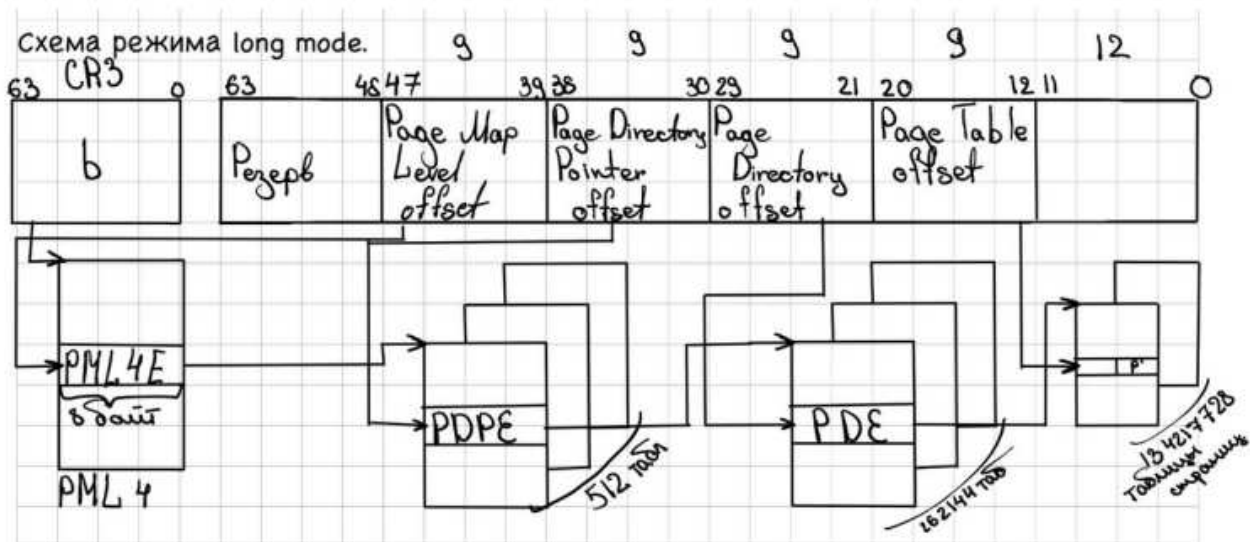


Рис. 8.14 Преобразование адреса страницы при комбинированном ассоциативно-прямом отображении.

- Идея: в ассоциативной памяти хранится лишь небольшая часть полной таблицы отображения страниц для процесса.
В строках этой таблицы отображаются только те страницы, к которым были последние ссылки, — это делается с учетом того эвристического соображения, что к странице, к которой осуществлялось обращение в недалеком прошлом, будет, по всей вероятности, новое обращение в ближайшем будущем.
- Программа обращается по виртуальному адресу (p, d) : сначала механизм преобразования адресов выполняет поиск страницы p в ассоциативной памяти (ассоциативном кэше)
- Если страница p найдена в таблице в ассоциативной памяти, то ассоциативная таблица выдает p' как номер кадра, соответствующего виртуальной странице p' , $r = p' + d$
- Если не найдено, то в таблице
- Обычно такой ассоциативный буфер удовлетворяет большинству запросов
 - В нем адреса страниц, к которым было последнее обращение
 - TLB буфер

PAE



- PTE — таблицы страниц
- PDE — каталоги таблиц страниц —
- PDPE — указатель на каталоги таблиц страниц — максимум 512 табл.
- PML4E — указатель на указатель — такая таблица в системе 1
- Дескрипторы инициализируются по мере надобности
- Размер дескриптора — 8 байт
- Таблицы по 2^9 строк $\Rightarrow$ каждая таблица в памяти занимает одну страницу
- Когда процесс создается — ему выделяется минимально необходимое число страниц.
- То есть выполняется отложено.

Например, при создании процесса создается 3 страницы: точка входа, страница сегмента данных, страница сегмента стека. Все остальные страницы создаются по мере надобности при возникновении системных прерываний.

Про кэш TLB:

У этого кеша есть блок достоверности LRU, в котором 3 бита нужны для реализации алгоритма псевдо-LRU. В кеш-памяти реализован метод сквозной зависимости. Замещение в кеше реализуется по алгоритму псевдо-LRU, для реализации которого определено 3 бита. Биты обновляются при каждом попадании в кеш-память или заполнении строки. Попадание в кеш-память — данная страница находится в физической памяти и её адрес находится в кеше.

Страница сначала всегда ищется в кеше. Если страница в кеше не найдена, то происходит обращение к таблицам страниц в оперативной памяти. Каждая строка кеша имеет бит достоверности. При очистке кеша или сбросе процессора все биты достоверности сбрасываются в 0. При необходимости записи в кеш нового физического адреса, выбирается любая недостоверная строка, при этом бит достоверности устанавливается в единицу. Если недостоверных строк нет, то выполняется замещение. При этом биты достоверности не устанавливаются для строк, являющихся частично достоверными.

- Наши машины могут работать в следующих режимах в связи с обратной совместимостью:
 - реальный режим — работает как компьютер на базе 8086 DOS 6.11, 16 разрядные регистры, 20 разрядная шина данных
 - Защищенный
32 разрядный, регистры 32х разрядные, 32 разрядная шина данных.
Многооперационный режим с поддержкой управления памятью:
 - сегментами по запросу
 - страницами по запросу (но отключить сегментное преобразование нельзя, используется плоская модель памяти. Каждому процессу создается адресное пространство и оно делится на страницы).

Тут впервые появилась схема PAE, но можно было использовать и дескрипторы для таблиц 4 уровня(?)

Стандартный защищенный режим: 4гб памяти, обычная двухуровневая схема преобразований: pte, pde

- long
 - 64 разрядный режим.
В этом режиме поддерживается режим обратной совместимости – можно выполнять 32 разрядные. Но нельзя выполнять 16 разрядные. В 64 разрядном режиме поддерживаются только аппаратные преобразования. Части регистров: 8, 16, 32 — аппаратно доступны => будут выполняться команды соответствующей разрядности.

Алгоритмы вытеснения страниц

Замещение страниц

- Физическая память имеет ограниченный размер
- Возникает ситуация, когда вся физическая память распределена, но процесс запрашивает путем страничного прерывания новую страницу, которую необходимо загрузить в память
- Значит, какую-то страницу придется вытолкнуть на диск

1. Выталкивание случайной страницы

- Выбирается первая попавшаяся страница
Выбирается случайная страница – то есть все страницы, находящиеся в основной памяти могут быть выбраны для выталкивания с равной вероятностью, в том числе даже следующая страница, что наиболее нецелесообразно
- Достоинства:
 - характеризуется малыми накладными расходами

- не является дискриминационной по отношению к каким-либо конкретным пользователям, НО
- Недостатки:
 - может быть вытолкнута часто используемая или только загруженная страница

2. FIFO

- Может быть реализован двумя способами
 - связный список
 - указание временной метки загрузки страницы в память
- Будут вытолкнуты страницы, дольше всего находящиеся в памяти
- Недостаток:
 - может быть вытолкнута часто используемая страница (Стратегия FIFO с достаточно большой вероятностью будет приводить к замещению активно используемых страниц, поскольку тот факт, что страница находится в основной памяти в течение длительного времени, вполне может означать, что она постоянно в работе.)
- Аномалия FIFO:
 - Казалось бы, чем больше фреймов выделяется процессу, тем с меньшим числом прерываний по отсутствию нужной страницы в памяти он будет выполняться.
 - Однако Билейди, Нельсон и Шедлер установили, что в стратегии FIFO определенные последовательности обращений к страницам приводят в действительности к увеличению количества прерываний по отсутствию страницы при увеличении количества страничных кадров, выделяемых процессу.

	1	2	3	4	5	6	7	8	9	10	11	12
	4	3	2	1	4	3	5	4	3	2	1	5
	4+	3+	2+	1+	4+	3+	5+	5	5	2+	1+	1
M=3		4	3	2	1	4	3	3	3	5	2	2
			④	③	②	①	4	4	④	③	5	5
F		+	+	+	+	+	+			+	+	

Частота срабатываний неудар $f = \frac{9}{12} = 75\%$

	1	2	3	4	5	6	7	8	9	10	11	12
	4	3	2	1	4	3	5	4	3	2	1	5
	4+	3+	2+	1+	4+	3+	5+	4+	3+	2+	1+	5+
M=4		4	3	2	2	2	1	5	4	3	2	1
			4	3	3	3	2	1	5	4	3	2
				4	4	④	③	②	①	⑤	④	3
F		+	+	+	+		+	+	+	+	+	+

$f = \frac{10}{12} \approx 83\%$

3. Least recently used (LRU)

- Вытаскиваются те, к которым дольше всего не обращаются
То есть для вытаскивания выбирается та страница, которая не использовалась дольше других.
- В голове списка страниц страницы, к которым дольше всего не было обращения
- Или используются временные метки, обновляемые при каждом обращении к странице

	1	2	3	4	5	6	7	8	9	10	11	12
	4	3	2	1	4	3	5	4	3	2	1	5
	4+	3+	2+	1+	4+	3+	5+	4	3	2+	1+	5+
$M=3$		4	3	2	1	4	3	5	4	3	2	1
			(4)	(3)	(2)	(1)	4	3	(5)	(4)	(3)	2
F	+	+	+	+	+	+	+			+	+	+

	1	2	3	4	5	6	7	8	9	10	11	12
	4	3	2	1	4	3	5	4	3	2	1	5
	4+	3+	2+	1+	4	3	5+	4	3	2+	1+	5+
$M=4$		4	3	2	1	4	3	5	4	3	2	1
			4	3	2	1	4	3	5	4	3	2
				4	3	(2)	(1)	(1)	(7)	(5)	(4)	3
F	+	+	+	+			+			+	+	+

- Стековые алгоритмы $\leftarrow$ LRU
- Обладают свойством локальности:
 - Если обращение было к странице, то наиболее вероятно, что следующее обращение будет к этой же странице
 - Следует из того, что в программах наиболее вероятно следование
- Но очень затратный — при каждом обращении редактируется временная метка

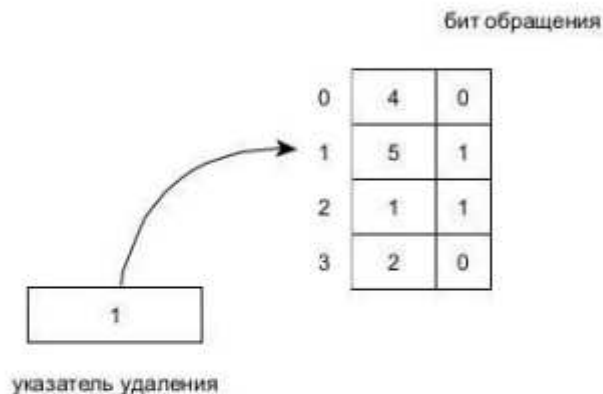
4. Least Frequently Used (LFU)

- Вытесняется наименее часто(наименее интенсивно) использовавшаяся страница
- Для реализации контролируется частота обращения к странице
- Недостаток: может быть вытолкнута только что загруженная страница, так как она еще не набрала обращений

5. Not Used Recently (NUR)

Выталкивание не использовавшейся в последнее время страницы

- К страницам, которые в последнее время не использовались, вряд ли будут обращения и в ближайшем будущем, так что их можно заменить на вновь поступающие страницы
- Желательно заменять ту страницу, которая в период нахождения в памяти не изменялась (почему....?) => вводятся два аппаратных бита-признака на страницу.
- Для каждой страницы записывается бит обращения



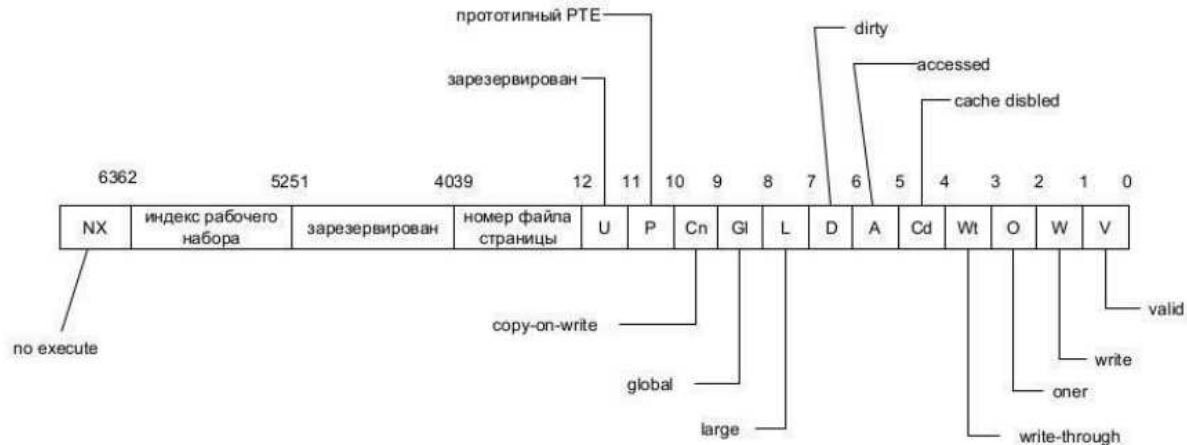
- Изначально биты-признаки обращения и модификации для всех страниц устанавливаются в 0.
- Бит обращения устанавливается при обращении к странице
- Страница для выгрузки в первую очередь ищется среди страниц, у которых бит обращения 0
- Если таких страниц нет, то ничего не остается, кроме как выталкивать страницу, к которой были обращения
- Память работает активно, и рано или поздно у большинства страниц признак обращения будет установлен, и мы не сможем отличать те страницы, которые наиболее целесообразно вытолкнуть
- => биты-признаки обращений периодически сбрасываются
- Если к странице были обращения, то проверяется, подвергалась ли она изменениям: бит модификации
- Если страница не модифицировалась, то выталкиваем ее, поскольку выгоднее выталкивать немодифицированные страницы, так как исключается копирование (на диске есть ее точная копия)

Описанный алгоритм предусматривает существование четырех групп страниц:

Группа	Бит обращения	Бит модификации
1	0	0
2	0	1
3	1	0
4	1	1

- Страницы группы с меньшими номерами следует выталкивать в первую очередь, а с большими - в последнюю.
- (Пояснение ко 2 строке) — модификация произведена до сбросов битов обращения
- Если нет соответствующей страницы в кэше при схеме с гиперстраницами, будет 2 обращения вместо 1
- Для реализации выталкивания большие затраты
- Виртуальная память требует больше затрат (4 уровня таблиц)
- От виртуальной памяти не отказались для увеличения мультизадачности (чтобы в памяти было большое число программ)

- Дескриптор PTE — дескриптор в таблице страниц
Первые 12 разрядов не участвуют в адресации => отводятся для флагов



- Аппаратное ограничение — не все биты решили использоваться для аппаратной адресации
- Access — аппроксимация LRU
Время от времени сбрасывается.
Устанавливается при загрузке страницы и при каждом обращении.
- Dirty — устанавливается, когда страница была изменена
(Обращение с целью записи в страницу приведет к установке)
- COPY ON WRITE — работа программного обеспечения. Когда страница определяется как разделяемая, для устанавливается этот флаг и права доступа только на чтение.
- При обращении к странице будет страничное исключение и флаг COW заставит менеджер памяти создать копию страницы в адресном пространстве процесса, который ее хотел изменить
- Менеджер памяти должен создать копию страницы — должен создать дескриптор страницы. После этого система может работать с этой страницей, выполнять действия над этой страницей.
- Для создания страницы может оказаться необходимым создать дескриптор таблицы страниц.

2. Unix: концепция процессов; иерархия процессов, процессы - «сироты», «зомби», «демоны», группы процессов. Программа с системным вызовом `exec()` (лаб. раб.).

Ответ:

2.2 Основные концепции Unix

2.2.1 Процессы и файлы

На первом месте поставили файловую систему. (Начали разработку юникс с разработки файловой системы)

Разработчики ввели ряд базовых абстракций, самой главной из них является процесс.

Определение 2.1 (Процесс)

Процесс — это программа в стадии выполнения.



! Основная парадигма Unix

В Unix всё — файл.

Устройство тоже файл. Эта идея позволила решить все задачи компактно, при том структурным кодом.

Linux — Unix-подобная операционная система, в которой реализованы все парадигмы.

Иерархия процессов поддерживается указателями. Процесс — программа в стадии выполнения (основная абстракция ОС).

- Что такое процесс-сирота? Процесс, у которого был завершен процесс-родитель
- Почему ввели понятие процесс "сирота" программистами системы? Сохранить иерархию процессов в отношении предок-потомок
- С помощью чего поддерживается иерархия в системе? Соотв. указателями в дескрипторе процесса
- Какой процесс наз. "зомби"? Процесс, у которого отобраны все ресурсы, кроме дескриптора

Демоны — это долгоживущие процессы. Зачастую они запускаются во время загрузки системы и завершают работу вместе с ней. Так как они не имеют управляющего терминала, говорят, что они работают в фоновом режиме. В системе UNIX демоны решают множество повседневных задач.

Группа процессов (process group) — это набор одного или нескольких процессов (? объединенных для целей управления сигналами).

`exec()` -- системный вызов, который переводит процесс на адресное пространство программы, которая была передана ему в качестве параметра

1. разбирает путь к файлу и осуществляет доступ к нему (для того чтобы проверить, существует ли этот файл)
2. проверяет, имеются ли полномочия у файла. проверяет, что он исполняемого проверяет у вызывающего процесса на выполнение исполняемого файла (выполняться может только исполняемый файл) (проверяется, что файл исполняемый, чтением заголовка файла)
3. освобождает ресурсы, которые были выделены при вызове `fork()` до вызова `exec()` (так как `fork` создал для процесса-потомка адресное пространство (таблицы страниц) и была выделена область свопинга)
4. создаются новые таблицы страниц (для процесса-потомка) (для программы, которая передаётся `exec` в качестве параметра)
5. заменяет адреса старых таблиц страниц на адрес новых таблиц страниц в дескрипторе процесса, в `task_struct` указатель `mm`
6. меняет аппаратный контекст: все регистры сбрасываются, а в счётчик команд записывается адрес точки входа (в переданную программу) (в регистр CR3 (в процессорах Intel) необходимо загрузить новый адрес корневой таблицы страниц)

```
int main(void) {
    char *apps[2] = { "./sub1.exe", "./sub2.exe" };
    pid_t childpid[2];
    int i;

    for (i = 0; i < 2; i++) {
        childpid[i] = fork();
        if (childpid[i] == -1) {
            perror("Error fork()");
            exit(1);
        }
        if (childpid[i] == 0) {
            printf("childpid[%d]: %d\n", i, getpid());
            if (execvp(apps[i], "args", NULL) == -1) {
                perror("Can't run program\n");
                exit(1);
            }
        }
    }

    int wstatus;
    for (i = 0; i < 2; i++) {
        waitpid(childpid[i], &wstatus, 0);
        if (WIFEXITED(wstatus))
            printf("Child PID=%d completed, status=%d\n", childpid[i], WEXITSTATUS(wstatus));
        else if (WIFSIGNALED(wstatus))
            printf("Child PID=%d completed with signal %d\n", childpid[i], WTERMSIG(wstatus));
        else if (WIFSTOPPED(wstatus))
            printf("Child PID=%d stopped with signal %d\n", childpid[i], WSTOPSIG(wstatus));
    }

    printf("Parent: PID = %d, group = %d, children = %d, %d\n", getpid(), getpgid(), childpid[0],
        childpid[1]);
}
```

```
    exit(0);  
}
```

1. Виртуальная память: управление памятью страницами по запросам - три схемы преобразования виртуального адреса к физическому. Алгоритмы вытеснения страниц: демонстрация особенностей на модели траектории страниц. Рабочее множество - определение, глобальное и локальное замещение. Схема PAE в x86-64, основные флаги в дескрипторе страницы.

10.15.2 Виртуальная память

Программа не должна целиком находиться в физической памяти. В физ.памяти должна находиться та часть программы, с которой производится работа в данный момент времени. То есть в памяти должны находиться те команды программы, к которым в текущий момент времени обращается процессор и данные, соответственно.

Это отправная точка идеи виртуальной памяти. Виртуальный — кажущийся, возможный.

Существует 3 схемы управления виртуальной памятью:

- 1) Страницами по запросу
- 2) Сегментами по запросу
- 3) Сегментами, поделёнными на страницы по запросу

В стремлении избавиться от фрагментации, от необходимости выделения непрерывных адресных пространств в физической памяти, пришли к идеи виртуальной памяти.

Но это очень затратная схема, очень много дополнительных действий необходимо выполнять для адресации команды данных. Какой от этого выигрыш, если это так затратно? Увеличение мультизадачности (мультипрограммности (multitasking)), это значит, что больше одной программы может одновременно находиться в физической памяти. Они могут выполняться, могут ждать своей очереди, но это и есть мультизадачность.

Это огромные затраты, но и колоссальный выигрыш. Но от идеи использования виртуальной памяти никто не отказался.

10.16 Управление памятью страницами по запросу

Во всех трёх частях содержится общая часть: «по запросу», то есть тогда, когда процессор в них возникает необходимость, когда процессор обращается к данным, отсутствующим в физической памяти. Реализуется это с помощью прерываний, в

частности, если мы говорим об управлении памятью страницами по запросу, то это будет страничное прерывание. Причём страничное прерывание в системах отнесено к исправимым исключениям.

В начальный момент времени, когда процесс порождён, то есть он получил идентификатор, часть его полей дескриптора проинициализирована системой, процессу выделяется физическая память. В современных системах это какое-то минимально необходимое для начала выполнения количество страниц физической памяти. Условно говоря, 3 страницы: **сегмента кода, сегмента данных и сегмента стека.**

После этого по диаграмме состояний, процесс переходит в очередь готовых процессов для получения процессорного времени. Получив процессорное время, процесс начинается выполняться команда за командой. В какой-то момент времени процессор обращается к данным, отсутствующим в физической памяти (под данными в общем случае понимаются и команды и собственно данные). Возникает страничное прерывание, в результате которого нужная страница будет загружена в физическую память, после чего процесс начнёт своё выполнение. То есть к затратам ещё добавляется обслуживание страничных прерываний, затраты ещё больше.

10.17 Схемы преобразования адресов в страничном распределении

10.17.1 Прямое отображение

Процессы имеют виртуальные адресные пространства. Виртуальный адрес делится на две части: **номер страницы и смещение.** В составе регистра процесса обязательно должен быть регистр базового адреса таблицы страниц.

Каждый процесс имеет собственную таблицу страниц, в которой находятся дескрипторы страниц.

Фактически номер страницы виртуального адреса служит смещением к дескриптору в таблице страниц конкретного процесса (?). Если страница загружена в физическую память, то из этой таблицы можно получить её адрес. Смещение одно и то же. Причём смещение берётся из программы, выполняется преобразование и получается физический адрес.

При этом такая таблица страниц находится в физической памяти. **То есть для того, чтобы выполнить преобразование вот в соответствии с этой схемой, сначала надо обратиться к таблице страниц, получить физический адрес страницы и затем выполнить преобразования для получения физического адреса данных.** Такое преобразование будет выполняться на каждой команде, а то и несколько раз — крайне затратные действия.

Обращение к физической памяти требует большого количества тактовых импульсов, так называемых циклов обращения к памяти. Это затратное действие. Стремясь

уменьшить эти накладные расходы, разработчики систем, основываясь на своих технических знаниях, предложили использовать ассоциативную память.

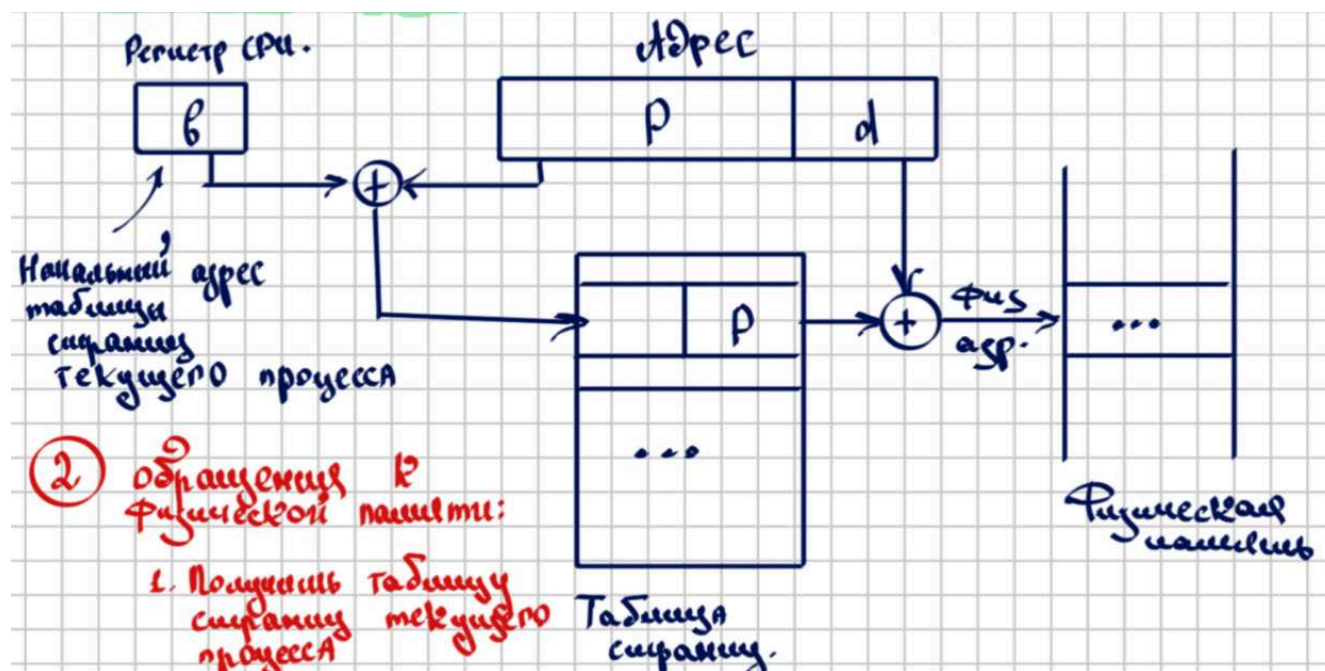


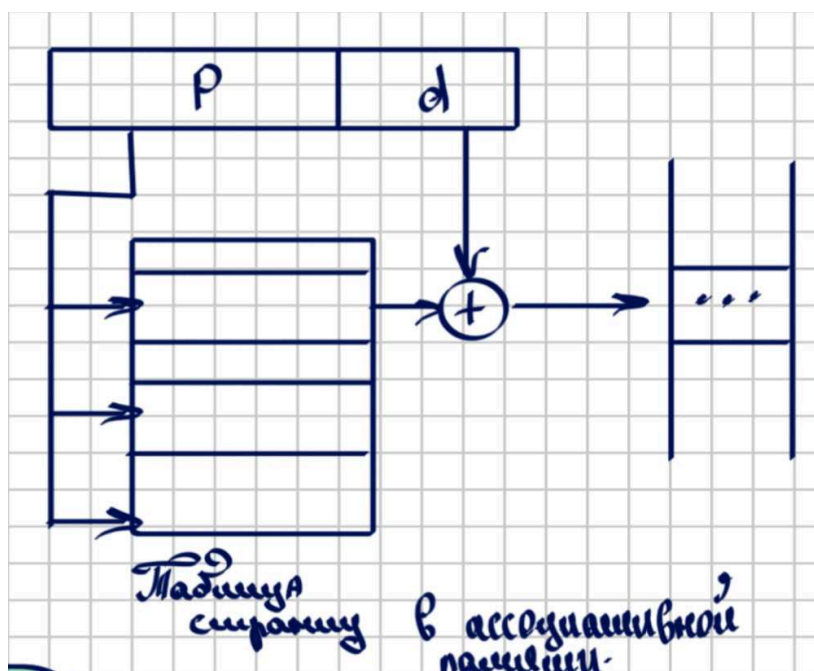
Рис. 36.

10.17.2 Ассоциативное отображение

Таблица страниц загружается в ассоциативную память.

Особенностью ассоциативной памяти является возможность получения данных за 1 такт за счёт схемы реализации ассоциативной памяти.

Важно понимать, почему эта схема не нашла своё физическое воплощение, а лишь частичное. Ассоциативная память очень дорогая, это всегда регистровая память.



Номер виртуальной страницы является ключом, ключ — 4 разряда. При совпадении ключа формируется импульс, который позволяет считать информацию. Каждый разряд каждого регистра дополнительно имеет схему совпадения (тут схема архитектурская).

Мало того, что это регистровая память (количество схем увеличивается в 2 раза), также увеличивается количество соединений. Это говорит о дороговизне ассоциативной памяти.

А с ростом размеров программ таблицы страниц увеличиваются, они становятся очень большими. Значит, необходимо иметь ассоциативную память очень большого размера, специальную ассоциативную память. То есть теоретически это возможно, но практически это сталкивается с огромными затратами, и с огромной стоимостью такой памяти.

10.17.3 Ассоциативно-прямое отображение

⚠ ВНИМАНИЕ

На картинке таблицы ассоциативной памяти написано, а она говорила просто ассоциативная память

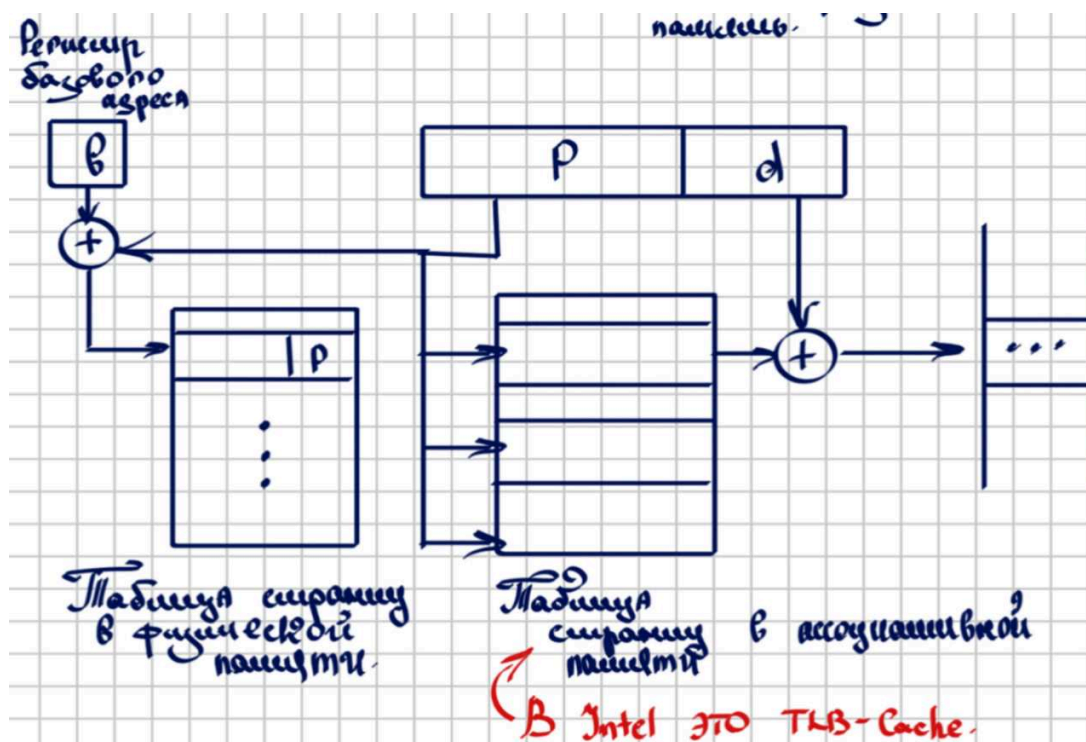


Рис. 38.

В физической памяти располагается таблица страниц. В процессоре есть регистр базового адреса таблицы страниц и ассоциативная память небольшого размера, в которое может находиться только небольшое количество дескрипторов страниц.

10.17.4 Схема с гиперстраницами (ЛИШНЕЕ НО НА ВСЯКИЙ)

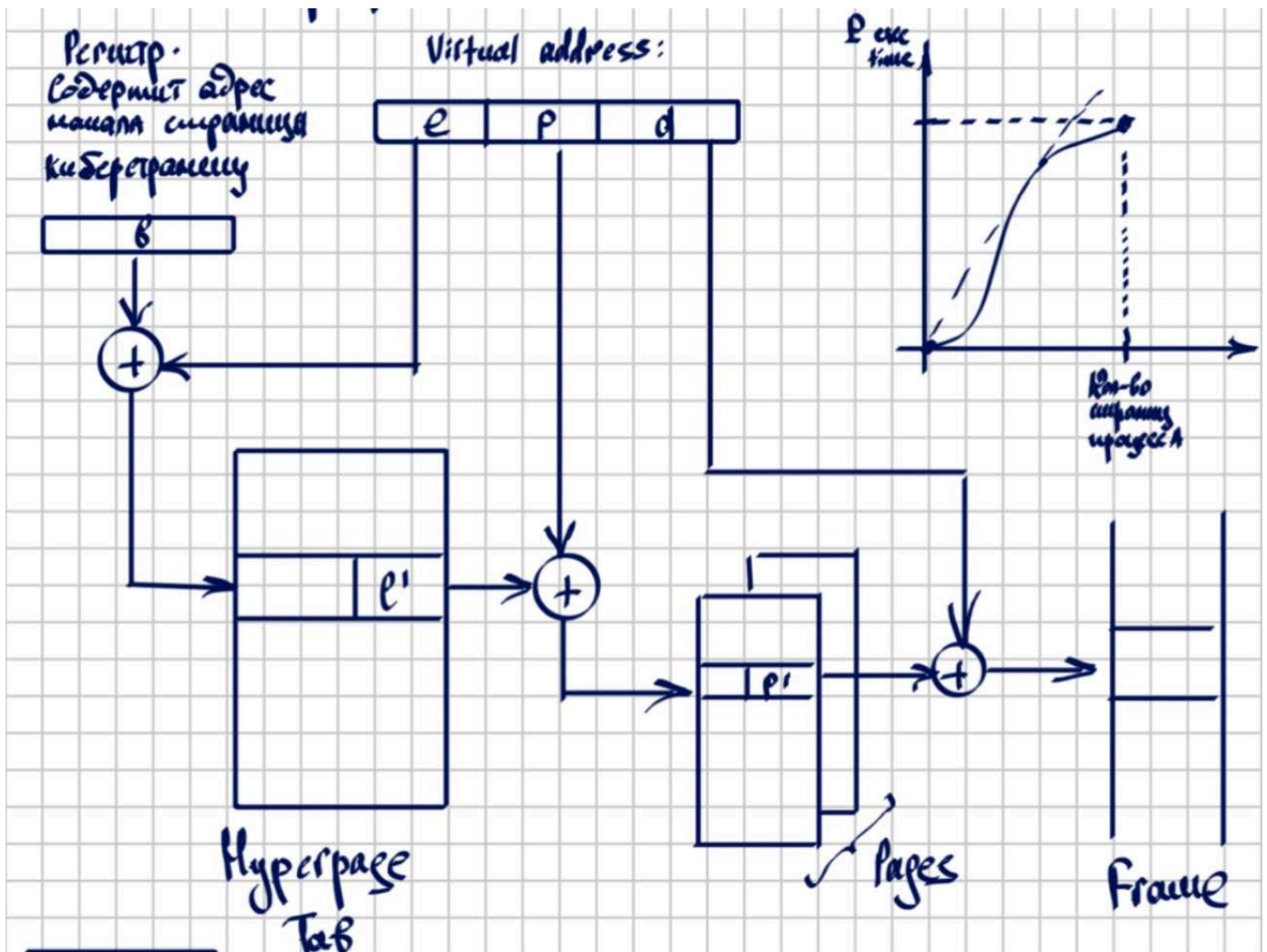


Рис. 39.

Адресное пространство процесса делится на гиперстраницы (грубо говоря, очень большие страницы), размер которых кратен одной странице, то есть это большая страница, содержащая другие страницы.

Это многоуровневая схема. Сначала таблица гиперстраниц, причем виртуальный адрес делится на 3 части: смещение, номер страницы, номер гиперстраницы. То есть теперь адресное пространство процесса описывается одной таблицей гиперстраниц и большим количеством таблиц страниц.

Вместо одного обращения к физической памяти для одного преобразования мы получаем два. Сюда так же нужно добавить ассоциативный кэш, которые сокращает затраты. К кэшу будет обращение по двум полям, *i* и *p*.

10.18 Замещение страниц

Процесс никогда не обращается одновременно ко всем своим страницам. Физическая память имеет ограниченный размер — это дефицитный ресурс в системе.

Процесс выполняется, процессор в какое-то время обращается к странице, которая отсутствует в физической памяти, а свободный фрейм отсутствует, то есть все стра-

ницы физической памяти заняты. Изначально различали страницу виртуального адресного пространства (тут сказала, что как-то назвали, но я не расслышала) и страницу физического адресного пространства, называемую фреймом. Что делать, если не хватает свободных физических страниц, все фреймы распределены? Надо выполнить замещение.

10.18.1 Замещение случайной страницы

Не является дискриминационным, то есть с равной вероятностью может быть вытеснена любая страница любого процесса, но также существует вероятность, что будет вытеснена только что загруженная страница или часто используемая. Реализация такого алгоритма малозатратна. Но из-за недостатков применяется редко.

10.18.2 Алгоритм FIFO

Для реализации данного алгоритма каждому фрейму присваивается или временная метка, или организуется связанный список. Вытеснена будет страница, которая дольше всего находилась в физической памяти. Но также такой страницей не окажется только что загруженная страница, но может оказаться часто используемая страница. Алгоритм FIFO по своим характеристикам может приблизиться к первому алгоритму замещения страниц в крайних случаях.

Но также было обнаружено, что алгоритм FIFO обладает свойством аномалии (аномалия алгоритма FIFO). Она заключается в том, что при увеличении объёма физической памяти, расчёт на то, что количество страничных прерываний сократится, не оправдывается. Количество страничных прерываний для некоторых траекторий страниц может увеличиться.

Размер памяти — 3 страницы. Траектория страниц — номера страниц, к которым выполняются обращения.

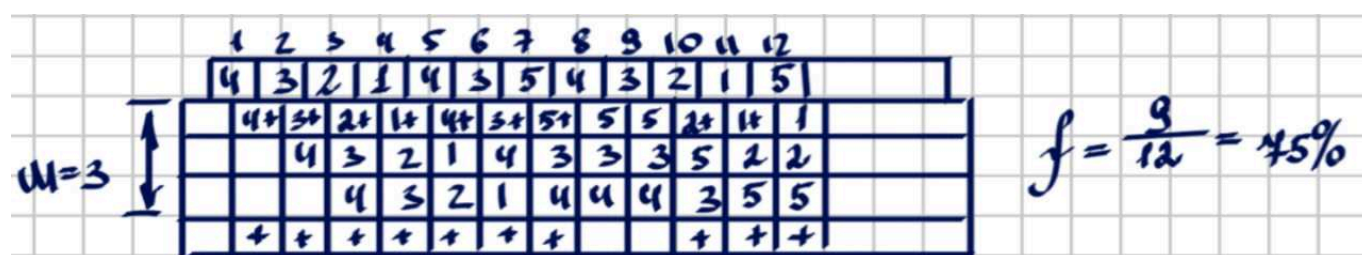


Рис. 40.

Соответственно, сначала надо загрузить четвёртую страницу. Обозначаем действие плюсом. Затем третью, четвёртая перемещается в голову. Затем вторую по алгоритму Фифо, четвертая перемещается в голову. И вот надо загрузить первую страницу. У нас нет свободных фреймов. Значит, по алгоритму Фифо будет выгружена четвёртая страница. Теперь надо загрузить четвёртую. Она в памяти. Так называемая страничная удача, но очередь не переформируется (остаётся в том же виде). Теперь нужно загрузить третью. Она также в памяти, также страничная удача. Очередь

не переформируется. Теперь надо вторую загрузить, выгрузится четвёртая. Теперь надо загрузить первую, выгрузится третья. И теперь надо загрузить пятую. Она в памяти. Очередь не переформируется.

Увеличим память на одну страницу.

10.18.3 Алгоритм LRU

LRU — least recently used, page replacement (наименее используемое последнее время)

Алгоритм относится к классу стековых алгоритмов. Алгоритм наиболее полно отвечает свойству локальности наших программ, но в реальных системах не используется, так как требует колоссальных затрат на реализацию. (Обращение к странице — на каждой команде, а то и несколько раз, если косвенный вызов)

Свойство 10.12 (Свойство локальности)

Свойство является следствием архитектуры фон Неймана. Если обращение было к странице, то наиболее вероятно, что следующие обращения будут к этой же странице. Это связано с тем, что программа в своём адресном пространстве располагается в последовательных адресах (в виртуальном адресном пространстве), команда за командой, при этом наиболее вероятной структурой является следование.

По этому алгоритму будет вытеснена страница, к которой дольше всего не было обращений. Это требует изменения работы или с временными метками или со связным списком. При каждом обращении к странице должна быть отредактирована временная метка. Если это связный список, то при каждом обращении страница перемещается в конец списка (меняется адрес). Это крайне затратные действия в системе, потому что обращения происходят на каждой команде, а то и несколько раз.

Сначала размер памяти возьмем 3 страницы, потом увеличим на единицу. Обращение к 4 (если находится в памяти – **страничная удача**), она есть, по алгоритму помещается в конец связного списка. Дальше обращение пополняется третьей странице. Она также находится в памяти, и она переносится в конец связанного списка.

LRU - least recently used

	1	2	3	4	5	6	7	8	9	10	11	12	
	4	3	2	1	4	3	5	4	3	2	1	5	
↑		4+	3+	2+	1+	4+	3+	5+	4	3	2+	1+	5+
M=3			4	3	2	1	4	3	5	4	3	2	1
			4	3	2	1	4	3	5	4	3	2	1
↓													
F		+	+	+	+	+	+			+	+	+	

	1	2	3	4	5	6	7	8	9	10	11	12	
	4	3	2	1	4	3	5	4	3	2	1	5	
↑		4+	3+	2+	1+	4	3	5+	4	3	2+	1+	5+
M=4			4	3	2	1	4	3	5	4	3	2	1
			4	3	2	1	4	3	5	4	3	2	1
↓													
F		+	+	+	+		+			+	+	+	

Рис. 41. LRU, пример

Как мы видим, количество страничных удач увеличилось (для 4 страниц), то есть не увеличилось число страничных прерываний. При этом обратим внимание, что первые три строки совпадают, поскольку алгоритм LRU относится к классу стековых алгоритмов (?).

10.18.4 Алгоритм LFU

LFU — least frequency used, page replacement (наименее часто используемая страница).

В этом алгоритме фактически контролируется частота обращения к странице. Это реализуется с помощью счётчика. Наименее используемой может оказаться только что загруженная страница. В результате она будет вытеснена, и поскольку она будет вытеснена, потому что не успела набрать счётчик обращений, её необходимо будет загрузить заново. Это большие накладные расходы, поэтому используется аппроксимация (алгоритм NUR).

10.18.5 Алгоритм NUR

NUR — not used recently (страница, не используемая в последнее время)

Это аппроксимация LRU

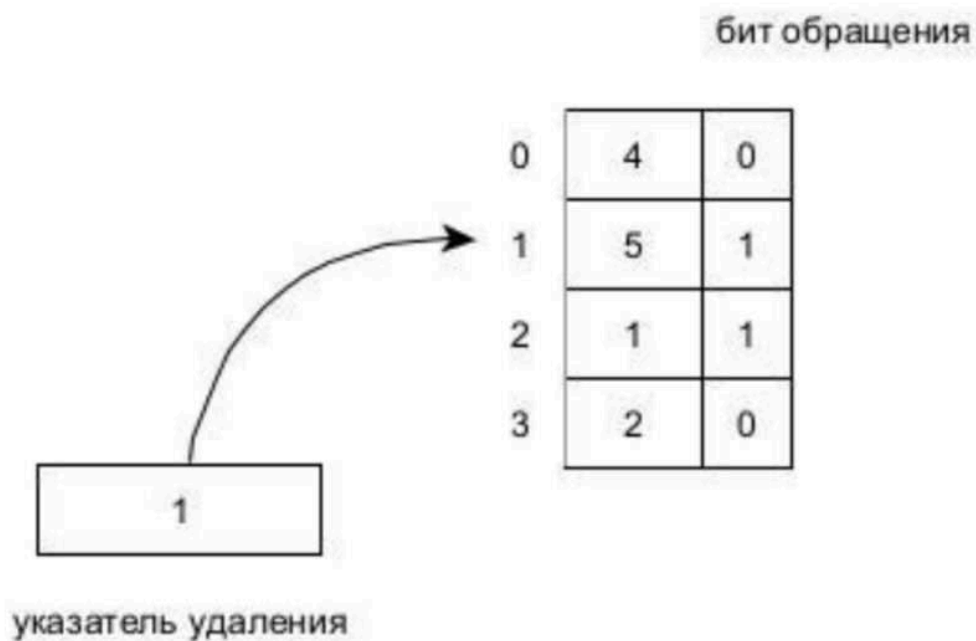


Рис. 42.

Имеется бит, который изменяется при каждом обращении к странице. При загрузке бит устанавливается в 1 и с этого момента будет рассматриваться удаление.

Именно этот способ реально используется в системах. В дескрипторе страницы имеется бит обращения access. Но кроме бита обращения вводится бит модификации.

2 бита, в результате возможны 4 ситуации:

1) 00

2) 01

Обращения не было, модификация была. Дело в том, что биты обращения время от времени сбрасываются в 0. При необходимости выгрузки страницы, страница для выгрузки ищется среди страниц, у которых бит обращения 0. Ввод бита модификации позволяет вытеснение выполнять более точно, и данное сочетание двух битов не абсурдно. В данном случае можно сказать, что модификация была до последнего сброса бита обращения в 0.

Возникает вопрос: какую страницу выгоднее вытеснять во вторичную память, модифицированную или не модифицированную? Не модифицированную, потому что её точная копия находится на диске и это не требует копирования. Копирование — сложный и затратный процесс.

3) 10

4) 11

Алгоритмы замещения

Если какую-то страницу нужно загрузить, значит одну надо вытолкнуть.

1. Выталкивание случайной страницы. Для замены выбирается любая случайная страница.

- Малые накладные расходы
- Может быть вытолкнута часто используемая или только что загруженная

2. Алгоритм FIFO: вытеснение страницы, которая дольше всего находится в памяти. Способы: временная метка (вытеснение с меньшей временной меткой) либо ведется связный список.

Этот алгоритм исключает возможность выгрузки только что загруженной страницы, но не исключает выгрузку часто используемой.

3. Алгоритм LRU (Least Recently Used): замещаем наименее используемую страницу.

Строится на эвристическом предположении, правиле, если некоторые страницы имеют большое число обращений, то следующее обращение будет к ним.

Необходимо или изменять временную метку при каждом обращении, или при каждом обращении помещать этот сегмент в начало связного списка.

Наиболее затратно. (большие накладные расходы)

4. Алгоритм NUR (Not Used Recently):

Колоссальные накладные расходы LRU привели к использованию аппроксимирующих его алгоритмов.

Приписывание каждой странице бита обращения.

- При добавлении бит обращения 0.
- Периодически все биты обращений сбрасываются в 0.
- При обращении к сегменту устанавливается в 1.

Когда нужно заместить, ищется страница с нулевым битом обращения.

Кроме бита обращения вводится бит модификации.

Выгоднее вытеснять не модифицированную страницу, потому что ее копия лежит на диске и не надо выполнять точное копирование.

5. Алгоритм LFU (Least Frequency Used): Наименее часто используемая страница.

При переполнении счетчик сбрасывается. Может быть вытеснена только что загруженная страница, не набравшая число обращений.

Рабочее множество

Рабочее множество

Рабочее множество – набор страниц, используемых процессом в настоящее время.

Вытеснение

Глобальное вытеснение – вытеснение любой страницы любого процесса. Локальное вытеснение – страница для вытеснения выбирается из пула загруженных страниц данного процесса.

10.19 Теория рабочего множества

В 1968 году Деннинг предложил в качестве локальной меры производительности взять число страниц, к которым обращается процесс за интервал времени Δt . Рабочее множество (working set) — функция $W(t, \Delta t)$. Величина показывает количество страниц, к которым обращается процесс. При этом выяснилось, что для эффективного выполнения процесса (процесс эффективно выполняется, когда нет страничных прерываний, то есть нет блокировок в ожидании загрузки нужной страницы), если в памяти находятся все страницы, к которым процесс обращается за некоторый интервал Δt , то есть всё рабочее множество.

Исследование IBM было проведено, потому что несмотря на затраты, никто не отказался от реализации идеи виртуальной памяти, потому что это существенно повышает уровень мультизадачности. При этом возникает вопрос: сколько физической памяти потенциально нужно предоставлять каждому процессу, то есть сколько физических страниц понадобится процессу для его эффективного выполнения, чтобы он смог загрузить в память своё рабочее множество. Эта теория отвечает на этот вопрос.

В результате исследований было подтверждено наличие рабочего множества. При этом в начальный момент времени, когда процесс порождён,

Процессу, чтобы он мог начать выполняться, выделяется минимальное необходимое количество страниц. Скажем, ему выделяется по одной странице кода, стека и данных. После этого процесс переходит в состояние готовности. Получив процессорное время, программа начинает выполняться, и процесс начинает интенсивно подгружать нужные ему страницы, то есть будут возникать страничные прерывания, система будет их обрабатывать и загружать нужные страницы в память.

В результате в какой-то момент времени процесс загрузит в память своё первое рабочее множество. Этот интервал Δt , в течение которого процесс будет обращаться к одним и тем же страницам, ни предсказать, ни измерить невозможно, { процессы выполняются по разным траекториям, зависят от динамических параметров.} Длительность тоже можно определить только в процессе выполнения. Затем процесс перейдёт к своему второму множеству. И так далее.

Существуют такие интервалы во времени жизни процесса, когда процесс выполняется без страничных прерываний. Для этого процесс должен иметь возможность загрузить в память всё своё рабочее множество, то есть количество физических страниц которые выделены процессу, например, как квота, должно позволять загружать ему рабочее множество. Если процесс не может загрузить в память своё рабочее множество, возникнет интенсивная подкачка одних и тех же страниц в результате страничных прерываний. Это получило название **trashing**.

В Windows рабочее множество — это квота. Каждому процессу изначально устанавливается квота — 10 физических страниц (то есть ему выделено 10 кадров, он может

загрузить 10 страниц.). Любая система контролирует число страничных прерываний. Если число страничных прерываний у процесса резко возрастает, это значит, что процесс не может загрузить в память всё своё рабочее множество. В этом случае квота может быть увеличена. Для этого нужно иметь возможность, например, отложить процесс на какое-то время, пока система не сможет добавить к его квоте ещё некоторое количество страниц, чтобы процесс смог загрузить в память своё рабочее множество.

10.19.1 hit rate

??

TLB Hit Rate?

hit rate paging — доля обращений, не потребовавших смена страницы.

Первый график отражает влияние изменения размера страниц при сохранении объёма оперативной памяти на число прерываний.

график, похожий на «купол» нормального распределения

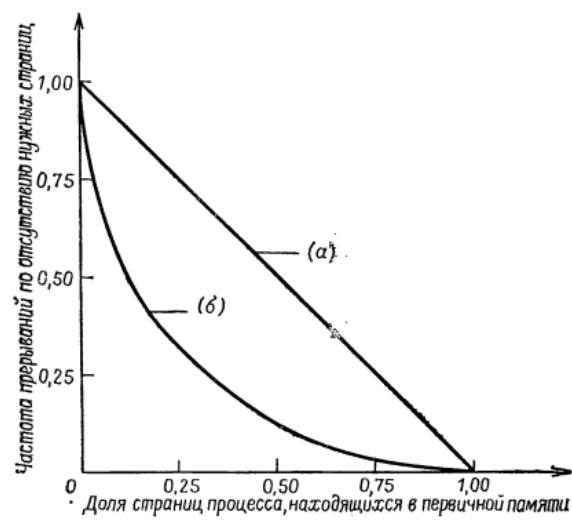
Рис. 43. первый график

Мы видим, что размер страницы увеличивается (в итоге может стать размером с память). Также видим, что число страничных прерываний растёт, сначала не очень интенсивно, затем резко, а потом падает. Рост числа страничных прерываний связан с тем, что чем больше страница, тем меньшее количество команд будет выполнено на этой странице. Когда страница достигнет размера, сопоставимого с размером команды, число страничных прерываний сократится.

Второй график — кривая времени жизни, график зависимости длительности периода между прерываниями. Размер программы — сегмент кода, сегмент данных, сегмент стека.

Теория рабочего множества

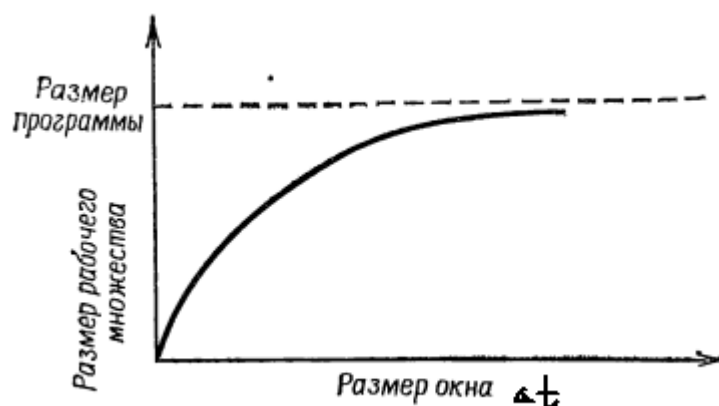
- Рассмотрим зависимость числа страничных прерываний от объема физической памяти



- Является мерой обобщенного поведения программ
- Неформально: рабочее множество – подмножество страниц, к которым процесс активно обращается. Для эффективного выполнения программы необходимо, чтобы ее рабочее множество находилось в первичной памяти
- В 1968 году Деннинг предложил в качестве локальной меры производительности взять число страниц, к которым обращается процесс за время Δt
- То есть, рабочее множество страниц процесса $W(t, \Delta t)$ в момент времени $t + \Delta t$ — набор страниц, к которым этот процесс обращается в течение интервала времени процесса Δt .
Время процесса – время, в течение которого процесс имеет в своем распоряжении центральный процессор.



- Чем больше Δt , тем к большему числу страниц обращается процесс
- При увеличении Δt размер рабочего множества стремится к пределу



- Программа должна иметь возможность загрузить в память рабочее множество страниц
 - То множество страниц, к которому в текущий интервал времени обращается процесс
- Если программа не имеет такой возможности, то пробуксовка (trashing)
 - Пробуксовка — загрузка и выгрузка одних и тех же страниц
- (Совершенная ОС стремится использовать все процессорное время на себя саму, а на приложения времени не остается – нужен компромисс)
- Page — виртуальная страница
- Frame — физическая страница

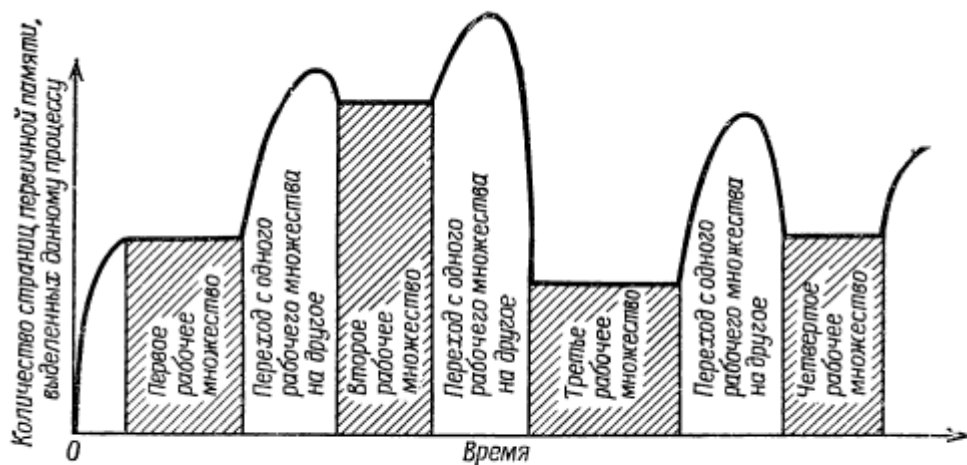


Рис. 9.6 Выделение основной памяти при управлении памятью на основе рабочих множеств.

1. Вначале, поскольку процесс запрашивает страницы своего рабочего множества не все сразу, а по одной, он постепенно получается достаточный объем памяти для размещения текущего рабочего множества.
2. Затем на какой-то период времени этот объем стабилизируется, поскольку процесс интенсивно обращается к страницам своего первого рабочего множества.
3. Со временем процесс перейдет на следующее рабочее множество, что показывает кривая линия от 1 ко 2 раб. множеству. Вначале эта кривая поднимается выше уровня первого рабочего набора, поскольку для процесса по его запросам быстро производится подкачка страниц нового рабочего множества.

При этом система не может сразу определить, то ли процесс просто расширяет свое рабочее множество, то ли процесс переходит на новое.

4. После того, как процесс начнет стабильно обращаться к страницам своего следующего рабочего множества, система видит, что в течение выбранного окна происходит обращение к меньшему количеству страниц, чем ранее, и сокращает объем памяти, выделяемой процессу, до числа страниц этого второго рабочего множества.
5. Каждый раз, когда осуществляется переход с одного рабочего множества на другое, кривая линия поднимается, а затем спадает, показывая, каким образом система адаптируется к подобному переходу.

Проблема: рабочие множества меняются во времени, причем следующее рабочее множество процесса может существенно отличаться от предыдущего. Стратегия управления памятью должна обязательно учитывать этот факт, чтобы избежать перегрузок первичной памяти и возникающего вследствие этого трешинга.

График зависимости длительности периода между прерываниями (кривая линии жизни)

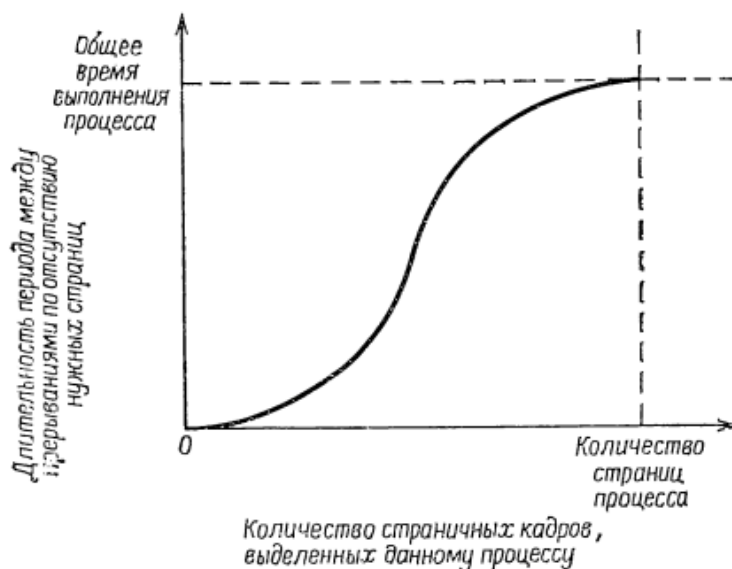
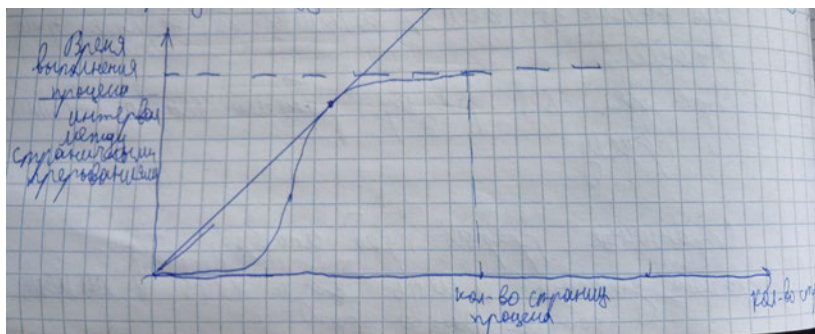


Рис. 9.12

- На графике показано, каким образом меняется среднее время между прерываниями по отсутствию нужной страницы в памяти при увеличении числа фреймов, выделяемых процессу.
- Кривая монотонно растет – чем больше фреймов имеет процесс, тем больше интервал между прерываниями по отсутствию нужных страниц.
- Перегиб происходит, так как в памяти оказывается все рабочее множество процесса
- Поначалу величина интервала между прерываниями растет очень быстро, поскольку растет часть рабочего множества, которая оказывается в первичной памяти.
- После того, как выделенный объем первичной памяти оказывается достаточным для размещения всего рабочего множества, кривая делает перегиб, указывая на то, что выделение дополнительных

фреймов не приведет к существенному увеличению интервала между прерываниями.

Влияние размера страниц при одном и том же размере физической памяти

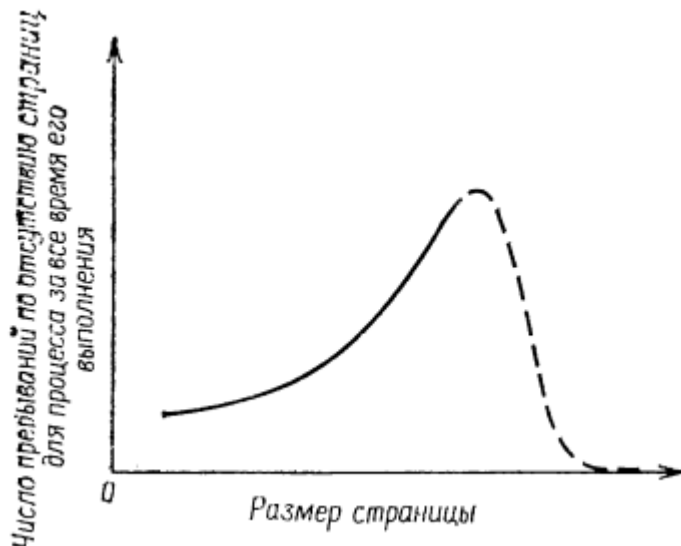
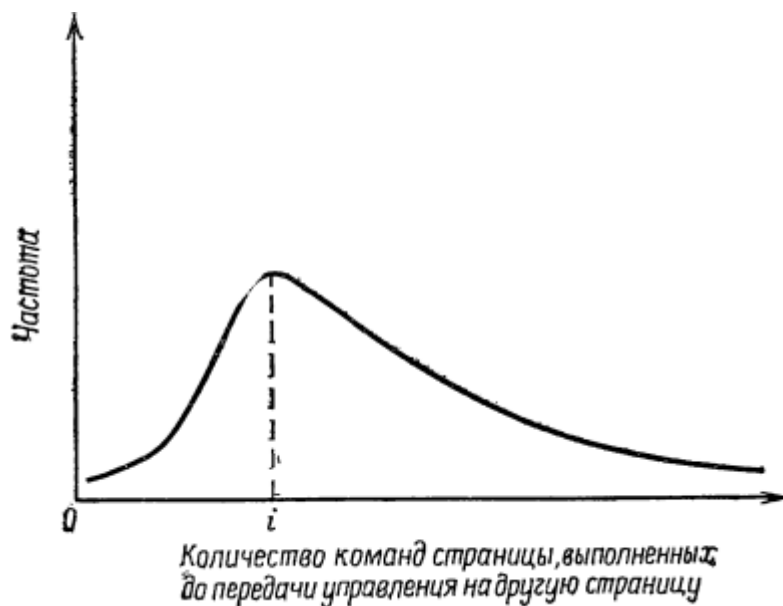


График показывает влияние размера страниц на число страничных прерываний

- С ростом размера страниц сначала число прерываний растет, так как в память попадет большее число процедур и данных, к которым не будет обращений. Так как никогда не выполняются все команды на странице, всегда выполняется только какой-то процент.
- Чем больше страница, тем меньше будет выполнено команд
- Падает, так как размер страницы стремится к размеру программы

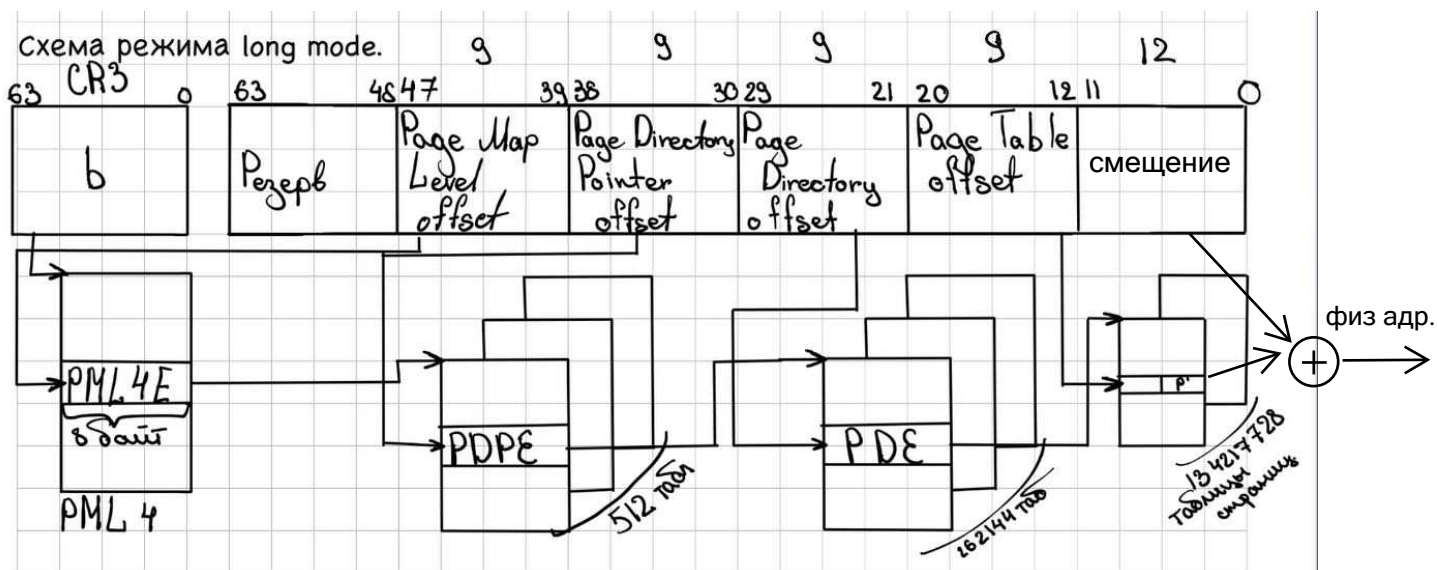
Зависимость частоты обращения к странице от количества команд на странице, выполненных до передачи управления на другую страницу



- На графике показано, какой процент команд данной страницы выполняется до того, как управление будет передано другой странице.
- Максимум указанной кривой пришелся на точку, соответствующую 200 командам страницы размеров 4096Б.
- По-видимому нужно уменьшать размер страниц – Дейтелл
- Рабочее множество от Windows называется квота
- Так как предсказать размер рабочего множества невозможно, то каждому процессу перед началом выполнения выдается квота — число страниц, которые процесс может запросить в ОЗУ.
 - Вопрос: сколько страниц выдавать?
- Производится контроль за числом страничных прерываний
 - Если идет рост страничных прерываний, то система работает неэффективно, так как страничные прерывания — затратные действия => квота для данного процесса мала
 - При малейшей возможности квота увеличивается, чтобы процесс мог их загрузить в память, то есть загрузить все рабочее множество.

- В режим long реализовано только управление памятью страницами по запросу.
- В режиме long — схема PAE (physical address extension)

PAE



- PTE — таблицы страниц
- PDE — каталоги таблиц страниц —
- PDPE — указатель на каталоги таблиц страниц — максимум 512 табл.
- PML4E — указатель на указатель — такая таблица в системе 1
- Дескрипторы инициализируются по мере надобности
- Размер дескриптора — 8 байт
- Таблицы по 2^9 строк $\Rightarrow$ каждая таблица в памяти занимает одну страницу
- Когда процесс создается — ему выделяется минимально необходимое число страниц.
- То есть выполняется отложено.

Например, при создании процесса создается 3 страницы: точка входа, страница сегмента данных, страница сегмента стека. Все остальные страницы создаются по мере надобности при возникновении системных прерываний.

2. Лабораторная работа по UNIX: системные вызовы `fork()`, `wait()`, `exec()`, `pipe()`, `signal()` и системный вызов POSIX `sigaction()`. Примеры пр-мм из лабораторной работы: `exec()`, `signal()`.

Что такое `fork()`?

`fork()` --- системный вызов, который создаёт новый процесс --- процесс-потомок, для которого создаётся собственное адресное пространство, т. е. создаются собственные карты трансляции АДРЕСОВ, дескрипторы СТРАНИЦ ТАБЛИЦ СТРАНИЦ потомка ссылаются на СТРАНИЦЫ адресного пространства предка И для страниц адресного пространства предка права доступа МЕНЯЮТСЯ на `only-read` и УСТАНАВЛИВАЕТСЯ флаг `copy-on-write` И если или предок, или потомок попытаются изменить какую-либо страницу, возникнет исключение по правам доступа И операционная система, обрабатывая это исключение, ОБНАРУЖИТ флаг `copy-on-write` и создаст копию страницы в адресном пространстве того процесса, который пытался её изменить И такая ситуация существует до тех пор, пока процесс-потомок не вызовет `exec()` или `exit()` (тут именно `exit()`, а не завершение процесса) И в результате страницам адресного пространства предка возвращаются обычные права доступа и флаг `copy-on-write` сбрасывается.

Что такое `exec()`?

`exec()` --- системный вызов, который переводит процесс на адресное пространство программы, которая была передана ему в качестве параметров

`exec()`:

1. разбирает путь к файлу, имеется в виду полное имя файла. система разбирает полное имя файла для того чтобы проверить существует ли этот файл
2. поскольку выполняться может только исполняемый файл, на выполнение этого файла у процесса должны быть соотв полномочия.
3. читает заголовок файла чтобы проверить что файл действительно исполняемый
4. требуется освобождение занятых ресурсов, поскольку это существующий процесс для которого создано адресное пространство и выделена область своппинга. эти ресурсы надо освободить
5. после этого для этого процесса потомка создаётся адресное пространство, можно сказать новое для программы которая передаётся `exec` в качестве параметра
6. после того как адресное пространство создано, необходимо поменять адресацию, т.е. в дескрипторе процесса указан адрес старых таблиц страниц. этот адрес надо заменить на адрес новых таблиц страниц
7. надо поменять аппаратный контекст, все регистры сбрасываются, а в счётчик команд пишется адрес точки входа

в процессорах Intel есть регистр `cr3` - регистр начального адреса каталога таблиц страниц (для 32х разрядных машин), адрес таблицы указателей (для PAE), адрес таблиц 4 уровня (для long). в этот регистр необходимо загрузить адрес таблиц страниц для того чтобы система могла выполнять соответствующие страничные преобразования.

Что такое `pipe()`?

`pipe()` --- системный вызов, который создаёт неименованный программный канал. **Где создаётся канал?** Канал создаётся в области данных ядра. **Почему он создаётся там?** Так как адресное пространство процесса ЗАЩИЩЁННОЕ, ни один процесс не может обратиться в адресное пространство другого процесса, единственный способ обмена данными между процессами --- через третье адресное пространство.

`pipe()`

Каналы реализуют потоковую передачу данных – симплексная связь.

Системный вызов `pipe()` создает неименованный программный канал. Неименованные программные каналы могут использоваться для обмена сообщениями между процессами родственниками. В отличие от именованных программных каналов неименованные не имеют идентификатора, но имеют дескриптор. Процесс-потомок наследует все дескрипторы открытых файлов процесса-предка, в том числе и неименованных программных каналов.

Что такое wait()?

Системный вызов wait() приостанавливает выполнение вызвавшего процесса до тех пор, пока не прекратит выполнение один из его потомков.

wait()

Данный системный вызов блокирует родительский процесс до момента завершения дочернего процесса. При этом процесс-предок получает статус завершения процесса-потомка. `pid_t wait(int *stat_loc)`; Вызов возвращает PID дочернего процесса. Обычно это дочерний процесс, который завершился. Сведения о состоянии позволяют родительскому процессу определить статус завершения дочернего процесса, т.е. значение, возвращенное из функции `main` потомка или переданное функции `exit()`. Если `stat_loc` не равен пустому указателю, информация о состоянии будет записана в то место, на которое указывает этот параметр. Интерпретировать информацию о состоянии процесса можно с помощью макросов:

Макрос	Описание
<code>WIFEXITED(stat_val)</code>	Ненулевой, если дочерний процесс завершен нормально
<code>WEXITSTATUS(stat_val)</code>	Если <code>WIFEXITED</code> ненулевой, возвращает код завершения дочернего процесса
<code>WIFSIGNALED(stat_val)</code>	Ненулевой, если дочерний процесс завершается не перехватываемым сигналом
<code>WTERMSIG(stat_val)</code>	Если <code>WIFSIGNALED</code> ненулевой, возвращает номер сигнала
<code>WIFSTOPPED(stat_val)</code>	Ненулевой, если дочерний процесс остановился
<code>WSTOPSIG(stat_val)</code>	Если <code>WIFSTOPPED</code> ненулевой, возвращает номер сигнала

```
int main(void)
{
    pid_t child_pid[2];
    for (size_t i = 0; i < 2; i++)
    {
        if ((child_pid[i] = fork()) == -1)
        {
            perror("Can't fork\n");
            exit(1);
        }
        else if (child_pid[i] == 0)
        {
            printf("Child: pid = %d, ppid = %d, gid = %d\n",
                   getpid(), getppid(), getpgrp());
            return 0;
        }
        printf("Parent. Children id: %d\n", child_pid[i]);
    }

    for (size_t i = 0; i < 2; i++)
    {
        int status;
        waitpid(child_pid[i], &status, 0);
        if (WIFEXITED(status))
        {
            printf("Child (pid: %d) exited with code %d\n",
                   child_pid[i], WEXITSTATUS(status));
        }
        else if (WIFSIGNALED(status))
        {
            printf("Child (pid: %d) received signal %d\n",
                   child_pid[i], WTERMSIG(status));
        }
        else if (WIFSTOPPED(status))
        {
            printf("Child (pid: %d) received signal %d\n",
                   child_pid[i], WSTOPSIG(status));
        }
    }
}
```

Что такое signal()?

Сигналы информируют процессы о событиях, происходящих вне этих процессов (об асинхронных событиях) и о синхронных событиях, т.е. происходящих внутри этого процесса.

Обрабатывать полученный сигнал можно 3мя способами:

1. процесс проигнорирует полученный сигнал
2. процесс реагирует способом, установленным по умолчанию (как в системе)
3. процесс сам реагирует, своим способом. Делается обработчиком сигнала, который процесс регистрирует в системе.

С помощью системного вызова `signal()` можно установить свой обработчик сигнала. Вешается хендлер на определенный сигнал.

Что такое sigaction()?

Системный вызов `sigaction()` позволяет установить обработчик сигнала, при этом он входит в стандарт POSIX.

```
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

Лабораторная работа.

`exec()`

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/wait.h>

#define CHILDS_CNT 2

int main(void)
{
    char *apps[CHILDS_CNT] = {"/progs/app1", "/progs/app_sleep"};
    pid_t childPIDS[CHILDS_CNT];
    int i;

    for (i = 0; i < CHILDS_CNT; i++)
    {
        childPIDS[i] = fork();
        if (childPIDS[i] == -1)
        {
            perror("Can't fork");
            exit(1);
        }
        if (childPIDS[i] == 0)
        {
            if (execlp(apps[i], "args", NULL) == -1)
            {
                perror("Can't exec\n");
                exit(1);
            }
        }
    }

    int status;
    for (i = 0; i < CHILDS_CNT; i++)
    {
        waitpid(childPIDS[i], &status, 0);
        if (WIFEXITED(status))
            printf("\t%d-Child exited with status = %d\n", childPIDS[i], WEXITSTATUS(status));
        else if (WIFSIGNALED(status))
            printf("\t%d-Child with (signal %d) killed\n", childPIDS[i], WTERMSIG(status));
        else if (WIFSTOPPED(status))
```

```

        printf("\t%d-Child with (signal %d) stopped\n", childPIDS[i], WSTOPSIG(status));
    else
        printf("\t%d-Unexpected status for Child (0x%x)\n", childPIDS[i], status);
    }

    printf("Parent: PID = %d, group = %d, children = %d, %d\n", getpid(), getpgrp(), childPIDS[0],
childPIDS[1]);

    exit(0);
}

```

signal()

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int flag = 0;
void sig_handler(int sig_num) {
    flag = 1;
}

int main(void)
{
    pid_t child_pid[2];
    char messages[2][32] = {"aaaaaaaa", \
                           "bbbbbbbbbbbbbbbbbbbbbbbbb"};

    int fd[2];

    if (signal(SIGINT, sig_handler) == SIG_ERR) {
        perror("Can't set signal handler\n");
        exit(1);
    }

    if (pipe(fd) == -1)
    {
        perror("Can't pipe\n");
        exit(1);
    }

    for (size_t i = 0; i < 2; i++)
    {
        if ((child_pid[i] = fork()) == -1)
        {
            perror("Can't fork\n");
            exit(1);
        }
        else if (child_pid[i] == 0)

```

```

{

    if (!flag) {
        sleep(5);
        if (!flag) {
            perror("Can't write\n");
            exit(1);
        }
    }
    if (close(fd[0]) == -1) {
        perror("Can't close\n");
        exit(1);
    }
    if (write(fd[1], messages[i], strlen(messages[i])) != -1) {
        perror("Can't write\n");
        exit(1);
    }
    return 0;
}

printf("Parent. Children id: %d\n", child_pid[i]);
}

if (close(fd[1]) == -1) {
    perror("Can't close");
    exit(1);
}

for (size_t i = 0; i < 2; i++)
{
    int status;
    waitpid(child_pid[i], &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child (pid: %d) exited with code %d\n", \
            child_pid[i], WEXITSTATUS(status));
    }
    else if (WIFSIGNALED(status))
    {
        printf("Child (pid: %d) received signal %d\n", \
            child_pid[i], WTERMSIG(status));
    }
    else if (WIFSTOPPED(status))
    {
        printf("Child (pid: %d) received signal %d\n", \
            child_pid[i], WSTOPSIG(status));
    }
}

printf("Messages from children: \n");

for (size_t i = 0; i < 2; i++)
{

```

```
    char buf[64] = "";  
    int k = read(fd[0], buf, strlen(messages[i]));  
    printf("%d %s\n", k, buf);  
}  
  
return 0;  
}
```

Экзаменационный билет 17

1. Виртуальная память: распределение памяти страницами по запросам, алгоритмы вытеснения страниц, свойство локальности программ, рабочее множества, анализ страничного поведения процессов. Схема страничного преобразования в процессорах Intel (x86-64) PAE; дескриптор страницы, формат дескриптора, основные флаги, размеры таблиц дескрипторов, обоснование использования многоуровневого преобразования, кэш TLB - структура и алгоритм, аппроксимирующий LRU (для процессора i486).

Ответ:

Виртуальная память

10.15.2 Виртуальная память

Программа не должна целиком находиться в физической памяти. В физ.памяти должна находиться та часть программы, с которой производится работа в данный момент времени. То есть в памяти должны находиться те команды программы, к которым в текущий момент времени обращается процессор и данные, соответственно.

Это отправная точка идеи виртуальной памяти. Виртуальный — кажущийся, возможный.

Существует 3 схемы управления виртуальной памятью:

- 1) Страницами по запросу
- 2) Сегментами по запросу
- 3) Сегментами, поделёнными на страницы по запросу

В стремлении избавиться от фрагментации, от необходимости выделения непрерывных адресных пространств в физической памяти, пришли к идеи виртуальной памяти.

Но это очень затратная схема, очень много дополнительных действий необходимо выполнять для адресации команды данных. Какой от этого выигрыш, если это так затратно? Увеличение мультизадачности (мультипрограммности (multitasking)), это значит, что больше одной программы может одновременно находиться в физической памяти. Они могут выполняться, могут ждать своей очереди, но это и есть мультизадачность.

162

Это огромные затраты, но и колоссальный выигрыш. Но от идеи использования виртуальной памяти никто не отказался.

Распределение памяти страницами по запросам

10.16 Управление памятью страницами по запросу

Во всех трёх частях содержится общая часть: «по запросу», то есть тогда, когда процессор в них возникает необходимость, когда процессор обращается к данным, отсутствующим в физической памяти. Реализуется это с помощью прерываний, в частности, если мы говорим об управлении памятью страницами по запросу, то это будет страничное прерывание. Причём страничное прерывание в системах отнесено к исправимым исключениям.

В начальный момент времени, когда процесс порождён, то есть он получил идентификатор, часть его полей дескриптора проинициализирована системой, процессу выделяется физическая память. В современных системах это какое-то минимально необходимое для начала выполнения количество страниц физической памяти. Условно говоря, 3 страницы: **сегмента кода, сегмента данных и сегмента стека.**

После этого по диаграмме состояний, процесс переходит в очередь готовых процессов для получения процессорного времени. Получив процессорное время, процесс начинается выполняться команда за командой. В какой-то момент времени процессор обращается к данным, отсутствующим в физической памяти (под данными в общем случае

понимаются и команды и собственно данные). Возникает страничное прерывание, в результате которого нужная страница будет загружена в физическую память, после чего процесс начнёт своё выполнение. То есть к затратам еще добавляется обслуживание страничных прерываний, затраты ещё больше.

10.17 Схемы преобразования адресов в страничном распределении

10.17.1 Прямое отображение

Процессы имеют виртуальные адресные пространства. Виртуальный адрес делится на две части: **номер страницы и смещение**. В составе регистра процесса обязательно должен быть регистр базового адреса таблицы страниц.

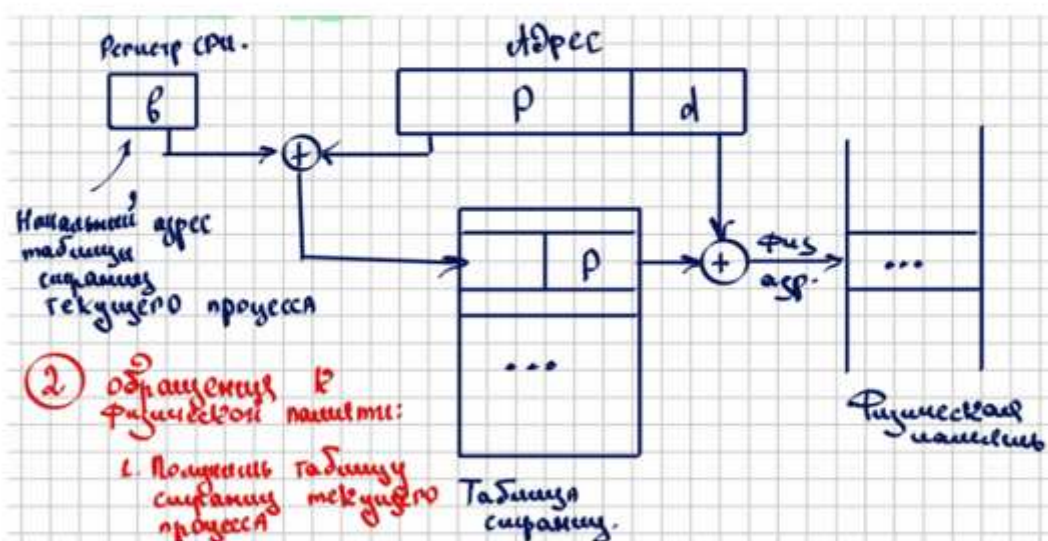
Каждый процесс имеет собственную таблицу страниц, в которой находятся дескрипторы страниц.

Фактически номер страницы виртуального адреса служит смещением к дескриптору в таблице страниц конкретного процесса (?). Если страница загружена в физическую память, то из этой таблицы можно получить её адрес. Смещение одно и то же. Причём смещение берётся из программы, выполняется преобразование и получается физический адрес.

При этом такая таблица страниц находится в физической памяти. **То есть для того, чтобы выполнить преобразование вот в соответствии с этой схемой, сначала надо обратиться к таблице страниц, получить физический**

адрес страницы и затем выполнить преобразования для получения физического адреса данных. Такое преобразование будет выполняться на каждой команде, а то и несколько раз — крайне затратные действия.

Обращение к физической памяти требует большого количества тактовых импульсов, так называемых циклов обращения к памяти. Это затратное действие. Стремясь уменьшить эти накладные расходы, разработчики систем, основываясь на своих технических знаниях, предложили использовать ассоциативную память.

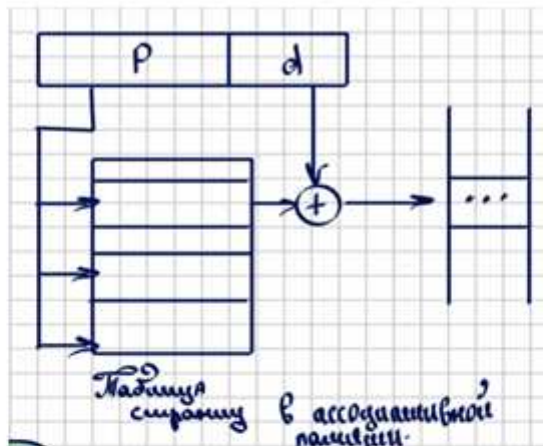


10.17.2 Ассоциативное отображение

Таблица страниц загружается в ассоциативную память.

Особенностью ассоциативной памяти является возможность получения данных за 1 такт за счёт схемы реализации ассоциативной памяти.

Важно понимать, почему эта схема не нашла своё физическое воплощение, а лишь частичное. Ассоциативная память очень дорогая, это всегда регистровая память.



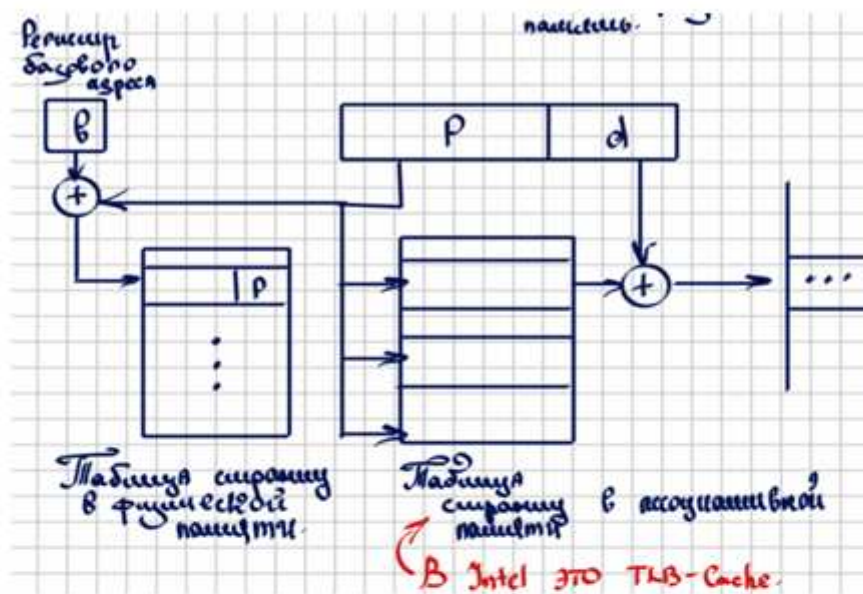
Номер виртуальной страницы является ключом, ключ - 4 разряда. При совпадении ключа формируется импульс, который позволяет считать информацию. Каждый разряд каждого регистра дополнительно имеет схему совпадения (тут схема архитектурская).

Мало того, что это регистровая память (количество схем увеличивается в 2 раза), также увеличивается количество соединений. Это говорит о дороговизне ассоциативной памяти.

А с ростом размеров программ таблицы страниц увеличиваются, они становятся очень большими. Значит, необходимо иметь ассоциативную память очень большого размера, специальную ассоциативную память. То есть теоретически это возможно, но практически это сталкивается с огромными затратами, и с огромной стоимостью такой памяти.

10.17.3 Ассоциативно-прямое отображение

ВНИМАНИЕ на картинке таблицы ассоциативной памяти написано, а она говорила просто ассоциативная память



В физической памяти располагается таблица страниц. В процессоре есть регистр базового адреса таблицы страниц и ассоциативная память небольшого размера, в которое может находиться только небольшое количество дескрипторов страниц.

При этом в такой ассоциативной памяти хранятся только дескрипторы страниц, к которым были последние обращения. Связано это со свойством локальности наших программ: если обращение было к странице, то наиболее вероятно, что следующие обращения будут к этой же странице. Это следует из того, что команды, данные хранятся в памяти в последовательных адресах, при этом наиболее вероятной структурой является следование (выполнение программы именно команды за командой). Ветвление

выполняется в программах реже, сюда же относим и ветвление в цикле. Кроме того, данные также хранятся в последовательных адресах. Но очевидно, что на странице никогда не выполняются все команды, как правило это от $\frac{1}{3}$ до $\frac{2}{3}$ команд. Даже кеш небольшого размера (от 16 строк) обеспечит скоростные показатели, составляющие 90% скоростных показателей при реализации полностью ассоциативной памяти.

КАК ПРОИСХОДИТ РАБОТА: Сначала страница ищется в ассоциативной памяти. Если её там нет, то обращение происходит к таблице страниц физической памяти. При этом будет выполнено замещение дескриптора (если дескриптора не оказалось в ассоциативной памяти в результате обращения, то будет выполнено замещение). В современных системах - Кеш TLB (Translation Lookaside Buffer), фирмы IBM, ассоциативный по множеству четырёхнаправленный кеш.

Размеры программ растут, размеры таблиц страниц увеличиваются. Решением задачи увеличения размеров стала схема с гиперссылками.

Алгоритмы вытеснения страниц

10.18 Замещение страниц

Процесс никогда не обращается одновременно ко всем своим страницам. Физическая память имеет ограниченный размер — это дефицитный ресурс в системе.

Процесс выполняется, процессор в какое-то время обращается к странице, которая отсутствует в физической памяти, а свободный фрейм отсутствует, то есть все страницы физической памяти заняты. Изначально различали страницу виртуального адресного пространства (тут сказала, что как-то назвали, но я не расслышала) и страницу физического адресного пространства, называемую фреймом. Что делать, если не хватает свободных физических страниц, все фреймы распределены? Надо выполнить замещение.

10.18.1 Замещение случайной страницы

Не является дискриминационным, то есть с равной вероятностью может быть вытеснена любая страница любого процесса, но также существует вероятность, что будет вытеснена только что загруженная страница или часто используемая. Реализация такого алгоритма малозатратна. Но из-за недостатков применяется редко.

10.18.2 Алгоритм FIFO

Для реализации данного алгоритма каждому фрейму присваивается или временная метка, или организуется связанный список. Вытеснена будет страница, которая

дольше всего находилась в физической памяти. Но также такой страницей не окажется только что загруженная страница, но может оказаться часто используемая страница. Алгоритм FIFO по своим характеристикам может приблизиться к первому алгоритму замещения страниц в крайних случаях.

Но также было обнаружено, что алгоритм FIFO обладает свойством аномалии (аномалия алгоритма FIFO). Она заключается в том, что при увеличении объёма физической памяти, расчёт на то, что количество страничных прерыва-

ний сократится, не оправдывается. Количество страничных прерываний для некоторых траекторий страниц может увеличиться.

Размер памяти - 3 страницы. Траектория страниц — номера страниц, к которым выполняются обращения.



Соответственно, сначала надо загрузить четвёртую страницу. Обозначаем действие плюсом. Затем третью, четвёртая перемещается в голову. Затем вторую по алгоритму Фифо, четвертая перемещается в голову. И вот надо загрузить первую страницу. У нас нет свободных фреймов. Значит, по алгоритму Фифо будет выгружена четвёртая страница. Теперь надо загрузить четвёртую. Она в памяти. Так называемая страничная удача, но очередь не переформируется

(остаётся в том же виде). Теперь нужно загрузить третью. Она также в памяти, также страничная удача. Очередь не переформируется. Теперь надо вторую загрузить, выгрузится четвёртая. Теперь надо загрузить первую, выгрузится третья. И теперь надо загрузить пятую. Она в памяти. Очередь не переформируется.

Увеличим память на одну страницу.

10.18.3 Алгоритм LRU

LRU — least recently used, page replacement (наименее используемое последнее время)

Алгоритм относится к классу стековых алгоритмов. Алгоритм наиболее полно отвечает свойству локальности наших программ, но в реальных системах не используется, так как требует колоссальных затрат на реализацию. (Обращение к странице - на каждой команде, а то и несколько раз, если косвенный вызов)

Свойство 10.12 (Свойство локальности)

Свойство является следствием архитектуры фон Неймана. Если обращение было к странице, то наиболее вероятно, что следующие обращения будут к этой же странице. Это связано с тем, что программа в своём адресном пространстве располагается в последовательных адресах (в виртуальном адресном пространстве), команда за командой, при этом наиболее вероятной структурой является следование.

По этому алгоритму будет вытеснена страница, к которой дольше всего не было обращений. Это требует изменения работы или с временными метками или со связным списком. При каждом обращении к страницы должна быть отредактирована временн*а*я метка. Если это связный список, то при каждом обращении страница перемещается в конец списка (меняется адрес). Это крайне затратные действия в системе, потому что обращения происходят на каждой команде, а то и несколько раз.

Сначала размер памяти возьмем 3 страницы, потом увеличим на единицу. Обращение к 4 (если находится в памяти – **страничная удача**), она есть, по алгоритму помещается в конец связного списка. Далее обращение пополняется третьей страницей. Она также находится в памяти, и она переносится в конец связанного списка.

LRU - least recently used

	1	2	3	4	5	6	7	8	9	10	11	12	
	4	3	2	1	4	3	5	4	3	2	1	5	
↑		4+	3+	2+	1+	4+	3+	5+	4	3	2+	1+	5+
M=3			4	3	2	1	4	3	5	4	3	2	1
			4	3	2	1	4	3	5	4	3	2	
↓													
F		+	+	+	+	+	+			+	+	+	

	1	2	3	4	5	6	7	8	9	10	11	12	
	4	3	2	1	4	3	5	4	3	2	1	5	
↑		4+	3+	2+	1+	4	3	5+	4	3	2+	1+	5+
M=4			4	3	2	1	4	3	5	4	3	2	1
			4	3	2	1	4	3	5	4	3	2	
↓													
F		+	+	+	+		+			+	+	+	

Рис. 37. LRU, пример

Как мы видим, количество страничных удач увеличилось (для 4 страниц), то есть не увеличилось число страничных прерываний. При этом обратим внимание, что первые три строки совпадают, поскольку алгоритм LRU относится к классу стековых алгоритмов(?).

10.18.4 Алгоритм LFU

LFU — least frequency used, page replacement (наименее часто используемая страница)

В этом алгоритме фактически контролируется частота обращения к странице. Это реализуется с помощью счётчика. Наименее используемой может оказаться только что

загруженная страница. В результате она будет вытеснена, и поскольку она будет вытеснена, потому что не успела набрать счётчик обращений, её необходимо будет загрузить заново. Это большие накладные расходы, поэтому используется аппроксимация (алгоритм NUR).

10.18.5 Алгоритм NUR

NUR — not used recently (страница, не используемая в последнее время)

Это аппроксимация LRU

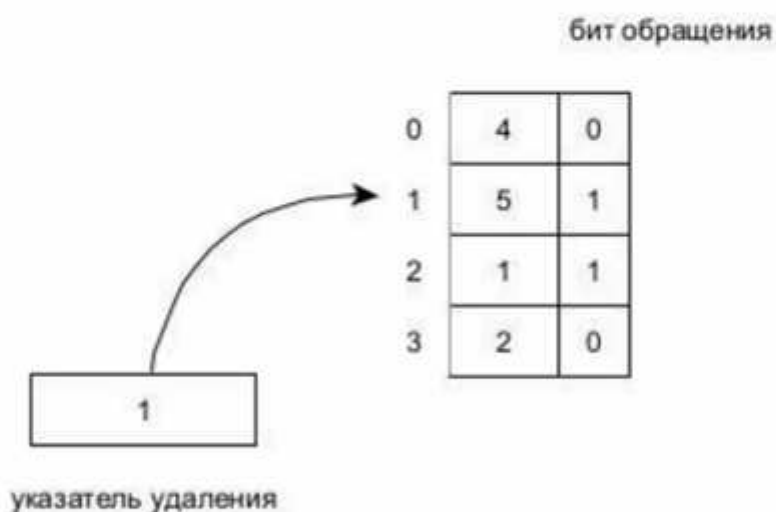


Рис. 38. рисунок

Имеется бит, который изменяется при каждом обращении к странице. При загрузке бит устанавливается в 1 и с этого момента будет рассматриваться удаление.

Именно этот способ реально используется в системах. В дескрипторе страницы имеется бит обращения access. Но кроме бита обращения вводится бит модификации.

2 бита, в результате возможны 4 ситуации:

1) 00

2) 01

Обращения не было, модификация была. Дело в том, что биты обращения время от времени сбрасываются в 0. При необходимости выгрузки страницы, страница для выгрузки ищется среди страниц, у которых бит обращения 0. Ввод бита модификации позволяет вытеснение выполнять более точно, и данное сочетание двух битов не абсурдно. В данном случае можно сказать, что модификация была до последнего сброса бита обращения в 0.

Возникает вопрос: какую страницу выгоднее вытеснять во вторичную память, модифицированную или не модифицированную? Не модифицированную, потому что её точная копия находится на диске и это не требует копирования. Копирование — сложный и затратный процесс.

3) 10

4) 11

Свойство локальности программ

Анализ страничного поведения процессов: свойство локальности, рабочее множество.

Свойство локальности:

Если обращение было к странице, то наиболее вероятно, что следующее обращение будет к этой же странице. (Если не слишком далеко заглядывать в будущее, то можно достаточно точно его прогнозировать исходя из прошлого.)

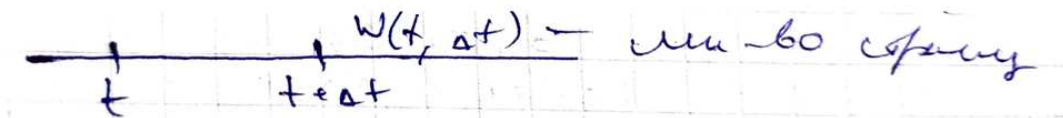
Страничное поведение программ – число страничных прерываний, происходящих в процессе.

Зависимость числа страничных прерывания от объема памяти является обобщенной мерой страничного поведения программы.

Программа одновременно не обращается ко всем своим страницам, позволяет использовать виртуальное пространство.

В разные моменты времени обращается к разным подмножествам множества страниц. В том числе в разные моменты времени программы обращаются к разному количеству страниц. Предсказать к каким страницам обратиться невозможно.

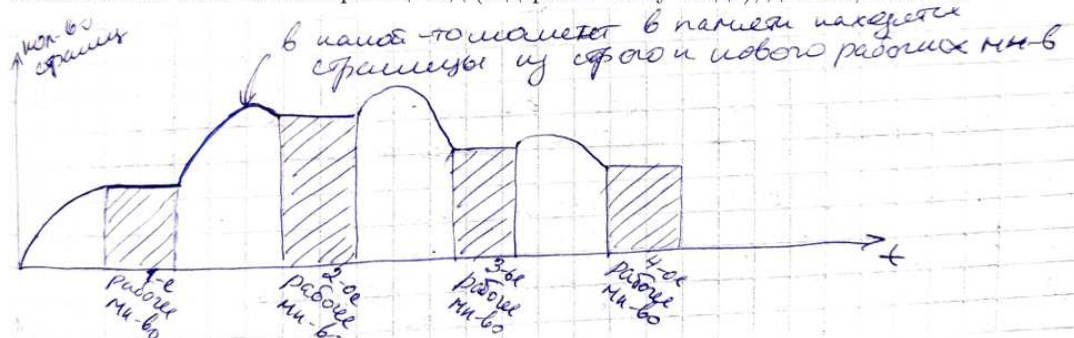
Деннинг в 1968 году предложил в качестве локальной меры производительности взять число страниц, к которым обращается программа за интервал времени Δt .



Размер рабочего множества является монотонной функцией от Δt , то есть при увеличении интервала Δt , число страниц будет стремиться к пределу L – которое определяет необходимое количество страниц для выполнения программы.

Нужно, чтобы рабочее множество находилось в оперативной памяти. Трешинг – подкачка одних и тех же страниц.

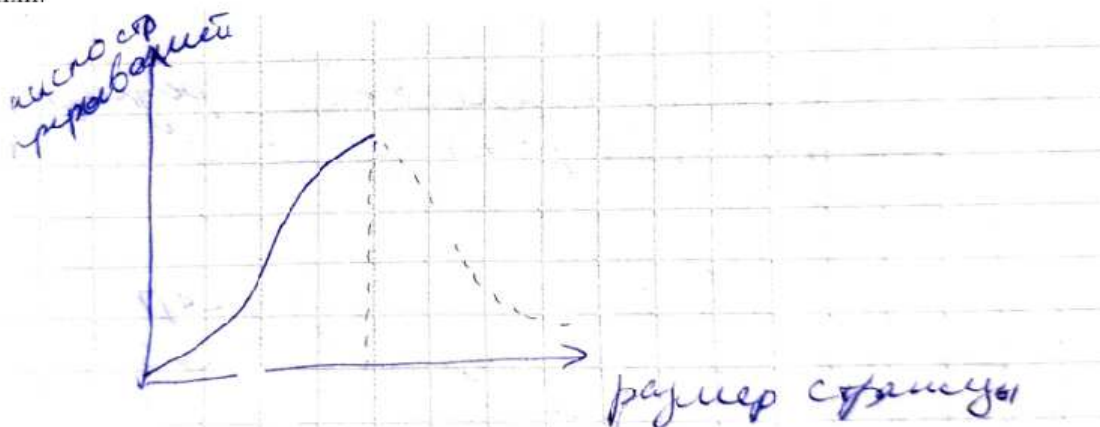
Минимальное количество страниц: код (содержит точку входа), данные, стек – 3.



Рабочее множество – набор страниц, используемых процессом в настоящее время.

Процесс удачных обращений – hit rate.

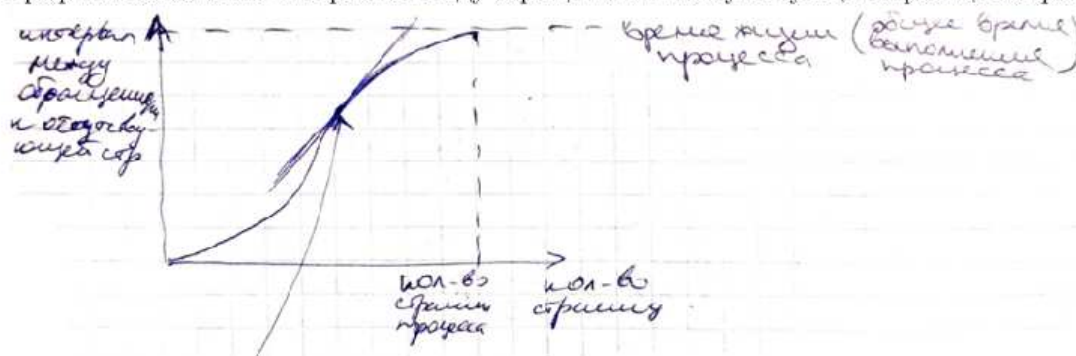
График. Влияние изменения размера страниц при сохранении объема оперативной памяти, на число прерываний.



С ростом размера страниц число страничных прерываний растет. Когда размер страницы становится соизмеримым с размером программы число страничных прерываний резко падает.

Чем меньше размер страницы тем больше страниц, но на маленьких страницах выполняется большое число команд.

График. Зависимость интервала между обращениями к отсутствующей странице за время жизни процесса.



Стрелка указывает на точку перегиба, в некоторый момент в оперативной памяти оказывается все рабочее множество процесса.

Система может контролировать количество страничных прерываний, если резко возросло, то означает что рабочее множество не может быть загружено в память.

Необходимо увеличить размер квоты на непродолжительный интервал времени.

Квота – каждому процессу может быть выделено определенное количество страниц, изначально.

Рабочее множество и анализ страничного поведения процессов

10.19 Теория рабочего множества

В 1968 году Деннинг предложил в качестве локальной меры производительности взять число страниц, к которым обращается процесс за интервал времени Δt . Рабочее

176

множество (working set) — функция $W(t, \Delta t)$. Величина показывает количество страниц, к которым обращается процесс. При этом выяснилось, что для эффективного выполнения процесса (процесс эффективно выполняется, когда нет страничных прерываний, то есть нет блокировок в ожидании загрузки нужной страницы), если в памяти находятся все страницы, к которым процесс обращается за некоторый интервал Δt , то есть всё рабочее множество.

Исследование IBM было проведено, потому что несмотря на затраты, никто не отказался от реализации идеи виртуальной памяти, потому что это существенно повышает уровень мультитзадачности. При этом возникает вопрос: сколько физической памяти потенциально нужно предоставлять каждому процессу, то есть сколько физических страниц понадобится процессу для его эффективного выполнения, чтобы он смог загрузить в память своё рабочее множество. Эта теория отвечает на этот вопрос.

В результате исследований было подтверждено наличие рабочего множества. При этом в начальный момент времени, когда процесс порождён,

Процессу, чтобы он мог начать выполняться, выделяется минимальное необходимое количество страниц. Скажем, ему выделяется по одной странице кода, стека и данных. После этого процесс переходит в состояние готовности.

Получив процессорное время, программа начинает выполняться, и процесс начинает интенсивно подгружать нужные ему страницы, то есть будут возникать страничные прерывания, система будет их обрабатывать и загружать нужные страницы в память.

В результате в какой-то момент времени процесс загрузит в память своё первое рабочее множество. Этот интервал Δt , в течение которого процесс будет обращаться к одним и тем же страницам, ни предсказать, ни измерить невозможно, процессы выполняются по разным траекториям, зависят от динамических параметров.} Длительность тоже можно определить только в процессе выполнения. Затем процесс перейдёт к своему второму множеству. И так далее.

Существуют такие интервалы во времени жизни процесса, когда процесс выполняется без страничных прерываний. Для этого процесс должен иметь возможность загрузить в память всё своё рабочее множество, то есть количество физических страниц которые выделены процессу, например, как квота, должно позволять загружать ему рабочее множество. Если процесс не может загрузить в память своё рабочее множество, возникнет интенсивная подкачка одних и тех же страниц в результате страничных прерываний. Это получило название **trashing**.

В Windows рабочее множество — это квота. Каждому процессу изначально устанавливается квота — 10 физических страниц (то есть ему выделено 10 кадров, он может загрузить 10 страниц.). Любая система контролирует число страничных прерываний. Если число страничных прерываний у процесса резко возрастает, это значит, что процесс не может загрузить в память всё своё рабочее множество. В этом случае квота может быть увеличена. Для этого нужно

178

иметь возможность, например, отложить процесс на какое-то время, пока система не сможет добавить к его квоте ещё некоторое количество страниц, чтобы процесс смог загрузить в память своё рабочее множество.

10.19.1 hit rate

?? TLB Hit Rate? hit rate paging — доля обращений, не потребовавших смена страницы.

Первый график отражает влияние изменения размера страниц при сохранении объёма оперативной памяти на число прерываний.

график, похожий на «купол» нормального распределения

Рис. 39. первый график

Мы видим, что размер страницы увеличивается (в итоге может стать размером с память). Также видим, что число страничных прерываний растёт, сначала не очень интенсивно, затем резко, а потом падает. Рост числа страничных прерываний связан с тем, что чем больше страница, тем меньшее количество команд будет выполнено на этой странице. Когда страница достигнет размера, сопоставимого с размером команды, число страничных прерываний сократится.

Второй график — кривая времени жизни, график зависимости длительности периода между прерываниями. Размер программы - сегмент кода, сегмент данных, сегмент стека.



Рис. 40. Рисунок

Время выполнения процесса — суммарное время, полученное процессом.

Перегиб происходит из-за того, что в некоторый момент времени в оперативной памяти процесс загрузил всё своё рабочее множество. Интервал между обращениями к отсутствующей странице назвали **временем жизни**.

Рис. 41. цилиндр

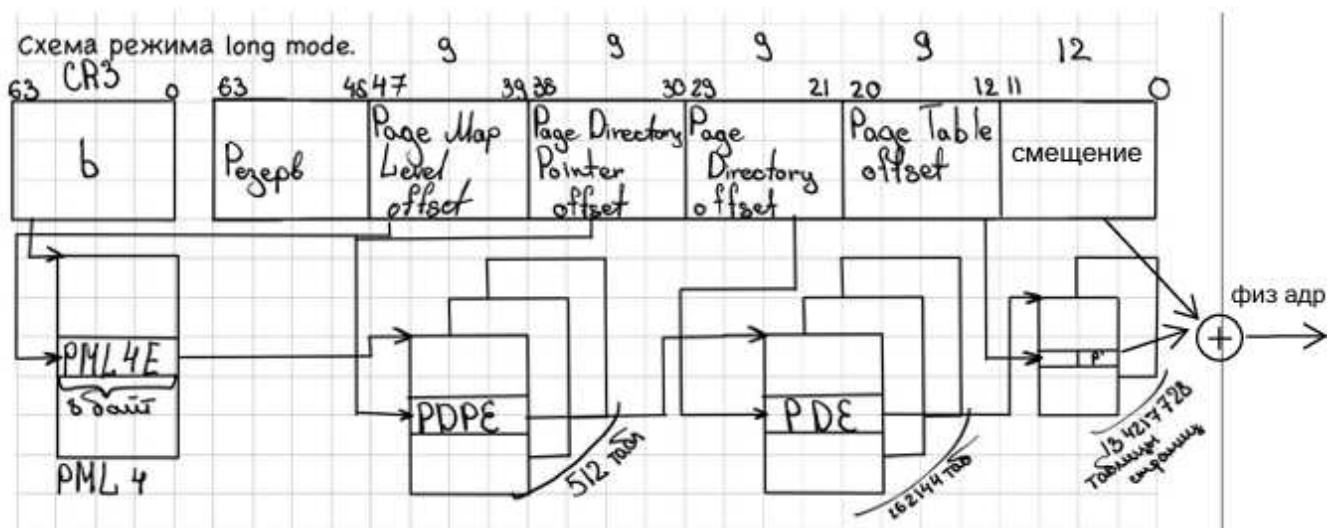
Программа в любом варианте находится во вторичной памяти. Когда она не выполняется, это просто файл. Когда мы запускаем программу на выполнение, это процесс, но его адресное пространство так же находится во вторичной памяти.

Любая программа считает, что она начинается с нулевого адреса, внутри неё есть смещения. Система хранит информацию о свободных, занятых кадрах, есть системная таблица, всё в ядре.

Схема станичного преобразования в процессорах Intel (x86-64) PAE

- В режим long реализовано только управление памятью страницами по запросу.
- В режиме long — схема PAE (physical address extension)

PAE



- PTE — таблицы страниц
- PDE — каталоги таблиц страниц —
- PDPE — указатель на каталоги таблиц страниц — максимум 512 табл.
- PML4E — указатель на указатель — такая таблица в системе 1
- Дескрипторы инициализируются по мере надобности
- Размер дескриптора — 8 байт
- Таблицы по 2^9 строк $\Rightarrow$ каждая таблица в памяти занимает одну страницу
- Когда процесс создается — ему выделяется минимально необходимое число страниц.
- То есть выполняется отложено.

Например, при создании процесса создается 3 страницы: точка входа, страница сегмента данных, страница сегмента стека.

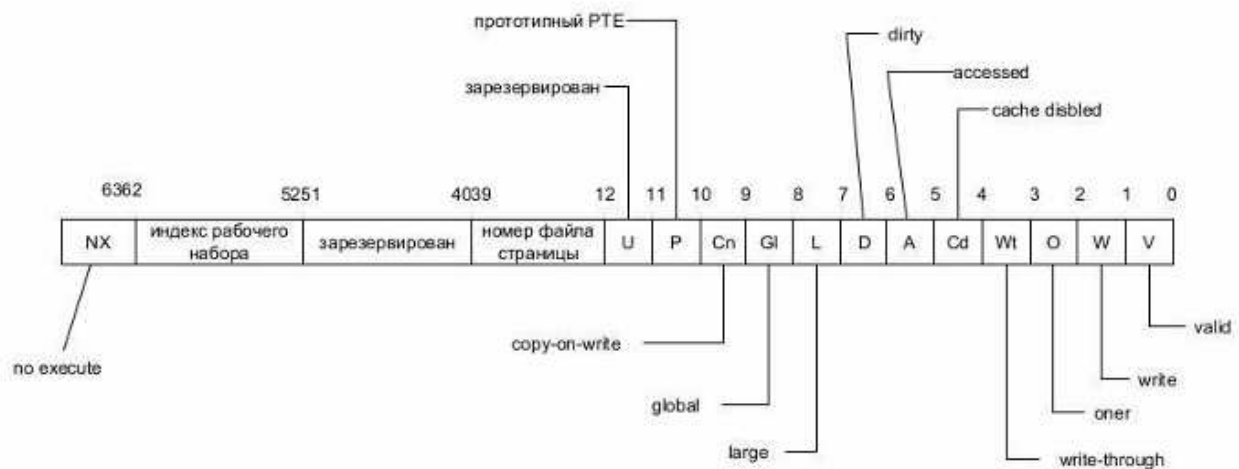
Все остальные страницы создаются по мере надобности при возникновении системных прерываний.

Дескриптор страницы, формат дескриптора

- PTE — таблицы страниц
- PDE — каталоги таблиц страниц –
- PDPE — указатель на каталоги таблиц страниц – максимум 512 табл.
- PML4E — указатель на указатель – такая таблица в системе 1
- Deskрипторы инициализируются по мере надобности
- **Размер** дескриптора – 8 байт
- Таблицы по 2^9 строк => каждая таблица в памяти занимает одну страницу
- Когда процесс создается — ему выделяется минимально необходимое число страниц.
- То есть выполняется отложено.
Например, при создании процесса создается 3 страницы: точка входа, страница сегмента данных, страница сегмента стека.
Все остальные страницы создаются по мере надобности при возникновении системных прерываний.

Основные флаги

- Дескриптор PTE — дескриптор в таблице страниц
Первые 12 разрядов не участвуют в адресации => отводятся для флагов



- Аппаратное ограничение — не все биты решили использоваться для аппаратной адресации
- Access — аппроксимация LRU
Время от времени сбрасывается.
Устанавливается при загрузке страницы и при каждом обращении.

Размеры таблиц дескрипторов (на схеме выше было)

Обоснование использования многоуровневого преобразования

Почему начали использовать страничную модель, несмотря на накладные расходы? Для оптимального использования оперативной памяти области ядра. Таблицы создаются там не сразу, могут быть выгружены во вторичную память. Таблицы хранятся в области ядра.

Виртуальный адрес - 64-х разрядный. Все таблицы имеют размер 4 КБ (одна страница). Это 12 разрядов, то есть смещение 2 в 12.

- 1) С 0 по 11 бит — смещение.
- 2) С 12 по 20 бит — номер страницы.
- 3) С 21 по 29 бит — в директории
- 4) С 30 по 38 бит — смещение в таблице страниц PDP
- 5) С 39 по 47 бит — смещение PML4
- 6) С 48 по 63 бит — Зарезервировано

Page Map Level 4 — 512 дескрипторов. Размер таблицы $512 \cdot 8 = 4$ КБ.

Это смещения в четырёхуровневой таблице страниц.

Начальный адрес этой таблицы находится в регистре CR3.

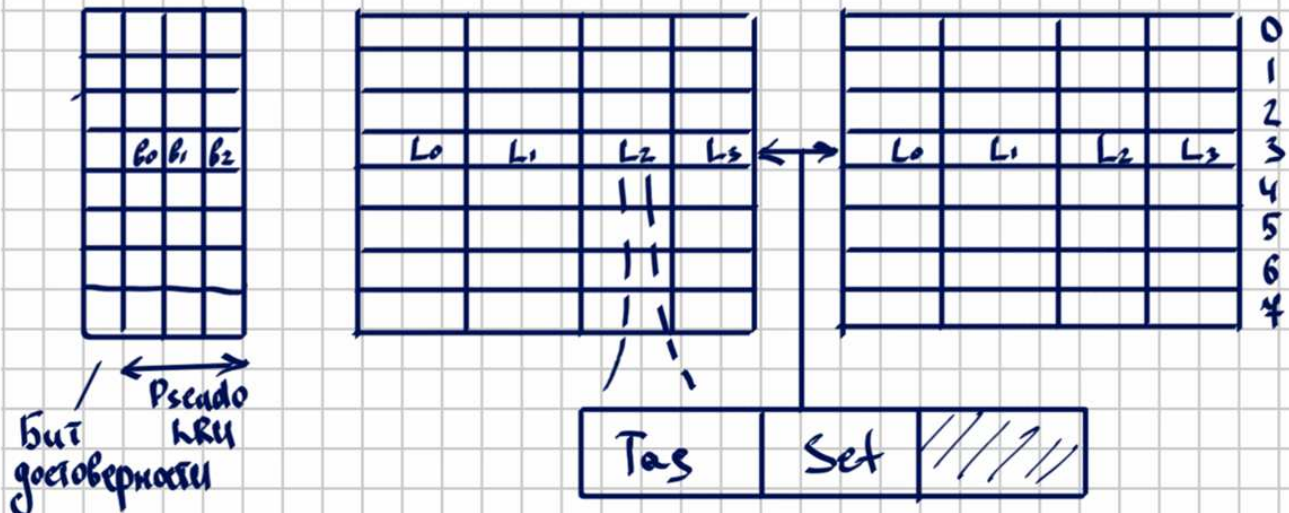
рисунок

Такая схема позволяет адресовать 256 ТБ адресного пространства. При этом указатель виртуальный адрес может в данной схеме занимать только 48 разрядов. На базе процессоров Intel существует возможность формировать адреса, используя 57 бит. В этом случае добавляется ещё одна таблица с 5 уровнями и адресация возрастает до 128 ПБ.

Кэш TLB - структура и алгоритм, аппроксимирующий LRU (для процессора i486)

Translation lookaside buffer.

(На примере i486)



При ошибке кэш все данные сбрасывается.

При явном сбое любой недостоверный сирокт
всем недостоверным или, или-то pseudo-LRU.

Всего битов обновляются при срабатывании
гиде.

$b_0 = 1$ last req to L_0/L_1

$b_0 = 0$ last req to L_2/L_3

$b_1 = 1$ last req to pair L_0-L_1 to L_0

$b_1 = 0$ last req to pair L_0-L_1 to L_1

$b_2 = 1$ last req to pair L_2-L_3 to L_2

$b_2 = 0$ last req to pair L_2-L_3 to L_3

b_0	b_1	b_2	
0	0	X	L_0
0	1	X	L_1
1	X	0	L_2
1	X	1	L_3

2. Принцип построения вычислительных систем: распараллеливание функций при управлении внешними устройствами. Трехшнная концептуальная архитектура. Программируемый контроллер прерываний. Адресация аппаратных прерываний в простейшей схеме (16-разрядный реальный режим). Функция обработчика прерывания от системного таймера в ОС разделения времени.

Ответ:

Принцип построения вычислительных систем

1.5 Третье поколение **вычислительных машин**

После 1960 года началась эпоха микросхем. Это третье поколение **вычислительных** машины.

Именно в машинах третьего поколения появилась полноценная архитектура **вычислительных** машин. Архитектура фон Неймана концептуальная, а в IBM 360 появилась аппаратная архитектура.

1.5.1 Канальная архитектура IBM 360

Во первых, элементная база процессоров изменилась, это больше не транзисторы, а интегральные схемы. А внешние устройства – механические, медленные. В результате если не предпринять действия, быстродействующий процессор будет простаивать, пока данные не будут введены. Это неэффективное использование дорогой техники. Чтобы сократить накладные расходы, в состав IBM 360 были введены специальные устройства, фактически это специальные процессоры, которые получили название канала. Т.е. IBM 360 была построена на основе канальной архитектуры. Соответственно когда процессу нужно было инициировать(?) операцию ввода-вывода, он посылал каналу так называемую канальную программу, получив которую канал начинал управлять внешним устройством. Процессор освобождался от управления внешним устройством, т.е. он мог перейти на выполнение другой работы.

В основу этой архитектуры положен **принцип распараллеливания функций**.

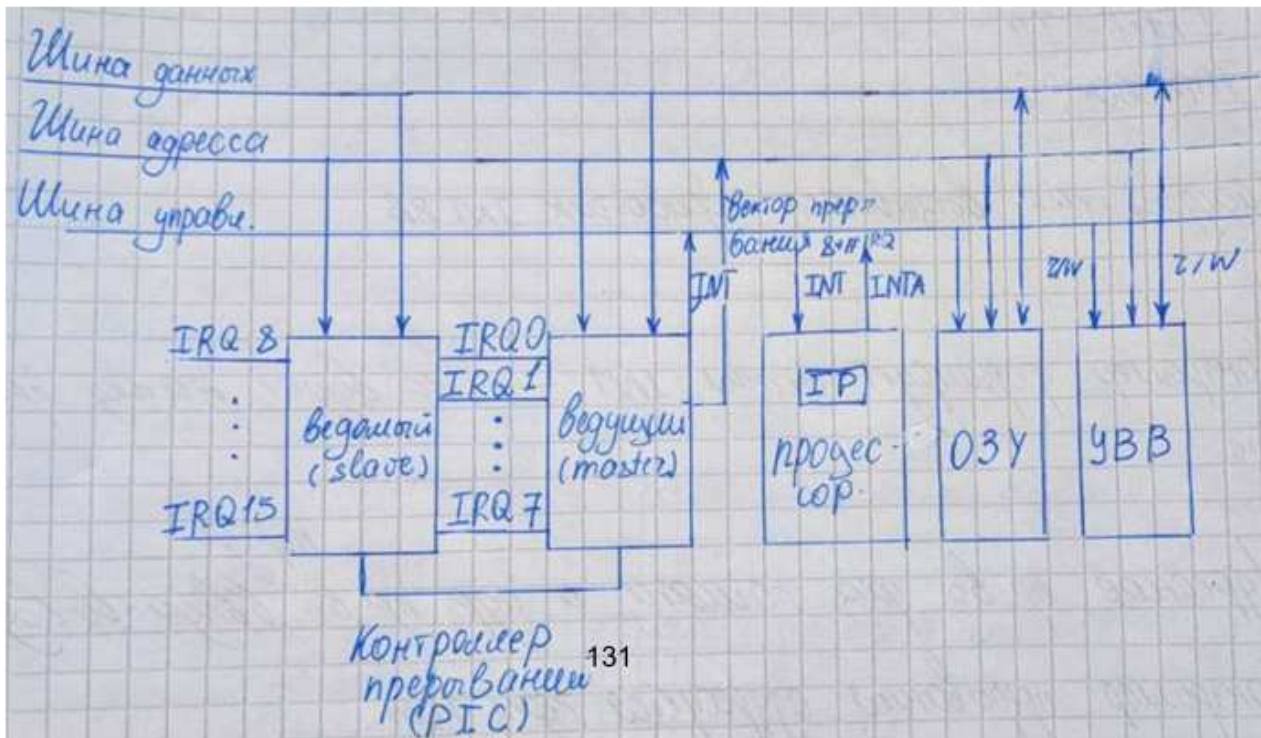
В наших компьютерах шинная архитектура. В современных компьютерах также реализован принцип РАСПАРАЛЛЕЛИВАНИЯ функций. Канал управляет внешним устройством, а параллельно процессор решает какие-то задачи. Архитектура не канальная, а шинная, но решается та же задача. но в наших компах внешними устройствами выполняют специальные устройства – контроллеры и адаптеры.

Контроллерами принято называть устройства, входящие в состав внешнего устройства, а **адаптерами** принято называть устройства, находящиеся на материнской плате. Это тоже программно управляемые устройства. Контроллеры

Трех шинная (трехшинная) концептуальная архитектура

- Когда на контроллер прерываний приходит прерывание от внешнего устройства, контроллер формирует INT, которые по шине управления поступает на выделенную ножку процессора.
- Цикл выполнения команды
 - → Выборка → Дешифрация → Выполнение
- В конце цикла выполнения команды ЦП проверяет наличие сигнала INT на выделенной ножке. Если он присутствует, то процессор по шине управления посылается INT A(answer)
- Получив этот сигнал от процессора(информирует о готовности обл. с к. д.) контроллер по шине данных процессору посылает вектор, который формируется как сумма базового вектора + номер линии IRQ.
- вектор = базовый вектор + №IRQ
- В реальном режиме базовый вектор ведущего контроллера равен 8, отсюда
INT8h — $8+0=8$;
INT9h — $8+1=9$; клавиатура

- В реальном режиме базовый вектор ведущего контроллера равен 8, отсюда
 $INT8h \rightarrow 8+0=8$;
 $INT9h \rightarrow 8+1=9$; клавиатура



программируемый контроллер прерываний

Адресация аппаратных прерываний в простейшей схеме (16-разрядный реальный режим)

???????

Прерывания

- Система прерываний:
 - системные вызовы (программные прерывания)
 - исключения

130

- аппаратные прерывания
 - от системного таймера (единственное периодическое)
 - от внешних устройств
 - от действий оператора (Ctrl+C)
- Система прерываний организуется через контроллер прерываний.
- Контроллер прерываний PIC(program interrupt controller) состоит из 2 микросхем, каждая по 8 входов, каскадное соединение.
Входы контроллера – линии IRQ(interrupt request) (линии прерываний). Имеют номера. IRQ2 используется для каскадного соединения.
 - Линии IRQ связаны с внешними устройствами
 - IRQ – interrupt request
 - IRQ0 – тики
 - IRQ1 – от клавиатуры PC/2
 - IRQ2 – от мыши
- 1 контроллер – ведущий, master
- 2 контроллер – ведомый, slave
- Когда на контроллер прерываний приходит прерывание от внешнего устройства, контроллер формирует INT, которые по шине управления поступает на выделенную ножку процессора.
- Когда на контроллер прерываний приходит прерывание от внешнего устройства, контроллер формирует INT, которые по шине управления поступает на выделенную ножку процессора.
- Цикл выполнения команды
 - → Выборка → Дешифрация → Выполнение
- В конце цикла выполнения команды ЦП проверяет наличие сигнала INT на выделенной ножке. Если он присутствует, то процессор по шине управления посылается INT A(answer)
- Получив этот сигнал от процессора(информирует о готовности обл. с к. д.) контроллер по шине данных процессору посылает вектор, который формируется как сумма базового вектора + номер линии IRQ.
- вектор = базовый вектор + №IRQ

1 Функции обработчика прерывания от системного таймера

1.1 Операционный системы семейства UNIX

По тикку

Функции:

- инкремент счётчика реального времени;
- декремент кванта текущего потока;
- декремент счётчиков времени до отправления на выполнение отложенных вызовов, при достижении счётчиков нуля происходит установка флага обработчика отложенных вызовов;
- инкремент счётчика процессорного времени, полученного процессом в режиме задачи и в режиме ядра.

В структуре `task_struct` присутствуют поля

```
u64    utime;  
u64    stime;
```

По главному тикку

Функции:

- инициализация отложенных вызовов функций, относящихся к работе планировщика;
- пробуждение системных процессов *swapper* и *pagedaemon*;
- декремент счётчика времени до отправления одного из следующих сигналов:
 - SIGALRM* - сигнал, посылаемый процессу по истечении времени, которое предварительно задано функцией *alarm()*;
 - SIGPROF* - сигнал, посылаемый процессу по истечении времени, которое задано в таймере профилирования;
 - SIGVTALRM* - сигнал, посылаемый процессу по истечении времени, которое задано в «виртуальном» таймере.

По кванту

Посылка сигнала *SIGXCPU* текущему процессу, если он превысил выделенный ему квант использования процессора.

Экзаменационный билет 18

1. Процессы: взаимодействие параллельных процессов, проблемы параллельного взаимодействия процессов, обоснование необходимости монопольного доступа к разделяемому ресурсу и методы взаимного исключения. Программный способ организации взаимодействия: флаги, алгоритм Деккера, алгоритм Лампорта "Булочная" (псевдокод), пример реализации на основе удаленного вызова процедуры (RPC) в Линукс.

Ответ:

Процессы: взаимодействие параллельных процессов

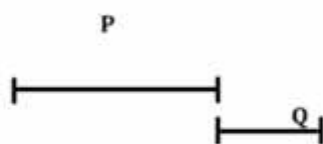
8 Взаимодействие параллельных процессов

8.1 Уровни наблюдения

Выделяют три варианта

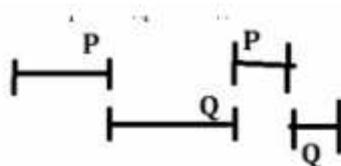
1) Последовательное выполнение

- Сначала выполняется первая программа от начала до конца, после этого начинает вторая от начала до конца и так далее.



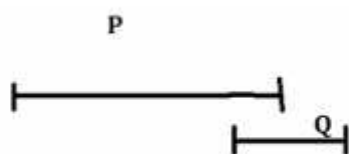
2) Квазипараллельное выполнение

- Связан с квантованием процессорного времени
- Для первой программы выделен квант, он истёк. Квант выделяется для второй программы и так далее.
- На уровне команд это последовательное выполнение, но пользователю кажется, что оно параллельное, отсюда и название.
- Это характерно для однопроцессорных систем (но в какие-то моменты может быть и в многоядерных системах).



3) Реальная параллельность

- Выполнение кодов совпадает по времени.
-



8.2 Проблемы параллельного выполнения

При этом это рассуждение проводится для того, что

проблемы, связанные с взаимодействием параллельных процессов, будут одни и те же для квазипараллельного выполнения и реальной параллельности.

8.2.1 Пример с банковским счётом

Рассмотрим пример. Два филиала банка или фирмы в конце дня должны перечислить на один и тот же счёт выручку. Нас не интересует интерфейс, а интересует только зачисление выручки.

(возможно в этом году было другое)

P1	P2
S := S + D1	S := S + D2
	mov
	add
	потерял квант
получил квант	
mov	
add	
загрузил в память	
	получил квант, загрузил в память
S = S + D1	S = S + D2

--

...Квант потерян. Сохраняется аппаратный контекст, т.е. сохраняется значение аккумулятора (AX). Начинает выполняться действие от другого банка, вырочку перевели в регистр, в аккумулятор записали значение суммы, сложили, записали в память, квант потерян. Квант получает второй филиал. Он из памяти уже все прочитал. Программа выполняется последовательно, в аккумуляторе уже сумма, осталось записать ее в ячейку памяти по адресу, в результате вырочка потеряна.

Два процесса выполняют они и те же действия. Это можно представить следующим образом:

1	P1: ...		P2: ...
2	mov eax, myvar		mov eax, myvar
3	inc eax		inc, eax
4	mov myvar, eax		mov myvar, eax

Листинг 1.

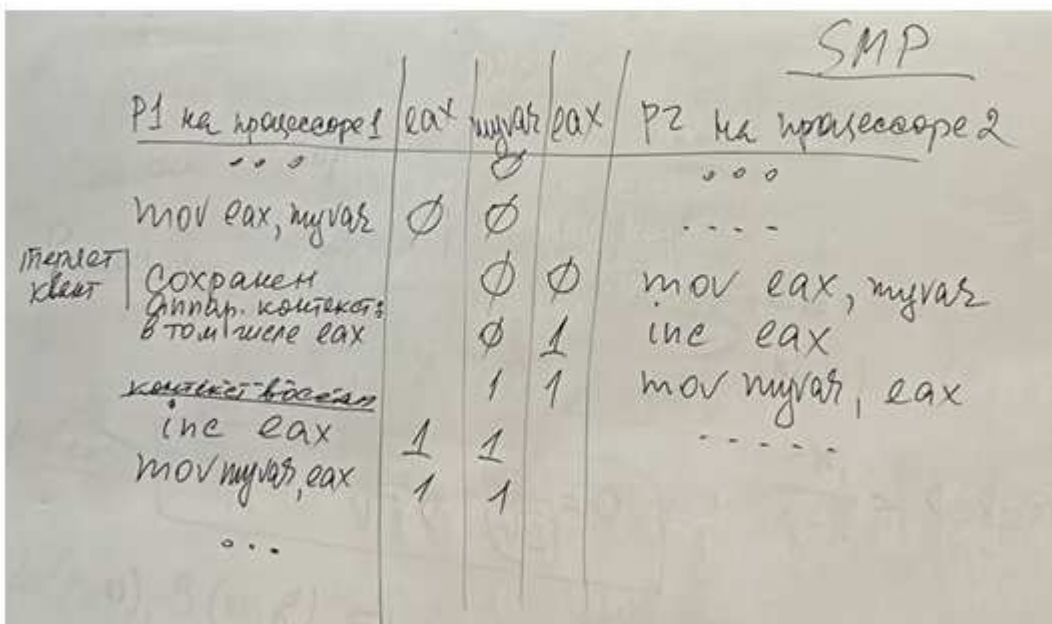


Рис. 13. Диаграмма выполнения на разных процессорах

- 1) Пусть первый процесс получает квант первым и выполняет команду и помещает переменную `muvar` в регистр `EAX`. Первый процесс теряет квант.
- 2) Второй получает процессорное время несколько позже, выполняет команды и переходит к другой работе.
- 3) Когда `P1` потерял квант, был сохранен аппаратный контекст, в том числе содержимое `EAX`. Получив квант, `P1` продолжает выполнение программы. После этого записывает `EAX` в `muvar`.
- 4) В итоге мы потеряли единицу $\Rightarrow$ потеряли данные, а это самое страшное в программировании

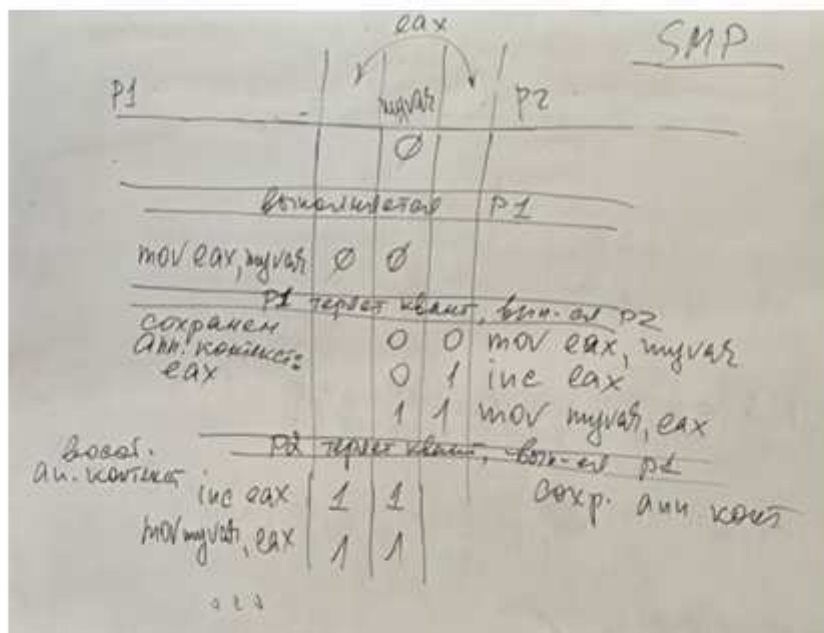


Рис. 14. Диаграмма для однопроцессорных машин

8.2.2 Критические секции и разделяемые ресурсы

Критическая секция — `mov eax, myvar; inc eax; mov myvar, eax` («вот этот вот код»)

Переменная `myvar` в данном случае называется разделяемой.

Чтобы не терять данные, работа с разделяемыми переменными или, в общем виде, с разделяемыми ресурсами должна выполняться в режиме монопольного доступа. Это значит, что если процесс зашёл в свой критический участок по разделяемой переменной, другой процесс в этот же критический участок по той же разделяемой переменной войти не должен.

Монопольный доступ обеспечивается методом взаимного исключения, то есть если процесс находится в критическом

участке, то другой процесс в этот же участок войти не сможет. Обеспечивается соответствующими средствами взаимного исключения.

8.3 Методы взаимного исключения

- 1) Программный способ
- 2) Аппаратный способ
- 3) С помощью семафоров
- 4) С помощью монитора

8.4 Программный способ взаимного исключения

8.4.1 Первая попытка с флагами

Это попытка обеспечить монопольный доступ процесса к разделяемым переменным чисто программным способом. Решение выполняется с помощью флагов.

```
1 fl1, fl2 : integer
2 P1: while (1)
3   {
4     while(fl2);
5     fl1 = 1;
6     CR1;
7     fl2 = 0;
8     PR1;
9   }
10 P2: while (1)
11   {
12     while(fl1);
13     fl1 = 1;
14     CR2;
15     fl2 = 0;
16     PR2;
17   }
18
19 main:
20   fl1 = 0
```

```

21 fl2 = 0
22 pasbegin P1; P;
23 pasend;
24 end;

```

Листинг 2. program №1

Пусть инициативу перехватил второй процесс (процессорное время). Вошел в цикл вайл и обнаружен что флаг 1 процесса сброшен зашел в крит секцию, установил флаг и потерял квант. Процессор 1 получил квант, обнаружил флаг, будет находится в цикле ожидания весь квант. Второй выполнил, сбросил. Все хорошо. Но что если второй зашел в крит секцию, но не успел установить флаг? Первый также обнаружил что флаг второго сброшен, выполнил действие (установил сбросил). P2 уже не проверяет флаг первого. Опять потеряли данные. Подход не всегда работает.

Пусть процессы сразу устанавливают свой флаг до цикла проверки флага.

```

1 fl1, fl2 : integer
2 P1: while (1)
3   {
4     fl1 = 1;
5     while(fl2);
6     CR1;
7     fl2 = 0;
8     PR1;
9   }
10 P2: while (1)
11   {
12     fl2 = 1;
13     while(fl1);
14     CR2;
15     fl2 = 0;
16     PR2;
17   }

```

```

18
19 main:
20   fl1 = 0
21   fl2 = 0
22   pasbegin P1; P;
23   pasend;
24   end;

```

Листинг 3. program №1 после изменений

- Второй процесс установил свой флаг и потерял квант.
- Первый получил квант, установил флаг и начал крутиться в цикле.
- Он потратил квант. Квант получил второй процесс.
- Так как флаг 1 процесса уже установлен, он начнет крутиться в цикле.
- И так по очереди. Процессы попали в тупик.
- Итог: флаги не работают в таком простейшем варианте.

8.4.2 Алгоритм Деккера

Задача решена только для двух параллельных процессов с помощью дополнительной переменной «чья очередь».

```

1 fl1, fl2m: logical;
2 queue : int;
3 P1: while (1)
4   {
5     fl1 = true;
6     while(fl2);
7     {
8       if (queue == 2) then
9       {
10        fl1 = false;
11        while (queue == 2)
12          fl1 = true;
13      }
14    }
15    CR1;

```

```

16  fl1 = false;
17  queue = 2;
18  PR1;
19  P2: while (1)
20  {
21      fl2 = true;
22      while(fl1);
23      {
24          if (queue == 1) then
25          {
26              f2 = false;
27              while (queue == 1)
28                  fl2 = true;
29          }
30      }
31      CR2;
32      fl2 = false;
33      queue = 1;
34      PR2;

```

Листинг 4.

- Проблема тупика решена за счет введения дополнительной переменной queue и если процесс обнаруживает что установлен флаг другого процесса, он проверяет, чья очередь. Если очередь второго процессора, то он (первый) свой флаг сбрасывает. Сбросил свой флаг он переходит в цикл ожидания по переменной queue. Когда выходит из своей критической секции, сбрасывает свой флаг, и устанавливает что следующим в критическую секцию войдет другой процесс.
- За счет того, что введена переменная чья очередь и за счет проверки этой переменной, процесс имеет возможность сбросить свой флаг если очередь другого процесса, но о всё равно входит в очередь ожидания, когда другой процесс, выйдя из своей критической секции, передаст инициативу другому процессу.

Но решена только для 2 процессов, ничего возня с флагами ничего не дала.

Мы увидели, что такое решение приводит к проверке значений переменных в цикле, то есть процессорное время тратится непроизводительно на проверку флагов. Это называется **активным ожиданием на процессоре, неэффективная трата процессорного времени**.

Есть и другое решение. «Деккер решает задачу достаточно сложным способом».

8.4.3 Алгоритм Bakery

Решение основывается на очереди.

9.5 Задача «Булочная» Лампорта (Lamport's Bakery Algorithm)

Решает задачу взаимного исключения для N параллельных процессов. Идею Лампорт заимствовал из принципа работы булочной. Каждому входящему в дверь булочной клиенту выдаётся листок с номером. Каждому вновь пришедшему клиенту выдаётся листок с номером на единицу больше всех выданных номеров. Продавец обслуживает клиентов в порядке полученных ими номеров.

При этом возможна ситуация когда в дверь булочной одновременно входят 2 клиента. Они должны получить один и тот же номер, но как их обслуживать, в каком порядке? Например, в соответствии с номером паспорта: у кого номер меньше, тот будет обслужен раньше.

```
1 var choosing: shared array[0..n-1] of boolean; // Массив флагов
2   number:   shared array[0..n-1] of integer; ...
3 repeat
4   choosing[i] := true;
5   number[i] := max(number[0], number[1], ..., number[n-1]) + 1;
6   choosing[i] := false;
7   for j := 0 to n-1 do begin
8     while choosing[j] do /* nothing */;
9     while number[j] <= 0 and
10       (number[j], j) < (number[i], i) do
11       /* nothing */;
```

```

12 end;
13 /* critical section */
14 number[i] := 0;
15 /* remainder section */
16 until false;

```

Листинг 11. ALGOL-подобный псевдокод

Под обслуживанием понимается код критической секции. Первыми обслуживаются клиенты с наименьшими номерами. обслуживание в порядке очереди. Расширение – есть информация о том, что текущий клиент уже получает обслуживаемое.

{ Причём здесь даже невозможно записать, сформализовать, что одновременно вошедшие в дверь клиенты получают один и тот же номер. По этой записи Макс + 1 они должны получить разные номера. Ну, ещё раз повторяю, Лампорт сформулировал, так как я продиктовала, формулировка Лампорта. Это как бы первая часть.}

- (Строки 4–6): Сначала входящий клиент получает максимальный номер из уже выданных (между true и false критический ресурс, разделяемый массив)
- (Строки 7–12): Затем он обслуживается в соответствии с полученным номером. При этом клиент будет ждать, пока клиенты с меньшими номерами не будут обслужены.
- (Строка 10): Был выбран процесс, который хочет войти в критическую секцию. Процесс P_i будет ждать пока имеются процессы с меньшими номерами. Если 2 процесса имеют один и тот же номер, то первым будет обслужен процесс с меньшим именем (с меньшим номером пас-

порта или каким-то ещё номером).

Это отношение называется лексикографическим порядком (см. ниже).

- (Строка 8): Указывает на то, что если процесс выбирает номер, когда P_i пытается так же его найти, P_i ждёт до тех пор, пока предыдущий процесс не выберет номер.
- (Строка 14): Показывает, что P_i больше не заинтересован во вхождении в критическую секцию.

Если процесс не пытается войти в свой критический участок, то их номера равны нулю.

Здесь нет таких средств взаимоисключения, как семафоры. Это взаимоисключение, которое обеспечивается порядком обслуживания. Если взять расширение, которое мы видим в работе аптеки , когда клиенты обслуживаются в порядке очереди, но разными провизорами, там необходимо будет использовать средства взаимоисключения, несмотря на то, что они обслуживаются в порядке очереди. Если провизор один, то уже ничего не нужно.

Определение 9.2 (Лексикографический порядок)

Запись $(a, b) < (c, d)$ называется лексикографическим порядком.

Она истинна если $a < c$ или $a = c$, но $b < d$

Булочная RPC

bakery.x

```
program BAKERY_PROG {  
  version BAKERY_VER {  
    int GET_TICKET(int client_id) = 1;  
    int GET_PROCESSING_TIME(int ticket_number) = 2;  
  } = 1;  
} = 0x30000001;
```

bakery_server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/time.h>
#include <unistd.h>
#include "bakery.h"

#define MAX_CLIENTS 100
#define MAX_TICKETS 1000

typedef struct {
    int ticket_number;
    long long processing_time_ns;
    int client_id;
    int is_processed;
    long long start_time_ns;
} TicketInfo;

// Глобальные переменные
TicketInfo tickets[MAX_TICKETS];
int ticket_counter = 0;
int next_ticket_number = 1;

// Функция для получения текущего времени в наносекундах
long long get_nanotime() {
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    return (long long)ts.tv_sec * 1000000000LL + ts.tv_nsec;
}

// Алгоритм булочной - получение номера
int get_ticket(int client_id) {
    int ticket_number = next_ticket_number++;

    // Находим свободный слот для хранения информации о заявке
    int slot = -1;
    for (int i = 0; i < MAX_TICKETS; i++) {
        if (tickets[i].ticket_number == 0) {
            slot = i;
            break;
        }
    }

    if (slot == -1) {
        slot = ticket_counter % MAX_TICKETS;
    }

    // Сохраняем информацию о заявке
    tickets[slot].ticket_number = ticket_number;
```

```

tickets[slot].client_id = client_id;
tickets[slot].is_processed = 0;
tickets[slot].processing_time_ns = 0;
tickets[slot].start_time_ns = get_nanotime();

printf("Ticket %d issued for client %d at time %lld\n",
       ticket_number, client_id, tickets[slot].start_time_ns);

return ticket_number;
}

void process_ticket(int ticket_number) {
    long long start_time = get_nanotime();

    // Ищем заявку
    for (int i = 0; i < MAX_TICKETS; i++) {
        if (tickets[i].ticket_number == ticket_number) {
            usleep(10000);

            long long end_time = get_nanotime();
            long long processing_time = end_time;

            tickets[i].processing_time_ns = processing_time;
            tickets[i].is_processed = 1;

            printf("Ticket %d processed in %lld ns\n", ticket_number, processing_time);
            break;
        }
    }
}

// Первый удаленный вызов: получение номера в очереди
int *get_ticket_1_svc(int *client_id, struct svc_req *rqstp) {
    (void)rqstp;
    static int result;

    int id = *client_id;

    if (id <= 0) {
        result = -1; // Неверный ID клиента
        return &result;
    }

    printf("\n=== Processing request from client %d ===\n", id);

    // 1. Получаем номер
    int ticket = get_ticket(id);
    printf("Client %d received ticket: %d\n", id, ticket);

    // 2. Немедленно обрабатываем заявку (однопоточный режим)
    printf("Processing ticket %d immediately...\n", ticket);
    process_ticket(ticket);
}

```

```

printf("Ticket %d completed for client %d\n", ticket, id);

result = ticket;
return &result;
}

// Второй удаленный вызов: получение времени обработки заявки
int *get_processing_time_1_svc(int *ticket_number, struct svc_req *rqstp) {
    (void)rqstp;
    static int result;

    int ticket = *ticket_number;

    if (ticket <= 0) {
        result = -1;
        return &result;
    }

    // Ищем заявку
    int found = 0;
    for (int i = 0; i < MAX_TICKETS; i++) {
        if (tickets[i].ticket_number == ticket) {
            found = 1;
            if (tickets[i].is_processed) {
                result = tickets[i].processing_time_ns;
                printf("Returning processing time for ticket %d: %d ns\n", ticket, result);
            } else {
                result = 0; // Заявка еще не обработана
                printf("Ticket %d is still processing\n", ticket);
            }
            break;
        }
    }

    if (!found) {
        result = -1; // Заявка не найдена
        printf("Ticket %d not found\n", ticket);
    }

    return &result;
}

// Обязательная функция
int bakery_prog_1_freeresult(SVCXPRT *transp, xdrproc_t xdr_result, caddr_t result) {
    (void)transp;
    xdr_free(xdr_result, result);
    return 1;
}

```

bakery_client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#include <sys/time.h>
#include "bakery.h"

// Функция для получения случайного ID клиента
int get_random_client_id() {
    return (rand() % 10) + 1; // Клиенты с ID от 1 до 10
}

// Функция для случайной задержки
void random_delay() {
    int delay_ms = (rand() % 500) + 100; // Задержка от 100 до 600 мс
    usleep(delay_ms * 1000);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s server_host [num_requests]\n", argv[0]);
        exit(1);
    }

    char *host = argv[1];
    int num_requests = 5; // По умолчанию 5 запросов

    if (argc > 2) {
        num_requests = atoi(argv[2]);
        if (num_requests <= 0) {
            num_requests = 5;
        }
    }

    CLIENT *clnt;

    // Создаем клиента
    clnt = clnt_create(host, BAKERY_PROG, BAKERY_VER, "tcp");
    if (clnt == NULL) {
        clnt_pcreateerror(host);
        exit(1);
    }

    srand(time(NULL));

    printf("=== Bakery Client: Automatic Requests ===\n");
    printf("Server: %s\n", host);
    printf("Number of requests: %d\n\n", num_requests);

    int successful_requests = 0;
    int tickets[num_requests];

```

```

// Фаза 1: Получаем заявки
printf("--- Phase 1: Requesting tickets ---\n");
for (int i = 0; i < num_requests; i++) {
    int client_id = get_random_client_id();

    printf("Request %d: Client %d requesting ticket... ", i+1, client_id);

    int *ticket_result = get_ticket_1(&client_id, clnt);
    if (ticket_result == NULL) {
        printf("FAILED (RPC error)\n");
        tickets[i] = -1;
        continue;
    }

    int ticket_number = *ticket_result;
    if (ticket_number == -1) {
        printf("FAILED (server error)\n");
        tickets[i] = -1;
    } else {
        printf("SUCCESS - Ticket %d\n", ticket_number);
        tickets[i] = ticket_number;
        successful_requests++;
    }

    // Случайная задержка между запросами
    if (i < num_requests - 1) {
        random_delay();
    }
}

// Небольшая пауза перед проверкой времени
printf("\nWaiting for processing to complete...\n");
sleep(2);

// Фаза 2: Проверяем время обработки
// printf("\n--- Phase 2: Checking processing times ---\n");
int checked_tickets = 0;

for (int i = 0; i < num_requests; i++) {
    if (tickets[i] != -1) {
        // printf("Checking ticket %d... ", tickets[i]);

        long long *time_result = get_processing_time_1(&tickets[i], clnt);
        if (time_result == NULL) {
            printf("FAILED (RPC error)\n");
            continue;
        }

        long long processing_time = *time_result;
        if (processing_time == -1) {
            printf("NOT FOUND\n");
        } else if (processing_time == 0) {
            printf("STILL PROCESSING\n");
        } else {

```

```

        // printf("%lld ns (%.3f ms)\n",
        //         processing_time, processing_time / 1000000.0);
        checked_tickets++;
    }

    // Случайная задержка между проверками
    if (i < num_requests - 1) {
        random_delay();
    }
}

// printf("\n=== Summary ===\n");
// printf("Total requests: %d\n", num_requests);
// printf("Successful ticket requests: %d\n", successful_requests);
// printf("Processing times checked: %d\n", checked_tickets);

cInt_destroy(cInt);
return 0;
}

```

2. Прерывания от устройств ввода-вывода: приоритеты ядра и режима пользователя Юникс/Линукс, пересчет пользовательских приоритетов, учитываемые значения, повышение приоритета при блокировках на устройствах ввода-вывода. Диаграмма "запрос приложения ввода-вывода".

Ответ:

Прерывания от устройств ввода-вывода

1.5.2 Система прерываний

Изменение ПО было кардинальным – появилось полноценная система прерываний, которая соответствует современной системе прерываний. Это значит, что если операция ввода-вывода направляет? управляет? канал, то необходимо сообщить процессору, что операция ввода-вывода завершена. Для этого были включены {аппаратные прерывания от внешних устройств прерывания от внешних устройств, которые относятся к аппаратным прерываниям}.

16

Т.е. по завершении операции ввода-вывода, канал информирует процессор о завершении ввода-вывода с помощью аппаратного прерывания, прерывания от устройства ввода-вывода.

Два типа каналов:

- мультиплексорный, позволяет обслуживать несколько медленных устройств, переключаясь между ними;
- селекторный, позволяет в каждый момент времени обслуживать одно быстрое внешнее устройство.

* Claude

Мультиплексорный канал переключается между несколькими медленными устройствами (терминалы, принтеры), обслуживая их почти одновременно. Селекторный канал работает с одним быстрым устройством (диск, магнитная лента) от начала до конца операции ввода-вывода.

В наших компьютерах шинная архитектура. В современных компьютерах также реализован принцип РАСПАРАЛЛЕЛИВАНИЯ функций. Канал управляет внешним устройством, а параллельно процессор решает какие-то задачи. получают от процессора команду и выполняя эту команду, они управляют работой внешнего устройства.

По завершении операции ввода-вывода формируется сигнал прерывания, который приходит к простейшей схеме на контроллер (?) прерывания. Эти прерывания необходимо обслуживать. Их обслуживают обработчики

прерывания. Т.е. прерывание возникают, в результате возникновения этого прерывания вызывается соответствующий обработчик прерывания. Обработчики прерывания входят в драйвера внешних устройств.

В систему прерываний¹ были включены:

- 1) Системный вызовы или программные прерывания.
Нам лучше называть это именно системными вызовами, потому что это более ёмкое название.
- 2) Исключения
- 3) Аппаратные прерывания. Их принято делить на
 - 3.1) Прерывания системного таймера.
 - 3.2) Прерывания от устройств ввода-вывода.
 - 3.3) Прерывания действия оператора.

Про аппаратные прерывания[new]. Каналы – программируемые, задача - управлять работой внешних устройств, освободить процессор от непроизводительного управления внешними устройствами (внешние - механические, значит медленные). Что значит процессор управляет? Значит, что процессор, например, ждет от внешнего устройства данные (запросил их) и он будет опрашивать готовность

¹Под interrupt понимается именно аппаратное прерывание.

устройства, например, клавиатуры, когда же запрошенные данные окажутся в буфере данной клавиатуры. Т.е. будет тратить кванты процессорного времени на опрос внешнего устройства и делать другую работу не может. Выполняется непроизводительная работа. Поэтому реализовано распараллеливание функций и управление устройствами передано каналам (канальная архитектура). В результате появилась группа аппаратных прерываний от устройств ввода вывода. При этом прерывание от устройств ввода вывода информирует процессор о **завершении** операции ввода вывода.

Также включены прерывания действий оператора.

Приоритеты ядра

5.2 Алгоритмы планирования (scheduling)

Алгоритмы планирования рассматриваются с точки зрения типов операционных систем. В мультипрограммных системах пакетной обработки.

Оценка эффективности машинок – количество выполненных заданий в единицу времени.

5.2.1 Классификация алгоритмов

Классификация алгоритмов:

1) Планирование без переключения (1)

Алгоритм FCFS (в современных системах). First Come - First Served (по принципу FIFO). Без переключения, без приоритетов, без вытеснения, процесс выполняется от начала до конца.

Процессы за время своей работы могут запросить ввод/вывод. Тогда процессы будут блокированы и не будут находится в очереди готовых процессов {тут сказала, что внесла что-то новое}.

2) Планирование с переключением

Необходимо выполнять планирование выполнения процессов, при этом необходимо учитывать, что процессы получают только отрезок процессорного времени и израсходовав квант, процесс должен быть снят с выполнения, чтобы квант мог быть предоставлен другому

процессу в очереди. Это планирование с переключением (процессор переключается между процессами)

Алгоритм RR (Round Robin), первый для систем разделения времени. Используется для систем разделения времени. Здесь добавляется линия, обозначающая что процесс потерял квант (процессор в результате того, что исчерпал квант). Без приоритетов.

Супостаты называют это вытеснением, но мы назовем это переключением. «Они убогие, они не различают.»

3) Планирование без приоритетов

4) Планирование с приоритетами

4.1) С вытеснением (3)

Алгоритм SRT (Shortest Remaining Time, Наименьшее Оставшееся Время). Процесс мог быть вытеснен с выполнения, если в очередь готовых процессов поступил процесс с меньшим оценочным временем выполнения, чем время, оставшееся выполняющемуся процессу до завершения. Он вытеснялся в очередь готовых процессов и из его оценочного времени вычиталось полученное им процессорное время. Но еще более усиленное бесконечное откладывание (не объяснила подробно).

4.2) Без вытеснения (2)

Алгоритм SJF (Shortest Job First), наикратчайшее задание первое. Должна быть априорная (до опыта) информация о времени выполнения задания. В

результате возникла ситуация: задания, которым требуется много процессорного времени, постоянно откладывались (потому что преимущество получали короткие задания, это приоритетное планирование). Возникла ситуация, которая получила название «бесконечное откладывание». Приоритет можно было купить, заплатив деньги (просто ее замечание о былых деньгах).

HRRN (Highest Response Ratio Next) (4). Был разработан для того чтобы убрать негативные последствия откладывания, здесь приоритет процесса вычисляется по формуле

$$P = \frac{t_w + t_s}{t_s}$$

t_s — запрошенное время выполнения

t_w — время простоя в очереди готовых процессора.

В результате учитывается время простоя и приоритет процесса увеличивается и в конечном счете он получит процессорное время.

Приоритеты режима пользователя Юникс/Линукс

6.2 События, переводящие систему в режим ядра

Фактически эта информация полностью коррелирует с системой прерываний. Три типа события, которые переводят систему из режима **пользователя** в режим ядра:

- 1) Системные вызовы (программные прерывания, Software Interrupts, Windows — traps, ловушки).
Это набор функций, которые предоставляются пользователю операционной системой, это не произвольные функции (говорили в POSIX).
- 2) Исключения (исключительные ситуации).
Исключительные ситуации в современных системах бывают двух типов:
 - 2.1) Исправимые — страничные прерывания. После того, как оно обработано системой, программа продолжает выполняться.
 - 2.2) Неисправимые — ошибки кода или системы (сбои).
При них программа завершается, как мы говорим — аварийно.
- 3) Аппаратные прерывания
Принято различать три типа:
 - 3.1) прерывания системного таймера (единственное периодическое прерывание системы)

45

3.2) прерывания от устройств ввода/вывода

3.3) прерывания от действий оператора

???

Пересчет пользовательских приоритетов, учитываемые значения, повышение приоритета при блокировках на устройствах ввода-вывода.

2 Пересчет динамических приоритетов

В операционных системах семейства UNIX и Windows пересчитываться могут только приоритеты пользовательских процессов.

2.1 UNIX

Планирование процессов в UNIX основано на приоритете процесса. Планировщик всегда выбирает процесс с наивысшим приоритетом. Приоритеты планирования пользовательских процессов изменяются с течением времени, то есть динамически системой, в зависимости от использования вычислительных ресурсов, времени ожидания запуска и текущего состояния процесса.

В современных системах UNIX ядро является вытесняемым – то есть процесс в режиме ядра может быть вытеснен более приоритетным процессом в режиме ядра. Это сделано, чтобы система могла обслуживать процессы реального времени, например видео и аудио.

Очередь процессов, готовых к выполнению, формируется согласно приоритетам и принципу вытесняющего циклического планирования, то есть сначала выполняются процессы с большим приоритетом, а процессы с одинаковым приоритетом выполняются в течении кванта времени друг за другом циклически. В случае, если процесс с более высоким приоритетом поступает в очередь процессов, готовых к выполнению, планировщик вытесняет текущий процесс и предоставляет ресурс более приоритетному процессу.

Приоритет процесса задается любым целым числом, которое лежит в диапазоне от 0 до 127 по правилу чем меньше число, тем выше приоритет. При этом числа в диапазоне от 0 до 49 являются зарезервированными для ядра, то есть они фиксированы. Числа в диапазоне от 50 до 127 являются приоритетами прикладных задач (приоритеты которых могут изменяться во времени).

Изменение приоритета прикладных задач зависит от двух факторов: фактор «любезности» и результат последней измеренной величины использования процессора. Фактор «любезности» – целое число в диапазоне от 0 до 39 со значением 20 по умолчанию. Увеличение значения приводит к уменьшению приоритета. Пользователи могут повлиять на приоритет процесса при помощи изменения значений этого фактора, но только суперпользователь может увеличить приоритет процесса. Фоновые процессы автоматически имеют более высокие значения этого фактора.

Дескриптор процесса UNIX BSD struct proc содержит следующие поля, которые относятся к приоритетам:

- p_pri – текущий приоритет планирования
- p_usrpri – приоритет режима задачи
- p_cpu – результат последнего измерения использования процессора
- p_nice – фактор «любезности», который устанавливается пользователем

p_pri используется планировщиком для принятия решения о том, какой процесс отправить на выполнение.

p_pri и **p_usrpri** равны, когда процесс находится в режиме задачи.

Значение **p_pri** может быть изменено (повышено) планировщиком для того, чтобы выполнить процесс в режиме ядра. В таком случае **p_usrpri** будет использоваться для хранения приоритета, который будет назначен процессу, когда тот вернется в режим задачи.

p_cpu инициализируется нулем при создании процесса (и на каждом тике обработчик таймера увеличивает это поле на 1, до максимального значения равного 127).

Ядро системы связывает приоритет сна с событием или ожидаемым ресурсом, из-за которого процесс может блокироваться (приоритет сна определяется для ядра, поэтому лежит в диапазоне 0 - 49). Когда процесс «просыпается», ядро устанавливает **p_pri**, равное приоритету сна события или ресурса, по которому произошла блокировка (значения приоритета сна для некоторых событий в системе 4.3 BSD представлены в таблице 2.1).

Таблица 2.1 — Таблица приоритетов в операционной системе 4.3BSD

Приоритет	Значение	Описание
PSWP	0	Свопинг
PSWP + 1	1	Страничный демон
PSWP + 1/2/4	1/2/4	Другие действия по обработке памяти
PINOD	10	Ожидание освобождения inode
PRIBIO	20	Ожидание дискового ввода-вывода
PRIBIO + 1	21	Ожидание освобождения буфера
PZERO	25	ый приоритет
TTIPRI	28	Ожидание ввода с терминала
TTOPRI	29	Ожидание вывода с терминала
PWAIT	30	Ожидание завершения процесса потомка
PLOCK	35	Консультативное ожидание блок. ресурса
PSLEP	40	Ожидание сигнала

В таблице 2.2 приведены значения приоритетов сна для систем *4.3BSD UNIX* и *SCO UNIX*. Такой подход позволяет системным вызовам быстрее завершать свою работу. По завершении процессом системного вызова его приоритет сбрасывается в значение текущего приоритета в режиме задачи. Если при этом приоритет окажется ниже, чем приоритет другого запущенного процесса, ядро произведет переключение контекста.

Каждую секунду ядро системы инициализирует отложенный вызов процедуры `schedcpu()` которая уменьшает значение `p_pri` каждого процесса исходя из фактора «полураспада» (в системе 4.3BSD считается по формуле 2.1)

$$decay = \frac{2 \cdot load_average}{2 \cdot load_average + 1}, \quad (2.1)$$

где `load_average` - это среднее количество процессов, находящихся в состоянии готовности к выполнению, за последнюю секунду.

Также процедура `schedcpu()` пересчитывает приоритеты для режима задачи всех процессов по формуле 2.2.

$$p_usrpri = PUSER + \frac{p_cpu}{2} + 2 \cdot p_nice, \quad (2.2)$$

где `PUSER` - базовый приоритет в режиме задачи, равный 50.

Таким образом, если процесс в последний раз использовал большое количество процессорного времени, то его `p_cpu` будет увеличен, следовательно будет рост значения `p_usrpri` и понижение приоритета. Чем дольше процесс простаивает в очереди на выполнение, тем больше фактор полураспада уменьшает его `p_cpu` и следовательно повышение его приоритета. Такая

Таблица 2.2 — Системные приоритеты сна

Событие	Приоритет в <i>4.3BSD UNIX</i>	Приоритет в <i>SCO UNIX</i>
Ожидание загрузки в память страницы	0	95
Ожидание индексного дескриптора	10	88
Ожидание ввода-вывода	20	81
Ожидание буфера	30	80
Ожидание терминального ввода	30	75
Ожидание терминального вывода	30	74
Ожидание завершения выполнения	30	73
Ожидание события	40	66

схема предотвращает бесконечное откладывание низкоприоритетных процессов. Применение данной схемы предпочтительно процессам, осуществляющим много операций ввода-вывода, в противоположность процессам, производящим много вычислений. То есть динамический пересчёт приоритетов приложений и пользовательских процессов позволяет избежать бесконечного откладывания.

Современные UNIX-системы являются многопоточными – и приоритеты уже пересчитываются не для процессов, а для потоков. Многопоточность в POSIX специфицирована стандартом IEEE 1003.1c, который описывает API для создания и синхронизации **потоков**. POSIX описывает лишь требуемое поведение библиотеки потоков.

В стандарте определены два основных типа данных для потоков:

- `pthread_t` – дескриптор потока;
- `pthread_attr_t` – набор атрибутов потока.

Определён следующий набор функций для управления потоками:

- `pthread_create()`;
- `pthread_exit()`;
- `pthread_cancel()` и др.

Имеются следующие примитивы синхронизации потоков с помощью мьютексов и условных переменных, а также функции для управления ими:

- `pthread_mutex_init()`;
- `pthread_mutex_destroy()`;
- `pthread_mutex_lock()`;
- `pthread_mutex_unlock()`;
- `pthread_cond_init()`;
- `pthread_cond_signal()`;
- `pthread_cond_wait()` и др.

Распределение приоритетов между потоками в Unix

Хотя исторически планировщик в Unix работал с процессами, в современных многопоточных системах единицей планирования является поток (нить, `thread`). В системах, поддерживающих POSIX Threads (pthreads), у каждого потока внутри процесса есть свой собственный приоритет и политика планирования. Это позволяет разным потокам одного процесса конкурировать за процессорное время независимо друг от друга.

Модели потоков и планирование

Существует две основных модели реализации потоков:

— **Пользовательские потоки (M:1):** Планирование осуществляется библиотекой в пользовательском пространстве, ядро «видит» лишь один процесс (один поток ядра — LWP). Пересчет и распределение приоритетов между такими потоками — задача библиотеки (например, NPTL в glibc), которая использует собственные алгоритмы, часто основанные на приоритетах, заданных программистом через `pthread_attr_t.schedparam`.

— **Потоки ядра (1:1):** Каждому пользовательскому потоку соответствует отдельный поток ядра (LWP). Именно эта модель является стандартом в современных Linux, FreeBSD, Solaris. Планировщик ядра работает с каждым таким потоком напрямую, назначая и пересчитывая приоритеты индивидуально для каждого.

Приоритеты и политики планирования (POSIX)

Стандарт POSIX определяет для потоков не только интерфейсы синхронизации, но и управление планированием через атрибуты потока (`pthread_attr_t`). Ключевую роль играют:

— **Политика планирования (scheduling policy):** Определяет алгоритм, по которому планировщик выбирает поток для выполнения.

`SCHED_OTHER` (стандартное циклическое планирование с разделением времени):

Используется для обычных потоков. В рамках этой политики и происходит динамич-

ческий пересчет приоритетов (например, на основе значения `nice`). Это политика по умолчанию.

SCHED_FIFO (планирование «первым пришел — первым обслужен»): Потоки с реальным временем. Приоритет статичен, назначенный поток выполняется, пока не завершится, не вытеснится потоком с более высоким приоритетом или не вызовет блокирующий системный вызов.

SCHED_RR (циклическое планирование для реального времени): Аналогично **SCHED_FIFO** но потоку выделяется квант времени, после чего он помещается в конец очереди для своего уровня приоритета.

— **Параметры планирования (scheduling parameters)**: Структура `sched_param`, содержащая как минимум поле `sched_priority` — статический приоритет потока. Для политик реального времени (**SCHED_FIFO**, **SCHED_RR**) это значение является фиксированным и определяющим. Для **SCHED_OTHER** это поле часто используется как базовый приоритет, относительно которого работает динамический пересчет.

Диаграмма "запрос приложения ввода-вывода"

3.1.2 Ввод-вывод

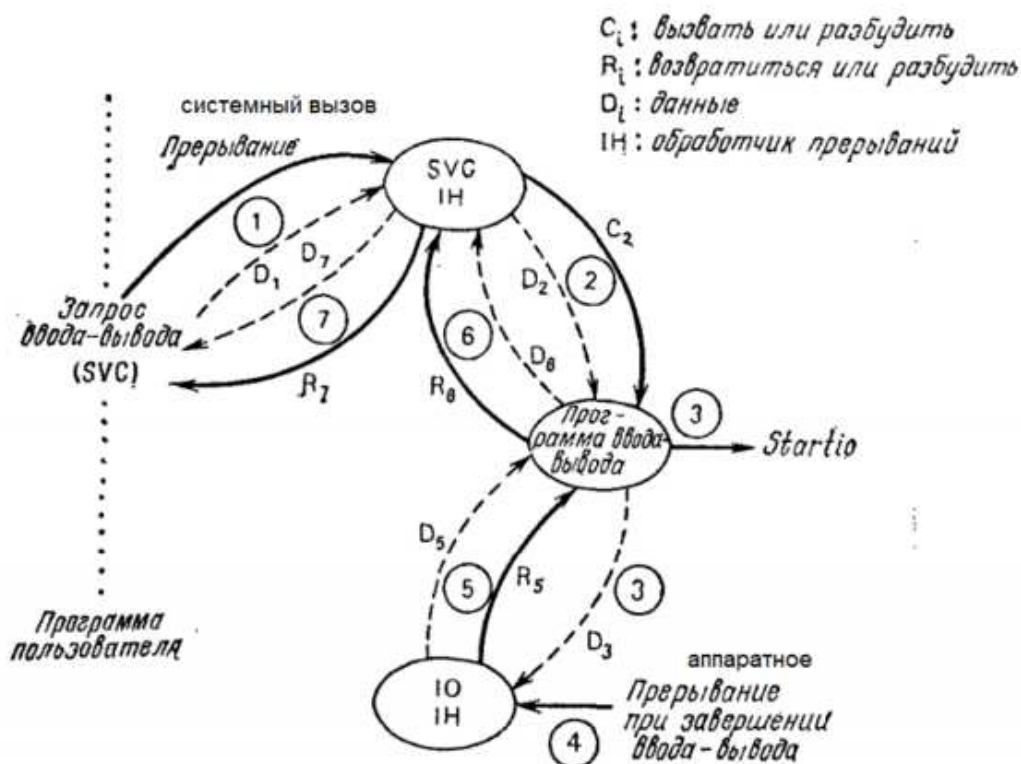


Рис. 1: Управление прерываниями в последовательности ввода-вывода

1 – системный вызов
D1 – передача данных

D5 – данные сохраняются в буфере ядра

Ю ИИ – выполняется на высоком уровне привилегий (должно быстро завершиться)

Программа ввода/вывода – драйвер

ОС в стадии выполнения – супервизор.

Обработчик системного вызова вызывает драйвер – код, который имеет много точек входа.

Драйвер предназначен для управления внешним устройством, его инициализация.

По шине данных посылается информация контроллеру.

Любой обработчик аппаратного прерывания входит в драйвер и является его точкой входа.

Экзаменационный билет 20

1. Процессы: взаимодействие параллельных процессов - монопольный доступ (обоснование необходимости) и взаимоисключение; программная реализация взаимоисключения - примеры, аппаратная реализация test-and-set, реализация spin-lock(); активное ожидание, семафоры Дейкстры Э.- определение, виды семафоров, примеры использования множественных семафоров из лабораторных работ «производство-потребление», «читатели-писатели» с обработчиком сигнала.

Ответ:

Процесс — это программа в стадии выполнения.

Проблемы многопоточного программирования. Одна из проблем разделение данных (Data Splitting):

- Имеется в виду работа потока с одними и теми же данными, то есть обращение к одним и тем же данным. Это требует специальных подходов, чтобы не терять данные, то есть не формировать ошибочные результаты.
- Это называется «взаимоисключение» или «монопольный доступ к данным»

Монопольный доступ и синхронизация – разные вещи!

Методы взаимоисключения:

1. Программный способ
2. Аппаратный способ
3. С помощью семафоров
4. С помощью монитора

8.5 Аппаратный способ взаимного исключения

8.5.1 Команда `test_and_set`

Начиная с машины IBM370 (с операционной системы, с системы команд), во всех архитектурах присутствует аппаратная команда `test_and_set(a,b);`.

- Это неделимая команда, её выполнение прервать нельзя.
- Она как правило (в архитектурах различных могут быть нюансы) читает значение логической переменной `b` и копирует его в `a`, затем устанавливает `b = истина`.

```
1 P1: while (1)
2   {
3     C1 = 1;
4     while (C1)
5       test_and_set(C1, fl);
6     CR1;
7     fl = 0;
8     PR1;
9   }
10 P2: while (1)
11   {
12     C2 = 1;
13     while (C2)
14       test_and_set(C2, fl);
15     CR2;
16     fl = 0;
17     PR2;
18   }
19 fl = 0;
20 pasbegin
21   P1, P2'
22 pasend
```

- Логическая переменная `fl` имеет значение «истина», когда любой из процессов находится в своём критическом участке, иначе ложь.
- Рассмотрим когда `P1` хочет войти в свой критический участок, а `P2` уже находится в своем крит. участке

- 1) P1 устанавливает в C1 единицу и входит в цикл проверки переменной fl командой test_and_set
 - 2) Так как P2 находится в критическом участке, то значение fl = 1
 - 3) Команда test_and_set обнаруживает этот факт и устанавливает в C1 единицу
 - 4) Таким образом P1 находится в цикле активного ожидания, пока P2 не выйдет из своего критического участка.
 - 5) Когда процесс выходит из критического участка и устанавливает в fl = false то скорее всего другой процесс сможет перехватить инициативу, установив в fl true
- Данный способ практически исключает захват критической секции одним и тем же процессом.

test_and_set лежит в основе spin_lock и семафоров.

8.5.2 Spin lock

test_and_set есть в макрокоманде spin_lock, которые активно используются в ядрах ОС, в ядрах Linux и Windows.

test_and_set используются для захвата в ядре критических секций, некоторые части кода ядра не могут блокироваться, на них можно устанавливать только спинлоки, это активное ожидание на процессоре. (ЭТО ПРИМЕЧАНИЕ ДЛЯ ВЕРСИИ ПРОСТО C while (test_and_set(*c) != 0))

Означает, что процессорное время используется неэффективно, плохо для системы.

Вот такой вариант лучше:

```

1 void spin_lock(spin_lock_t *c)
2 {
3     while (test_and_set(*c) != 0)
4         /* ресурс занят */
5         while (*c != 0);
6 }
7
8 void spin_unlock(spin_lock_t *c)
9 {
10    *c = 0;
11 }

```

Листинг 6.

- test_and_set приводит к занятию шины одним процессом или потоком и, следовательно, к снижению отзывчивости системы.
- Если ресурс занят, то дальше выполняется простая проверка c, и её можно прервать. То есть захват на длительное время локальной шины памяти не выполняется.

9 Семафоры Дейкстры

9.1 Определение семафора

Dijkstra EW.

Работа, связанная с семафорами и взаимным исключением была опубликована в 1965 году. Уже в 1965 году решение проблем взаимодействия параллельных процессов назрело, стало очень важным.

Определение 9.1 (Семафор)

Семафор — это неотрицательная защищённая переменная, на которой определены 2 операции:

- 1) P — Первая буква нидерландского слова «Proberen» (пытаться) — декремент, захват семафора. Можно выполнить только если $S > 0$. Иначе процесс блокируется в ожидании освобождения семафора.

$$S = S - 1 \quad S > 0$$

- 2) V — Первая буква нидерландского слова «Verhogen» (поднимать выше) — инкремент. Разблокирует процесс, ожидающий на семафоре.

$$S = S + 1$$

S — Semaphore.

Семафоры бывают:

- 1) Бинарные (0 и 1), определены только две неделимые операции (P и V)
- 2) Считающие (целые положительные значения и 0)

9.2 Реализация семафоров

В своих работах Дейкстра обозначил именно этими буквами операции на семафоре.

Изменять семафор можно только двумя неделимыми операциями.

Дейкстра определил это, чтобы уйти от активного ожидания на процессоре, чтобы не тратить процессорное время на проверку флагов. Но за всё надо платить. От активного ожидания удалось избавиться, но за счёт того, что операции P и S являются системными вызовами и переводят систему в режим ядра. Только ядро может заблокировать и разблокировать процесс. Переход в режим ядра несёт расходы, связанные с переключением программного контекста.

Изменение положения нескольких семафоров при этом современные приложения как правило многопоточные, но даже в те времена выполнялось распараллеливание действий. Изменение в таких приложениях набора нескольких семафоров между собой не связанных могло привести к тупику.

9.4 Задачи синхронизации

9.4.1 Задача производства — потребления (Producer — Consumer)

Есть 2 типа процессов:

- 1) Процессы-производители, которые производят единицу каких-то данных и помещают их в ячейки буфера / хранилища, т.е. производители могут только производить и помещать произведенные данные в буфер.
- 2) Процессы-потребители, которые могут только выбирать данные из буфера. В результате ячейка освобождается, и второй раз из этой же ячейки ничего взять невозможно.

Решение задачи предложил Дейкстра. Его решение основывается на 3 семафорах: двух считающих и одном бинарном.

- Считающие семафоры: *Empty*, *Sfull*
- Бинарные семафор: *Sbinary*

1	// Начальные значения:
2	// <i>Empty</i> = N
3	// <i>Sfull</i> = 0
4	// <i>Sbinary</i> = 1
5	producer:
6	P(<i>Empty</i>)
7	P(<i>Sbinary</i>)
8	N = N + 1
9	V(<i>Sbinary</i>)
10	V(<i>Sfull</i>)
11	
12	consumer:
13	P(<i>Sfull</i>)
14	P(<i>Sbinary</i>)
15	N = N - 1
16	V(<i>Sbinary</i>)
17	V(<i>Empty</i>)

- 1) Производитель пытается декрементировать семафор *empty* (проверка наличия свободных ячеек). Если ему это удалось, он пытается декрементировать (захватить) бинарный семафор для монопольного доступа к буферу, увеличивает количество занятых ячеек буфера, поскольку ему удалось положить что-то в ячейку буфера, после этого освобождает бинарный семафор и инкрементирует семафор *sfull*.
- 2) Потребитель пытается декрементировать *sfull* (проверка наличия заполненных ячеек). Если удалось, пытается захватить бинарный семафор, если удалось, освобождает ячейку, освобождает бинарный семафор и инкрементирует *empty*.

Producer будет блокирован на `empty`, если нет свободных ячеек в буфере. Он не может ни в одну ячейку не может положить произведённые данные. Он будет блокирован, пока Consumer не освободит хотя бы одну ячейку буфера. Если Producer удалось положить данные в ячейку буфера, он декрементирует количество свободных ячеек. Когда он выходит из своей критической секции, он увеличивает количество заполненных ячеек буфера.

Бинарный семафор обеспечивает монопольный доступ к буферу.

Consumer наоборот будет блокирован, если нет заполненных ячеек в буфере. Если есть хотя бы одна заполненная ячейка, Consumer декрементирует семафор `sfull`, захватывает семафор, декрементирует количество заполненных ячеек, освобождает семафор и инкрементирует `empty` количество свободных ячеек буфера.

Смысл в том что процессы являются асинхронными, то есть процессы выполняются с собственной скоростью. Предсказать, когда процесс придёт в свою определённую точку невозможно. Поэтому возможна ситуация когда потребители быстрее освобождают ячейку буфера, чем производители их заполняют, то есть возможны ситуация когда все ячейки буфера пустые, тогда потребитель будет ждать производителей.

И наоборот, возможна ситуация, когда производители работают быстрее потребителей, заполняют все ячейки буфера, и вынуждены ждать, когда потребители освободят все ячейки буфера. Таким образом происходит синхронизация работы производителей и потребителей.

Несмотря на то, что тут три семафора, тут тупик не возникнет, поскольку такая задача грамотно решённая и она очень показательна именно с точки зрения, что тут используются считающие семафоры.

«производство-потребление»

```
int flag = 1;
void sig_handler(int signal)
{
    flag = 0;
    printf("catch sig: %d, PID: %d\n", signal, getpid());
}

// Структуры для операций с семафорами:
// sem[0] - свободные места (n)
// sem[1] - занятые места (0)
// sem[2] - мьютекс (1)

// Захват свободного места и мьютекса
struct sembuf start_prod[2] = { { 0, -1, 0 }, { 2, -1, 0 } };
// Освобождение мьютекса и увеличение занятых
struct sembuf end_prod[2] = { { 2, 1, 0 }, { 1, 1, 0 } };
// Захват занятого места и мьютекса
struct sembuf start_cons[2] = { { 1, -1, 0 }, { 2, -1, 0 } };
// Освобождение мьютекса и увеличение свободных
```

```
struct sembuf end_cons[2] = { { 2, 1, 0 }, { 0, 1, 0 } };

void producer(int semid, char **prod_ptr, char *symb)
{
    while (flag)
    {
        // Захват семафоров для производителя
        if (semop(semid, start_prod, 2) == -1)
        {
            printf("semop, %d, errno: %d\n", getpid(), errno);
            exit(1);
        }

        // Критическая секция - производство
        **prod_ptr = *symb;
        printf("producer: %d, %c\n", getpid(), **prod_ptr);
        // ...

        // Освобождение семафоров
        if (semop(semid, end_prod, 2) == -1)
        {
            printf("semop, %d, errno: %d\n", getpid(), errno);
            exit(1);
        }
    }
}

void consumer(int semid, char **cons_ptr)
{
    while (flag)
    {
        // Захват семафоров для потребителя
        if (semop(semid, start_cons, 2) == -1)
        {
            printf("semop, %d, errno: %d\n", getpid(), errno);
            exit(1);
        }

        // Критическая секция - потребление
        printf("consumer: %d, %c\n", getpid(), **cons_ptr);
        // ...

        // Освобождение семафоров
        if (semop(semid, end_cons, 2) == -1)
        {
            printf("semop, %d, errno: %d\n", getpid(), errno);
            exit(1);
        }
    }
}

int main()
{
    ...
}
```

```
// Установка обработчика сигнала для корректного завершения
if (signal(SIGALRM, sig_handler) == (void (*)(int)) -1)
{
    printf("signal, errno: %d\n", errno);
    exit(1);
}

// Создание массива семафоров (3 семафора)
if ((semid = semget(key, 3, IPC_CREAT | perms)) == -1)
{
    printf("semget, errno: %d\n", errno);
    exit(1);
}

// Инициализация семафоров:
if (semctl(semid, 0, SETVAL, n) == -1)    // Свободные места = n
if (semctl(semid, 1, SETVAL, 0) == -1)    // Занятые места = 0
if (semctl(semid, 2, SETVAL, 1) == -1)    // Мьютекс = 1 (свободен)

// Создание дочерних процессов
for (int i = 0; i < 6; ++i)
{
    if ((childs[i] = fork()) == -1) { ... }
    else if (childs[i] == 0)
    {
        if (i % 2 == 0)
            producer(semid, prod_ptr, symb);
        else
            consumer(semid, cons_ptr);
        exit(0);
    }
}

// Ожидание завершения всех процессов
...

// Удаление семафоров
if (semctl(semid, 0, IPC_RMID, NULL) == -1)
{
    printf("semctl, errno: %d\n", errno);
    exit(1);
}

return 0;
}
```

«читатели-писатели»

```
#include <sys/sem.h>
#include <sys/shm.h>
#include <stdio.h>
```

```
#include <unistd.h>
#include <stdlib.h>
#include <signal.h>

// Определение семафоров:
#define WAIT_WRITERS 0 // Ожидющие писатели
#define WAIT_READERS 1 // Ожидющие читатели
#define ACTIVE_WRITER 2 // Активный писатель (0 или 1)
#define ACTIVE_READERS 3 // Количество активных читателей

// Операции для читателя:
struct sembuf start_read[] = {
    // Увеличить счетчик ожидающих читателей
    {WAIT_READERS, 1, 0},
    // Проверить, что нет активного писателя (ожидание = 0)
    {ACTIVE_WRITER, 0, 0},
    // Проверить, что нет ожидающих писателей (ожидание = 0)
    {WAIT_WRITERS, 0, 0},
    // Уменьшить счетчик ожидающих читателей
    {WAIT_READERS, -1, 0},
    // Увеличить счетчик активных читателей
    {ACTIVE_READERS, 1, 0},
};

struct sembuf stop_read[] = {
    // Уменьшить счетчик активных читателей
    {ACTIVE_READERS, -1, 0}
};

// Операции для писателя:
struct sembuf start_write[] = {
    // Увеличить счетчик ожидающих писателей
    {WAIT_WRITERS, 1, 0},
    // Проверить, что нет активных читателей (ожидание = 0)
    {ACTIVE_READERS, 0, 0},
    // Проверить, что нет активного писателя (ожидание = 0)
    {ACTIVE_WRITER, 0, 0},
    // Уменьшить счетчик ожидающих писателей
    {WAIT_WRITERS, -1, 0},
    // Установить активного писателя (увеличить с 0 до 1)
    {ACTIVE_WRITER, 1, 0}
};

struct sembuf stop_write[] = {
    // Сбросить активного писателя (уменьшить с 1 до 0)
    {ACTIVE_WRITER, -1, 0}
};

int flag = 1;

void sig_handler(int signal) {
    flag = 0;
    printf("catch sig: %d\n", signal);
}
```

```
void reader(key_t semfd, char *var) {
    while (flag) {
        // Захват ресурса для чтения
        // (могут читать несколько читателей одновременно)
        if (semop(semfd, start_read, 5) == -1) {
            if (errno == EINTR) {
                printf("errno EINTR: PID %d\n", getpid());
                exit(1);
            }
            printf("semop: %s\n", strerror(errno));
            exit(1);
        }

        // Критическая секция чтения
        printf("reader: %d read %c\n", getpid(), *var);

        // Освобождение ресурса
        if (semop(semfd, stop_read, 1) == -1) {
            printf("semop: %s\n", strerror(errno));
            exit(1);
        }
    }
}

void writer(key_t semfd, char *var) {
    while (flag) {
        // Захват ресурса для записи (только один писатель)
        if (semop(semfd, start_write, 5) == -1) {
            if (errno == EINTR) {
                printf("errno EINTR: PID %d\n", getpid());
                exit(1);
            }
            printf("semop: %s\n", strerror(errno));
            exit(1);
        }

        // Критическая секция записи (эксклюзивный доступ)
        (*var)++;
        if (*var == 'z' + 1)
            *var = 'a';
        printf("writer: %d wrote %c\n", getpid(), *var);

        // Освобождение ресурса
        if (semop(semfd, stop_write, 1) == -1) {
            printf("semop: %s\n", strerror(errno));
            exit(1);
        }
    }
}

int main(void) {
    ...
}
```

```
// Установка обработчика сигнала
if (signal(SIGALRM, sig_handler) == (void (*)(int)) -1) {
    printf("signal: %s\n", strerror(errno));
    exit(1);
}

// Создание массива из 4 семафоров
int semfd = semget(IPC_PRIVATE, 4, IPC_CREAT | perms);
if (semfd == -1) {
    printf("semget: %s\n", strerror(errno));
    exit(1);
}

// Инициализация семафоров:
if (semctl(semfd, 3, SETVAL, 0) == -1) // ACTIVE_READERS = 0
if (semctl(semfd, 2, SETVAL, 0) == -1) // ACTIVE_WRITER = 0
if (semctl(semfd, 0, SETVAL, 0) == -1) // WAIT_WRITERS = 0
if (semctl(semfd, 1, SETVAL, 0) == -1) // WAIT_READERS = 0

// Создание процессов читателей и писателей
for (int i = 0; i < readers + writers; ++i) {
    if ((wchlds[i] = fork()) == -1) {
        printf("fork: %s\n", strerror(errno));
        exit(1);
    }
    if (wchlds[i] == 0) {
        if (i % 2 == 0)
            reader(semfd, buf);
        else
            writer(semfd, buf);
        exit(0);
    }
}

// Ожидание завершения процессов
...

// Удаление семафоров
if (semctl(semfd, IPC_RMID, 0) == -1) {
    printf("semctl: %s\n", strerror(errno));
    exit(1);
}

exit(0);
}
```

9.6.1 Отличия от семафоров

От перечисленных средств семафоры отличаются тем, что у семафора нет хозяина. Мьютекс может освободить только тот процесс, который его захватил; это касается и

других средств взаимного исключения. Семафор же может освободить другой процесс, который, зная идентификатор, может получить дескриптор семафора.

В качестве отдельного средства взаимодействия процессов, рассматриваются сообщения и системные вызовы отправки и приёма сообщений. Сообщение — специальный метод обмена данными, соответственно, используются специальные системные вызовы.

Также, кроме перечисленных ранее способов взаимного исключения (программный, аппаратный, с помощью семафоров, мониторов), существует рандеву.

2. Пересчет динамических приоритетов в ОС и Windows - процессы и потоки: сравнительная характеристика процессов и потоков. Лабораторная работа: многопоточный сервер «калькулятор» (основные действия).

Ответ:

1. Общие принципы

Обе ОС:

- Являются **системами разделения времени** с вытесняющим планированием.
- Пересчет динамических приоритетов применяется **только к пользовательским процессам/потокам**.
- Приоритеты ядра фиксированы (в UNIX — 0–49, в Windows — IRQL).
- Цель: предотвратить **бесконечное откладывание** низкоприоритетных задач.

2. Единица планирования

Критерий	UNIX	Windows
Исторически	Процесс	Поток
Современно	Поток (LWP, 1:1 модель)	Поток
Приоритет у процесса	Есть (унаследованный)	Есть (базовый)
Приоритет у потока	Собственный (pthread)	Относительный (от процесса + сдвиг)

3. Динамический пересчет приоритетов в UNIX

3.1. Основа

- Диапазон приоритетов пользовательских процессов: 50–127 (чем меньше число, тем выше приоритет).
- Факторы влияния:
 1. `p_cpu` — загрузка CPU (растет на каждом тике, макс. 127);
 2. `p_nice` — «любезность» (0–39, умолч. 20);
 3. Время простоя (через фактор полураспада).

3.2. Алгоритм

- Каждую секунду вызывается `schedcpu()` :
 - `p_cpu` уменьшается по формуле:

$$decay = \frac{2 \cdot load_average}{2 \cdot load_average + 1}$$

- Пересчет приоритета режима задачи:

$$p_usrpri = PUSER + \frac{p_cpu}{2} + 2 \cdot p_nice$$

где `PUSER = 50` .

3.3. Особенности

- Приоритет связан с событием (приоритет сна) — при пробуждении процесс получает приоритет события.
- После системного вызова приоритет сбрасывается в `p_usrpri` .
- Чем дольше процесс ждет — тем выше его приоритет.



4. Динамический пересчет приоритетов в Windows

4.1. Основа

- Диапазон приоритетов потоков: 0–31.
 - 0–15 — динамический диапазон;
 - 16–31 — реального времени (статичны);
 - 0 — зарезервирован (обнуление страниц).
- Приоритет процесса = класс приоритета (Real-time, High, Above Normal, Normal, Below Normal, Idle).
- Приоритет потока = базовый приоритет процесса + относительный сдвиг (Time-critical ... Idle).

4.2. Причины повышения приоритета

1. Владелец блокировки (при освобождении ресурса);
2. Завершение ввода-вывода (приращение зависит от устройства: диск = 1, сеть = 2, мышь/клавиатура = 6, звук = 8);
3. Ввод из пользовательского интерфейса;
4. Длительное ожидание ресурса;
5. Ожидание объекта ядра;
6. Диспетчер настройки баланса:
 - Если поток ждет ≈ 4 секунды — повышение до **15** (макс. динамического уровня);
 - После отработки кванта — возврат к базовому.
7. MMCSS (мультимедиа):
 - Категории: High (23–26), Medium (16–22), Low (8–15), Exhausted (1–7);
 - Временное повышение, затем снижение до Exhausted.



5. Сравнительная таблица: процессы и потоки

Характеристика	UNIX	Windows
Объект планирования	Поток (ядро 1:1)	Поток
Приоритет процесса	Хранится в <code>p_usrpri</code>	Класс приоритета (6 уровней)
Приоритет потока	<code>p_pri</code> (текущий), <code>p_usrpri</code> (при возврате в user)	Базовый (процесс) + сдвиг
Динамический диапазон	50–127	1–15
Пересчет по времени	Каждую секунду (фактор полураспада)	При событиях (I/O, ожидание, баланс)
Факторы пересчета	<code>p_cpu</code> , <code>p_nice</code> , <code>load_average</code>	I/O, UI, блокировки, ММ, балансировка
Приоритет сна	Есть (0–49)	Нет (в IRQL)
Влияние пользователя	<code>nice</code>	Изменение класса/сдвига (ограничено)
Мультимедиа	Через приоритеты реального времени	MMCSS (категории)
Процессы ядра	Фиксированные приоритеты (0–49)	IRQL (0–31)

calculator_client.c

```
const char ops[] = { '+', '-', '*', '/' };
const int NUM_OPS = 4;
void calculator_prog_1_auto(char *host)
{
    CLIENT *clnt;
    float *result_1;
    struct CALCULATOR args;
    unsigned long request_id = 0;
    srand(time(NULL));
    clnt = clnt_create(host, CALCULATOR_PROG, CALCULATOR_VER, "tcp");
    if (clnt == NULL) {
        clnt_pcreateerror(host);
        exit(1);
    }
    printf("Подключено к %s\n", host);
    while (1) {
        float a = (rand() % 201) - 100.0f;
```

```

float b = (rand() % 201) - 100.0f;
if (b == 0.0f) b = 1.0f;
int op_index = rand() % NUM_OPS;
char op_char = ops[op_index];
args.arg1 = a;
args.arg2 = b;
switch(op_char) {
    case '+': args.op = ADD; break;
    case '-': args.op = SUB; break;
    case '*': args.op = MUL; break;
    case '/': args.op = DIV; break;
}
result_1 = calculator_proc_1(&args, clnt);
if (result_1 == NULL) {
    clnt_perror(clnt, "RPC вызов не удался");
    continue;
}
request_id++;
printf("3anpoc %lu: %7.2f %c %7.2f = %8.2f\n", request_id, a, op_char, b, *result_1);
}
clnt_destroy(clnt);
}

int main(int argc, char *argv[])
{
    if (argc < 2) {
        printf("usage: %s server_host\n", argv[0]);
        exit(1);
    }
    calculator_prog_1_auto(argv[1]);
    return 0;
}

```

calculator_server.c

```

float *
calculator_proc_1_svc(struct CALCULATOR *argp, struct svc_req *rqstp)
{
    static __thread float result;

    (void)rqstp;
    switch (argp->op) {
    case ADD:
        result = argp->arg1 + argp->arg2;
        break;
    case SUB:
        result = argp->arg1 - argp->arg2;
        break;
    case MUL:
        result = argp->arg1 * argp->arg2;
        break;
    case DIV:

```

```
        result = (argp->arg2 != 0.0f) ? (argp->arg1 / argp->arg2) : 0.0f;
        break;
    default:
        result = 0.0f;
        break;
    }
    return &result;
}
```

Экзаменационный билет 21

1. Тупики: типы ресурсов в теории тупиков, четыре условия возникновения тупика, стратегии борьбы с тупиками. Обход тупиков алгоритм Э. Дейкстры "Алгоритм банкира» (недостатки) и алгоритм Хабермана с примером предварительного анализа запросов состояния системы относительно тупика

Ответ:

Определение 10.5 (Тупик / тупиковая ситуация)

Тупик или тупиковая ситуация — ситуация, которая возникает в результате монопольного использования разделяемых ресурсов, когда процесс, владея ресурсом, запрашивает другой ресурс, занятый непосредственно или через цепочку запросов другим процессом, ожидающим освобождение ресурса, занятого первым процессом.



С точки зрения тупиков ресурсы делятся на два типа:

1) Повторно используемые ресурсы

- 1.1) Аппаратная составляющая компьютера (процессоры, физическая память, устройства ввода/вывода)
- 1.2) Реентерабельные коды ОС
- 1.3) Объекты ядра (семафоры, программные каналы, разделяемая память, очереди сообщений)

Получение процессом такого ресурса никак не влияет на этот ресурс. В результате использования его характеристики не меняются, в систему он возвращается в том же виде, в котором был получен.

Таких ресурсов в системе очень много, причём они разнообразные.

Количество ресурсов ограничено, их количество постоянно для некоторого периода взаимодействия процессов (например, для семафоров).

2) Потребляемые

- 2.1) Сообщения

В процессе получения эти ресурсы перестают существовать.

Наибольшую проблему представляет в распределённых системах, когда может существовать большое количество процессов, создающих сообщения и принимающих их.

Любой процесс может создать и потребить любое количество сообщений

10.13.1 Условия возникновения тупика

Была предложена стратегия Хейвендера, в которой сначала формулируется 4 условия возникновения тупика:

1) Взаимоисключение (mutual exclusion)

Процессы монопольно используют предоставляемые им ресурсы. Монопольное использование предполагает взаимодействие: если один процесс получил в своё распоряжение ресурс, то другой процесс получить этот же ресурс не может, пока первый процесс его не освободит.

2) Ожидание (hold and wait)

Процессы удерживают занятые ими ресурсы, ожидая получения дополнительных ресурсов, чтобы продолжить своё выполнение.

3) Не перераспределяемость (no preemption)

Ресурсы нельзя отобрать у процессов до их завершения или освобождения этих ресурсов самим процессом.

4) Круговое ожидание (circular wait)

То, что видели на графе, это и есть круговое ожидание. Существует цепь процессов, в которой каждый процесс занимает необходимый ему ресурс для дальнейшего выполнения.

Часть из этих условий является необходимыми, часть – достаточными.

10.13.2 Методы борьбы с тупиками

1) Предотвращение тупиков (исключение тупиков)

Необходимо создать в системе такие условия, при которых возникновение тупиков невозможно.

2) Обход тупиков (недопущение тупиков)

82

Тупики теоретически возможны (для этого есть условия), но система выполняет такие действия, чтобы тупик не возник. Выполняемые действия позволяют обойти тупиковую ситуацию.

У этого метода есть классический способ обхода, который был предложен Дейкстрой — алгоритм банкира (раздел 10.13.4).

3) Обнаружение тупиков и восстановление работоспособности системы

Обнаружение всегда требует восстановления работоспособности системы попавшей в тупиковую ситуацию.

10.13.4 Алгоритм банкира

Дейкстра усмотрел в этом методе действия, схожие с действием банкира. Банкир имеет какой-то капитал. К нему обращаются заёмщики, а банкир, выдав им сумма, рассчитывает на возвращение выданных денег. Часто заёмщик, чтобы вернуть взятые в долг деньги, нуждается в дополнительной сумме. Банкир должен проанализировать, кому из заёмщиков он может выделить дополнительные деньги, учитывая возможность возвращения выделенных сумм.

В качестве банкира выступает менеджер ресурсов.

Заёмщики — процессы, которые делают заявки на ресурсы. Чтобы менеджер мог проанализировать ситуацию, необходимо, чтобы процессы в заявках указывали свою максимальную потребность в каждом из ресурсов (максимальное количество единиц каждого ресурса). При этом процесс не может запросить единиц ресурса больше, чем указано в его заявке.

Анализируя возникающую в системе ситуацию, менеджер ресурсов на основе заявок процессов может принять решения, какому процессу выделять ресурсы. В такой системе число процессов фиксировано, число ресурсов фиксировано и это позволит менеджеру ресурсов принять решение.

Процессы	Текущее распределение	Свободные единицы ресурса	Заявка
P1	4		12
P2	2		4
P3	4		8
	$\sum 10$	2	

Анализируя текущую ситуацию в системе, можно сделать вывод, что процесс P2 может гарантировано завершиться, так как он уже получил 2 единицы ресурса, его максимальная потребность составляет 4 единицы, то есть он может запросить только 2 единицы, и у системы эти единицы имеются.

Завершившись, процесс P2 вернёт системе занимаемые им ресурсы, и в сумме со свободными единицами система сможет удовлетворить максимальную потребность в ресурсах процесса P3, который также завершится и освободит занимаемые им ресурсы, которые в сумме со свободными единицами позволят системе удовлетворить максимальную потребность в единицах ресурса процесса P1.

В результате мы имеем последовательность процессов, которая может успешно завершиться. Такое состояние в системе называется **безопасным относительно тупика**, то есть существует последовательность процессов, каждый из которых может успешно завершиться, освобождая занимаемые им ресурсы, что позволит удовлетворить потребность других процессов в ресурсах.

Процессы	Текущее распределение	Свободные единицы ресурса	Заявка
P1	5		12
P2	2		4
P3	4		8
	$\sum 11$	1	

Допустим, в системе возникла ситуация, когда процесс P1 получил 5 единиц и у системы осталась одна свободная единица ресурсов. Это состояние является **небезопасным**. Мы не можем найти последовательность процессов, которые гарантированно будут завершаться. Однако данное небезопасное состояние необя-

зательно приведёт к тупику, так как процессы могут не запросить ранее заявленное максимальное количество ресурсов.

С другой стороны, безопасное относительно тупика необязательно приведёт так же при выделении ресурса к следующему безопасному состоянию. Если состояние системы безопасно относительно тупика, то следующее состояние может быть небезопасным.

Алгоритм Дейкстры (сам непосредственно)

{Формализация алгоритма} Формально, состояние является безопасным, если существует последовательность процессов такая, что:

- 1) Первый (не по номеру) процесс в найденной последовательности обязательно завершится, так как даже если он запросит максимальное заявленное количество единиц ресурса, у системы имеется необходимое количество свободных единиц для удовлетворения запроса.
- 2) Второй процесс в последовательности может завершиться, если завершился первый и вернул системе все занятые им единицы ресурса, что в сумме со свободными единицами позволит удовлетворить максимальную потребность в единицах ресурса второго процесса в последовательности. И так далее...
- 3) i -й процесс в последовательности может завершиться, если все предыдущие $i - 1$ процессы успешно завершились и сумма освобождённых единиц ресурса сможет удовлетворить максимальную потребность i -го процесса в последовательности.

Таким образом, всякий раз, когда процесс делает новый запрос, менеджер ресурсов должен найти последовательность процессов, которая может завершиться, то есть где каждый процесс может успешно завершиться. Только в этом случае запрос процесса может быть удовлетворён. В результате проведения такого анализа можно гарантировать, что система не попадёт в тупик. Но для этого необходимо исследовать $N!$ последовательностей.

В связи с этим алгоритм банкира имеет теоретический, а не практический интерес. Он показывает, что действительно можно обойти тупик, но затраты колоссальные.

Самый популярный — алгоритм банкира. Самый известный аппроксимирующий его алгоритм — алгоритм Хабермана.

10.13.6 Алгоритм Хабермана

Рассмотрим алгоритм Хабермана. И в том, и в другом случае речь идёт о матрицах. Только на основе представления системы в виде матриц, можно выполнить определённые действия. В данном случае алгоритм выполняется вручную. Его можно реализовать программно, поскольку формализованы все необходимые действия.

Выполнение необходимых действий начинается с заданного состояния системы. При этом имитируется выполнение каждого процесса в плане запросов процессов на необходимое ему количество ресурсов. При этом процесс запрашивает максимальное количество ресурсов, которое ему необходимо. Если в результате анализа все процессы успешно завершаются, то состояние системы называется безопасным. Анализируется N процессов и M типов ресурсов. Для решения задачи необходимо иметь матрицу свободных (доступных) ресурсов.

90

Такая матрица называется `avail_matrix`. Обозначим её как $A = (a_1, a_2, \dots, a_n)$. В этом векторе содержатся свободные единицы каждого типа свободных ресурсов.

Определение 10.9 (Матрица заявок)

Матрица заявок — матрица, в которой указана максимальная потребность каждого процесса в каждом типе ресурса (*claim_matrix*).

$$P = \begin{pmatrix} b_{11} & \dots & b_{1m} \\ \vdots & & \vdots \\ b_{n1} & \dots & b_{nm} \end{pmatrix} = \begin{pmatrix} B_1 \\ B_2 \\ \vdots \\ B_n \end{pmatrix}$$

Определение 10.10 (Матрица распределений)

Матрица распределений (*allocation_matrix*) —

$$C = \begin{pmatrix} C_{11} & \dots & C_{1m} \\ \vdots & & \vdots \\ C_{n1} & \dots & C_{nm} \end{pmatrix} = \begin{pmatrix} C_1 \\ \vdots \\ C_n \end{pmatrix}$$

c_{ij} — количество ресурсов r_j , выделенное процессу p_i .

При этом

- 1) все k для которых $B_k \leq A$, могут быть удовлетворены, но главное, что процесс не может требовать количество единиц ресурса больше, чем имеется в системе или больше, чем доступное количество;
- 2) $C \leq B$ — указывает на то, что процесс не может запросить больше ресурсов, чем указано в его заявке;
- 3) $\sum_{k=1}^n$ — в системе невозможно выделить больше ресурсов, чем имеется у системы;
- 4) матрица доступных ресурсов может быть сформирована следующим образом:

$$D = (d_1 \dots d_m) = A - \sum_{k=1}^n C_k$$

У системы имеется количество единиц ресурсов, которое представляется вектором A , но чтобы анализировать текущую ситуацию, мы должны определить, сколько в текущий момент в системе имеется свободных единицы каждого ресурса. Для этого мы выполняем вычитание и для каждого ресурса суммируем уже выделенное количество ресурсов. Мы получаем матрицу свободных единиц ресурсов в текущий момент времени.

5) Матрица возможных запросов:

91

$$E = \begin{pmatrix} e_{11} & \dots & e_{1m} \\ \vdots & & \vdots \\ e_{n1} & \dots & e_{nm} \end{pmatrix} = B - C = \begin{pmatrix} E_1 \\ \vdots \\ E_n \end{pmatrix}$$

Из заявки вычитаем полученные единицы, в результате формируем матрицу возможных запросов

6) Матрица текущих запросов (Current Request Matrix)

$$F = \begin{pmatrix} f_{11} & \dots & f_{1m} \\ \vdots & & \vdots \\ f_{n1} & \dots & f_{nm} \end{pmatrix} = \begin{pmatrix} F_1 \\ \vdots \\ F_n \end{pmatrix}$$

7) Можно определить предварительное состояние. Предположим, что удовлетворяются все запросы и проведём анализ ситуации

$$D \leftarrow \underbrace{D}_{\text{Доступно}} - \underbrace{F_i}_{\text{Запросы}}$$

D доступно, F — запросы. В результате запросов D изменится, то есть мы заранее анализируем запросы. C_i так же будет изменено, если предположить, что запросы будут удовлетворены, то есть

$$C_i \leftarrow C_i + F_i$$

E_i также изменится, т.к. что мы предполагаем, что запросы были удовлетворены и количество единиц ресурса, которое в последствии процесс может запросить, изменится

$$E_i \leftarrow E_i - F_i$$

Пример относительно 1 типа ресурса

Процессы	Текущее распределение	Свободные единицы ресурса	Заявка
P1	4		12
P2	2		4
P3	4		8
-	-	2	-

- Показанное распределение безопасно, так как есть последовательность процессов (P2, P3, P1), которая может завершиться
- P2 не может запросить больше 2 единиц ресурса, так как он уже удерживает 2 единицы, а в заявке указано 4 единицы, и в системе есть 2 свободные единицы ресурса =>

- P2 завершится, освободит занимаемые им единицы ресурса, и вместе со свободным ресурсом это сможет удовлетворить максимальную потребность P3, так как максимальная потребность $P3 = 8$, то есть P3 может запросить не более 4 единиц, которые будут у системы
- Завершившись, P3 вернет в пул ресурсов ресурсы, и сможет удовлетворить запрос P1, которому нужно не более 8 единиц ресурса.
- Но безопасное состояние не всегда абсолютно.
Безопасное состояние может перейти в небезопасное относительно тупика, если процессу P1 выделена 1 единица ресурса (т.о. их всего 5), а у системы останется 1 свободная единица ресурса.

Процессы	Текущее распределение	Свободные единицы ресурса	Заявка
P1	5		12
P2	2		4
P3	4		8
-	-	1	-

- Нет необходимой последовательности
- Небезопасное состояние относительно тупика не всегда приведет к тупику, т.к. выполняющийся процесс может не запросить максимальное количество ресурсов
- Состояние системы является безопасным, если существует последовательность процессов, такая что:
 - первый процесс в найденной последовательности обязательно завершится, так как если он запросит даже максимальное заявленное число единиц ресурса, у системы имеется необходимое количество свободных единиц ресурса для удовлетворения запроса
 - второй процесс последовательности может завершиться, если завершился первый процесс и вернул ОС все занимаемые им единицы ресурса, что в сумме со свободными единицами

ресурса позволит удовлетворить максимальную потребность второго процесса

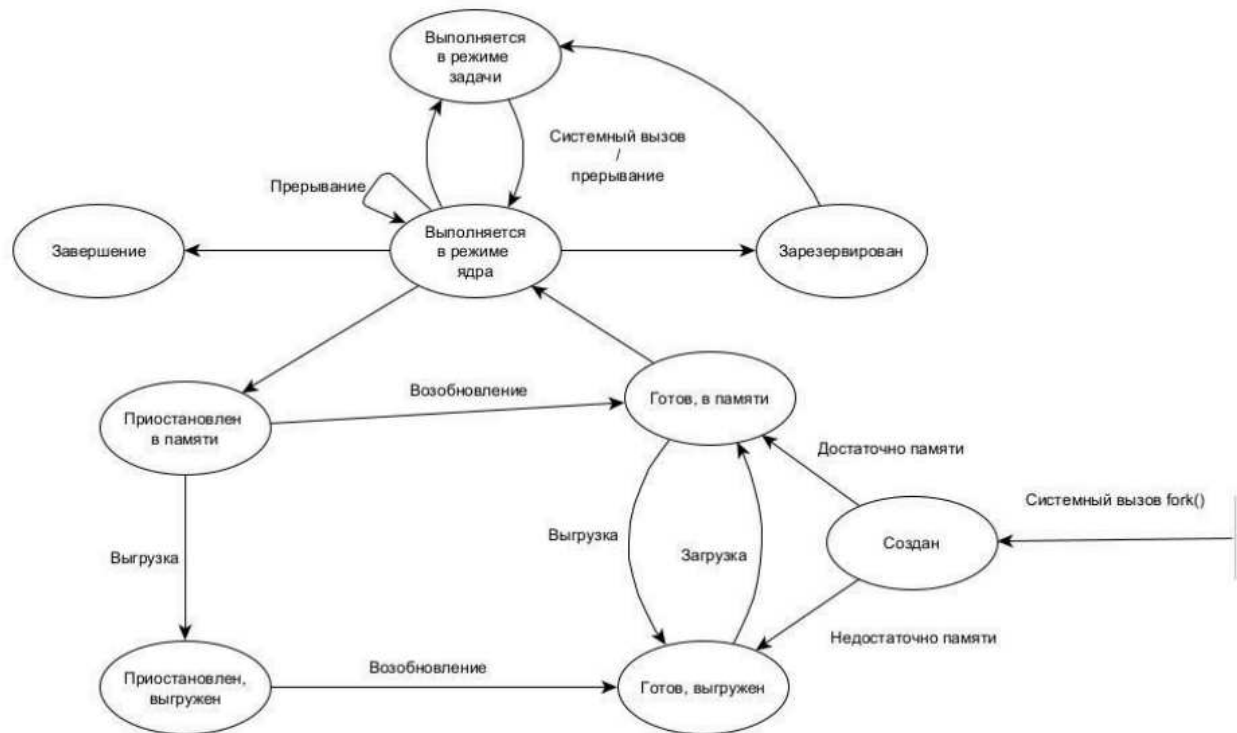
- i процесс в последовательности может завершиться, если все предыдущие $i-1$ процессы успешно завершатся, и вернут системе занимаемые им единицы ресурса, что позволит удовлетворить потребность i в ресурсе.
- Для определения состояния системы относительно тупика необходимо исследовать $n!$ последовательностей, прежде чем состояние системы будет отнесено к безопасному или небезопасному
=> алгоритм банкира имеет **неоспоримое(показал суть решения)** теоретическое значение, но его практическое применение затруднено в силу выполнения $n!$ проверок. Также очень много ограничений. Однако фактически задача обойти тупик решена.
- Так как существуют системы, которые не могут попадать в тупики, алгоритм Дейкстры привел к ряду аппроксимаций. Одна из аппроксимаций — алгоритм Хаберманна (Габерманна).

2. Процессы Unix создание процесса в ОС Unix и запуск новой программы: `fork()`, `exec()`, `wait()`, взаимодействие процессов через программные каналы: именованные и неименованные. Сигналы System V назначение, реакция процесса из сигнала. Примеры из лабораторной работы (коды). Сигналы в лабораторной "Демон". Обработчики сигналов в процессах-родственника

Ответ:

Процессы UNIX

- Диаграмма состояния процессов в UNIX



- Каждый процесс в UNIX проходит через состояние “зомби”

`fork()` --- системный вызов, который создаёт новый процесс --- процесс-потомок, для которого создаётся собственное адресное пространство, т. е. создаются собственные карты трансляции АДРЕСОВ, дескрипторы СТРАНИЦ ТАБЛИЦ СТРАНИЦ потомка ссылаются на СТРАНИЦЫ адресного пространства предка И для страниц адресного пространства предка права доступа МЕНЯЮТСЯ на `only-read` и УСТАНОВЛИВАЕТСЯ флаг `copy-on-write` И если или предок, или потомок попытаются изменить какую-либо страницу, возникнет исключение по правам доступа И операционная система, обрабатывая это исключение, ОБНАРУЖИТ флаг `copy-on-write` и создаст копию страницы в адресном пространстве того процесса, который пытался её изменить И такая ситуация существует до тех пор, пока процесс-потомок не вызовет `exec()` или `exit()` (тут именно `exit()`, а не завершение процесса) И в результате страницам адресного пространства предка возвращаются обычные права доступа и флаг `copy-on-write` сбрасывается.

`exec()` -- системный вызов, который переводит процесс на адресное пространство программы, которАЯ была передана ему в качества параметра

1. разбирает путь к файлу и осуществляет доступ к нему (для того чтобы проверить, существует ли этот файл)
2. проверяет, имеются ли полномочия у файла. проверяет, что он исполняемого проверяет у вызывающего процесса на выполнение исполняемого файла (выполняться может только исполняемый файл) (проверяется, что файл исполняемый, чтением заголовка файла)
3. освобождает ресурсы, которые были выделены при вызове `fork()` до вызова `exec()` (так как `fork` создал для процесса-потомка адресное пространство (таблицы страниц) и была выделена область свопинга)
4. создаются новые таблицы страниц (для процесса-потомка) (для программы, которая передаётся `exec` в качестве параметра)
5. заменяет адреса старых таблиц страниц на адрес новых таблиц страниц в дескрипторе процесса, в `task_struct` указатель `mm`
6. меняет аппаратный контекст: все регистры сбрасываются, а в счётчик команд записывается адрес точки входа (в переданную программу) (в регистр CR3 (в процессорах Intel) необходимо загрузить новый адрес корневой таблицы страниц)

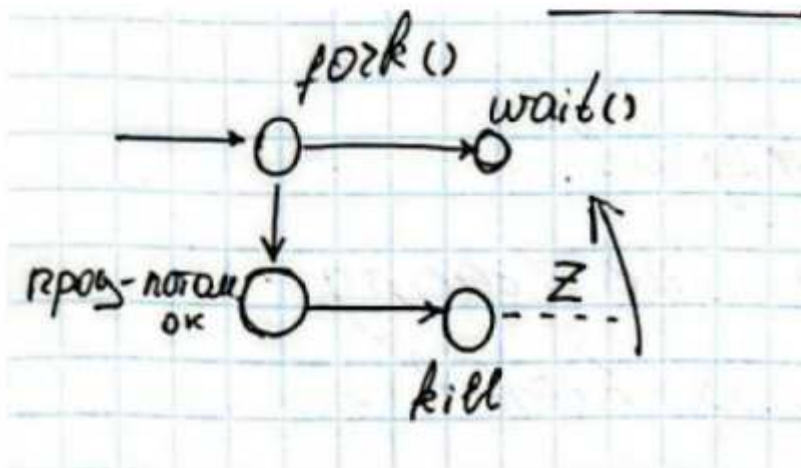
Чтобы процессы-предки не завершались раньше процессов-потомков, в UNIX имеется системный вызов `wait()`.

- **wait(&status)** — блокирует процесс до завершения процесса-потомка.
Система не контролирует, вызывал ли процесс wait.
- **waitpid(childpid, &status, WUNTRACED)**
- Смысл wait — получить статус завершения потомка, проанализировать его и в зависимости от него ветвиться

```

if (WIFEXITED(status))
{
    printf("exited, status=%d\n", WEXITSTATUS(status));
}
else if (WIFSIGNALED(status))
{
    printf("killed by signal %d\n", WTERMSIG(status));
}
else if (WIFSTOPPED(status))
{
    printf("stopped by signal %d\n", WSTOPSIG(status));
}

```



- Итог: процесс не может получить статус завершения и заблокируется навсегда, что плохо, так как занимает ресурсы
=> в Unix введено состояние зомби
- Зомби — процесс, у которого отобраны все ресурсы кроме последнего (дескриптора, чтобы предок мог получить статус завершения потомка и продолжить выполнение)

- Что такое неименованный программный канал -- буфер, который находится в области данных ядра
- Какие процессы могут общаться по неименованным программным каналам? Родственные
- Почему? Потому что при вызове `fork()` процесс-потомок наследует дескрипторы открытых файлов процесса-предка

Программные каналы (pipe)

Linux так же, как Unix поддерживает программные каналы двух типов: именованные и неименованные. **Именованные программные каналы** создаются командой **mknod**. Программный канал - это специальный файл, в который можно «писать» информацию и из которого эту информацию можно «читать». Причем порядок записи информации и последующего чтения – FIFO (очередь).

Именованный канал имеет имя, которое указывается при вызове команды `mknode`:

#mknod [опции] <имя> p

Использовать именованный канал может любой процесс, «знающий» имя канала.

Сигнал — это программное средство информирования процесса о событиях, которые могут произойти как внутри процесса, так и вне его (то есть в системе). Информация хранится в `signal.h`

SIGHUP	1	Разрыв связи с терминалом
SIGINT	2	Завершение выполнения процесса, посылается при нажатии Ctrl+C
SIGKILL	9	Системный вызов kill
SIGSEGV	11	Выход процесса за его адресное пространство
SIGPIPE	13	При записи в Pipe, когда нет читающего процесса. Информирует о недопустимой операции
SIGALARM	14	Прерывание от системного таймера
SIGTERM	15	Выполнение команды kill в командном режиме. К адресуемому PID
SIGUSR1	16	При системном вызове kill. К адресуемому PID
SIGUSR2	17	При системном вызове kill. К адресуемому PID
SIGCLD	18	Информирование процесса о завершении процесса-потомка
SIGPWR	19	Падение напряжения. Система переходит к выполнению необходимых действий для сохранения необходимых значений
SIGPOOL	22	Событие в stream-устройстве

- Два макроса
 - SIG_DFL
 - SIG_IGN

```
int kill(int pid, int sig)
```

- `pid` в `kill` — не обязательно `id` процесса.
Если это `Id` процесса, то сигнал получит конкретный процесс.
Если `pid <= 1`, то сигнал будет послан группе процессов:
Если `pid` равен 0, то она отправляется каждому процессу в `process`

`group` вызывающего процесса.

<https://man7.org/linux/man-pages/man2/kill.2.html>

- Процессы в UNIX определяются в группы по разным критериям.
Например, терминальная группа процессов
- Родитель является лидером группы, создает группу процессов.
- `kill(getpid(), SIGALARM);` — сигнал будет послан процессу, который вызвал `kill`.

```
void *signal(int sig, void (*handk(int)))
```

- `handle` – стандартное название обработчика
- Реакцией процесса, вызвавшего `signal()` с аргументом `handl`, на сигнал `sig` будет вызов этого обработчика
То есть с помощью этого системного вызова можно на сигнал установить собственный обработчик.
- Системный вызов `signal` не входит в POSIX 1
- POSIX 1 перечисляет около тысячи системных вызовов, которые должна поддерживать операционная система, чтобы выполнить переносимость ПО.
Это не единственный стандарт. Наиболее интересны стандарты, связанные с аппаратным ПО: X/Open, XPG4
- Системный вызов `signal` возвращает указатель на предыдущий обработчик данного сигнала
 - => его можно использовать для восстановления обработчика
- Сигналы возникают в ответ на внешние к процессу события
- Процесс будет реагировать на внешние асинхронные события.
Внешние асинхронные события сопровождаются сигналами, которые получает процесс. Процесс может поменять ход своего выполнения.
 - Асинхронно — независимо от того, что делает процесс:
Пример: процесс что-то делает, нажимаем `ctrl+c`. Независимо от того, что делает процесс, он меняет ход выполнения.
- Одни и те же сигналы могут получать процессы одной группы
- Процесс, вызвавший `fork()`, создает свою группу

```
sigset_t mask;

void *thr_fn(void *arg) {
    int signo;

    for (;;) {
        // Ожидание поступления сигнала
        if (sigwait(&mask, &signo) != 0) {
            syslog(LOG_ERR, "sigwait error ");
            exit(1);
        }

        // Обработка полученных сигналов
```

```
        switch (signo) {
        case SIGHUP:
            syslog(LOG_INFO, "signal SIGHUP ");
            // Перезапуск конфигурации и т.п.
            break;
        case SIGTERM:
            syslog(LOG_INFO, "signal SIGTERM ");
            pthread_exit(0); // Завершение потока и демона
            break;
        default:
            syslog(LOG_INFO, "received signal: %d\n", signo);
        }
    }
    return 0;
}

void daemonize(const char *cmd) {
    struct sigaction sa;

    // Игнорирование SIGHUP при создании демона
    sa.sa_handler = SIG_IGN;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    if (sigaction(SIGHUP, &sa, NULL) < 0) {
        err_quit("%s: sigaction error ", cmd);
    }
    // ... остальной код демонизации
}

int main(void) {
    int err;
    pthread_t tid;
    struct sigaction sa;

    // Переход в режим демона
    daemonize(DAEMONNAME);

    // Восстановление действия по умолчанию для SIGHUP
    sa.sa_handler = SIG_DFL;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    if (sigaction(SIGHUP, &sa, NULL) < 0) {
        err_quit("%s: sigaction error");
    }

    // Блокировка всех сигналов в главном потоке
    sigfillset(&mask);
    if ((err = pthread_sigmask(SIG_BLOCK, &mask, NULL)) != 0) {
        err_exit(err, "pthread_sigmask error ");
    }

    // Создание отдельного потока для обработки сигналов
    err = pthread_create(&tid, NULL, thr_fn, 0);
    if (err != 0) {
```

```
    err_exit(err, "pthread_create error ");
}

// Основная работа демона
while (1) {
    sleep(10);
    // ... работа демона
}

return 0;
}
```

Правила программирования демона

1) Вызывается функция `umask(0)`

- a) сбрасывает маску режима создания файла
- b) `main` – предок, в котором вызовем функцию `daemonise`
- c) у него предком является `shell`
- d) потомок наследует сигнальную маску и обработчики сигналов, дескрипторы открытых файлов и маску режимов создания файлов.
- e) сбрасываем маску режима создания файлов, чтобы демон мог создавать файлы с любыми правами доступа.
Сигнальную маску не сбрасываем

Экзаменационный билет 22

1. Процессы: взаимодействие параллельных процессов - монопольный доступ и взаимоисключение; программная и аппаратная реализации взаимоисключения командой test-and-set, спин-блокировка - реализация и активное ожидание, семафоры Дейкстры Э.: определение, взаимоисключение с помощью семафоров, алгоритм «Производство-потребление» - решение Дейкстры (псевдокод), сравнение семафоров и мьютексов.

Ответ:

Процесс — это программа в стадии выполнения.

Проблемы многопоточного программирования. Одна из проблем разделение данных (Data Splitting):

- Имеется в виду работа потока с одними и теми же данными, то есть обращение к одним и тем же данным. Это требует специальных подходов, чтобы не терять данные, то есть не формировать ошибочные результаты.
- Это называется «взаимоисключение» или «монопольный доступ к данным»

Монопольный доступ и синхронизация – разные вещи!

Методы взаимоисключения:

1. Программный способ
2. Аппаратный способ
3. С помощью семафоров
4. С помощью монитора

8.5 Аппаратный способ взаимного исключения

8.5.1 Команда `test_and_set`

Начиная с машины IBM370 (с операционной системы, с системы команд), во всех архитектурах присутствует аппаратная команда `test_and_set(a,b);`.

- Это неделимая команда, её выполнение прервать нельзя.
- Она как правило (в архитектурах различных могут быть нюансы) читает значение логической переменной `b` и копирует его в `a`, затем устанавливает `b = истина`.

```
1 P1: while (1)
2   {
3     C1 = 1;
4     while (C1)
5       test_and_set(C1, fl);
6     CR1;
7     fl = 0;
8     PR1;
9   }
10 P2: while (1)
11   {
12     C2 = 1;
13     while (C2)
14       test_and_set(C2, fl);
15     CR2;
16     fl = 0;
17     PR2;
18   }
19 fl = 0;
20 pasbegin
21   P1, P2'
22 pasend
```

- Логическая переменная `fl` имеет значение «истина», когда любой из процессов находится в своём критическом участке, иначе ложь.
- Рассмотрим когда `P1` хочет войти в свой критический участок, а `P2` уже находится в своем крит. участке

- 1) P1 устанавливает в C1 единицу и входит в цикл проверки переменной fl командой test_and_set
- 2) Так как P2 находится в критическом участке, то значение fl = 1
- 3) Команда test_and_set обнаруживает этот факт и устанавливает в C1 единицу
- 4) Таким образом P1 находится в цикле активного ожидания, пока P2 не выйдет из своего критического участка.
- 5) Когда процесс выходит из критического участка и устанавливает в fl = false то скорее всего другой процесс сможет перехватить инициативу, установив в fl true

— Данный способ практически исключает захват критической секции одним и тем же процессом.

test_and_set лежит в основе spin_lock и семафоров.

8.5.2 Spin lock

test_and_set есть в макрокоманде spin_lock, которые активно используются в ядрах ОС, в ядрах Linux и Windows.

test_and_set используются для захвата в ядре критических секций, некоторые части кода ядра не могут блокироваться, на них можно устанавливать только спинлоки, это активное ожидание на процессоре. (ЭТО ПРИМЕЧАНИЕ ДЛЯ ВЕРСИИ ПРОСТО C while (test_and_set(*c) != 0))

Означает, что процессорное время используется неэффективно, плохо для системы.

Вот такой вариант лучше:

```

1 void spin_lock(spin_lock_t *c)
2 {
3     while (test_and_set(*c) != 0)
4         /* ресурс занят */
5         while (*c != 0);
6 }
7
8 void spin_unlock(spin_lock_t *c)
9 {
10    *c = 0;
11 }

```

Листинг 6.

- test_and_set приводит к занятию шины одним процессом или потоком и, следовательно, к снижению отзывчивости системы.
- Если ресурс занят, то дальше выполняется простая проверка c, и её можно прервать. То есть захват на длительное время локальной шины памяти не выполняется.

9 Семафоры Дейкстры

9.1 Определение семафора

Dijkstra EW.

Работа, связанная с семафорами и взаимным исключением была опубликована в 1965 году. Уже в 1965 году решение проблем взаимодействия параллельных процессов назрело, стало очень важным.

Определение 9.1 (Семафор)

Семафор — это неотрицательная защищённая переменная, на которой определены 2 операции:

- 1) P — Первая буква нидерландского слова «Proberen» (пытаться) — декремент, захват семафора. Можно выполнить только если $S > 0$. Иначе процесс блокируется в ожидании освобождения семафора.

$$S = S - 1 \quad S > 0$$

- 2) V — Первая буква нидерландского слова «Verhogen» (поднимать выше) — инкремент. Разблокирует процесс, ожидающий на семафоре.

$$S = S + 1$$

S — Semaphore.

Семафоры бывают:

- 1) Бинарные (0 и 1), определены только две неделимые операции (P и V)
- 2) Считающие (целые положительные значения и 0)

9.2 Реализация семафоров

В своих работах Дейкстра обозначил именно этими буквами операции на семафоре.

Изменять семафор можно только двумя неделимыми операциями.

Дейкстра определил это, чтобы уйти от активного ожидания на процессоре, чтобы не тратить процессорное время на проверку флагов. Но за всё надо платить. От активного ожидания удалось избавиться, но за счёт того, что операции P и S являются системными вызовами и переводят систему в режим ядра. Только ядро может заблокировать и разблокировать процесс. Переход в режим ядра несёт расходы, связанные с переключением программного контекста.

Изменение положения нескольких семафоров при этом современные приложения как правило многопоточные, но даже в те времена выполнялось распараллеливание действий. Изменение в таких приложениях набора нескольких семафоров между собой не связанных могло привести к тупику.

9.3 Взаимоисключение с помощью семафоров

1	S: Semaphore
2	P1:
3	P(S)
4	CR1
5	V(S)
6	PR1
7	P2:
8	P(S)
9	CR2
10	V(S)
11	PR2

Листинг 7. Взаимоисключение с помощью семафоров

P2 получил инициативу, выполнил декремент, P1 получив инициативу, будет блокирован, пока P2 не инкрементирует S. Это взаимодействие с использованием бинарных семафоров.

1	S1, S2: Semaphore
2	P1:
3	P(S1)
4	P(S2)
5	CR1
6	P(S2) // как будто должно быть V(S2)
7	P(S1) // как будто должно быть V(S1)
8	PR1
9	P2:
10	P(S2)
11	P(S1)
12	CR2
13	P(S1) // как будто должно быть V(S1)
14	P(S2) // как будто должно быть V(S2)
15	PR2

Листинг 8. Пример взаимной блокировки процессов (deadlock)

Процесс изменяет несколько семафоров. 2 семафора. В обоих семафорах нуждаются оба процесса. Каждый из процессов захватил семафор, но для продолжения работы нужно захватить ещё один семафор, который занят другим процессом. Никто свой семафор освободить не может, это тупик.

В современных системах определены **наборы** считающих семафоров. Семантически наборы определены в ОС как массивы.

9.4 Задачи синхронизации

9.4.1 Задача производства — потребления (Producer — Consumer)

Есть 2 типа процессов:

- 1) Процессы-производители, которые производят единицу каких-то данных и помещают их в ячейки буфера / хранилища, т.е. производители могут только производить и помещать произведенные данные в буфер.
- 2) Процессы-потребители, которые могут только выбирать данные из буфера. В результате ячейка освобождается, и второй раз из этой же ячейки ничего взять невозможно.

Решение задачи предложил Дейкстра. Его решение основывается на 3 семафорах: двух считающих и одном бинарном.

- Считающие семафоры: *Empty*, *Sfull*
- Бинарные семафор: *Sbinary*

1	// Начальные значения:
2	// <i>Empty</i> = N
3	// <i>Sfull</i> = 0
4	// <i>Sbinary</i> = 1
5	producer:
6	P(<i>Empty</i>)
7	P(<i>Sbinary</i>)
8	N = N + 1
9	V(<i>Sbinary</i>)
10	V(<i>Sfull</i>)
11	
12	consumer:
13	P(<i>Sfull</i>)
14	P(<i>Sbinary</i>)
15	N = N - 1
16	V(<i>Sbinary</i>)
17	V(<i>Empty</i>)

- 1) Производитель пытается декрементировать семафор *empty* (проверка наличия свободных ячеек). Если ему это удалось, он пытается декрементировать (захватить) бинарный семафор для монопольного доступа к буферу, увеличивает количество занятых ячеек буфера, поскольку ему удалось положить что-то в ячейку буфера, после этого освобождает бинарный семафор и инкрементирует семафор *sfull*.
- 2) Потребитель пытается декрементировать *sfull* (проверка наличия заполненных ячеек). Если удалось, пытается захватить бинарный семафор, если удалось, освобождает ячейку, освобождает бинарный семафор и инкрементирует *empty*.

Producer будет блокирован на `empty`, если нет свободных ячеек в буфере. Он не может ни в одну ячейку не может положить произведённые данные. Он будет блокирован, пока Consumer не освободит хотя бы одну ячейку буфера. Если Producer удалось положить данные в ячейку буфера, он декрементирует количество свободных ячеек. Когда он выходит из своей критической секции, он увеличивает количество заполненных ячеек буфера.

Бинарный семафор обеспечивает монопольный доступ к буферу.

Consumer наоборот будет блокирован, если нет заполненных ячеек в буфере. Если есть хотя бы одна заполненная ячейка, Consumer декрементирует семафор `sfull`, захватывает семафор, декрементирует количество заполненных ячеек, освобождает семафор и инкрементирует `Sempty` количество свободных ячеек буфера.

Смысл в том что процессы являются асинхронными, то есть процессы выполняются с собственной скоростью. Предсказать, когда процесс придёт в свою определённую точку невозможно. Поэтому возможна ситуация когда потребители быстрее освобождают ячейку буфера, чем производители их заполняют, то есть возможны ситуация когда все ячейки буфера пустые, тогда потребитель будет ждать производителей.

И наоборот, возможна ситуация, когда производители работают быстрее потребителей, заполняют все ячейки буфера, и вынуждены ждать, когда потребители освободят все ячейки буфера. Таким образом происходит синхронизация работы производителей и потребителей.

Несмотря на то, что тут три семафора, тут тупик не возникнет, поскольку такая задача грамотно решённая и она очень показательна именно с точки зрения, что тут используются считающие семафоры.

9.6.1 Отличия от семафоров

От перечисленных средств семафоры отличаются тем, что у семафора нет хозяина. Мьютекс может освободить только тот процесс, который его захватил; это касается и

других средств взаимного исключения. Семафор же может освободить другой процесс, который, зная идентификатор, может получить дескриптор семафора.

В качестве отдельного средства взаимодействия процессов, рассматриваются сообщения и системные вызовы отправки и приёма сообщений. Сообщение — специальный метод обмена данными, соответственно, используются специальные системные вызовы.

Также, кроме перечисленных ранее способов взаимного исключения (программный, аппаратный, с помощью семафоров, мониторов), существует рандеву.

2. Процессы: бесконечное откладывание, зависание, тупиковая ситуация - анализ на примере задачи об обедающих философах и примеры аналогичных ситуаций в ОС. Множественные семафоры в Linux: системные вызовы, определенные на семафорах, и поддержка семафоров в ОС Unix/Linux; примеры использования из лабораторных работ: «производство-потребление» и «читатели-писатели» с обработчиками сигнала

Ответ:

9.4.2 Задача обедающих философов

В современных операционных системах реализованы наборы считающих семафоров. Работу наборов продемонстрирует задача обедающих философов, которую также сформулировал Дейкстра

На круглом столе стоят 5 тарелок, между которым лежит по одному прибору. Философ может поесть только если ему удалось захватить сразу два прибора: левый и правый. Иначе он будет «голодать». Он думает некоторое время, потом пытается поесть. То есть пытается захватить приборы, захватив, ест. После этого освобождает приборы, опять какое-то время думает.

Существуют 3 варианта действия философов:

54

- 1) Каждый философ пытается захватить сразу два прибора: левый и правый. Если ему удаётся, он может начать есть. Поев, он освобождает сразу оба прибора.
- 2) Каждый философ сначала захватывает сначала правый прибор. Если ему это удалось, захватывает левый.
- 3) Каждый философ пытается захватить левый прибор. Если удалось, захватить, начинает есть. Если не удалось — освобождает левый.

Первый вариант может привести к «голодной смерти». Умереть может только 1 философ.

Скажем, сначала начальный момент времени второму и пятому удалось захватить оба прибора. Соответственно, первый, третий и четвёртый захватить оба прибора уже не могут. Поев, скажем, второй, освобождает оба прибора. Значит, третий может захватить оба прибора. Второй, первый, четвёртый захватить не мог. Если третий освободит, то, скажем, второй опять может захватить. Четвёртый не может, потому что пятый удерживает. Понятно? Если пятая освобождает, то четвёртый может захватить. Поев, освободит пятый может захватить. Значит, в итоге вот он первый, он никак не может захватить. То второй удерживает, то пятый.

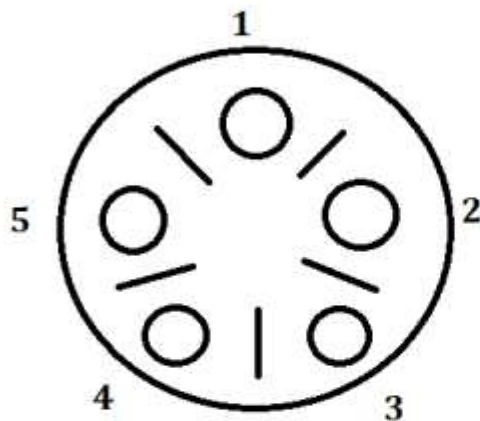
Второй вариант. Каждому удаётся взять правые приборы, левые взять не могут. Всё расхватили. Тупик, блокируют друг друга.

И третий вариант. Взяли правый прибор, не могут взять левый, кладут правый. Опять взяли правый, не могут взять левый, кладут. Захваты, освобождения одних и тех же ресурсов.

Первый вариант — это бесконечное откладывание. Процесс не может захватить необходимые ему ресурсы. Второй вариант — это тупик. И третий вариант — захват и освобождения одних и тех же ресурсов. Работа выполняется системой, но полезные действия при этом отсутствуют. Ничего полезного не делается, всё тратится на захват и освобождение одних и тех же ресурсов.

Обедающие философы

За круглым столом сидят 5 философов. Перед каждым философом стоит тарелка. Между тарелками лежит по 1 палочке (для еды нужно 2).



Существует три способа взаимодействия философов:

- 1) Каждый философ пытается взять сразу оба прибора (левый и правый) и начать есть. Поев, возвращает оба прибора
- 2) Каждый из философов берет правый прибор и пытается взять левый, удерживая правый
- 3) Если не может взять левый, возвращает правый

Проблемы:

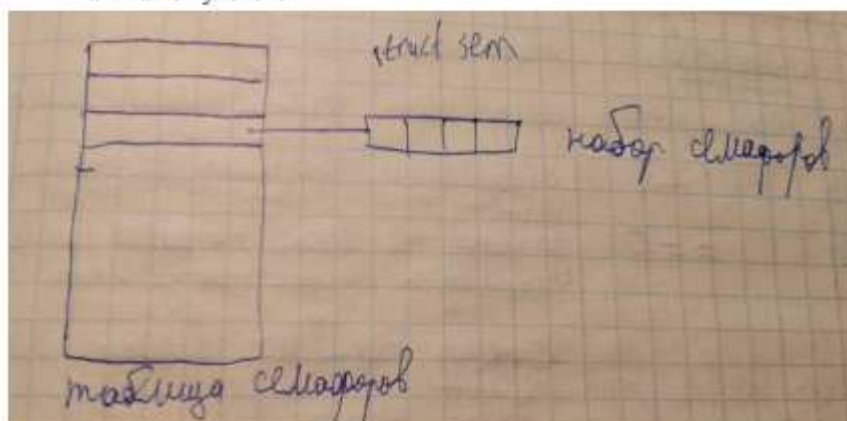
- 1) Приводит к бесконечному откладыванию
 - a) 1 философ умрет голодной смертью
 - b) 2 и 5 берут оба прибора (1, 3, 4 ждут)
 - c) 5 положил, 4 может взять (1, 3, 5 ждут)
 - d) 2 положил, 3 может взять (1, 2, 4 ждут)
 - e) 1 постоянно ждет
- 2) Все взяли правый прибор, пытаются взять левый — классический пример блокировки deadlock
- 3) Все взяли правый, не могут взять левый => возвращают правый и повторяют эти действия.
 - a) В системе процессы захватывают и освобождают одни и те же ресурсы => полезная работа не выполняется

Семафоры UNIX

- Семафоры UNIX вносят дополнения в семафоры Дейкстры.
- В современных системах поддерживаются наборы семафоров
- Наборы семафоров представлены массивами семафоров
- В UNIX LINUX семафоры поддерживаются системной таблицей(находится в адресном пространстве системы) семафоров. Каждый элемент таблицы описывает один набор семафоров (то есть это дескриптор)
- Дескриптор набора семафоров хранит:
 - имя (целое число) – присваивается процессом, который создал набор семафоров. Другие процессы по этому имени могут получить доступ к нему.
 - uid
 - идентификатор создателя наборов семафоров и идентификатор его группы.

Процесс, uid которого совпадает с uid созд. может удалить набор и изменить его существующие права(?)

- права доступа
 - количество семафоров в наборе
 - время изменения 1 или нескольких семафоров последним процессом
 - время последнего изменения управляющих параметров набора каким-либо процессом
 - указатель на массив семафоров
- => в наборе семафоров семафоры обозначаются индексами начиная с нулевого



- На семафорах определены системные вызовы:
 - `semget()` — создание набора семафоров
 - `semctl()` — изменение управляющих параметров набора семафоров
 - `semop()` — операции на наборе семафоров или на семафоре
- Для определения операций на семафорах определена структура

```
struct sembuf
{
    ushort sem_num; //index
    short sem_op; // операция
    short sem_fl; // флаги
}
```

-
- В отличие от семафоров Дейкстры, на семафорах UNIX определено 3 операции
 - `sem_op`:
 - `> 0` — инкремент, освобождение семафора
 - `< 0` — декремент, захват семафора
 - блокирование процесса, если декремент невозможен
 - `== 0` — захват не выполняется, процесс проверяет, свободен ресурс или занят
 - будет переведен в состояние ожидания освобождения ресурса. После его освобождения декремент не выполн.

- Флаги sem_fl:
 - IPC_NOWAIT — информирует систему о нежелании процесса переходить в состояние ожидания (освобождения семафора)
 - Наличие данного флага объясняется желанием избежать блокирования всех процессов, находящихся в очереди процессов, в случае, если захвативший ресурс процесс завершится аварийно или получит сигнал kill
 - Так как kill невозможно перехватить, процесс не может освободить семафор
 - Блокированный процесс не получает процессорного времени
 - SEM_UNDO — данный флаг указывает ядру, что необходимо отслеживать изменение значения семафора в результате вызова semop.
 - Указывает ядру, что при завершении процесса, вызвавшего semop, необходимо ликвидировать сделанные изменения, чтобы другие процессы не были заблокированы на семафорах навечно
- semget возвращает файловый дескриптор созданного набора семафоров
- Семафоры могут служить как источник информации — процесс освобождает семафор как событие. Но это одно значение — свободен или занят. Но есть и другие средства передачи данных.

«производство-потребление»

```
int flag = 1;
void sig_handler(int signal)
{
    flag = 0;
    printf("catch sig: %d, PID: %d\n", signal, getpid());
}

// Структуры для операций с семафорами:
// sem[0] - свободные места (n)
// sem[1] - занятые места (0)
// sem[2] - мьютекс (1)

// Захват свободного места и мьютекса
struct sembuf start_prod[2] = { { 0, -1, 0 }, { 2, -1, 0 } };
// Освобождение мьютекса и увеличение занятых
struct sembuf end_prod[2] = { { 2, 1, 0 }, { 1, 1, 0 } };
// Захват занятого места и мьютекса
struct sembuf start_cons[2] = { { 1, -1, 0 }, { 2, -1, 0 } };
```

```
// Освобождение мьютекса и увеличение свободных
struct sembuf end_cons[2] = { { 2, 1, 0 }, { 0, 1, 0 } };

void producer(int semid, char **prod_ptr, char *symb)
{
    while (flag)
    {
        // Захват семафоров для производителя
        if (semop(semid, start_prod, 2) == -1)
        {
            printf("semop, %d, errno: %d\n", getpid(), errno);
            exit(1);
        }

        // Критическая секция - производство
        **prod_ptr = *symb;
        printf("producer: %d, %c\n", getpid(), **prod_ptr);
        // ...

        // Освобождение семафоров
        if (semop(semid, end_prod, 2) == -1)
        {
            printf("semop, %d, errno: %d\n", getpid(), errno);
            exit(1);
        }
    }
}

void consumer(int semid, char **cons_ptr)
{
    while (flag)
    {
        // Захват семафоров для потребителя
        if (semop(semid, start_cons, 2) == -1)
        {
            printf("semop, %d, errno: %d\n", getpid(), errno);
            exit(1);
        }

        // Критическая секция - потребление
        printf("consumer: %d, %c\n", getpid(), **cons_ptr);
        // ...

        // Освобождение семафоров
        if (semop(semid, end_cons, 2) == -1)
        {
            printf("semop, %d, errno: %d\n", getpid(), errno);
            exit(1);
        }
    }
}

int main()
{
```

```

...

// Установка обработчика сигнала для корректного завершения
if (signal(SIGALRM, sig_handler) == (void (*)(int)) -1)
{
    printf("signal, errno: %d\n", errno);
    exit(1);
}

// Создание массива семафоров (3 семафора)
if ((semid = semget(key, 3, IPC_CREAT | perms)) == -1)
{
    printf("semget, errno: %d\n", errno);
    exit(1);
}

// Инициализация семафоров:
if (semctl(semid, 0, SETVAL, n) == -1)    // Свободные места = n
if (semctl(semid, 1, SETVAL, 0) == -1)    // Занятые места = 0
if (semctl(semid, 2, SETVAL, 1) == -1)    // Мьютекс = 1 (свободен)

// Создание дочерних процессов
for (int i = 0; i < 6; ++i)
{
    if ((childs[i] = fork()) == -1) { ... }
    else if (childs[i] == 0)
    {
        if (i % 2 == 0)
            producer(semid, prod_ptr, symb);
        else
            consumer(semid, cons_ptr);
        exit(0);
    }
}

// Ожидание завершения всех процессов
...

// Удаление семафоров
if (semctl(semid, 0, IPC_RMID, NULL) == -1)
{
    printf("semctl, errno: %d\n", errno);
    exit(1);
}

return 0;
}

```

«читатели-писатели»

```

#include <sys/sem.h>
#include <sys/shm.h>

```

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <signal.h>

// Определение семафоров:
#define WAIT_WRITERS 0 // Ожидющие писатели
#define WAIT_READERS 1 // Ожидющие читатели
#define ACTIVE_WRITER 2 // Активный писатель (0 или 1)
#define ACTIVE_READERS 3 // Количество активных читателей

// Операции для читателя:
struct sembuf start_read[] = {
    // Увеличить счетчик ожидающих читателей
    {WAIT_READERS, 1, 0},
    // Проверить, что нет активного писателя (ожидание = 0)
    {ACTIVE_WRITER, 0, 0},
    // Проверить, что нет ожидающих писателей (ожидание = 0)
    {WAIT_WRITERS, 0, 0},
    // Уменьшить счетчик ожидающих читателей
    {WAIT_READERS, -1, 0},
    // Увеличить счетчик активных читателей
    {ACTIVE_READERS, 1, 0},
};

struct sembuf stop_read[] = {
    // Уменьшить счетчик активных читателей
    {ACTIVE_READERS, -1, 0}
};

// Операции для писателя:
struct sembuf start_write[] = {
    // Увеличить счетчик ожидающих писателей
    {WAIT_WRITERS, 1, 0},
    // Проверить, что нет активных читателей (ожидание = 0)
    {ACTIVE_READERS, 0, 0},
    // Проверить, что нет активного писателя (ожидание = 0)
    {ACTIVE_WRITER, 0, 0},
    // Уменьшить счетчик ожидающих писателей
    {WAIT_WRITERS, -1, 0},
    // Установить активного писателя (увеличить с 0 до 1)
    {ACTIVE_WRITER, 1, 0}
};

struct sembuf stop_write[] = {
    // Сбросить активного писателя (уменьшить с 1 до 0)
    {ACTIVE_WRITER, -1, 0}
};

int flag = 1;

void sig_handler(int signal) {
    flag = 0;
    printf("catch sig: %d\n", signal);
}
```

```
}

void reader(key_t semfd, char *var) {
    while (flag) {
        // Захват ресурса для чтения
        // (могут читать несколько читателей одновременно)
        if (semop(semfd, start_read, 5) == -1) {
            if (errno == EINTR) {
                printf("errno EINTR: PID %d\n", getpid());
                exit(1);
            }
            printf("semop: %s\n", strerror(errno));
            exit(1);
        }

        // Критическая секция чтения
        printf("reader: %d read %c\n", getpid(), *var);

        // Освобождение ресурса
        if (semop(semfd, stop_read, 1) == -1) {
            printf("semop: %s\n", strerror(errno));
            exit(1);
        }
    }
}

void writer(key_t semfd, char *var) {
    while (flag) {
        // Захват ресурса для записи (только один писатель)
        if (semop(semfd, start_write, 5) == -1) {
            if (errno == EINTR) {
                printf("errno EINTR: PID %d\n", getpid());
                exit(1);
            }
            printf("semop: %s\n", strerror(errno));
            exit(1);
        }

        // Критическая секция записи (эксклюзивный доступ)
        (*var)++;
        if (*var == 'z' + 1)
            *var = 'a';
        printf("writer: %d wrote %c\n", getpid(), *var);

        // Освобождение ресурса
        if (semop(semfd, stop_write, 1) == -1) {
            printf("semop: %s\n", strerror(errno));
            exit(1);
        }
    }
}

int main(void) {
    ...
}
```

```
// Установка обработчика сигнала
if (signal(SIGALRM, sig_handler) == (void (*)(int)) -1) {
    printf("signal: %s\n", strerror(errno));
    exit(1);
}

// Создание массива из 4 семафоров
int semfd = semget(IPC_PRIVATE, 4, IPC_CREAT | perms);
if (semfd == -1) {
    printf("semget: %s\n", strerror(errno));
    exit(1);
}

// Инициализация семафоров:
if (semctl(semfd, 3, SETVAL, 0) == -1) // ACTIVE_READERS = 0
if (semctl(semfd, 2, SETVAL, 0) == -1) // ACTIVE_WRITER = 0
if (semctl(semfd, 0, SETVAL, 0) == -1) // WAIT_WRITERS = 0
if (semctl(semfd, 1, SETVAL, 0) == -1) // WAIT_READERS = 0

// Создание процессов читателей и писателей
for (int i = 0; i < readers + writers; ++i) {
    if ((wchlds[i] = fork()) == -1) {
        printf("fork: %s\n", strerror(errno));
        exit(1);
    }
    if (wchlds[i] == 0) {
        if (i % 2 == 0)
            reader(semfd, buf);
        else
            writer(semfd, buf);
        exit(0);
    }
}

// Ожидание завершения процессов
...

// Удаление семафоров
if (semctl(semfd, IPC_RMID, 0) == -1) {
    printf("semctl: %s\n", strerror(errno));
    exit(1);
}

exit(0);
}
```